# Tina4 for PHP Developers

This is not a framework

Andre van Zuydam

v3.1.0 - 2026

38 features. Zero dependencies. 65,000 words.

# Table of Contents

1. Chapter 1: Getting Started with Tina4 PHP
2. Chapter 2: Routing
3. Chapter 3: Request & Response
4. Chapter 4: Templates
5. Chapter 5: Database
6. Chapter 6: ORM
7. Chapter 7: Authentication
8. Chapter 8: Middleware
9. Chapter 9: Sessions & Cookies
10. Chapter 10: API Documentation with Swagger
11. Chapter 11: Queue System
12. Chapter 12: Real-time with WebSocket
13. Chapter 13: Email with Messenger
14. Chapter 14: Caching
15. Chapter 15: Frontend with tina4css
16. Chapter 16: GraphQL
17. Chapter 17: Testing
18. Chapter 18: Dev Tools
19. Chapter 19: CLI & Scaffolding
20. Chapter 20: Deployment
21. Chapter 21: Building a Complete App

# Chapter 1: Getting Started with Tina4 PHP

## 1. What Is Tina4 PHP

Tina4 PHP is a zero-dependency web framework for PHP 8.1+ that gives you routing, an ORM, a template engine, authentication, queues, WebSocket, and 70 other features in a single Composer package under 5,000 lines of code.

It is part of the Tina4 family -- four identical frameworks in Python, PHP, Ruby, and Node.js. Everything you learn here transfers directly to any of the other three languages. The project structure is the same. The template syntax is the same. The CLI commands are the same. The `.env` variables are the same.

Tina4 PHP uses `camelCase` for method names (`fetchOne()`, `softDelete()`, `hasMany()`) following PHP community conventions. Class names are `PascalCase`. Constants are `UPPER_SNAKE_CASE`.

By the end of this chapter, you will have a working Tina4 PHP project with an API endpoint and a rendered HTML page.

## 2. Prerequisites and Installation

### *What You Need*

Before installing Tina4, make sure you have:

1. **PHP 8.1 or later** -- check with:

```
php -v
```

You should see output like:

```
PHP 8.3.4 (cli) (built: Feb 13 2026 09:27:45) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.3.4, Copyright (c) Zend Technologies
    with Zend OPcache v8.3.4, Copyright (c), by Zend Technologies
```

If you see a version lower than 8.1, upgrade PHP first.

2. **Composer** -- PHP's package manager. Check with:

```
composer --version
```

You should see:

```
Composer version 2.7.2 2024-03-11 17:12:18
```

If Composer is not installed, get it from [https://getcomposer.org](https://getcomposer.org).

3. **The Tina4 CLI** -- a Rust-based binary that manages all four Tina4 frameworks:

```
# macOS (Homebrew)
brew install tina4stack/tap/tina4

# Linux / macOS (install script)
curl -fsSL https://raw.githubusercontent.com/tina4stack/tina4/main/install.sh | bash

# Windows (PowerShell)
irm https://raw.githubusercontent.com/tina4stack/tina4/main/install.ps1 | iex
```

Verify the CLI is installed:

```
tina4 --version
```

```
tina4 0.1.0
```

4. **Required PHP extensions** -- these ship with most PHP installations:

• `ext-json` (bundled since PHP 8.0)

• `ext-mbstring`

• `ext-openssl`

• `ext-sqlite3` (for the default SQLite database)

• `ext-fileinfo`

Check if they are available:

```
php -m | grep -E "json|mbstring|openssl|sqlite3|fileinfo"
```

```
fileinfo
json
mbstring
openssl
sqlite3
```

If any are missing, install them via your OS package manager (e.g., `apt install php8.3-mbstring` on Ubuntu).

### *Creating a New Project*

Use the Tina4 CLI to scaffold a new project:

```
tina4 init my-store
```

You should see:

```
Creating Tina4 project in ./my-store ...
  Detected language: PHP (composer.json)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created src/migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-store
// ...
```

Now install the PHP dependencies:

```
cd my-store
composer install
```

```
Installing dependencies from lock file
  - Installing tina4/tina4-php (v3.0.0): Extracting archive
Generating autoload files
1 package installed
```

That is right -- **one package**. No dependency tree. No version conflicts. Just `tina4/tina4-php`.

### *Starting the Dev Server*

```
tina4 serve
```

```
 _____ _             _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|

  Tina4 PHP v3.0.0
  Server running at http://0.0.0.0:7145
  Debug mode: ON
  Database: sqlite:///data/app.db
  Press Ctrl+C to stop
```

Open your browser to `http://localhost:7145`. You should see the Tina4 welcome page.

Open `http://localhost:7145/health` in your browser or curl it:

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 12,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

Your Tina4 PHP project is running.

## 3. Project Structure Walkthrough

Let us look at what `tina4 init` created:

```
my-store/
■■■ .env                    # Your configuration (gitignored)
■■■ .env.example            # Template for other developers
■■■ .gitignore              # Pre-configured
■■■ composer.json           # Composer package definition
■■■ composer.lock           # Locked dependency versions
■■■ vendor/                 # Composer packages (gitignored)
■■■ src/
■   ■■■ routes/             # Your route handlers go here
■   ■■■ orm/                # Your ORM model classes go here
■   ■■■ migrations/         # SQL migration files
■   ■■■ seeds/              # Database seed files
■   ■■■ templates/          # Frond/Twig templates
■   ■   ■■■ errors/         # Custom 404.html, 500.html
■   ■■■ public/             # Static files (CSS, JS, images)
■   ■   ■■■ js/
■   ■   ■   ■■■ frond.js    # Auto-provided JS helper library
```

```
■   ■   ■■■ css/
■   ■   ■   ■■■ tina4.css   # Built-in CSS utility framework
■   ■   ■■■ scss/
■   ■   ■■■ images/
■   ■   ■■■ icons/
■   ■■■ locales/            # Translation files
■       ■■■ en.json
■■■ data/                   # SQLite databases (gitignored)
// ...
```

**Key directories:**

• **`src/routes/`** -- Every `.php` file here is auto-loaded at startup. Put your route definitions here. Organize into subdirectories if you want.

• **`src/orm/`** -- Every `.php` file here is auto-loaded. Put your ORM model classes here.

• **`src/templates/`** -- Frond looks here when you call `$response->render("my-page.html", $data)`.

• **`src/public/`** -- Files served directly. `src/public/images/logo.png` is available at `/images/logo.png`.

• **`data/`** -- The default SQLite database (`app.db`) lives here. Gitignored because databases should not be in version control.

## 4. Your First Route

Let us create an API endpoint that returns a JSON greeting.

Create the file `src/routes/greeting.php`:

```php
<?php
use Tina4\Route;

Route::get("/api/greeting/{name}", function ($request, $response) {
    $name = $request->params["name"];
    return $response->json([
        "message" => "Hello, " . $name . "!",
        "timestamp" => date("c")
    ]);
});
```

Save the file. If the dev server is running with live reload, it picks up the change automatically. If not, restart the server with `tina4 serve`.

### *Test It*

Open your browser to:

```
http://localhost:7145/api/greeting/Alice
```

You should see:

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

Or use curl:

```
curl http://localhost:7145/api/greeting/Alice
```

```
{"message":"Hello, Alice!","timestamp":"2026-03-22T14:30:00+00:00"}
```

Notice the difference: the browser shows pretty-printed JSON (because of browser extensions or dev mode), while curl shows compact JSON. You can force pretty output by adding `?pretty=true`:

```
curl "http://localhost:7145/api/greeting/Alice?pretty=true"
```

```
{
  "message": "Hello, Alice!",
  "timestamp": "2026-03-22T14:30:00+00:00"
}
```

### *Understanding What Happened*

1. You created a file in `src/routes/`. Tina4 auto-discovered it at startup.

2. `Route::get("/api/greeting/{name}", ...)` registered a GET route with a path parameter `{name}`.

3. When you requested `/api/greeting/Alice`, the router matched the pattern and called your handler function.

4. `$request->params["name"]` gave you the value `"Alice"` from the URL.

5. `$response->json(...)` serialized the array to JSON, set `Content-Type: application/json`, and returned a `200 OK` response.

### *Adding More HTTP Methods*

Let us add a POST endpoint. Update `src/routes/greeting.php`:

```
<?php
use Tina4\Route;

Route::get("/api/greeting/{name}", function ($request, $response) {
    $name = $request->params["name"];
    return $response->json([
        "message" => "Hello, " . $name . "!",
        "timestamp" => date("c")
    ]);
});

Route::post("/api/greeting", function ($request, $response) {
    $name = $request->body["name"] ?? "World";
    $language = $request->body["language"] ?? "en";

    $greetings = [
        "en" => "Hello",
        "es" => "Hola",
        "fr" => "Bonjour",
        "de" => "Hallo",
        "ja" => "Konnichiwa"
    ];

    $greeting = $greetings[$language] ?? $greetings["en"];

// ...
```

Test the POST endpoint:

```
curl -X POST http://localhost:7145/api/greeting \
  -H "Content-Type: application/json" \
  -d '{"name": "Carlos", "language": "es"}'
```

```
{"message":"Hola, Carlos!","language":"es"}
```

The HTTP status code is `201 Created` (the second argument to `$response->json()`).

# 5. Your First Template

Tina4 uses the **Frond** template engine -- a zero-dependency, Twig-compatible engine built from scratch. If you have used Twig, Jinja2, or Nunjucks, this will feel familiar.

## *Create a Base Layout*

Create `src/templates/base.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        .product-card { border: 1px solid #ddd; border-radius: 8px; padding: 16px; margin: 8px 0; }
        .product-card h3 { margin-top: 0; }
        .price { color: #2d8f2d; font-weight: bold; font-size: 1.2em; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.8em; }
        .badge-success { background: #d4edda; color: #155724; }
        .badge-danger { background: #f8d7da; color: #721c24; }
        nav { background: #333; color: white; padding: 12px 20px; }
        nav a { color: white; text-decoration: none; margin-right: 16px; }
    </style>
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
    </nav>
// ...
```

This base layout defines two blocks (`title` and `content`) that child templates can override. It includes `tina4.css` (the built-in CSS framework) and `frond.js` (the built-in JS helper library).

## *Create a Product Listing Page*

Create `src/templates/products.html`:

```
{% extends "base.html" %}

{% block title %}Products - My Store{% endblock %}

{% block content %}
    <h1>Our Products</h1>
    <p>Showing {{ products | length }} product{{ products | length != 1 ? "s" : "" }}</p>

    {% if products | length > 0 %}
        {% for product in products %}
            <div class="product-card">
                <h3>{{ product.name }}</h3>
                <p>{{ product.description }}</p>
                <p class="price">${{ product.price | number_format(2) }}</p>
                {% if product.in_stock %}
                    <span class="badge badge-success">In Stock</span>
                {% else %}
```

```
                    <span class="badge badge-danger">Out of Stock</span>
                {% endif %}
                {% if not loop.last %}
                    {# Don't add separator after the last item #}
                {% endif %}
            </div>
        {% endfor %}
    {% else %}
// ...
```

### Create the Route That Renders the Template

Create `src/routes/pages.php`:

```
<?php
use Tina4\Route;

Route::get("/products", function ($request, $response) {
    $products = [
        [
            "name" => "Wireless Keyboard",
            "description" => "Ergonomic wireless keyboard with backlit keys.",
            "price" => 79.99,
            "in_stock" => true
        ],
        [
            "name" => "USB-C Hub",
            "description" => "7-port USB-C hub with HDMI, SD card reader, and Ethernet.",
            "price" => 49.99,
            "in_stock" => true
        ],
        [
            "name" => "Monitor Stand",
            "description" => "Adjustable aluminum monitor stand with cable management.",
            "price" => 129.99,
            "in_stock" => false
        ],
        [
            "name" => "Mechanical Mouse",
// ...
```

### See It in the Browser

Open `http://localhost:7145/products` in your browser. You should see:

• A dark navigation bar at the top with "Home" and "Products" links

• The heading "Our Products"

• A subheading showing "Showing 4 products"

• Four product cards, each with a name, description, price, and stock badge

• The "Monitor Stand" card shows a red "Out of Stock" badge

• The other three show green "In Stock" badges

### How Template Rendering Works

1. `$response->render("products.html", ["products" => $products])` tells Frond to render `src/templates/products.html` with the given data.

2. Frond sees `{% extends "base.html" %}` and loads the base template.

3. The `{% block content %}` in `products.html` replaces the same block in `base.html`.

4. `{{ product.name }}` outputs the value, auto-escaped for HTML safety.

5. `{{ product.price | number_format(2) }}` formats the number with 2 decimal places.

6. `{% for product in products %}` loops through the array.

7. `{% if product.in_stock %}` conditionally renders the stock badge.

8. `{{ products | length }}` returns the count of items in the array.

### *About tina4css*

The `tina4.css` file included in the base template is Tina4's built-in CSS utility framework. It provides layout utilities, typography, and common UI patterns without needing Bootstrap or Tailwind. It is auto-provided when you scaffold a project -- you do not need to download it separately.

## 6. Understanding .env

Open the `.env` file at the root of your project:

```
TINA4_DEBUG=true
```

That is likely all you see. The scaffold creates a minimal `.env` with debug mode enabled. Everything else uses defaults.

The important defaults for development:

| Variable | Default Value | What It Means |

|----------|---------------|---------------|

| `TINA4_PORT` | `7145` | Server runs on port 7145 |

| `DATABASE_URL` | `sqlite:///data/app.db` | SQLite database in the `data/` directory |

| `TINA4_LOG_LEVEL` | `ALL` | All log messages are output |

| `CORS_ORIGINS` | `*` | All origins allowed (fine for development) |

| `TINA4_RATE_LIMIT` | `60` | 60 requests per minute per IP |

To change the port, add it to `.env`:

```
TINA4_DEBUG=true
TINA4_PORT=8080
```

Restart the server (`Ctrl+C`, then `tina4 serve`). It now runs on port 8080.

For the complete `.env` reference with all 68 variables, see Book 0, Chapter 4.

## 7. The Dev Dashboard

With `TINA4_DEBUG=true`, Tina4 provides a built-in development dashboard. First, enable it by adding a console token to your `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
```

```
TINA4_CONSOLE_TOKEN=my-dev-token
```

Restart the server and navigate to:

```
http://localhost:7145/tina4/console
```

Enter your token (`my-dev-token`) when prompted. You will see:

• **System Overview** -- framework version, PHP version, uptime, memory usage, database status

• **Request Inspector** -- recent HTTP requests with method, path, status, duration, and request ID. Click any request to see full headers, body, database queries, and template renders.

• **Error Log** -- unhandled exceptions with stack traces and occurrence counts

• **Queue Manager** -- queue status (pending, reserved, failed, dead-letter messages)

• **WebSocket Monitor** -- active WebSocket connections with metadata

• **Routes** -- all registered routes with their methods, paths, and middleware

The console is a powerful debugging tool. It shows you exactly what your application is doing without adding print statements or log calls to your code.

Additionally, when you visit any HTML page (like `/products`), you will see a **debug overlay** -- a toolbar at the bottom of the page showing:

• Request details (method, URL, duration)

• Database queries executed (with timing)

• Template renders (with timing)

• Session data

• Recent log entries

This overlay is only visible when `TINA4_DEBUG=true`. It is never shown in production.

## 8. Exercise: Greeting API + Product List Template

Build the following two features from scratch, without looking at the examples above.

### *Exercise Part A: Greeting API*

Create an API endpoint at `GET /api/greet` that:

1. Accepts a query parameter `name` (e.g., `/api/greet?name=Sarah`)

2. If `name` is missing, defaults to `"Stranger"`

3. Returns JSON like:

```
{
  "greeting": "Welcome, Sarah!",
  "time_of_day": "afternoon"
}
```

4. The `time_of_day` should be calculated from the server's current hour:

• 5:00 - 11:59 = "morning"

• 12:00 - 16:59 = "afternoon"

• 17:00 - 20:59 = "evening"

• 21:00 - 4:59 = "night"

**Test your endpoint with:**

```
curl "http://localhost:7145/api/greet?name=Sarah"
curl "http://localhost:7145/api/greet"
```

### *Exercise Part B: Product List Page*

Create a page at `GET /store` that:

1. Displays a list of at least 5 products (hardcoded for now)

2. Each product has: name, category, price, and a boolean `featured` flag

3. Featured products should be visually highlighted (different background color, border, or badge)

4. The page should show the total number of products and the number of featured products

5. Use template inheritance -- create a layout template and a page template that extends it

6. Include `tina4.css` and `frond.js`

**Your products data should look like this in your route handler:**

```
$products = [
    ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "featured" => true],
    ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "featured" => false],
    ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "featured" => true],
    ["name" => "Noise-Canceling Headphones", "category" => "Electronics", "price" => 199.99, "featured" => tru
    ["name" => "Water Bottle", "category" => "Fitness", "price" => 24.99, "featured" => false]
];
```

**Expected browser output:**

• A page titled "Our Store"

• Text showing "5 products, 3 featured"

• A list of product cards with name, category, price, and a "Featured" badge on the highlighted items

• Featured products have a distinct visual style (your choice -- different border color, background, star icon, etc.)

## 9. Solutions

### *Solution A: Greeting API*

Create `src/routes/greet.php`:

```
<?php
use Tina4\Route;

Route::get("/api/greet", function ($request, $response) {
    $name = $request->query["name"] ?? "Stranger";
    $hour = (int) date("G");

    if ($hour >= 5 && $hour < 12) {
        $timeOfDay = "morning";
```

```
    } elseif ($hour >= 12 && $hour < 17) {
        $timeOfDay = "afternoon";
    } elseif ($hour >= 17 && $hour < 21) {
        $timeOfDay = "evening";
    } else {
        $timeOfDay = "night";
    }

    return $response->json([
        "greeting" => "Welcome, " . $name . "!",
        "time_of_day" => $timeOfDay
    ]);
});
```

**Test:**

```
curl "http://localhost:7145/api/greet?name=Sarah"

{"greeting":"Welcome, Sarah!","time_of_day":"afternoon"}

curl "http://localhost:7145/api/greet"

{"greeting":"Welcome, Stranger!","time_of_day":"afternoon"}
```

### *Solution B: Product List Page*

Create `src/templates/store-layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Store{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 960px; margin: 0 auto; padding: 20px; }
        header { background: #1a1a2e; color: white; padding: 16px 20px; }
        header h1 { margin: 0; }
        .stats { color: #888; margin: 8px 0 20px; }
        .product-grid { display: grid; gap: 16px; }
        .product-card { background: white; border: 2px solid #e0e0e0; border-radius: 8px; padding: 16px; }
        .product-card.featured { border-color: #ffc107; background: #fffdf0; }
        .product-name { font-size: 1.2em; font-weight: bold; margin: 0 0 4px; }
        .product-category { color: #666; font-size: 0.9em; }
        .product-price { color: #2d8f2d; font-weight: bold; font-size: 1.1em; margin-top: 8px; }
        .featured-badge { background: #ffc107; color: #333; padding: 2px 8px; border-radius: 4px; font-size: 
    </style>
</head>
<body>
    <header>
        <h1>{% block header %}Store{% endblock %}</h1>
// ...
```

Create `src/templates/store.html`:

```
{% extends "store-layout.html" %}

{% block title %}Our Store{% endblock %}
{% block header %}Our Store{% endblock %}

{% block content %}
    <p class="stats">{{ products | length }} products, {{ featured_count }} featured</p>

    <div class="product-grid">
```

```
        {% for product in products %}
            <div class="product-card{{ product.featured ? ' featured' : '' }}">
                <p class="product-name">
                    {{ product.name }}
                    {% if product.featured %}
                        <span class="featured-badge">Featured</span>
                    {% endif %}
                </p>
                <p class="product-category">{{ product.category }}</p>
                <p class="product-price">${{ product.price | number_format(2) }}</p>
            </div>
        {% endfor %}
    </div>
{% endblock %}
```

Create `src/routes/store.php`:

```
<?php
use Tina4\Route;

Route::get("/store", function ($request, $response) {
    $products = [
        ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "featured" => true],
        ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "featured" => false],
        ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "featured" => true],
        ["name" => "Noise-Canceling Headphones", "category" => "Electronics", "price" => 199.99, "featured" =>
        ["name" => "Water Bottle", "category" => "Fitness", "price" => 24.99, "featured" => false]
    ];

    $featuredCount = count(array_filter($products, fn($p) => $p["featured"]));

    return $response->render("store.html", [
        "products" => $products,
        "featured_count" => $featuredCount
    ]);
});
```

**Open `http://localhost:7145/store` in your browser.** You should see:

• A dark header reading "Our Store"

• Text showing "5 products, 3 featured"

• Five product cards in a grid

• Three cards (Espresso Machine, Standing Desk, Noise-Canceling Headphones) have a yellow border, light yellow background, and a "Featured" badge

• Two cards (Yoga Mat, Water Bottle) have a standard white background with gray border

• Each card shows the product name, category, and price formatted with two decimal places

## 10. Gotchas

### *1. File not auto-discovered*

**Problem:** You created a route file but nothing happens when you visit the URL.

**Cause:** The file is not in `src/routes/`. Double-check the path. It must be inside `src/routes/` (or a subdirectory of it), and the file must end with `.php`.

**Fix:** Move the file to `src/routes/your-file.php` and restart the server.

### 2. "Class not found" errors

**Problem:** `Class 'Tina4\Route' not found` or similar.

**Cause:** Missing `use` statement or Composer autoload is out of date.

**Fix:** Make sure your route file starts with `<?php` and includes `use Tina4\Route;`. Run `composer dump-autoload` if the error persists.

### 3. JSON response shows HTML

**Problem:** Your JSON endpoint returns HTML instead of JSON.

**Cause:** You returned a string instead of using `$response->json()`. If you return a plain string, Tina4 treats it as HTML.

**Fix:** Always use `$response->json($data)` for JSON endpoints, not `echo json_encode($data)`.

### 4. Template not found

**Problem:** `Template "my-page.html" not found` error.

**Cause:** The template file is not in `src/templates/`, or there is a typo in the filename.

**Fix:** Check that the file exists at `src/templates/my-page.html`. The name in `$response->render()` is relative to `src/templates/`.

### 5. Port already in use

**Problem:** `Error: Address already in use (port 7145)`

**Cause:** Another process (or another Tina4 instance) is using port 7145.

**Fix:** Either stop the other process, or change the port:

```
TINA4_PORT=8080
```

Or use the CLI flag: `tina4 serve --port 8080`.

### 6. Changes not reflected

**Problem:** You edited a file but the browser shows the old version.

**Cause:** In some setups, live reload may not be active. Also, browser caching can serve stale versions.

**Fix:** Hard-refresh the browser (`Ctrl+Shift+R` or `Cmd+Shift+R`). If that does not help, restart the dev server with `Ctrl+C` and `tina4 serve`.

### 7. .env not loaded

**Problem:** Environment variables seem to have no effect.

**Cause:** The `.env` file must be at the project root (same directory as `composer.json`). If it is in a subdirectory, Tina4 will not find it.

**Fix:** Move `.env` to the project root.

### 8. Debug mode in production

**Problem:** Your production site shows stack traces and query details.

**Cause:** `TINA4_DEBUG=true` in production.

**Fix:** Set `TINA4_DEBUG=false` in your production `.env`. This hides all debug information, enables HTML minification, and activates `.broken` file health checks.

# Chapter 2: Routing

## 1. How Routing Works in Tina4

Every web application maps URLs to code. You type `/products` in your browser, the framework finds the function that handles `/products`, runs it, and sends back the result. That mapping is called routing.

In Tina4, you define routes in PHP files inside `src/routes/`. Every `.php` file in that directory (and its subdirectories) is auto-loaded when the server starts. You do not need to register files or update a central config -- just drop a file in and it works.

Here is the simplest possible route:

```
<?php
use Tina4\Route;

Route::get("/hello", function ($request, $response) {
    return $response->json(["message" => "Hello, World!"]);
});
```

Save that as `src/routes/hello.php`, start the server with `tina4 serve`, and visit `http://localhost:7145/hello`:

```
{"message":"Hello, World!"}
```

That is it. One line registers the route, one line handles the request.

## 2. HTTP Methods

Tina4 supports all five standard HTTP methods. Each one has a static method on the `Route` class:

```
<?php
use Tina4\Route;

Route::get("/products", function ($request, $response) {
    return $response->json(["action" => "list all products"]);
});

Route::post("/products", function ($request, $response) {
    return $response->json(["action" => "create a product"], 201);
});

Route::put("/products/{id}", function ($request, $response) {
    $id = $request->params["id"];
    return $response->json(["action" => "replace product " . $id]);
});

Route::patch("/products/{id}", function ($request, $response) {
    $id = $request->params["id"];
    return $response->json(["action" => "update product " . $id]);
});

Route::delete("/products/{id}", function ($request, $response) {
    $id = $request->params["id"];
    return $response->json(["action" => "delete product " . $id]);
});
```

Test each one:

```
curl http://localhost:7145/products
```

```
{"action":"list all products"}
```

```
curl -X POST http://localhost:7145/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget"}'
```

```
{"action":"create a product"}
```

```
curl -X PUT http://localhost:7145/products/42
```

```
{"action":"replace product 42"}
```

```
curl -X PATCH http://localhost:7145/products/42
```

```
{"action":"update product 42"}
```

```
curl -X DELETE http://localhost:7145/products/42
```

```
{"action":"delete product 42"}
```

Use `GET` for reading, `POST` for creating, `PUT` for full replacement, `PATCH` for partial updates, and `DELETE` for removal. This follows the REST convention and makes your API predictable.

## 3. Path Parameters

Path parameters let you capture values from the URL. Wrap the parameter name in curly braces:

```
<?php
use Tina4\Route;

Route::get("/users/{id}/posts/{postId}", function ($request, $response) {
    $userId = $request->params["id"];
    $postId = $request->params["postId"];

    return $response->json([
        "user_id" => $userId,
        "post_id" => $postId
    ]);
});
```

```
curl http://localhost:7145/users/5/posts/99
```

```
{"user_id":"5","post_id":"99"}
```

Notice that `user_id` came back as the string `"5"`, not the integer `5`. Path parameters are always strings by default.

### *Typed Parameters*

You can enforce a type by adding a colon and the type after the parameter name:

```
<?php
use Tina4\Route;

Route::get("/orders/{id:int}", function ($request, $response) {
    $id = $request->params["id"]; // This is now an integer
    return $response->json([
        "order_id" => $id,
        "type" => gettype($id)
    ]);
});
```

```
curl http://localhost:7145/orders/42
```

```
{"order_id":42,"type":"integer"}
```

If you pass a non-integer value, the route will not match and you will get a 404:

```
curl http://localhost:7145/orders/abc
```

```
{"error":"Not found","path":"/orders/abc","status":404}
```

Supported types:

| Type | Matches | Example |

|------|---------|---------|

| `int` | Digits only | `{id:int}` matches `42` but not `abc` |

| `float` | Decimal numbers | `{price:float}` matches `19.99` |

| `alpha` | Letters only | `{slug:alpha}` matches `hello` but not `hello123` |

| `alphanumeric` | Letters and digits | `{code:alphanumeric}` matches `abc123` |

## 4. Query Parameters

Query parameters are the key-value pairs after the `?` in a URL. Access them via `$request->query`:

```
<?php
use Tina4\Route;

Route::get("/search", function ($request, $response) {
    $q = $request->query["q"] ?? "";
    $page = (int) ($request->query["page"] ?? 1);
    $limit = (int) ($request->query["limit"] ?? 10);

    return $response->json([
        "query" => $q,
        "page" => $page,
        "limit" => $limit,
        "offset" => ($page - 1) * $limit
    ]);
});
```

```
curl "http://localhost:7145/search?q=keyboard&page=2&limit=20"
```

```
{"query":"keyboard","page":2,"limit":20,"offset":20}
```

If a query parameter is missing, `$request->query["key"]` will not exist, so always use the null coalescing operator (`??`) to provide defaults.

## 5. Route Groups

When you have a set of routes that share a common prefix, use `Route::group()` to avoid repeating yourself:

```
<?php
use Tina4\Route;

Route::group("/api/v1", function () {

    Route::get("/users", function ($request, $response) {
```

```
        return $response->json(["users" => []]);
    });

    Route::get("/users/{id:int}", function ($request, $response) {
        $id = $request->params["id"];
        return $response->json(["user" => ["id" => $id, "name" => "Alice"]]);
    });

    Route::post("/users", function ($request, $response) {
        return $response->json(["created" => true], 201);
    });

    Route::get("/products", function ($request, $response) {
        return $response->json(["products" => []]);
    });
});
```

The routes above register as `/api/v1/users`, `/api/v1/users/{id}`, and `/api/v1/products`. You write short paths inside the group, and Tina4 prepends the prefix automatically.

```
curl http://localhost:7145/api/v1/users

{"users":[]}

curl http://localhost:7145/api/v1/products

{"products":[]}
```

Groups can be nested:

```
<?php
use Tina4\Route;

Route::group("/api", function () {
    Route::group("/v1", function () {
        Route::get("/status", function ($request, $response) {
            return $response->json(["version" => "1.0"]);
        });
    });

    Route::group("/v2", function () {
        Route::get("/status", function ($request, $response) {
            return $response->json(["version" => "2.0"]);
        });
    });
});

curl http://localhost:7145/api/v1/status

{"version":"1.0"}

curl http://localhost:7145/api/v2/status

{"version":"2.0"}
```

## 6. Middleware

Middleware is code that runs before (or after) your route handler. Use it for authentication, logging, rate limiting, input validation, or anything that should happen on multiple routes.

### *Middleware on a Single Route*

Pass middleware as the third argument to any route method:

```php
<?php
use Tina4\Route;

function logRequest($request, $response, $next) {
    $start = microtime(true);
    error_log("[" . date("Y-m-d H:i:s") . "] " . $request->method . " " . $request->path);

    $result = $next($request, $response);

    $duration = round((microtime(true) - $start) * 1000, 2);
    error_log("  Completed in " . $duration . "ms");

    return $result;
}

Route::get("/api/data", function ($request, $response) {
    return $response->json(["data" => [1, 2, 3]]);
}, "logRequest");
```

The middleware function receives `$request`, `$response`, and `$next`. Call `$next($request, $response)` to continue to the route handler. If you do not call `$next`, the route handler never runs -- useful for blocking unauthorized requests.

### *Blocking Middleware*

Here is middleware that checks for an API key:

```php
<?php
use Tina4\Route;

function requireApiKey($request, $response, $next) {
    $apiKey = $request->headers["X-API-Key"] ?? "";

    if ($apiKey !== "my-secret-key") {
        return $response->json(["error" => "Invalid API key"], 401);
    }

    return $next($request, $response);
}

Route::get("/api/secret", function ($request, $response) {
    return $response->json(["secret" => "The answer is 42"]);
}, "requireApiKey");
```

```
curl http://localhost:7145/api/secret
```

```
{"error":"Invalid API key"}
```

The response status is `401 Unauthorized`.

```
curl http://localhost:7145/api/secret -H "X-API-Key: my-secret-key"
```

```
{"secret":"The answer is 42"}
```

### *Middleware on a Group*

Apply middleware to an entire group by passing it as the third argument to `Route::group()`:

```php
<?php
use Tina4\Route;

function requireAuth($request, $response, $next) {
    $token = $request->headers["Authorization"] ?? "";

    if (empty($token)) {
```

```
        return $response->json(["error" => "Authentication required"], 401);
    }

    return $next($request, $response);
}

Route::group("/api/admin", function () {

    Route::get("/dashboard", function ($request, $response) {
        return $response->json(["page" => "admin dashboard"]);
    });

    Route::get("/users", function ($request, $response) {
        return $response->json(["page" => "user management"]);
    });

}, "requireAuth");
```

Every route inside the group now requires the `Authorization` header. You do not need to add the middleware to each route individually.

### *Multiple Middleware*

Chain multiple middleware by passing an array:

```
Route::get("/api/important", function ($request, $response) {
    return $response->json(["data" => "important stuff"]);
}, ["logRequest", "requireApiKey", "requireAuth"]);
```

Middleware runs in order: `logRequest` first, then `requireApiKey`, then `requireAuth`, then the route handler. If any middleware does not call `$next`, the chain stops there.

## 7. Route Decorators: @noauth and @secured

Tina4 provides two special decorators for controlling authentication on routes.

### *@noauth -- Public Routes*

When your application has global authentication middleware, use the `@noauth` annotation to mark specific routes as public:

```
<?php
use Tina4\Route;

/**
 * @noauth
 */
Route::get("/api/public/info", function ($request, $response) {
    return $response->json([
        "app" => "My Store",
        "version" => "1.0.0"
    ]);
});
```

The `@noauth` decorator tells Tina4 to skip authentication checks for this route, even if global auth middleware is configured in `.env` or applied to the parent group.

### *@secured -- Protected GET Routes*

The `@secured` annotation explicitly marks a GET route as requiring authentication:

```php
<?php
use Tina4\Route;

/**
 * @secured
 */
Route::get("/api/profile", function ($request, $response) {
    // $request->user is populated by the auth middleware
    return $response->json([
        "user" => $request->user
    ]);
});
```

By default, `POST`, `PUT`, `PATCH`, and `DELETE` routes are considered secured. `GET` routes are not -- they are public unless you add `@secured`. This matches the common pattern where reading data is public but modifying data requires authentication.

## 8. Wildcard and Catch-All Routes

### *Wildcard Routes*

Use `*` at the end of a path to match anything after it:

```php
<?php
use Tina4\Route;

Route::get("/docs/*", function ($request, $response) {
    $path = $request->params["*"] ?? "";
    return $response->json([
        "section" => "docs",
        "path" => $path
    ]);
});
```

```
curl http://localhost:7145/docs/getting-started
```

```
{"section":"docs","path":"getting-started"}
```

```
curl http://localhost:7145/docs/api/authentication/jwt
```

```
{"section":"docs","path":"api/authentication/jwt"}
```

### *Catch-All Route (Custom 404)*

Register a catch-all to handle any unmatched URL:

```php
<?php
use Tina4\Route;

Route::get("/*", function ($request, $response) {
    return $response->json([
        "error" => "Page not found",
        "path" => $request->path
    ], 404);
});
```

This route should be defined last (or in a file that sorts alphabetically after your other route files) so it does not shadow your real routes. Tina4 matches routes in the order they are registered -- the first match wins.

Alternatively, you can create a custom 404 page by placing a template at `src/templates/errors/404.html`:

```
{% extends "base.html" %}
```

```
{% block title %}Not Found{% endblock %}

{% block content %}
    <h1>404 - Page Not Found</h1>
    <p>The page you are looking for does not exist.</p>
    <a href="/">Go back home</a>
{% endblock %}
```

Tina4 automatically uses this template for any unmatched route when the template file exists.

## 9. Route Listing via CLI

As your application grows, you will want to see all registered routes at a glance. Use the Tina4 CLI:

```
tina4 routes

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /hello                        -                   public
GET      /products                     -                   public
POST     /products                     -                   secured
PUT      /products/{id}                -                   secured
PATCH    /products/{id}                -                   secured
DELETE   /products/{id}                -                   secured
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/admin/dashboard          requireAuth         public
GET      /api/admin/users              requireAuth         public
GET      /api/public/info              -                   @noauth
GET      /api/profile                  -                   @secured
GET      /search                       -                   public
GET      /docs/*                       -                   public
```

The `Auth` column shows whether a route is public, secured (default for non-GET methods), explicitly `@noauth`, or explicitly `@secured`.

You can also filter by method:

```
tina4 routes --method POST

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
POST     /products                     -                   secured
POST     /api/v1/users                 -                   secured
```

Or search for a specific path pattern:

```
tina4 routes --filter users

Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /api/v1/users                 -                   public
GET      /api/v1/users/{id:int}        -                   public
POST     /api/v1/users                 -                   secured
GET      /api/admin/users              requireAuth         public
```

## 10. Organizing Route Files

You are free to organize route files any way you like. Tina4 loads every `.php` file in `src/routes/` recursively. Here are two common patterns:

### *Pattern 1: One File Per Resource*

```
src/routes/
■■■ products.php     # All product routes
■■■ users.php        # All user routes
■■■ orders.php       # All order routes
■■■ pages.php        # HTML page routes
```

### *Pattern 2: Subdirectories by Feature*

```
src/routes/
■■■ api/
■   ■■■ products.php
■   ■■■ users.php
■   ■■■ orders.php
■■■ admin/
■   ■■■ dashboard.php
■   ■■■ settings.php
■■■ pages/
    ■■■ home.php
    ■■■ about.php
```

Both patterns work identically. The directory structure has no effect on the URL paths -- only the route definitions inside the files matter. Choose whichever pattern keeps your project navigable.

## 11. Exercise: Build a Full CRUD API for Products

Build a complete REST API for managing products. All data is stored in a PHP array (no database yet -- we will add that in Chapter 5).

### *Requirements*

Create a file `src/routes/product-api.php` with the following routes:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/products` | List all products. Support `?category=` filter. |

| `GET` | `/api/products/{id:int}` | Get a single product by ID. Return 404 if not found. |

| `POST` | `/api/products` | Create a new product. Return 201. |

| `PUT` | `/api/products/{id:int}` | Replace a product. Return 404 if not found. |

| `DELETE` | `/api/products/{id:int}` | Delete a product. Return 204 with no body. |

Each product has: `id` (int), `name` (string), `category` (string), `price` (float), `in_stock` (bool).

Start with this seed data:

```
$products = [
    ["id" => 1, "name" => "Wireless Keyboard", "category" => "Electronics", "price" => 79.99, "in_stock" => tr
    ["id" => 2, "name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => true],
    ["id" => 3, "name" => "Coffee Grinder", "category" => "Kitchen", "price" => 49.99, "in_stock" => false],
    ["id" => 4, "name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock" => true],
    ["id" => 5, "name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock" => true]
```

```
];
```

Test with:

```
# List all
curl http://localhost:7145/api/products

# Filter by category
curl "http://localhost:7145/api/products?category=Fitness"

# Get one
curl http://localhost:7145/api/products/3

# Create
curl -X POST http://localhost:7145/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Desk Lamp", "category": "Office", "price": 39.99, "in_stock": true}'

# Update
curl -X PUT http://localhost:7145/api/products/3 \
  -H "Content-Type: application/json" \
  -d '{"name": "Burr Coffee Grinder", "category": "Kitchen", "price": 59.99, "in_stock": true}'

# Delete
curl -X DELETE http://localhost:7145/api/products/3

# Not found
curl http://localhost:7145/api/products/999
```

## 12. Solution

Create `src/routes/product-api.php`:

```
<?php
use Tina4\Route;

// In-memory product store (resets on server restart)
$products = [
    ["id" => 1, "name" => "Wireless Keyboard", "category" => "Electronics", "price" => 79.99, "in_stock" => tr
    ["id" => 2, "name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => true],
    ["id" => 3, "name" => "Coffee Grinder", "category" => "Kitchen", "price" => 49.99, "in_stock" => false],
    ["id" => 4, "name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock" => true],
    ["id" => 5, "name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock" => true]
];

$nextId = 6;

// List all products, optionally filter by category
Route::get("/api/products", function ($request, $response) use (&$products) {
    $category = $request->query["category"] ?? null;

    if ($category !== null) {
        $filtered = array_values(array_filter(
            $products,
            fn($p) => strtolower($p["category"]) === strtolower($category)
        ));
        return $response->json(["products" => $filtered, "count" => count($filtered)]);
    }
// ...
```

**Expected output for the test commands:**

List all:

```
{"products":[{"id":1,"name":"Wireless Keyboard","category":"Electronics","price":79.99,"in_stock":true},{"id"
```

Filter by category:

```
{"products":[{"id":2,"name":"Yoga Mat","category":"Fitness","price":29.99,"in_stock":true},{"id":5,"name":"Run
```

Get one:

```
{"id":3,"name":"Coffee Grinder","category":"Kitchen","price":49.99,"in_stock":false}
```

Create:

```
{"id":6,"name":"Desk Lamp","category":"Office","price":39.99,"in_stock":true}
```

(Status: `201 Created`)

Update:

```
{"id":3,"name":"Burr Coffee Grinder","category":"Kitchen","price":59.99,"in_stock":true}
```

Delete: empty response with status `204 No Content`.

Not found:

```
{"error":"Product not found","id":999}
```

(Status: `404 Not Found`)

## 13. Gotchas

### *1. Trailing Slashes Matter*

**Problem:** `/products` works but `/products/` returns a 404 (or vice versa).

**Cause:** Tina4 treats `/products` and `/products/` as different routes by default.

**Fix:** Pick one convention and stick with it. If you want both to work, register the route without a trailing slash -- Tina4 will redirect `/products/` to `/products` automatically when `TINA4_TRAILING_SLASH_REDIRECT=true` is set in `.env`.

### *2. Parameter Names Must Be Unique in a Path*

**Problem:** `/users/{id}/posts/{id}` does not work as expected -- both parameters have the same name.

**Cause:** The second `{id}` overwrites the first in `$request->params`.

**Fix:** Use distinct names: `/users/{userId}/posts/{postId}`.

### *3. Method Conflicts*

**Problem:** You defined `Route::get("/items/{id}", ...)` and `Route::get("/items/{action}", ...)` and the wrong handler runs.

**Cause:** Both patterns match `/items/42`. The first one registered wins.

**Fix:** Use typed parameters to disambiguate: `Route::get("/items/{id:int}", ...)` will only match integers, leaving `/items/export` free for the other route. Alternatively, restructure your paths: `/items/{id:int}` and `/items/actions/{action}`.

### *4. Route Handler Must Return a Response*

**Problem:** Your route handler runs but the browser shows an empty page or a 500 error.

**Cause:** You forgot the `return` statement. Without `return`, the handler returns `null` and Tina4 does not know what to send back.

**Fix:** Always `return $response->json(...)` or `return $response->html(...)` or `return $response->render(...)`. Every handler must return something.

### *5. Route Files Must Start with `<?php`*

**Problem:** Your route file is ignored. No errors, just no routes registered.

**Cause:** The file does not start with `<?php`. Without the PHP opening tag, the file is not parsed as PHP code.

**Fix:** Make sure every route file starts with `<?php` on the very first line.

### *6. Middleware Function Must Be a String Name*

**Problem:** Passing a closure directly as middleware causes an error.

**Cause:** Tina4 expects middleware to be referenced by function name (a string), not as an inline closure.

**Fix:** Define your middleware as a named function and pass the name as a string: `"myMiddleware"`, not `function ($request, $response, $next) { ... }`.

### *7. Group Prefix Must Start with a Slash*

**Problem:** `Route::group("api/v1", ...)` produces routes like `/api/v1/users` but they do not match.

**Cause:** The group prefix should start with `/` for consistency. While Tina4 may auto-correct this, it is better to be explicit.

**Fix:** Always start group prefixes with `/`: `Route::group("/api/v1", ...)`.

# Chapter 3: Request & Response

## 1. The Two Objects You Always Get

Every route handler in Tina4 receives two arguments: `$request` and `$response`. The request tells you what the client sent. The response is how you send something back. Together they are the entire HTTP conversation.

```
<?php
use Tina4\Route;

Route::get("/echo", function ($request, $response) {
    return $response->json([
        "method" => $request->method,
        "path" => $request->path,
        "your_ip" => $request->ip
    ]);
});
```

```
curl http://localhost:7145/echo
```

```
{"method":"GET","path":"/echo","your_ip":"127.0.0.1"}
```

That is the pattern for every route: inspect the request, build the response, return it.

## 2. The Request Object

The `$request` object gives you access to everything the client sent. Here is the complete list of properties.

### *method*

The HTTP method as an uppercase string: `"GET"`, `"POST"`, `"PUT"`, `"PATCH"`, or `"DELETE"`.

```
$request->method // "GET"
```

### *path*

The URL path without query parameters:

```
// Request to /api/users?page=2
$request->path // "/api/users"
```

### *params*

Path parameters from the URL pattern (see Chapter 2):

```
// Route: /users/{id}/posts/{postId}
// Request: /users/5/posts/42
$request->params["id"]     // "5" (or 5 if typed as {id:int})
$request->params["postId"] // "42"
```

### *query*

Query string parameters as an associative array:

```
// Request: /search?q=keyboard&page=2&sort=price
$request->query["q"]    // "keyboard"
$request->query["page"] // "2"
$request->query["sort"] // "price"
```

### *body*

The parsed request body. For JSON requests, this is an associative array. For form submissions, it contains the form fields:

```
// POST with {"name": "Widget", "price": 9.99}
$request->body["name"]  // "Widget"
$request->body["price"] // 9.99
```

### *headers*

Request headers as an associative array. Header names are normalized to their original casing:

```
$request->headers["Content-Type"]  // "application/json"
$request->headers["Authorization"] // "Bearer eyJhbGci..."
$request->headers["X-Custom"]      // "my-value"
```

### *ip*

The client's IP address:

```
$request->ip // "127.0.0.1"
```

Tina4 respects `X-Forwarded-For` and `X-Real-IP` headers when behind a reverse proxy.

### *cookies*

Cookies sent by the client:

```
$request->cookies["session_id"]  // "abc123"
$request->cookies["preferences"] // "dark-mode"
```

### *files*

Uploaded files (covered in detail in section 7):

```
$request->files["avatar"] // File object with name, type, size, tmpPath
```

### *Inspecting the Full Request*

Here is a route that dumps everything:

```
<?php
use Tina4\Route;

Route::post("/debug/request", function ($request, $response) {
    return $response->json([
        "method" => $request->method,
        "path" => $request->path,
        "params" => $request->params,
        "query" => $request->query,
        "body" => $request->body,
        "headers" => $request->headers,
        "ip" => $request->ip,
        "cookies" => $request->cookies
    ]);
});
```

```
curl -X POST "http://localhost:7145/debug/request?page=1" \
  -H "Content-Type: application/json" \
  -H "X-Custom: hello" \
  -d '{"name": "test"}'
```

```
{
  "method": "POST",
  "path": "/debug/request",
```

```
  "params": {},
  "query": {"page": "1"},
  "body": {"name": "test"},
  "headers": {
    "Content-Type": "application/json",
    "X-Custom": "hello",
    "Host": "localhost:7145",
    "User-Agent": "curl/8.4.0",
    "Accept": "*/*",
    "Content-Length": "16"
  },
  "ip": "127.0.0.1",
  "cookies": {}
}
```

## 3. The Response Object

The `$response` object is your toolkit for sending data back to the client. Every method on it returns the response, so you can chain calls together.

### *json() -- JSON Response*

The most common response for APIs. Pass any array or value and it becomes JSON:

```
return $response->json(["name" => "Alice", "age" => 30]);
```

```
{"name":"Alice","age":30}
```

Pass a status code as the second argument:

```
return $response->json(["id" => 7, "name" => "Widget"], 201);
```

This returns `201 Created` with the JSON body.

### *html() -- Raw HTML Response*

Return an HTML string directly:

```
return $response->html("<h1>Hello</h1><p>This is HTML.</p>");
```

Sets `Content-Type: text/html; charset=utf-8` automatically.

### *text() -- Plain Text Response*

Return plain text:

```
return $response->text("Just a plain string.");
```

Sets `Content-Type: text/plain; charset=utf-8`.

### *render() -- Template Response*

Render a Frond template with data (covered in depth in Chapter 4):

```
return $response->render("products.html", [
    "products" => $products,
    "title" => "Our Products"
]);
```

Tina4 looks for the template in `src/templates/`, renders it with the provided data, and returns the HTML.

### *redirect() -- Redirect Response*

Send the client to a different URL:

```
return $response->redirect("/login");
```

This sends a `302 Found` redirect by default. Pass a different status code for permanent redirects:

```
return $response->redirect("/new-location", 301);
```

### *file() -- File Download Response*

Send a file to the client for download:

```
return $response->file("/path/to/report.pdf");
```

This sets the appropriate `Content-Type` based on the file extension and adds a `Content-Disposition` header so the browser downloads the file rather than displaying it.

You can set a custom filename:

```
return $response->file("/path/to/report.pdf", "monthly-report-march-2026.pdf");
```

## 4. Status Codes

Every response method accepts a status code. Here are the most common ones you will use:

| Code | Meaning | When to Use |

|------|---------|-------------|

| `200` | OK | Default. Successful GET, PUT, PATCH. |

| `201` | Created | Successful POST that created a resource. |

| `204` | No Content | Successful DELETE. No body needed. |

| `301` | Moved Permanently | URL has permanently changed. |

| `302` | Found | Temporary redirect. |

| `400` | Bad Request | Invalid input from the client. |

| `401` | Unauthorized | Missing or invalid authentication. |

| `403` | Forbidden | Authenticated but not allowed. |

| `404` | Not Found | Resource does not exist. |

| `409` | Conflict | Duplicate or conflicting data. |

| `422` | Unprocessable Entity | Valid JSON but fails business rules. |

| `500` | Internal Server Error | Something went wrong on the server. |

You can also set the status explicitly with the `status()` method and chain it:

```
return $response->status(201)->json(["id" => 7, "created" => true]);
```

This is equivalent to `$response->json(["id" => 7, "created" => true], 201)` but some developers find the chained form more readable.

## 5. Custom Headers

Set response headers with the `header()` method:

```
Route::get("/api/data", function ($request, $response) {
    return $response
        ->header("X-Request-Id", uniqid())
        ->header("X-Rate-Limit-Remaining", "57")
        ->header("Cache-Control", "no-cache")
        ->json(["data" => [1, 2, 3]]);
});
```

```
curl -v http://localhost:7145/api/data 2>&1 | grep "< X-"
```

```
< X-Request-Id: 65f3a7b8c1234
< X-Rate-Limit-Remaining: 57
```

Headers are case-insensitive in HTTP, but it is conventional to use `Title-Case` for custom headers and prefix them with `X-`.

### *CORS Headers*

Tina4 handles CORS automatically based on the `CORS_ORIGINS` setting in `.env`. The default `*` allows all origins. For production, restrict it:

```
CORS_ORIGINS=https://myapp.com,https://admin.myapp.com
```

You rarely need to set CORS headers manually, but you can if needed:

```
return $response
    ->header("Access-Control-Allow-Origin", "https://myapp.com")
    ->header("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE")
    ->header("Access-Control-Allow-Headers", "Content-Type, Authorization")
    ->json(["data" => "value"]);
```

## 6. Cookies

Set cookies on the response:

```
Route::post("/login", function ($request, $response) {
    // After validating credentials...
    return $response
        ->cookie("session_id", "abc123xyz", [
            "httpOnly" => true,
            "secure" => true,
            "sameSite" => "Strict",
            "maxAge" => 3600,       // 1 hour in seconds
            "path" => "/"
        ])
        ->json(["message" => "Logged in"]);
});
```

Cookie options:

| Option | Type | Default | Description |

|--------|------|---------|-------------|

| `httpOnly` | bool | `false` | Cannot be accessed by JavaScript |

| `secure` | bool | `false` | Only sent over HTTPS |

| `sameSite` | string | `"Lax"` | `"Strict"`, `"Lax"`, or `"None"` |

| `maxAge` | int | session | Lifetime in seconds |

| `path` | string | `"/"` | URL path scope |

| `domain` | string | current | Domain scope |

Read cookies from the request:

```
Route::get("/profile", function ($request, $response) {
    $sessionId = $request->cookies["session_id"] ?? null;

    if ($sessionId === null) {
        return $response->json(["error" => "Not logged in"], 401);
    }

    return $response->json(["session" => $sessionId]);
});
```

Delete a cookie by setting its `maxAge` to `0`:

```
return $response
    ->cookie("session_id", "", ["maxAge" => 0, "path" => "/"])
    ->json(["message" => "Logged out"]);
```

## 7. File Uploads

Uploaded files are available via `$request->files`. Each file is an object with properties for the file's metadata and a temporary path.

### *Handling a Single File Upload*

```
<?php
use Tina4\Route;

Route::post("/api/upload", function ($request, $response) {
    if (empty($request->files["image"])) {
        return $response->json(["error" => "No file uploaded"], 400);
    }

    $file = $request->files["image"];

    return $response->json([
        "name" => $file->name,        // "photo.jpg"
        "type" => $file->type,        // "image/jpeg"
        "size" => $file->size,        // 245760 (bytes)
        "tmp_path" => $file->tmpPath  // Temporary file location
    ]);
});
```

Test with curl:

```
curl -X POST http://localhost:7145/api/upload \
  -F "image=@/path/to/photo.jpg"

{
  "name": "photo.jpg",
  "type": "image/jpeg",
  "size": 245760,
  "tmp_path": "/tmp/tina4_upload_abc123"
}
```

### *Saving the Uploaded File*

The uploaded file is stored in a temporary location. Move it to a permanent location:

```
<?php
use Tina4\Route;

Route::post("/api/upload", function ($request, $response) {
    if (empty($request->files["image"])) {
        return $response->json(["error" => "No file uploaded"], 400);
    }

    $file = $request->files["image"];

    // Validate file type
    $allowedTypes = ["image/jpeg", "image/png", "image/gif", "image/webp"];
    if (!in_array($file->type, $allowedTypes)) {
        return $response->json(["error" => "Invalid file type. Allowed: JPEG, PNG, GIF, WebP"], 400);
    }

    // Validate file size (max 5MB)
    $maxSize = 5 * 1024 * 1024;
    if ($file->size > $maxSize) {
        return $response->json(["error" => "File too large. Maximum size: 5MB"], 400);
    }

    // Generate a unique filename
    $extension = pathinfo($file->name, PATHINFO_EXTENSION);
    $filename = uniqid("img_") . "." . $extension;
// ...
```

```
curl -X POST http://localhost:7145/api/upload \
  -F "image=@/path/to/photo.jpg"
```

```
{
  "message": "File uploaded successfully",
  "filename": "img_65f3a7b8c1234.jpg",
  "url": "/uploads/img_65f3a7b8c1234.jpg",
  "size": 245760
}
```

The uploaded file is now available at `http://localhost:7145/uploads/img_65f3a7b8c1234.jpg`.

### *Handling Multiple Files*

When the HTML form uses `multiple` or you have multiple file inputs:

```
Route::post("/api/upload-many", function ($request, $response) {
    $results = [];

    foreach ($request->files as $key => $file) {
        $extension = pathinfo($file->name, PATHINFO_EXTENSION);
        $filename = uniqid("file_") . "." . $extension;
        $destination = __DIR__ . "/../../public/uploads/" . $filename;

        if (!is_dir(dirname($destination))) {
            mkdir(dirname($destination), 0755, true);
        }

        rename($file->tmpPath, $destination);

        $results[] = [
            "original_name" => $file->name,
            "saved_as" => $filename,
            "url" => "/uploads/" . $filename
        ];
```

```
    }

    return $response->json(["uploaded" => $results, "count" => count($results)], 201);
});
```

## 8. File Downloads

Send files to the client using `$response->file()`:

```
<?php
use Tina4\Route;

Route::get("/api/reports/{filename}", function ($request, $response) {
    $filename = $request->params["filename"];
    $filepath = __DIR__ . "/../../data/reports/" . $filename;

    if (!file_exists($filepath)) {
        return $response->json(["error" => "Report not found"], 404);
    }

    return $response->file($filepath);
});
```

The browser will download the file. Tina4 automatically detects the MIME type from the file extension and sets the appropriate headers.

To force a specific download filename:

```
return $response->file($filepath, "Q1-2026-Sales-Report.pdf");
```

## 9. Content Negotiation

Sometimes the same endpoint should return different formats based on what the client asks for. Check the `Accept` header:

```
<?php
use Tina4\Route;

Route::get("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $product = [
        "id" => $id,
        "name" => "Wireless Keyboard",
        "price" => 79.99
    ];

    $accept = $request->headers["Accept"] ?? "application/json";

    if (strpos($accept, "text/html") !== false) {
        return $response->render("product-detail.html", ["product" => $product]);
    }

    if (strpos($accept, "text/plain") !== false) {
        $text = "Product #" . $id . ": " . $product["name"] . " - $" . $product["price"];
        return $response->text($text);
    }

    // Default to JSON
    return $response->json($product);
```

```
});

# JSON (default)
curl http://localhost:7145/api/products/1

{"id":1,"name":"Wireless Keyboard","price":79.99}

# Plain text
curl http://localhost:7145/api/products/1 -H "Accept: text/plain"

Product #1: Wireless Keyboard - $79.99

# HTML (would render the template)
curl http://localhost:7145/api/products/1 -H "Accept: text/html"

<!DOCTYPE html>
<html>...rendered template...</html>
```

## 10. Exercise: Build an Image Upload API

Build an API that handles image uploads and serves them back. Create two endpoints:

### *Requirements*

| Method | Path | Description |

|--------|------|-------------|

| `POST` | `/api/images` | Upload an image. Validate type and size. Return the image URL. |

| `GET` | `/api/images/{filename}` | Return the uploaded image file. Return 404 if not found. |

Rules:

1. Only accept JPEG, PNG, and WebP files

2. Maximum file size: 2MB

3. Save files to `src/public/uploads/` with a unique filename

4. Return the original filename, the saved filename, file size in KB, and the URL

5. The GET endpoint should serve the file directly (not JSON)

### *Test with:*

```
# Upload
curl -X POST http://localhost:7145/api/images \
  -F "image=@/path/to/photo.jpg"

# Download
curl http://localhost:7145/api/images/img_65f3a7b8c1234.jpg --output downloaded.jpg
```

## 11. Solution

Create `src/routes/images.php`:

```
<?php
use Tina4\Route;

Route::post("/api/images", function ($request, $response) {
```

```
    // Check if a file was uploaded
    if (empty($request->files["image"])) {
        return $response->json(["error" => "No image file provided. Use field name 'image'."], 400);
    }

    $file = $request->files["image"];

    // Validate file type
    $allowedTypes = ["image/jpeg", "image/png", "image/webp"];
    if (!in_array($file->type, $allowedTypes)) {
        return $response->json([
            "error" => "Invalid file type",
            "received" => $file->type,
            "allowed" => $allowedTypes
        ], 400);
    }

    // Validate file size (max 2MB)
    $maxSize = 2 * 1024 * 1024;
    if ($file->size > $maxSize) {
        return $response->json([
// ...
```

**Expected output for upload:**

```
{
  "message": "Image uploaded successfully",
  "original_name": "photo.jpg",
  "saved_name": "img_65f3a7b8c1234.jpg",
  "size_kb": 240.0,
  "type": "image/jpeg",
  "url": "/uploads/img_65f3a7b8c1234.jpg"
}
```

(Status: `201 Created`)

**Expected output for invalid type:**

```
{
  "error": "Invalid file type",
  "received": "application/pdf",
  "allowed": ["image/jpeg", "image/png", "image/webp"]
}
```

(Status: `400 Bad Request`)

**Expected output for file too large:**

```
{
  "error": "File too large",
  "size_bytes": 5242880,
  "max_bytes": 2097152
}
```

(Status: `400 Bad Request`)

**The GET endpoint** returns the raw image file with the correct `Content-Type` header. The browser displays the image directly, and curl with `--output` saves it to disk.

## 12. Gotchas

### *1. Forgetting `return`*

**Problem:** Your handler runs (you can see log output) but the browser shows an empty response or a 500 error.

**Cause:** You wrote `$response->json([...])` without `return`.

**Fix:** Always write `return $response->json([...])`. The response object needs to be returned from the handler for Tina4 to send it to the client.

### *2. Body Is Null for JSON Requests*

**Problem:** `$request->body` is `null` or empty even though you are sending JSON.

**Cause:** You forgot the `Content-Type: application/json` header in your request. Without it, Tina4 does not know to parse the body as JSON.

**Fix:** Always include `-H "Content-Type: application/json"` when sending JSON with curl. In frontend JavaScript, `fetch()` with `JSON.stringify()` requires `headers: {"Content-Type": "application/json"}`.

### *3. Content-Type Mismatch*

**Problem:** You call `$response->json()` but the client receives HTML, or you call `$response->html()` but the client gets plain text.

**Cause:** A middleware or error handler is overwriting the response. Or you are returning a string instead of using a response method.

**Fix:** Make sure your handler returns `$response->json(...)`, `$response->html(...)`, or another response method. Do not use `echo` -- it bypasses the response object entirely.

### *4. File Uploads Return Empty*

**Problem:** `$request->files` is empty even though you are uploading a file.

**Cause:** The form is not using `enctype="multipart/form-data"`, or the curl command is using `-d` instead of `-F`.

**Fix:** For HTML forms, use `<form enctype="multipart/form-data">`. For curl, use `-F "field=@file.jpg"` (with `@`), not `-d`.

### *5. Redirect Loops*

**Problem:** The browser shows "too many redirects" or hangs.

**Cause:** You have a route that redirects to another route, which redirects back to the first one. For example, `/login` redirects to `/dashboard`, and `/dashboard` redirects to `/login` because the user is not authenticated.

**Fix:** Check your redirect logic carefully. Use the browser's network inspector to trace the redirect chain. Make sure your auth check does not redirect authenticated users away from pages they should access.

### *6. Cookie Not Set*

**Problem:** You called `$response->cookie(...)` but the browser does not show the cookie.

**Cause:** If `secure` is `true`, the cookie is only sent over HTTPS. During local development with `http://localhost`, the cookie is silently dropped.

**Fix:** Set `"secure" => false` during development, or use `"secure" => ($_ENV["TINA4_DEBUG"] ?? "false") !== "true"` to auto-switch based on environment.

### *7. Large Request Body Rejected*

**Problem:** POST requests with large bodies return a 413 error.

**Cause:** The request body exceeds the configured maximum size.

**Fix:** Increase `TINA4_MAX_BODY_SIZE` in `.env`. The default is `10mb`. For file upload endpoints, you may need `50mb` or more:

```
TINA4_MAX_BODY_SIZE=50mb
```

# Chapter 4: Templates

## 1. Why Templates

In Chapter 1, you saw `$response->render("products.html", $data)` and it produced a full HTML page. That rendering was done by **Frond**, Tina4's built-in template engine. Frond is a zero-dependency, Twig-compatible engine that ships with the framework. If you have used Twig, Jinja2, or Nunjucks, you already know 90% of Frond.

Templates live in `src/templates/`. When you call `$response->render("page.html", $data)`, Frond loads `src/templates/page.html`, processes the tags and expressions, and returns the final HTML.

This chapter covers every feature of the template engine so you can build real pages.

## 2. Variables and Expressions

Output a variable with double curly braces:

```
<h1>Hello, {{ name }}!</h1>
```

Route handler:

```
<?php
use Tina4\Route;

Route::get("/welcome", function ($request, $response) {
    return $response->render("welcome.html", [
        "name" => "Alice"
    ]);
});
```

Create `src/templates/welcome.html`:

```
<!DOCTYPE html>
<html>
<head><title>Welcome</title></head>
<body>
    <h1>Hello, {{ name }}!</h1>
</body>
</html>
```

**Expected browser output:**

```
Hello, Alice!
```

### *Accessing Nested Data*

Use dot notation for nested arrays:

```
$data = [
    "user" => [
        "name" => "Alice",
        "email" => "alice@example.com",
        "address" => [
            "city" => "Cape Town",
            "country" => "South Africa"
        ]
    ]
];
```

```
return $response->render("profile.html", $data);
```

```
<p>{{ user.name }} lives in {{ user.address.city }}, {{ user.address.country }}.</p>
```

**Output:**

```
Alice lives in Cape Town, South Africa.
```

### *Expressions*

You can use basic expressions inside `{{ }}`:

```
<p>Total: ${{ price * quantity }}</p>
<p>Discounted: ${{ price * 0.9 }}</p>
<p>Full name: {{ first_name ~ " " ~ last_name }}</p>
```

The `~` operator concatenates strings.

## 3. Template Inheritance

Template inheritance is the most powerful feature. Define a base layout once, then extend it in every page.

### *Base Layout*

Create `src/templates/base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}My App{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    {% block head %}{% endblock %}
</head>
<body>
    <nav>
        <a href="/">Home</a>
        <a href="/about">About</a>
        <a href="/contact">Contact</a>
    </nav>

    <main>
        {% block content %}{% endblock %}
    </main>

    <footer>
        <p>&copy; 2026 My App. All rights reserved.</p>
    </footer>

    <script src="/js/frond.js"></script>
// ...
```

This base defines four blocks: `title`, `head`, `content`, and `scripts`. Child templates override only the blocks they need.

### *Child Template*

Create `src/templates/about.twig`:

```
{% extends "base.twig" %}
```

```
{% block title %}About Us{% endblock %}

{% block content %}
    <h1>About Us</h1>
    <p>We have been building things since {{ founded_year }}.</p>
    <p>Our team has {{ team_size }} members across {{ office_count }} offices.</p>
{% endblock %}
```

Route handler:

```
<?php
use Tina4\Route;

Route::get("/about", function ($request, $response) {
    return $response->render("about.twig", [
        "founded_year" => 2020,
        "team_size" => 12,
        "office_count" => 3
    ]);
});
```

**Expected browser output:** A full HTML page with the nav, the "About Us" content, and the footer. The `<title>` tag says "About Us". The `head` and `scripts` blocks are empty because the child template did not override them.

### *Using `{{ parent() }}`*

Sometimes you want to add to a block rather than replace it entirely. Use `{{ parent() }}`:

```
{% extends "base.twig" %}

{% block head %}
    {{ parent() }}
    <link rel="stylesheet" href="/css/contact-form.css">
{% endblock %}

{% block content %}
    <h1>Contact Us</h1>
    <form>...</form>
{% endblock %}
```

The `head` block now contains everything from the base template's `head` block plus the extra stylesheet.

## 4. Includes

Break your templates into reusable pieces with `{% include %}`:

Create `src/templates/partials/header.twig`:

```
<header>
    <div class="logo">{{ site_name | default("My App") }}</div>
    <nav>
        <a href="/">Home</a>
        <a href="/products">Products</a>
        <a href="/contact">Contact</a>
    </nav>
</header>
```

Create `src/templates/partials/product-card.twig`:

```
<div class="product-card{{ product.featured ? ' featured' : '' }}">
    <h3>{{ product.name }}</h3>
```

```
    <p class="price">${{ product.price | number_format(2) }}</p>
    {% if product.in_stock %}
        <span class="badge-success">In Stock</span>
    {% else %}
        <span class="badge-danger">Out of Stock</span>
    {% endif %}
</div>
```

Use them in a page:

```
{% extends "base.twig" %}

{% block content %}
    {% include "partials/header.twig" %}

    <h1>Products</h1>
    {% for product in products %}
        {% include "partials/product-card.twig" %}
    {% endfor %}
{% endblock %}
```

The `product` variable is automatically available inside the included template because it exists in the current scope (the for loop).

### *Passing Variables to Includes*

You can pass specific variables with `with`:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} %}
```

Use `only` to isolate the included template from the parent scope:

```
{% include "partials/header.twig" with {"site_name": "Cool Store"} only %}
```

With `only`, the included template can only access `site_name` -- it cannot see any other variables from the parent.

## 5. For Loops

Loop through arrays with `{% for %}`:

```
<ul>
{% for item in items %}
    <li>{{ item }}</li>
{% endfor %}
</ul>
```

### *The `loop` Variable*

Inside a for loop, Frond provides a special `loop` variable:

| Property | Type | Description |

|----------|------|-------------|

| `loop.index` | int | Current iteration (1-based) |

| `loop.index0` | int | Current iteration (0-based) |

| `loop.first` | bool | True on the first iteration |

| `loop.last` | bool | True on the last iteration |

| `loop.length` | int | Total number of items |

| `loop.revindex` | int | Iterations remaining (1-based) |

```
<table>
    <thead>
        <tr><th>#</th><th>Name</th><th>Price</th></tr>
    </thead>
    <tbody>
    {% for product in products %}
        <tr class="{{ loop.index is odd ? 'row-light' : 'row-dark' }}">
            <td>{{ loop.index }}</td>
            <td>{{ product.name }}</td>
            <td>${{ product.price | number_format(2) }}</td>
        </tr>
    {% endfor %}
    </tbody>
</table>
```

### *Empty Lists*

Handle empty lists with `{% else %}`:

```
{% for product in products %}
    <div class="product-card">
        <h3>{{ product.name }}</h3>
    </div>
{% else %}
    <p>No products found.</p>
{% endfor %}
```

If `products` is empty (or not defined), the `else` block renders instead.

### *Looping Over Key-Value Pairs*

```
{% for key, value in metadata %}
    <dt>{{ key }}</dt>
    <dd>{{ value }}</dd>
{% endfor %}
```

## 6. Conditionals

### *if / elseif / else*

```
{% if user.role == "admin" %}
    <a href="/admin">Admin Panel</a>
{% elseif user.role == "editor" %}
    <a href="/editor">Editor Dashboard</a>
{% else %}
    <a href="/profile">My Profile</a>
{% endif %}
```

### *Ternary Operator*

For inline conditionals:

```
<span class="{{ is_active ? 'text-green' : 'text-gray' }}">
    {{ is_active ? 'Active' : 'Inactive' }}
</span>
```

### *Testing for Existence*

Use `is defined` to check if a variable exists:

```
{% if error_message is defined %}
    <div class="alert alert-danger">{{ error_message }}</div>
{% endif %}
```

### *Truthiness*

These values are considered false: `false`, `null`, `0`, `""` (empty string), `[]` (empty array). Everything else is true.

```
{% if items %}
    <p>{{ items | length }} items found.</p>
{% else %}
    <p>No items.</p>
{% endif %}
```

## 7. Filters

Filters transform values. Apply them with the `|` (pipe) character:

```
{{ name | upper }}
```

Here are the most commonly used filters:

### *Text Filters*

| Filter | Input | Output | Description |

|--------|-------|--------|-------------|

| `upper` | `"hello"` | `"HELLO"` | Uppercase |

| `lower` | `"HELLO"` | `"hello"` | Lowercase |

| `capitalize` | `"hello world"` | `"Hello world"` | Capitalize first letter |

| `title` | `"hello world"` | `"Hello World"` | Capitalize each word |

| `trim` | `" hello "` | `"hello"` | Remove whitespace |

| `striptags` | `"<b>bold</b>"` | `"bold"` | Remove HTML tags |

### *Number Filters*

| Filter | Input | Output | Description |

|--------|-------|--------|-------------|

| `number_format(2)` | `1234.5` | `"1,234.50"` | Format number |

| `round` | `3.7` | `4` | Round to nearest integer |

| `round(2)` | `3.14159` | `3.14` | Round to N decimal places |

| `abs` | `-5` | `5` | Absolute value |

### *Array Filters*

| Filter | Input | Output | Description |

|--------|-------|--------|-------------|

| `length` | `[1,2,3]` | `3` | Count items |

| `join(", ")` | `["a","b","c"]` | `"a, b, c"` | Join with separator |

| `first` | `[1,2,3]` | `1` | First item |

| `last` | `[1,2,3]` | `3` | Last item |

| `reverse` | `[1,2,3]` | `[3,2,1]` | Reverse order |

| `sort` | `[3,1,2]` | `[1,2,3]` | Sort ascending |

| `slice(1, 2)` | `[1,2,3,4]` | `[2,3]` | Slice from offset, length |

### *Date Filter*

```
<p>Published: {{ created_at | date("F j, Y") }}</p>
<p>Time: {{ created_at | date("H:i") }}</p>
```

With a PHP timestamp or date string as input:

• `date("F j, Y")` outputs `"March 22, 2026"`

• `date("H:i")` outputs `"14:30"`

• `date("Y-m-d")` outputs `"2026-03-22"`

### *The `default` Filter*

Provide a fallback value when a variable is null or undefined:

```
<p>{{ subtitle | default("No subtitle") }}</p>
<p>{{ user.nickname | default(user.name) | default("Anonymous") }}</p>
```

### *The `escape` and `raw` Filters*

By default, all `{{ }}` output is auto-escaped for HTML safety. This prevents XSS attacks.

```
{{ user_input }}
{# If user_input is "<script>alert('xss')</script>", outputs:
   &lt;script&gt;alert('xss')&lt;/script&gt; #}
```

If you trust the content and need raw HTML:

```
{{ trusted_html | raw }}
```

Use `raw` sparingly. Only apply it to content you fully control, never to user input.

### *Chaining Filters*

Filters chain left to right:

```
{{ name | trim | lower | capitalize }}
{# "  ALICE SMITH  " -> "Alice smith" #}
```

## 8. Macros

Macros are reusable template functions. Think of them as components you define once and call many times.

### *Defining a Macro*

Create `src/templates/macros.twig`:

```
{% macro button(text, url, style) %}
    <a href="{{ url | default('#') }}" class="btn btn-{{ style | default('primary') }}">
        {{ text }}
    </a>
{% endmacro %}

{% macro alert(message, type) %}
    <div class="alert alert-{{ type | default('info') }}">
        {{ message }}
    </div>
{% endmacro %}

{% macro input(name, label, type, value) %}
    <div class="form-group">
        <label for="{{ name }}">{{ label | default(name | capitalize) }}</label>
        <input type="{{ type | default('text') }}" id="{{ name }}" name="{{ name }}" value="{{ value | defaul
    </div>
{% endmacro %}
```

### *Using Macros*

Import macros in your template:

```
{% from "macros.twig" import button, alert, input %}

{% extends "base.twig" %}

{% block content %}
    {{ alert("Your profile has been updated.", "success") }}

    <form method="POST" action="/profile">
        {{ input("name", "Full Name", "text", user.name) }}
        {{ input("email", "Email Address", "email", user.email) }}
        {{ input("phone", "Phone Number", "tel", user.phone) }}

        {{ button("Save Changes", "", "primary") }}
        {{ button("Cancel", "/dashboard", "secondary") }}
    </form>
{% endblock %}
```

**Expected output** (simplified):

```
<div class="alert alert-success">
    Your profile has been updated.
</div>

<form method="POST" action="/profile">
    <div class="form-group">
        <label for="name">Full Name</label>
        <input type="text" id="name" name="name" value="Alice">
    </div>
    <div class="form-group">
        <label for="email">Email Address</label>
        <input type="email" id="email" name="email" value="alice@example.com">
    </div>
    <div class="form-group">
        <label for="phone">Phone Number</label>
        <input type="tel" id="phone" name="phone" value="">
    </div>

    <a href="#" class="btn btn-primary">Save Changes</a>
    <a href="/dashboard" class="btn btn-secondary">Cancel</a>
```

```
</form>
```

# 9. Special Tags

### *{% raw %} -- Literal Output*

When you need to output literal `{{ }}` or `{% %}` (for example, in a Vue.js or Angular template):

```
{% raw %}
    <div id="app">
        {{ message }}
    </div>
{% endraw %}
```

This outputs the literal text `{{ message }}` without trying to process it as a Frond expression.

### *{% spaceless %} -- Remove Whitespace*

Remove whitespace between HTML tags:

```
{% spaceless %}
    <div>
        <span>Hello</span>
    </div>
{% endspaceless %}
```

**Output:**

```
<div><span>Hello</span></div>
```

Useful for inline elements where whitespace creates unwanted gaps.

### *{% autoescape %} -- Control Escaping*

Override the auto-escaping behavior for a block:

```
{% autoescape false %}
    {{ trusted_html }}
{% endautoescape %}
```

Everything inside the block outputs without HTML escaping. Equivalent to using `| raw` on every variable, but more convenient for large blocks of trusted content.

### *Comments*

Template comments are invisible in the output:

```
{# This comment will not appear in the HTML output #}

{#
    Multi-line comments work too.
    Use them to document template logic.
#}
```

# 10. tina4css Integration

Every Tina4 project includes `tina4.css`, a built-in CSS utility framework. It is available at `/css/tina4.css` and provides layout, typography, and common UI patterns without external dependencies.

Include it in your base template:

```
<link rel="stylesheet" href="/css/tina4.css">
```

### *Layout Classes*

```
<div class="container">
    <div class="row">
        <div class="col-6">Left half</div>
        <div class="col-6">Right half</div>
    </div>
</div>
```

### *Common Components*

```
<!-- Buttons -->
<button class="btn btn-primary">Primary</button>
<button class="btn btn-secondary">Secondary</button>
<button class="btn btn-danger">Danger</button>

<!-- Cards -->
<div class="card">
    <div class="card-header">Title</div>
    <div class="card-body">Content here</div>
    <div class="card-footer">Footer</div>
</div>

<!-- Alerts -->
<div class="alert alert-success">Operation completed.</div>
<div class="alert alert-danger">Something went wrong.</div>
<div class="alert alert-warning">Please review your input.</div>

<!-- Forms -->
<div class="form-group">
    <label for="name">Name</label>
    <input type="text" id="name" class="form-control">
</div>
```

### *Utility Classes*

```
<p class="text-center">Centered text</p>
<p class="text-right">Right-aligned text</p>
<div class="mt-4">Margin top</div>
<div class="p-3">Padding all around</div>
<span class="text-muted">Gray text</span>
<span class="text-primary">Primary color text</span>
```

You do not need Bootstrap or Tailwind. If you prefer those, you can replace `tina4.css` with your preferred framework -- just change the `<link>` tag.

## 11. Exercise: Build a Product Catalog Page

Build a product catalog page with a base layout, product cards, category filters, and a reusable card macro.

### *Requirements*

1. Create a base layout at `src/templates/catalog-base.twig` with blocks for `title`, `content`, and `scripts`

2. Create a macro file at `src/templates/catalog-macros.twig` with:

• A `productCard(product)` macro that renders a styled card with name, category, price, stock status, and optional featured badge

• A `categoryFilter(categories, active)` macro that renders filter buttons

3. Create a page template at `src/templates/catalog.twig` that:

• Extends the base layout

• Uses the macros

• Shows a heading with the total product count

• Shows category filter buttons (All, and one per unique category)

• Shows product cards in a grid

• Shows featured products with a distinct style

• Handles the case when no products match the filter

4. Create a route at `GET /catalog` that accepts an optional `?category=` filter

### *Data*

Use this product list in your route handler:

```
$products = [
    ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "in_stock" => true, "featured"
    ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => true, "featured" => false
    ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock" => true, "featured" => 
    ["name" => "Blender", "category" => "Kitchen", "price" => 89.99, "in_stock" => false, "featured" => false
    ["name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock" => true, "featured" =>
    ["name" => "Desk Lamp", "category" => "Office", "price" => 39.99, "in_stock" => true, "featured" => true],
    ["name" => "Cast Iron Skillet", "category" => "Kitchen", "price" => 44.99, "in_stock" => true, "featured"
];
```

### *Test with:*

```
http://localhost:7145/catalog
http://localhost:7145/catalog?category=Kitchen
http://localhost:7145/catalog?category=Fitness
```

## 12. Solution

Create `src/templates/catalog-base.twig`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Product Catalog{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; margin: 0; pad
        .container { max-width: 1000px; margin: 0 auto; padding: 20px; }
        .header { background: #2c3e50; color: white; padding: 20px; margin-bottom: 24px; }
        .header h1 { margin: 0; }
        .header p { margin: 4px 0 0; opacity: 0.8; }
        .filters { margin-bottom: 20px; }
        .filter-btn { display: inline-block; padding: 6px 14px; margin: 0 6px 6px 0; border-radius: 20px; tex
```

```
        .filter-btn.active { background: #2c3e50; color: white; border-color: #2c3e50; }
        .product-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(280px, 1fr)); gap: 16p
        .product-card { background: white; border: 2px solid #e9ecef; border-radius: 8px; padding: 16px; trans
        .product-card:hover { border-color: #adb5bd; }
        .product-card.featured { border-color: #f39c12; background: #fef9e7; }
        .product-name { font-size: 1.1em; font-weight: 600; margin: 0 0 4px; }
        .product-category { font-size: 0.85em; color: #6c757d; margin: 0 0 8px; }
        .product-price { font-size: 1.2em; font-weight: bold; color: #27ae60; }
        .badge { display: inline-block; padding: 2px 8px; border-radius: 4px; font-size: 0.75em; font-weight:
        .badge-featured { background: #f39c12; color: white; }
// ...
```

Create `src/templates/catalog-macros.twig`:

```
{% macro productCard(product) %}
    <div class="product-card{{ product.featured ? ' featured' : '' }}">
        <p class="product-name">
            {{ product.name }}
            {% if product.featured %}
                <span class="badge badge-featured">Featured</span>
            {% endif %}
        </p>
        <p class="product-category">{{ product.category }}</p>
        <p class="product-price">
            ${{ product.price | number_format(2) }}
            {% if product.in_stock %}
                <span class="badge badge-stock">In Stock</span>
            {% else %}
                <span class="badge badge-nostock">Out of Stock</span>
            {% endif %}
        </p>
    </div>
{% endmacro %}

{% macro categoryFilter(categories, active) %}
    <div class="filters">
        <a href="/catalog" class="filter-btn{{ active is not defined or active == '' ? ' active' : '' }}">All
        {% for cat in categories %}
            <a href="/catalog?category={{ cat }}" class="filter-btn{{ active == cat ? ' active' : '' }}">{{ ca
// ...
```

Create `src/templates/catalog.twig`:

```
{% extends "catalog-base.twig" %}

{% from "catalog-macros.twig" import productCard, categoryFilter %}

{% block title %}{{ active_category | default("All") }} Products - Catalog{% endblock %}

{% block content %}
    <div class="header">
        <h1>Product Catalog</h1>
        <p>{{ products | length }} product{{ products | length != 1 ? 's' : '' }}{% if active_category %} in
    </div>

    <div class="container">
        {{ categoryFilter(categories, active_category) }}

        {% if products | length > 0 %}
            <div class="product-grid">
                {% for product in products %}
                    {{ productCard(product) }}
                {% endfor %}
            </div>
```

```
        {% else %}
            <div class="empty-state">
                <h2>No products found</h2>
                <p>Try a different category or <a href="/catalog">view all products</a>.</p>
// ...
```

Create `src/routes/catalog.php`:

```
<?php
use Tina4\Route;

Route::get("/catalog", function ($request, $response) {
    $allProducts = [
        ["name" => "Espresso Machine", "category" => "Kitchen", "price" => 299.99, "in_stock" => true, "featu
        ["name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => true, "featured" => fa
        ["name" => "Standing Desk", "category" => "Office", "price" => 549.99, "in_stock" => true, "featured"
        ["name" => "Blender", "category" => "Kitchen", "price" => 89.99, "in_stock" => false, "featured" => fa
        ["name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock" => true, "featured"
        ["name" => "Desk Lamp", "category" => "Office", "price" => 39.99, "in_stock" => true, "featured" => tr
        ["name" => "Cast Iron Skillet", "category" => "Kitchen", "price" => 44.99, "in_stock" => true, "featu
    ];

    // Get unique categories
    $categories = array_unique(array_column($allProducts, "category"));
    sort($categories);

    // Filter by category if specified
    $activeCategory = $request->query["category"] ?? "";
    if (!empty($activeCategory)) {
        $products = array_values(array_filter(
            $allProducts,
            fn($p) => strtolower($p["category"]) === strtolower($activeCategory)
        ));
// ...
```

**Expected browser output for `/catalog`:**

• A dark header with "Product Catalog" and "7 products"

• Filter buttons: All (active), Fitness, Kitchen, Office

• A grid of 7 product cards

• Three cards (Espresso Machine, Standing Desk, Desk Lamp) have a gold border and "Featured" badge

• The Blender card shows an "Out of Stock" badge in red

**Expected browser output for `/catalog?category=Kitchen`:**

• Header shows "3 products in Kitchen"

• The Kitchen filter button is active

• Only three cards: Espresso Machine, Blender, Cast Iron Skillet

## 13. Gotchas

### *1. `{% extends %}` Must Be the First Tag*

**Problem:** Template inheritance does not work. The page renders without the base layout.

**Cause:** `{% extends "base.twig" %}` must be the very first tag in the template. If there is any text, whitespace, or even a comment before it, inheritance breaks.

**Fix:** Make `{% extends %}` the absolute first thing in the file. Move any `{% from %}` imports to after the extends tag.

### *2. Undefined Variables Show Nothing*

**Problem:** `{{ username }}` renders as empty instead of showing an error.

**Cause:** Frond silently outputs nothing for undefined variables. This is by design (like Twig), but it can hide bugs.

**Fix:** Use the `default` filter: `{{ username | default("Guest") }}`. Or check first with `{% if username is defined %}`.

### *3. Auto-Escaping Prevents HTML Output*

**Problem:** You pass HTML content like `"<strong>bold</strong>"` but it appears as literal text in the page.

**Cause:** Auto-escaping converts `<` to `<` and `>` to `>` for security.

**Fix:** If the content is trusted, use `{{ content | raw }}`. Never use `raw` on user-supplied input.

### *4. Variable Scope in Includes*

**Problem:** A variable defined inside a `{% for %}` loop is not accessible after the loop ends.

**Cause:** Loop variables are scoped to the loop. They do not leak into the outer scope.

**Fix:** If you need to accumulate values, use `{% set %}` before the loop and update it inside. Or restructure your template to keep all logic within the loop.

### *5. Macro Arguments Are Positional*

**Problem:** Calling `{{ button("Click", style="danger") }}` does not work as expected.

**Cause:** Frond macros use positional arguments, not keyword arguments. The order matters.

**Fix:** Pass arguments in the order they are defined: `{{ button("Click", "/url", "danger") }}`. If you have many optional arguments, consider passing a single object.

### *6. Template File Extension Does Not Matter*

**Problem:** You are not sure whether to use `.html`, `.twig`, or `.tpl`.

**Cause:** Frond does not care about the file extension. It processes any file in `src/templates/`.

**Fix:** Pick one extension and be consistent. This book uses `.twig` for templates with Twig syntax and `.html` for simple HTML files. Both work identically.

### *7. Filters Are Not PHP Functions*

**Problem:** You try `{{ items | count }}` or `{{ name | strtoupper }}` and get an error.

**Cause:** Frond filters are not PHP functions. The filter names follow Twig conventions, not PHP conventions.

**Fix:** Use `{{ items | length }}` instead of `count`, `{{ name | upper }}` instead of `strtoupper`, `{{ text | lower }}` instead of `strtolower`. See the filter table in section 7 for the full list.

# Chapter 5: Database

## 1. From Arrays to Real Data

In Chapters 2 and 3, all our data lived in PHP arrays that reset every time the server restarted. That is fine for learning routing and responses, but real applications need persistent storage. This chapter covers Tina4's database layer -- raw queries, parameterised queries, transactions, schema inspection, helper methods, and migrations.

Tina4 supports five database engines: SQLite, PostgreSQL, MySQL, Microsoft SQL Server, and Firebird. The API is the same across all of them. You switch databases by changing one line in `.env`.

## 2. Connecting to a Database

### *The Default: SQLite*

When you scaffold a project with `tina4 init`, Tina4 creates a SQLite database at `data/app.db`. The default `.env` includes:

```
TINA4_DEBUG=true
```

With no explicit `DATABASE_URL`, Tina4 defaults to `sqlite:///data/app.db`. That is why the health check at `/health` already shows `"database": "connected"` without any configuration.

### *Connection Strings for Other Databases*

Set `DATABASE_URL` in `.env` to use a different engine:

```
# SQLite (explicit)
DATABASE_URL=sqlite:///data/app.db

# PostgreSQL
DATABASE_URL=postgres://localhost:5432/myapp

# MySQL
DATABASE_URL=mysql://localhost:3306/myapp

# Microsoft SQL Server
DATABASE_URL=mssql://localhost:1433/myapp

# Firebird
DATABASE_URL=firebird://localhost:3050/path/to/database.fdb
```

### *Separate Credentials*

If you prefer to keep credentials out of the connection string (recommended for production), use separate environment variables:

```
DATABASE_URL=postgres://localhost:5432/myapp
DATABASE_USERNAME=myuser
DATABASE_PASSWORD=secretpassword
```

Tina4 merges these with the connection string at startup. The credentials in the separate variables take precedence over any embedded in the URL.

### *Verifying the Connection*

After updating `.env`, restart the server and check:

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 3,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

If the database is not reachable, you will see `"database": "disconnected"` with an error message.

## 3. Getting the Database Object

In your route handlers, access the database via the global `Tina4\Database` class:

```
<?php
use Tina4\Route;
use Tina4\Database;

Route::get("/api/test-db", function ($request, $response) {
    $db = Database::getConnection();

    $result = $db->fetch("SELECT 1 + 1 AS answer");

    return $response->json($result);
});
```

```
curl http://localhost:7145/api/test-db
```

```
{"answer": 2}
```

`Database::getConnection()` returns the active database connection. You call methods like `fetch()`, `execute()`, and `fetchOne()` on this object.

## 4. Raw Queries

### *fetch() -- Get Multiple Rows*

```
$db = Database::getConnection();

// Returns an array of associative arrays
$products = $db->fetch("SELECT * FROM products WHERE price > 50");
```

Each row is an associative array with column names as keys:

```
// $products looks like:
[
    ["id" => 1, "name" => "Keyboard", "price" => 79.99],
    ["id" => 4, "name" => "Standing Desk", "price" => 549.99]
]
```

### *fetchOne() -- Get a Single Row*

```
$product = $db->fetchOne("SELECT * FROM products WHERE id = 1");
// Returns: ["id" => 1, "name" => "Keyboard", "price" => 79.99]
```

If no row matches, `fetchOne()` returns `null`.

### *execute() -- Run a Statement*

For INSERT, UPDATE, DELETE, and DDL statements that do not return rows:

```
$db->execute("INSERT INTO products (name, price) VALUES ('Widget', 9.99)");
$db->execute("UPDATE products SET price = 89.99 WHERE id = 1");
$db->execute("DELETE FROM products WHERE id = 5");
$db->execute("CREATE TABLE IF NOT EXISTS logs (id INTEGER PRIMARY KEY, message TEXT, created_at TEXT)");
```

### *Full Example: A Simple Query Route*

```
<?php
use Tina4\Route;
use Tina4\Database;

Route::get("/api/products", function ($request, $response) {
    $db = Database::getConnection();

    $products = $db->fetch("SELECT * FROM products ORDER BY name");

    return $response->json([
        "products" => $products,
        "count" => count($products)
    ]);
});
```

```
curl http://localhost:7145/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Keyboard", "price": 79.99, "in_stock": 1},
    {"id": 2, "name": "Mouse", "price": 29.99, "in_stock": 1},
    {"id": 3, "name": "Monitor", "price": 399.99, "in_stock": 0}
  ],
  "count": 3
}
```

## 5. Parameterised Queries

Never concatenate user input into SQL strings. This is how SQL injection attacks happen:

```
// NEVER do this:
$db->fetch("SELECT * FROM products WHERE name = '" . $userInput . "'");
```

Instead, use parameterised queries. Pass parameters as the second argument:

```
$db = Database::getConnection();

// Named parameters
$product = $db->fetchOne(
    "SELECT * FROM products WHERE id = :id",
    ["id" => 42]
);

// Positional parameters
$products = $db->fetch(
    "SELECT * FROM products WHERE price BETWEEN ? AND ? ORDER BY price",
    [10.00, 100.00]
);
```

The database driver handles escaping. Your input is never part of the SQL string.

### *A Safe Search Endpoint*

```php
<?php
use Tina4\Route;
use Tina4\Database;

Route::get("/api/products/search", function ($request, $response) {
    $db = Database::getConnection();

    $q = $request->query["q"] ?? "";
    $maxPrice = (float) ($request->query["max_price"] ?? 99999);

    if (empty($q)) {
        return $response->json(["error" => "Query parameter 'q' is required"], 400);
    }

    $products = $db->fetch(
        "SELECT * FROM products WHERE name LIKE :query AND price <= :maxPrice ORDER BY name",
        ["query" => "%" . $q . "%", "maxPrice" => $maxPrice]
    );

    return $response->json([
        "query" => $q,
        "max_price" => $maxPrice,
        "results" => $products,
        "count" => count($products)
    ]);
// ...
```

```bash
curl "http://localhost:7145/api/products/search?q=key&max_price=100"
```

```json
{
  "query": "key",
  "max_price": 100,
  "results": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "in_stock": 1}
  ],
  "count": 1
}
```

## 6. Transactions

When you need multiple operations to succeed or fail together, use transactions:

```php
<?php
use Tina4\Route;
use Tina4\Database;

Route::post("/api/orders", function ($request, $response) {
    $db = Database::getConnection();
    $body = $request->body;

    try {
        $db->startTransaction();

        // Create the order
        $db->execute(
            "INSERT INTO orders (customer_id, total, status) VALUES (:customerId, :total, 'pending')",
            ["customerId" => $body["customer_id"], "total" => $body["total"]]
        );

        // Get the new order ID
```

```
        $order = $db->fetchOne("SELECT last_insert_rowid() AS id");
        $orderId = $order["id"];

        // Create order items
        foreach ($body["items"] as $item) {
            $db->execute(
                "INSERT INTO order_items (order_id, product_id, quantity, price) VALUES (:orderId, :productId,
// ...
```

If any step fails, `rollback()` undoes everything. The database is never left in a half-finished state.

**Important:** You must call `commit()` to save the changes. If you forget, the transaction will be rolled back when the connection closes.

## 7. Schema Inspection

Tina4 provides methods to inspect your database structure at runtime:

### *getTables()*

```
$db = Database::getConnection();
$tables = $db->getTables();
```

Returns an array of table names:

```
["orders", "order_items", "products", "users"]
```

### *getColumns()*

```
$columns = $db->getColumns("products");
```

Returns an array of column definitions:

```
[
    ["name" => "id", "type" => "INTEGER", "nullable" => false, "primary" => true],
    ["name" => "name", "type" => "TEXT", "nullable" => false, "primary" => false],
    ["name" => "price", "type" => "REAL", "nullable" => true, "primary" => false],
    ["name" => "in_stock", "type" => "INTEGER", "nullable" => true, "primary" => false]
]
```

### *tableExists()*

```
if ($db->tableExists("products")) {
    // Table exists, safe to query
}
```

### *A Schema Info Endpoint*

```
<?php
use Tina4\Route;
use Tina4\Database;

Route::get("/api/schema", function ($request, $response) {
    $db = Database::getConnection();
    $tables = $db->getTables();

    $schema = [];
    foreach ($tables as $table) {
        $schema[$table] = $db->getColumns($table);
    }

    return $response->json(["tables" => $schema]);
```

```
});
```

## 8. Batch Operations with executeMany()

Insert or update many rows efficiently:

```
$db = Database::getConnection();

$products = [
    ["name" => "Widget A", "price" => 9.99],
    ["name" => "Widget B", "price" => 14.99],
    ["name" => "Widget C", "price" => 19.99],
    ["name" => "Widget D", "price" => 24.99]
];

$db->executeMany(
    "INSERT INTO products (name, price) VALUES (:name, :price)",
    $products
);
```

`executeMany()` prepares the statement once and executes it for each item in the array. This is significantly faster than calling `execute()` in a loop because the SQL only needs to be parsed once.

## 9. Helper Methods: insert(), update(), delete()

Tina4 provides shorthand methods so you do not have to write SQL for simple operations.

### *insert()*

```
$db = Database::getConnection();

// Insert a single row
$db->insert("products", [
    "name" => "Wireless Mouse",
    "price" => 34.99,
    "in_stock" => 1
]);

// Insert multiple rows
$db->insert("products", [
    ["name" => "USB Cable", "price" => 9.99, "in_stock" => 1],
    ["name" => "HDMI Cable", "price" => 14.99, "in_stock" => 1],
    ["name" => "DisplayPort Cable", "price" => 19.99, "in_stock" => 0]
]);
```

### *update()*

```
// Update rows matching a filter
$db->update("products", ["price" => 39.99, "in_stock" => 1], "id = :id", ["id" => 7]);
```

The third argument is the WHERE clause, and the fourth is the parameters for it.

### *delete()*

```
// Delete rows matching a filter
$db->delete("products", "id = :id", ["id" => 7]);
```

These helper methods generate the SQL for you. They are convenient for simple CRUD operations but do not replace raw queries for complex joins, subqueries, or aggregations.

# 10. Migrations

Migrations are versioned SQL scripts that evolve your database schema over time. Instead of manually running `CREATE TABLE` statements, you write migration files and Tina4 applies them in order.

## *Creating a Migration*

Use the CLI:

```
tina4 migrate:create create_products_table
```

```
Created migration: src/migrations/20260322143000_create_products_table.sql
```

The filename starts with a timestamp so migrations always run in chronological order.

Edit the generated file `src/migrations/20260322143000_create_products_table.sql`:

```
-- UP
CREATE TABLE products (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    category TEXT NOT NULL DEFAULT 'Uncategorized',
    price REAL NOT NULL DEFAULT 0.00,
    in_stock INTEGER NOT NULL DEFAULT 1,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS products;
```

The `-- UP` section runs when applying the migration. The `-- DOWN` section runs when rolling back.

## *Running Migrations*

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322143000_create_products_table.sql
Migrations complete. 1 applied.
```

## *Checking Migration Status*

```
tina4 migrate:status
```

```
Migration                                      Status     Applied At
---------                                      ------     ----------
20260322143000_create_products_table.sql        applied    2026-03-22 14:30:00
20260322150000_create_orders_table.sql          pending    -
```

## *Rolling Back*

```
tina4 migrate:rollback
```

```
Rolling back last migration...
  [ROLLED BACK] 20260322150000_create_orders_table.sql
Rollback complete. 1 rolled back.
```

This runs the `-- DOWN` section of the most recently applied migration.

## *A Real Migration Sequence*

Here is what a typical project's migrations look like:

```
src/migrations/
■■■ 20260322143000_create_products_table.sql
■■■ 20260322143100_create_users_table.sql
■■■ 20260322143200_create_orders_table.sql
■■■ 20260322143300_create_order_items_table.sql
■■■ 20260323091500_add_email_index_to_users.sql
```

The last migration might look like:

```
-- UP
CREATE INDEX idx_users_email ON users (email);

-- DOWN
DROP INDEX IF EXISTS idx_users_email;
```

Migrations are applied in filename order. Each migration runs only once -- Tina4 tracks which ones have been applied in a `_migrations` table.

## 11. Query Caching

For read-heavy applications, enable query caching:

```
TINA4_DB_CACHE=true
```

When enabled, Tina4 caches the results of `fetch()` and `fetchOne()` calls. Identical queries with identical parameters return cached results instead of hitting the database again.

The cache is automatically invalidated when you call `execute()`, `insert()`, `update()`, or `delete()` on the same table.

You can also control caching per-query:

```
// Force a fresh query (bypass cache)
$products = $db->fetch("SELECT * FROM products", [], false); // third arg = use cache

// Clear the entire cache
$db->clearCache();
```

## 12. Exercise: Build a Notes App

Build a notes application backed by SQLite. Create the database table via a migration and build a full CRUD API.

### *Requirements*

1. Create a migration that creates a `notes` table with columns:

• `id` -- integer, primary key, auto-increment

• `title` -- text, not null

• `content` -- text, not null

• `tag` -- text, default "general"

• `created_at` -- text, default current timestamp

• `updated_at` -- text, default current timestamp

2. Build these API endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/notes` | List all notes. Support `?tag=` and `?search=` filters. |

| `GET` | `/api/notes/{id:int}` | Get a single note. 404 if not found. |

| `POST` | `/api/notes` | Create a note. Validate title and content are not empty. |

| `PUT` | `/api/notes/{id:int}` | Update a note. 404 if not found. |

| `DELETE` | `/api/notes/{id:int}` | Delete a note. 204 on success, 404 if not found. |

### *Test with:*

```
# Create
curl -X POST http://localhost:7145/api/notes \
  -H "Content-Type: application/json" \
  -d '{"title": "Shopping List", "content": "Milk, eggs, bread", "tag": "personal"}'

# List all
curl http://localhost:7145/api/notes

# Search
curl "http://localhost:7145/api/notes?search=shopping"

# Filter by tag
curl "http://localhost:7145/api/notes?tag=personal"

# Get one
curl http://localhost:7145/api/notes/1

# Update
curl -X PUT http://localhost:7145/api/notes/1 \
  -H "Content-Type: application/json" \
  -d '{"title": "Updated Shopping List", "content": "Milk, eggs, bread, butter"}'

# Delete
curl -X DELETE http://localhost:7145/api/notes/1
```

## 13. Solution

### *Migration*

Create `src/migrations/20260322143000_create_notes_table.sql`:

```
-- UP
CREATE TABLE notes (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    content TEXT NOT NULL,
    tag TEXT NOT NULL DEFAULT 'general',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS notes;
```

Run the migration:

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322143000_create_notes_table.sql
Migrations complete. 1 applied.
```

## *Routes*

Create `src/routes/notes.php`:

```
<?php
use Tina4\Route;
use Tina4\Database;

// List all notes with optional filters
Route::get("/api/notes", function ($request, $response) {
    $db = Database::getConnection();

    $tag = $request->query["tag"] ?? "";
    $search = $request->query["search"] ?? "";

    $sql = "SELECT * FROM notes";
    $params = [];
    $conditions = [];

    if (!empty($tag)) {
        $conditions[] = "tag = :tag";
        $params["tag"] = $tag;
    }

    if (!empty($search)) {
        $conditions[] = "(title LIKE :search OR content LIKE :search)";
        $params["search"] = "%" . $search . "%";
    }

// ...
```

**Expected output for create:**

```
{
  "id": 1,
  "title": "Shopping List",
  "content": "Milk, eggs, bread",
  "tag": "personal",
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

(Status: `201 Created`)

**Expected output for list:**

```
{
  "notes": [
    {
      "id": 1,
      "title": "Shopping List",
      "content": "Milk, eggs, bread",
      "tag": "personal",
      "created_at": "2026-03-22 14:30:00",
      "updated_at": "2026-03-22 14:30:00"
    }
  ],
```

```
  "count": 1
}
```

**Expected output for search:**

```
{
  "notes": [
    {
      "id": 1,
      "title": "Shopping List",
      "content": "Milk, eggs, bread",
      "tag": "personal",
      "created_at": "2026-03-22 14:30:00",
      "updated_at": "2026-03-22 14:30:00"
    }
  ],
  "count": 1
}
```

**Expected output for validation error:**

```
{"errors": ["Title is required", "Content is required"]}
```

(Status: `400 Bad Request`)

# 14. Gotchas

### *1. Forgetting commit()*

**Problem:** You call `startTransaction()`, run your queries, but the changes disappear on the next request.

**Cause:** Without `commit()`, the transaction is rolled back when the connection closes.

**Fix:** Always call `$db->commit()` after your transaction succeeds. Use a try/catch block with `$db->rollback()` in the catch.

### *2. Connection String Formats*

**Problem:** The database will not connect and you see a cryptic error about the connection string.

**Cause:** Each database engine expects a specific URL format. A common mistake is using `mysql://user:pass@host/db` when the engine expects the port.

**Fix:** Always include the port: `mysql://localhost:3306/mydb`. Here are the default ports:

| Engine | Default Port |

|--------|-------------|

| PostgreSQL | 5432 |

| MySQL | 3306 |

| MSSQL | 1433 |

| Firebird | 3050 |

| SQLite | (file path, no port) |

### *3. SQLite File Paths*

**Problem:** SQLite creates a new empty database instead of using the existing one.

**Cause:** The path in `DATABASE_URL` is relative and resolves to the wrong directory, or you used `sqlite://` (two slashes) instead of `sqlite:///` (three slashes).

**Fix:** Use three slashes for a relative path: `sqlite:///data/app.db`. For an absolute path, use four slashes: `sqlite:////var/data/app.db`. The third slash separates the scheme from the path; the fourth starts the absolute path.

### *4. Parameterised Queries with LIKE*

**Problem:** `WHERE name LIKE :q` with `["q" => "%search%"]` works, but `WHERE name LIKE '%:q%'` does not.

**Cause:** Parameters inside quotes are treated as literal text, not as placeholders.

**Fix:** Include the `%` wildcards in the parameter value, not in the SQL: `["q" => "%" . $search . "%"]`. The SQL should be `WHERE name LIKE :q`.

### *5. Boolean Values in SQLite*

**Problem:** You insert `true` or `false` but the database stores `1` or `0`. When you read it back, you get integers, not booleans.

**Cause:** SQLite does not have a native boolean type. It stores booleans as integers.

**Fix:** Cast in your PHP code: `"in_stock" => (bool) $row["in_stock"]`. Or accept that `1` and `0` work as truthy/falsy in PHP.

### *6. Migration Already Applied*

**Problem:** You edited a migration file and ran `tina4 migrate` again, but nothing changed.

**Cause:** Tina4 tracks applied migrations by filename. Once applied, a migration will not run again even if you change its contents.

**Fix:** Create a new migration for schema changes. Do not edit applied migrations. If you are in early development and want to start fresh, use `tina4 migrate:rollback` to undo and then `tina4 migrate` to reapply.

### *7. fetch() Returns Empty Array, Not Null*

**Problem:** You check `if ($result === null)` but it never matches, even when the table is empty.

**Cause:** `fetch()` always returns an array. An empty result is `[]`, not `null`. Only `fetchOne()` returns `null` when no row matches.

**Fix:** Check with `if (empty($result))` or `if (count($result) === 0)`.

# Chapter 6: ORM

## 1. From SQL to Objects

In Chapter 5 you wrote raw SQL for every operation. That works, but it gets tedious. You end up writing the same `INSERT INTO products (name, price, ...) VALUES (:name, :price, ...)` patterns over and over. The ORM (Object-Relational Mapper) lets you work with PHP classes instead. You define a class, map it to a table, and call methods like `save()`, `load()`, and `delete()`.

Tina4's ORM is minimal by design. It does not try to hide SQL completely -- it gives you convenient methods for common operations and stays out of the way when you need raw queries.

## 2. Defining a Model

ORM models live in `src/orm/`. Every `.php` file in that directory is auto-loaded, just like route files.

Create `src/orm/Product.php`:

```
<?php
use Tina4\ORM;

class Product extends ORM
{
    public int $id;
    public string $name;
    public string $category = "Uncategorized";
    public float $price = 0.00;
    public bool $inStock = true;
    public string $createdAt;
    public string $updatedAt;

    // Map to the "products" table
    public string $tableName = "products";

    // Primary key field
    public string $primaryKey = "id";
}
```

That is a complete model. Let us break it down:

• **Extends `ORM`** -- This gives you `save()`, `load()`, `delete()`, `select()`, and other methods.

• **Public properties** -- Each property maps to a database column. The property name is `camelCase` and the column name is `snake_case`. Tina4 converts between them automatically: `inStock` maps to `in_stock`, `createdAt` maps to `created_at`.

• **`$tableName`** -- The database table this model maps to. If you omit it, Tina4 infers it from the class name: `Product` becomes `products`, `OrderItem` becomes `order_items`.

• **`$primaryKey`** -- The primary key column. Defaults to `"id"`.

• **Default values** -- Properties with defaults (like `$category = "Uncategorized"`) are used when creating new records without specifying those fields.

## 3. Field Types

Use PHP type declarations on your properties. Tina4 uses these types when generating DDL and validating data:

| PHP Type | Database Type (SQLite) | Database Type (PostgreSQL) | Notes |

|----------|----------|----------|-------|

| `int` | INTEGER | INTEGER | Whole numbers |

| `string` | TEXT | VARCHAR(255) | Text fields |

| `float` | REAL | DOUBLE PRECISION | Decimal numbers |

| `bool` | INTEGER | BOOLEAN | SQLite stores as 0/1 |

| `?string` | TEXT (nullable) | VARCHAR(255) NULL | Nullable with `?` prefix |

### *Nullable Fields*

Use PHP's nullable type syntax:

```
public ?string $description = null;
public ?float $discount = null;
```

A nullable property allows `null` values in the database column.

### *Primary Keys and Auto-Increment*

By default, Tina4 treats the `$primaryKey` field as auto-incrementing. When you call `save()` on a new object (where the primary key is not set), the database generates the ID:

```
$product = new Product();
$product->name = "Widget";
$product->price = 9.99;
$product->save();

echo $product->id; // Auto-generated: 1, 2, 3, ...
```

## 4. Creating and Saving Records

### *save() -- Insert or Update*

The `save()` method inserts a new record or updates an existing one, depending on whether the primary key is set:

```
<?php
use Tina4\Route;

Route::post("/api/products", function ($request, $response) {
    $body = $request->body;

    $product = new Product();
    $product->name = $body["name"];
    $product->category = $body["category"] ?? "Uncategorized";
    $product->price = (float) ($body["price"] ?? 0);
    $product->inStock = (bool) ($body["in_stock"] ?? true);
    $product->save();

    return $response->json($product->toDict(), 201);
```

```
});
```

```
curl -X POST http://localhost:7145/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Wireless Keyboard", "category": "Electronics", "price": 79.99}'
```

```
{
  "id": 1,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

### *Updating an Existing Record*

When `id` is already set, `save()` performs an UPDATE:

```
Route::put("/api/products/{id:int}", function ($request, $response) {
    $product = new Product();
    $product->load($request->params["id"]);

    if (empty($product->id)) {
        return $response->json(["error" => "Product not found"], 404);
    }

    $body = $request->body;
    $product->name = $body["name"] ?? $product->name;
    $product->price = (float) ($body["price"] ?? $product->price);
    $product->category = $body["category"] ?? $product->category;
    $product->save();

    return $response->json($product->toDict());
});
```

## 5. Loading Records

### *load() -- Get by Primary Key*

```
$product = new Product();
$product->load(42);

if (empty($product->id)) {
    // Product with ID 42 not found
}
```

`load()` populates the object's properties from the database row matching the primary key. If no row matches, the properties remain at their default values (or unset).

### *A Simple Get Endpoint*

```
Route::get("/api/products/{id:int}", function ($request, $response) {
    $product = new Product();
    $product->load($request->params["id"]);

    if (empty($product->id)) {
        return $response->json(["error" => "Product not found"], 404);
    }

    return $response->json($product->toDict());
```

```
});
```

```
curl http://localhost:7145/api/products/1
```

```
{
  "id": 1,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

## 6. Deleting Records

### *delete()*

```
Route::delete("/api/products/{id:int}", function ($request, $response) {
    $product = new Product();
    $product->load($request->params["id"]);

    if (empty($product->id)) {
        return $response->json(["error" => "Product not found"], 404);
    }

    $product->delete();

    return $response->json(null, 204);
});
```

`delete()` removes the row from the database. The object still exists in memory but the database row is gone.

## 7. Querying with select()

The `select()` method lets you find records with filters, ordering, and pagination:

### *Basic Select*

```
$product = new Product();
$products = $product->select("*");
```

Returns an array of Product objects with all records.

### *Filtering*

```
$product = new Product();

// Simple filter
$electronics = $product->select("*", "category = :category", ["category" => "Electronics"]);

// Multiple conditions
$affordable = $product->select("*", "price < :maxPrice AND in_stock = :inStock", [
    "maxPrice" => 100,
    "inStock" => 1
]);
```

### *Ordering*

```
$product = new Product();
$sorted = $product->select("*", "", [], "price DESC");
```

The fourth argument is the ORDER BY clause.

### *Pagination*

```
$product = new Product();

$page = 1;
$perPage = 10;
$offset = ($page - 1) * $perPage;

$products = $product->select("*", "", [], "name ASC", $perPage, $offset);
```

The fifth argument is LIMIT, the sixth is OFFSET.

### *A Full List Endpoint with Filters*

```
<?php
use Tina4\Route;

Route::get("/api/products", function ($request, $response) {
    $product = new Product();

    $category = $request->query["category"] ?? "";
    $minPrice = (float) ($request->query["min_price"] ?? 0);
    $maxPrice = (float) ($request->query["max_price"] ?? 999999);
    $page = (int) ($request->query["page"] ?? 1);
    $perPage = (int) ($request->query["per_page"] ?? 20);
    $sort = $request->query["sort"] ?? "name";
    $order = strtoupper($request->query["order"] ?? "ASC");

    // Build filter
    $conditions = [];
    $params = [];

    if (!empty($category)) {
        $conditions[] = "category = :category";
        $params["category"] = $category;
    }

    $conditions[] = "price >= :minPrice AND price <= :maxPrice";
    $params["minPrice"] = $minPrice;
// ...
```

```
curl "http://localhost:7145/api/products?category=Electronics&sort=price&order=DESC&page=1&per_page=5"
```

```
{
  "products": [
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": true},
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": true}
  ],
  "page": 1,
  "per_page": 5,
  "count": 2
}
```

## 8. Creating Tables from Models

Instead of writing a migration manually, you can generate the table from your model:

```
$product = new Product();
$product->createTable();
```

This reads the properties, types, and defaults from the model class and generates the appropriate `CREATE TABLE` statement for your database engine.

You can also use the CLI:

```
tina4 orm:create-table Product
```

```
Created table "products" with 7 columns.
```

This is convenient during early development. For production, use migrations (Chapter 5) so schema changes are versioned and reversible.

# 9. Relationships

### *hasMany -- One-to-Many*

A user has many posts:

Create `src/orm/User.php`:

```
<?php
use Tina4\ORM;

class User extends ORM
{
    public int $id;
    public string $name;
    public string $email;
    public string $createdAt;

    public string $tableName = "users";
    public string $primaryKey = "id";

    public function posts(): array
    {
        return $this->hasMany(Post::class, "user_id");
    }
}
```

Create `src/orm/Post.php`:

```
<?php
use Tina4\ORM;

class Post extends ORM
{
    public int $id;
    public int $userId;
    public string $title;
    public string $body;
    public string $createdAt;

    public string $tableName = "posts";
    public string $primaryKey = "id";

    public function user(): ?User
    {
        return $this->belongsTo(User::class, "user_id");
    }
```

```
    public function comments(): array
    {
        return $this->hasMany(Comment::class, "post_id");
    }
}
```

The second argument to `hasMany()` is the foreign key column on the related table. `$this->hasMany(Post::class, "user_id")` means: find all rows in `posts` where `user_id` equals this user's ID.

### *hasOne -- One-to-One*

```
public function profile(): ?Profile
{
    return $this->hasOne(Profile::class, "user_id");
}
```

`hasOne()` works like `hasMany()` but returns a single object instead of an array.

### *belongsTo -- Inverse Relationship*

The inverse of `hasMany`. A post belongs to a user:

```
public function user(): ?User
{
    return $this->belongsTo(User::class, "user_id");
}
```

`belongsTo(User::class, "user_id")` means: load the User where `users.id` equals `this->user_id`.

### *Using Relationships*

```
Route::get("/api/users/{id:int}", function ($request, $response) {
    $user = new User();
    $user->load($request->params["id"]);

    if (empty($user->id)) {
        return $response->json(["error" => "User not found"], 404);
    }

    $posts = $user->posts();

    return $response->json([
        "user" => $user->toDict(),
        "posts" => array_map(fn($p) => $p->toDict(), $posts),
        "post_count" => count($posts)
    ]);
});
```

```
curl http://localhost:7145/api/users/1
```

```
{
  "user": {"id": 1, "name": "Alice", "email": "alice@example.com"},
  "posts": [
    {"id": 1, "user_id": 1, "title": "First Post", "body": "Hello world!"},
    {"id": 3, "user_id": 1, "title": "Second Post", "body": "Another one."}
  ],
  "post_count": 2
}
```

## 10. Eager Loading

Calling relationship methods inside a loop creates the N+1 query problem. If you load 100 users and then call `$user->posts()` for each one, that is 101 queries (1 for users + 100 for posts).

Use the `include` parameter with `select()` to eager-load relationships:

```
$user = new User();
$users = $user->select("*", "", [], "name ASC", 20, 0, ["posts"]);
```

The seventh argument is an array of relationship names to include. This runs just 2 queries (one for users, one for all related posts) and stitches the results together.

### *toDict() with Nested Includes*

When eager loading is active, `toDict()` includes the related data:

```
$user = new User();
$users = $user->select("*", "", [], "", 0, 0, ["posts"]);

$result = array_map(fn($u) => $u->toDict(), $users);

return $response->json($result);
```

```
[
  {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "posts": [
      {"id": 1, "title": "First Post", "body": "Hello world!"},
      {"id": 3, "title": "Second Post", "body": "Another one."}
    ]
  },
  {
    "id": 2,
    "name": "Bob",
    "email": "bob@example.com",
    "posts": [
      {"id": 2, "title": "Bob's Post", "body": "Hi there."}
    ]
  }
]
```

### *Nested Eager Loading*

Load multiple levels deep with dot notation:

```
$user = new User();
$users = $user->select("*", "", [], "", 0, 0, ["posts", "posts.comments"]);
```

This loads users, their posts, and each post's comments in 3 queries total.

## 11. Soft Delete

If your model has a `deletedAt` property, Tina4 supports soft delete -- marking records as deleted without actually removing them from the database:

```
<?php
use Tina4\ORM;

class Post extends ORM
{
```

```
    public int $id;
    public string $title;
    public string $body;
    public ?string $deletedAt = null;

    public string $tableName = "posts";
    public string $primaryKey = "id";
    public bool $softDelete = true;
}
```

With `$softDelete = true`:

• `$post->delete()` sets `deleted_at` to the current timestamp instead of deleting the row

• `select()` automatically excludes rows where `deleted_at` is not null

• `$post->forceDelete()` permanently removes the row

### *Restoring Soft-Deleted Records*

```
$post = new Post();
$post->load(5); // Load even if soft-deleted
$post->deletedAt = null;
$post->save();
```

### *Including Soft-Deleted Records in Queries*

```
$post = new Post();
$allPosts = $post->select("*", "", [], "", 0, 0, [], true); // eighth arg = include deleted
```

## 12. NumericField for Prices

When working with money, floating-point arithmetic causes rounding errors. Use the `NumericField` type for precise decimal values:

```
<?php
use Tina4\ORM;
use Tina4\NumericField;

class Product extends ORM
{
    public int $id;
    public string $name;
    public NumericField $price;
    public NumericField $discount;

    public string $tableName = "products";
    public string $primaryKey = "id";
}
```

`NumericField` maps to `DECIMAL` or `NUMERIC` in the database and handles precision correctly. When you access `$product->price`, you get a value that compares and calculates accurately for financial operations.

## 13. Auto-CRUD

One of Tina4's most powerful features is auto-CRUD -- it automatically generates REST endpoints for any ORM model.

Add the `$autoCrud` property to your model:

```
<?php
use Tina4\ORM;

class Product extends ORM
{
    public int $id;
    public string $name;
    public string $category = "Uncategorized";
    public float $price = 0.00;
    public bool $inStock = true;

    public string $tableName = "products";
    public string $primaryKey = "id";
    public bool $autoCrud = true;
}
```

With `$autoCrud = true`, Tina4 automatically registers these routes:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/products` | List all with pagination |

| `GET` | `/api/products/{id}` | Get one by ID |

| `POST` | `/api/products` | Create a new record |

| `PUT` | `/api/products/{id}` | Update a record |

| `DELETE` | `/api/products/{id}` | Delete a record |

The endpoint prefix is derived from the table name: `products` becomes `/api/products`.

```
curl http://localhost:7145/api/products
```

```
{
  "data": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": true},
    {"id": 2, "name": "Yoga Mat", "category": "Fitness", "price": 29.99, "in_stock": true}
  ],
  "total": 2,
  "page": 1,
  "per_page": 20
}
```

Auto-CRUD supports query parameters for filtering, sorting, and pagination out of the box:

```
curl "http://localhost:7145/api/products?category=Electronics&sort=price&order=desc&page=1&per_page=10"
```

You can still define custom routes alongside auto-CRUD. Your custom routes take precedence over the auto-generated ones.

## 14. Exercise: Build a Blog

Build a blog with three models: User, Post, and Comment. Use relationships, eager loading, and auto-CRUD.

### *Requirements*

1. Create three models in `src/orm/`:

**User** -- `users` table:

• `id` (int, primary key)

• `name` (string)

• `email` (string)

• `createdAt` (string)

• Has many posts

**Post** -- `posts` table:

• `id` (int, primary key)

• `userId` (int, foreign key)

• `title` (string)

• `body` (string)

• `published` (bool, default false)

• `createdAt` (string)

• Belongs to user, has many comments

**Comment** -- `comments` table:

• `id` (int, primary key)

• `postId` (int, foreign key)

• `authorName` (string)

• `body` (string)

• `createdAt` (string)

• Belongs to post

2. Create migrations for all three tables.

3. Build custom endpoints:

| Method | Path | Description |

|--------|------|-------------|

| `GET` | `/api/blog/posts` | List published posts with author info (eager load user) |

| `GET` | `/api/blog/posts/{id:int}` | Get a post with author and comments (eager load both) |

| `POST` | `/api/blog/posts/{id:int}/comments` | Add a comment to a post |

4. Enable auto-CRUD on the User model for admin access at `/api/users`.

### *Test with:*

```
# Create a user (via auto-CRUD)
curl -X POST http://localhost:7145/api/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com"}'
```

```
# Create a post
curl -X POST http://localhost:7145/api/blog/posts \
  -H "Content-Type: application/json" \
  -d '{"user_id": 1, "title": "My First Post", "body": "Hello world!", "published": true}'

# List posts
curl http://localhost:7145/api/blog/posts

# Add a comment
curl -X POST http://localhost:7145/api/blog/posts/1/comments \
  -H "Content-Type: application/json" \
  -d '{"author_name": "Bob", "body": "Great post!"}'

# Get post with comments
curl http://localhost:7145/api/blog/posts/1
```

## 15. Solution

### *Migrations*

Create `src/migrations/20260322150000_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS users;
```

Create `src/migrations/20260322150100_create_posts_table.sql`:

```
-- UP
CREATE TABLE posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    title TEXT NOT NULL,
    body TEXT NOT NULL,
    published INTEGER NOT NULL DEFAULT 0,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id)
);

-- DOWN
DROP TABLE IF EXISTS posts;
```

Create `src/migrations/20260322150200_create_comments_table.sql`:

```
-- UP
CREATE TABLE comments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    post_id INTEGER NOT NULL,
    author_name TEXT NOT NULL,
    body TEXT NOT NULL,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (post_id) REFERENCES posts(id)
);

-- DOWN
```

```
DROP TABLE IF EXISTS comments;
```

Run them:

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322150000_create_users_table.sql
  [APPLIED] 20260322150100_create_posts_table.sql
  [APPLIED] 20260322150200_create_comments_table.sql
Migrations complete. 3 applied.
```

### *Models*

Create `src/orm/User.php`:

```
<?php
use Tina4\ORM;

class User extends ORM
{
    public int $id;
    public string $name;
    public string $email;
    public string $createdAt;

    public string $tableName = "users";
    public string $primaryKey = "id";
    public bool $autoCrud = true;

    public function posts(): array
    {
        return $this->hasMany(Post::class, "user_id");
    }
}
```

Create `src/orm/Post.php`:

```
<?php
use Tina4\ORM;

class Post extends ORM
{
    public int $id;
    public int $userId;
    public string $title;
    public string $body;
    public bool $published = false;
    public string $createdAt;

    public string $tableName = "posts";
    public string $primaryKey = "id";

    public function user(): ?User
    {
        return $this->belongsTo(User::class, "user_id");
    }

    public function comments(): array
    {
        return $this->hasMany(Comment::class, "post_id");
    }
}
```

Create `src/orm/Comment.php`:

```php
<?php
use Tina4\ORM;

class Comment extends ORM
{
    public int $id;
    public int $postId;
    public string $authorName;
    public string $body;
    public string $createdAt;

    public string $tableName = "comments";
    public string $primaryKey = "id";

    public function post(): ?Post
    {
        return $this->belongsTo(Post::class, "post_id");
    }
}
```

### *Routes*

Create `src/routes/blog.php`:

```php
<?php
use Tina4\Route;

// List published posts with author
Route::get("/api/blog/posts", function ($request, $response) {
    $post = new Post();
    $posts = $post->select("*", "published = :published", ["published" => 1], "created_at DESC", 0, 0, ["user

    $results = array_map(fn($p) => $p->toDict(), $posts);

    return $response->json([
        "posts" => $results,
        "count" => count($results)
    ]);
});

// Get a single post with author and comments
Route::get("/api/blog/posts/{id:int}", function ($request, $response) {
    $post = new Post();
    $post->load($request->params["id"]);

    if (empty($post->id)) {
        return $response->json(["error" => "Post not found"], 404);
    }

// ...
```

**Expected output for GET /api/blog/posts/1:**

```json
{
  "id": 1,
  "user_id": 1,
  "title": "My First Post",
  "body": "Hello world!",
  "published": true,
  "created_at": "2026-03-22 15:00:00",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
```

```
  },
  "comments": [
    {
      "id": 1,
      "post_id": 1,
      "author_name": "Bob",
      "body": "Great post!",
      "created_at": "2026-03-22 15:01:00"
    }
  ],
  "comment_count": 1
}
```

## 16. Gotchas

### *1. Table Naming Convention*

**Problem:** Your model class is `OrderItem` but queries fail because the table does not exist.

**Cause:** Tina4 converts `OrderItem` to `order_items` (plural, snake_case). If your table is named `order_item` (singular), it will not match.

**Fix:** Set `$tableName` explicitly: `public string $tableName = "order_item";`. Or rename your table to match the convention.

### *2. Null Handling*

**Problem:** A field that should be nullable causes errors when the value is null.

**Cause:** The PHP property is declared as `string` instead of `?string`. PHP 8.1+ enforces type declarations strictly.

**Fix:** Use nullable types: `public ?string $description = null;`. The `?` prefix allows null values.

### *3. Relationship Foreign Key Direction*

**Problem:** You write `$this->hasMany(Post::class, "id")` and get wrong results.

**Cause:** The foreign key argument is the column on the related table, not the current table. `hasMany(Post::class, "user_id")` means "find posts where posts.user_id = this.id", not "find posts where posts.id = this.user_id".

**Fix:** The foreign key is always on the "many" side. For `hasMany`, it is the column on the child table. For `belongsTo`, it is the column on the current table.

### *4. camelCase to snake_case Mapping*

**Problem:** You have a property `$userId` but the database column is `user_id`. Queries return null for this field.

**Cause:** Tina4 automatically converts between `camelCase` (PHP) and `snake_case` (database). If your database column is `userid` (no underscore), the mapping breaks.

**Fix:** Use consistent naming. PHP properties should be `camelCase` and database columns should be `snake_case`. If your column names differ, you may need to adjust them or override the mapping.

### *5. Forgetting save()*

**Problem:** You set properties on a model but the database does not change.

**Cause:** You forgot to call `$model->save()`. Setting properties only changes the in-memory object.

**Fix:** Always call `save()` after modifying properties that should be persisted.

### *6. Auto-CRUD Endpoint Conflicts*

**Problem:** Your custom route at `/api/products/{id}` does not work after enabling auto-CRUD on the Product model.

**Cause:** Both your custom route and the auto-CRUD route match the same path. The first one registered wins.

**Fix:** Custom routes defined in `src/routes/` files are loaded before auto-CRUD routes, so they take precedence. If that is not the behavior you want, use a different path for your custom route (e.g., `/api/shop/products/{id}`).

### *7. select() Returns Objects, Not Arrays*

**Problem:** You try to use array syntax (`$result["name"]`) on the result of `select()` and get an error.

**Cause:** `select()` returns an array of model objects, not associative arrays. Each item is an instance of your model class.

**Fix:** Access properties with object syntax: `$result->name`. Or convert to an array with `$result->toDict()`.

# Chapter 7: Authentication

## 1. Locking the Door

Up to now, every endpoint you have built is public. Anyone with the URL can read, create, update, and delete data. That is fine for a tutorial, but a real application needs to know who is making a request and whether they are allowed to make it.

This chapter covers Tina4's authentication system: JWT tokens, password hashing, middleware-based route protection, CSRF tokens for forms, and session management.

## 2. JWT Tokens

Tina4 uses JSON Web Tokens (JWT) for authentication. A JWT is a signed string that contains a payload (like a user ID and role). The server creates the token at login, the client sends it with every request, and the server verifies it without needing to look anything up in a database.

### *Generating a Token*

```
<?php
use Tina4\Auth;

$payload = [
    "user_id" => 42,
    "email" => "alice@example.com",
    "role" => "admin"
];

$token = Auth::getToken($payload);
```

`getToken()` signs the payload with a secret key and returns a JWT string like:


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo0MiwiZW1haWwiOiJhbGljZUBleGFtcGxlLmNvbSIsInJvbGUiOiJhZG1pb
```


The token has three parts separated by dots: header, payload, and signature. The signature ensures the token has not been tampered with.

### *Token Expiry*

By default, tokens expire after 1 hour. Configure this in `.env`:

```
TINA4_JWT_EXPIRY=3600
```

The value is in seconds. Common settings:

| Value | Duration |

|-------|----------|

| `900` | 15 minutes |

| `3600` | 1 hour (default) |

| `86400` | 24 hours |

| `604800` | 7 days |

### *Validating a Token*

```
$isValid = Auth::validToken($token);
// Returns true if the token is valid and not expired, false otherwise
```

### *Reading the Payload*

```
$payload = Auth::getPayload($token);
```

Returns the original payload array:

```
[
    "user_id" => 42,
    "email" => "alice@example.com",
    "role" => "admin",
    "iat" => 1711112600,  // issued at (Unix timestamp)
    "exp" => 1711116200   // expires at (Unix timestamp)
]
```

If the token is invalid or expired, `getPayload()` returns `null`.

### *The Secret Key*

Tina4 uses a secret key to sign and verify tokens. On first run, it automatically generates a random key and stores it in `secrets/jwt.key`. You can also set it explicitly:

```
TINA4_JWT_SECRET=my-very-long-and-random-secret-key-at-least-32-chars
```

Keep this key secret. If someone gets it, they can forge tokens.

## 3. Password Hashing

Never store passwords in plain text. Tina4 provides two functions for secure password handling:

### *Hashing a Password*

```
use Tina4\Auth;

$hash = Auth::hashPassword("my-secure-password");
// Returns: "$2y$10$abc123...long-hash-string..."
```

This uses PHP's `password_hash()` with bcrypt under the hood. Each hash includes a random salt, so hashing the same password twice produces different results.

### *Checking a Password*

```
$isCorrect = Auth::checkPassword("my-secure-password", $storedHash);
// Returns true if the password matches the hash
```

### *Registration Example*

```
<?php
use Tina4\Route;
use Tina4\Auth;
use Tina4\Database;

Route::post("/api/register", function ($request, $response) {
    $body = $request->body;

    // Validate input
    if (empty($body["name"]) || empty($body["email"]) || empty($body["password"])) {
        return $response->json(["error" => "Name, email, and password are required"], 400);
    }
```

```
    if (strlen($body["password"]) < 8) {
        return $response->json(["error" => "Password must be at least 8 characters"], 400);
    }

    $db = Database::getConnection();

    // Check if email already exists
    $existing = $db->fetchOne("SELECT id FROM users WHERE email = :email", ["email" => $body["email"]]);
    if ($existing !== null) {
        return $response->json(["error" => "Email already registered"], 409);
    }

// ...
```

```
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "user": {"id": 1, "name": "Alice", "email": "alice@example.com"}
}
```

## 4. The Login Flow

Here is the complete login flow: the client sends credentials, the server validates them, and returns a JWT token.

```
<?php
use Tina4\Route;
use Tina4\Auth;
use Tina4\Database;

/**
 * @noauth
 */
Route::post("/api/login", function ($request, $response) {
    $body = $request->body;

    if (empty($body["email"]) || empty($body["password"])) {
        return $response->json(["error" => "Email and password are required"], 400);
    }

    $db = Database::getConnection();

    // Find the user
    $user = $db->fetchOne(
        "SELECT id, name, email, password_hash FROM users WHERE email = :email",
        ["email" => $body["email"]]
    );

    if ($user === null) {
        return $response->json(["error" => "Invalid email or password"], 401);
// ...
```

Notice the `@noauth` annotation. The login endpoint must be public -- you cannot require a token to get a token.

```
curl -X POST http://localhost:7145/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com"
  }
}
```

The client stores this token (in localStorage, a cookie, or memory) and sends it with subsequent requests.

## 5. Using Tokens in Requests

The client sends the token in the `Authorization` header with the `Bearer` prefix:

```
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

## 6. Protecting Routes

### *Auth Middleware*

Create a reusable auth middleware:

```
<?php
use Tina4\Auth;

function authMiddleware($request, $response, $next) {
    $authHeader = $request->headers["Authorization"] ?? "";

    if (empty($authHeader) || !str_starts_with($authHeader, "Bearer ")) {
        return $response->json(["error" => "Authorization header required"], 401);
    }

    $token = substr($authHeader, 7); // Remove "Bearer " prefix

    if (!Auth::validToken($token)) {
        return $response->json(["error" => "Invalid or expired token"], 401);
    }

    $payload = Auth::getPayload($token);
    $request->user = $payload; // Attach user data to the request

    return $next($request, $response);
}
```

### *Applying Middleware to Routes*

```
<?php
use Tina4\Route;

Route::get("/api/profile", function ($request, $response) {
    return $response->json([
        "user_id" => $request->user["user_id"],
        "email" => $request->user["email"],
        "name" => $request->user["name"]
    ]);
}, "authMiddleware");
```

```
# Without token -- 401
curl http://localhost:7145/api/profile

{"error":"Authorization header required"}

# With valid token -- 200
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

{
  "user_id": 1,
  "email": "alice@example.com",
  "name": "Alice"
}
```

### *Applying Middleware to a Group*

```
Route::group("/api/admin", function () {

    Route::get("/stats", function ($request, $response) {
        return $response->json(["active_users" => 42]);
    });

    Route::get("/logs", function ($request, $response) {
        return $response->json(["logs" => []]);
    });

}, "authMiddleware");
```

Every route in the group requires a valid token.

## 7. @noauth and @secured Decorators

As introduced in Chapter 2, these decorators control authentication at the route level.

### *@noauth -- Skip Authentication*

Use `@noauth` for public endpoints that should bypass any global or group-level auth:

```
/**
 * @noauth
 */
Route::get("/api/public/health", function ($request, $response) {
    return $response->json(["status" => "ok"]);
});

/**
 * @noauth
 */
Route::post("/api/login", function ($request, $response) {
    // Login logic
});

/**
 * @noauth
 */
Route::post("/api/register", function ($request, $response) {
    // Registration logic
});
```

### *@secured -- Require Authentication for GET Routes*

By default, POST, PUT, PATCH, and DELETE routes are considered secured. GET routes are public. Use `@secured` to explicitly protect a GET route:

```
/**
 * @secured
 */
Route::get("/api/me", function ($request, $response) {
    // This GET route requires authentication
    return $response->json($request->user);
});
```

### *Role-Based Authorization*

Combine auth middleware with role checks:

```
<?php
use Tina4\Auth;

function requireRole($role) {
    return function ($request, $response, $next) use ($role) {
        // First check authentication
        $authHeader = $request->headers["Authorization"] ?? "";
        if (empty($authHeader) || !str_starts_with($authHeader, "Bearer ")) {
            return $response->json(["error" => "Authorization required"], 401);
        }

        $token = substr($authHeader, 7);
        if (!Auth::validToken($token)) {
            return $response->json(["error" => "Invalid or expired token"], 401);
        }

        $payload = Auth::getPayload($token);

        // Check role
        if (($payload["role"] ?? "") !== $role) {
            return $response->json(["error" => "Forbidden. Required role: " . $role], 403);
        }

        $request->user = $payload;
        return $next($request, $response);
// ...
```

Note that `requireRole()` returns a closure. To use it as middleware, you need to register the returned function:

```
$adminOnly = requireRole("admin");

Route::delete("/api/users/{id:int}", function ($request, $response) {
    // Only admins can delete users
    return $response->json(["deleted" => true]);
}, $adminOnly);
```

## 8. CSRF Protection

For traditional form-based applications (not SPAs), Tina4 provides CSRF protection with form tokens.

### *Generating a Token*

In your template, include the CSRF token in every form:

```
<form method="POST" action="/profile/update">
```

```
    {{ form_token() }}

    <div class="form-group">
        <label for="name">Name</label>
        <input type="text" name="name" id="name" value="{{ user.name }}">
    </div>

    <button type="submit">Update Profile</button>
</form>
```

`{{ form_token() }}` renders a hidden input field:

```
<input type="hidden" name="_token" value="abc123randomtoken456">
```

### *Validating the Token*

In your route handler, check the token:

```
<?php
use Tina4\Route;
use Tina4\Auth;

Route::post("/profile/update", function ($request, $response) {
    // Validate CSRF token
    if (!Auth::validateFormToken($request->body["_token"] ?? "")) {
        return $response->json(["error" => "Invalid form token. Please refresh and try again."], 403);
    }

    // Process the form...
    return $response->redirect("/profile");
});
```

The CSRF token is tied to the user's session and expires after a single use. This prevents cross-site request forgery attacks where a malicious site tricks the user's browser into submitting a form.

### *When to Use CSRF Tokens*

Use CSRF tokens for:

• HTML forms submitted by browsers

• Any POST/PUT/DELETE from server-rendered pages

You do not need CSRF tokens for:

• API endpoints that use JWT (the Bearer token already proves the request is intentional)

• Single-page applications that use `fetch()` with custom headers (the `Authorization` header cannot be set by cross-origin forms)

## 9. Sessions

Tina4 supports server-side sessions for storing per-user state between requests. Sessions work alongside JWT tokens -- use JWTs for API authentication and sessions for stateful web pages.

### *Session Configuration*

Set the session backend in `.env`:

```
# File-based sessions (default)
TINA4_SESSION_DRIVER=file
```

```
# Redis
TINA4_SESSION_DRIVER=redis
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379

# MongoDB
TINA4_SESSION_DRIVER=mongodb
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=27017

# Valkey
TINA4_SESSION_DRIVER=valkey
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
```

File-based sessions work out of the box with no additional dependencies. Use Redis or Valkey for production deployments with multiple servers, so sessions are shared across instances.

### *Using Sessions*

Access session data via `$request->session`:

```php
<?php
use Tina4\Route;

Route::post("/login-form", function ($request, $response) {
    // After validating credentials...
    $request->session["user_id"] = 42;
    $request->session["user_name"] = "Alice";
    $request->session["logged_in"] = true;

    return $response->redirect("/dashboard");
});

Route::get("/dashboard", function ($request, $response) {
    if (empty($request->session["logged_in"])) {
        return $response->redirect("/login");
    }

    return $response->render("dashboard.html", [
        "user_name" => $request->session["user_name"]
    ]);
});

Route::post("/logout", function ($request, $response) {
    // Clear all session data
    $request->session = [];
// ...
```

### *Session Options*

```
TINA4_SESSION_LIFETIME=3600       # Session lifetime in seconds (default: 3600)
TINA4_SESSION_NAME=tina4_session  # Cookie name for the session ID
```

## 10. Exercise: Build Login, Register, and Profile

Build a complete authentication system with registration, login, profile viewing, and password changing.

### *Requirements*

1. Create a `users` table migration with: `id`, `name`, `email` (unique), `password_hash`, `role` (default "user"), `created_at`

2. Build these endpoints:

| Method | Path | Auth | Description |

|--------|------|------|-------------|

| `POST` | `/api/register` | @noauth | Create an account. Validate name, email, password (min 8 chars). |

| `POST` | `/api/login` | @noauth | Login. Return JWT token. |

| `GET` | `/api/profile` | secured | Get current user's profile from token. |

| `PUT` | `/api/profile` | secured | Update name and email. |

| `PUT` | `/api/profile/password` | secured | Change password. Require current password. |

3. Create auth middleware that extracts the user from the JWT and attaches it to `$request->user`.

### *Test with:*

```
# Register
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'

# Login
curl -X POST http://localhost:7145/api/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securePass123"}'

# Save the token from login response, then:

# Get profile
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE"

# Update profile
curl -X PUT http://localhost:7145/api/profile \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Smith"}'

# Change password
curl -X PUT http://localhost:7145/api/profile/password \
  -H "Authorization: Bearer YOUR_TOKEN_HERE" \
// ...
```

## 11. Solution

### *Migration*

Create `src/migrations/20260322160000_create_auth_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
```

```
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS users;
```

```
tina4 migrate
```

### Middleware

Create `src/routes/middleware.php`:

```
<?php
use Tina4\Auth;

function authMiddleware($request, $response, $next) {
    $authHeader = $request->headers["Authorization"] ?? "";

    if (empty($authHeader) || !str_starts_with($authHeader, "Bearer ")) {
        return $response->json(["error" => "Authorization required. Send: Authorization: Bearer <token>"], 401
    }

    $token = substr($authHeader, 7);

    if (!Auth::validToken($token)) {
        return $response->json(["error" => "Invalid or expired token. Please login again."], 401);
    }

    $payload = Auth::getPayload($token);
    $request->user = $payload;

    return $next($request, $response);
}
```

### Routes

Create `src/routes/auth.php`:

```
<?php
use Tina4\Route;
use Tina4\Auth;
use Tina4\Database;

/**
 * @noauth
 */
Route::post("/api/register", function ($request, $response) {
    $body = $request->body;

    // Validate input
    $errors = [];
    if (empty($body["name"])) {
        $errors[] = "Name is required";
    }
    if (empty($body["email"])) {
        $errors[] = "Email is required";
    }
    if (empty($body["password"])) {
        $errors[] = "Password is required";
    } elseif (strlen($body["password"]) < 8) {
        $errors[] = "Password must be at least 8 characters";
    }

// ...
```

**Expected output for register:**

```
{
  "message": "Registration successful",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "role": "user",
    "created_at": "2026-03-22 16:00:00"
  }
}
```

(Status: `201 Created`)

**Expected output for login:**

```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice",
    "email": "alice@example.com",
    "role": "user"
  }
}
```

**Expected output for profile without token:**

```
{"error":"Authorization required. Send: Authorization: Bearer <token>"}
```

(Status: `401 Unauthorized`)

**Expected output for profile with token:**

```
{
  "id": 1,
  "name": "Alice",
  "email": "alice@example.com",
  "role": "user",
  "created_at": "2026-03-22 16:00:00"
}
```

**Expected output for password change with wrong current password:**

```
{"error":"Current password is incorrect"}
```

(Status: `401 Unauthorized`)

**Expected output for successful password change:**

```
{"message":"Password changed successfully"}
```

# 12. Gotchas

## *1. Token Expiry Confusion*

**Problem:** Tokens that worked yesterday now return 401.

**Cause:** The default token lifetime is 1 hour. After that, the token is invalid even if the signature is correct.

**Fix:** Issue a new token at login. If your application needs long-lived sessions, use refresh tokens: a short-lived access token (15 minutes) paired with a long-lived refresh token (7 days) that can be used to get a new access token without re-entering credentials.

### *2. Secret Key Management*

**Problem:** Tokens generated on one server are invalid on another, or tokens stop working after a deployment.

**Cause:** Each server generated its own random `secrets/jwt.key` file. Or the key file was deleted/regenerated during deployment.

**Fix:** Set `TINA4_JWT_SECRET` in `.env` explicitly and use the same value across all servers. Store it in your deployment secrets manager (not in version control). If the key changes, all existing tokens become invalid and users must log in again.

### *3. CORS with Authentication*

**Problem:** Frontend requests with the `Authorization` header fail with a CORS error, even though `CORS_ORIGINS=*` is set.

**Cause:** When the browser sends an `Authorization` header, it first sends a preflight `OPTIONS` request. The server must respond to the OPTIONS request with the correct CORS headers, including `Access-Control-Allow-Headers: Authorization`.

**Fix:** Tina4 handles preflight requests automatically. Make sure `CORS_ORIGINS` is set correctly. If it is still failing, check that you are not overriding CORS headers in middleware.

### *4. Storing Tokens in localStorage*

**Problem:** Your token is stolen via an XSS attack because it was stored in `localStorage`.

**Cause:** Any JavaScript on the page can read `localStorage`, including injected scripts from an XSS vulnerability.

**Fix:** Store tokens in `httpOnly` cookies when possible -- they cannot be accessed by JavaScript. For SPAs that must use `localStorage`, implement strict Content Security Policy headers and sanitize all user input.

### *5. Forgetting @noauth on Login*

**Problem:** Your login endpoint returns 401 -- you cannot log in because the endpoint requires authentication.

**Cause:** If you have global auth middleware, the login endpoint needs the `@noauth` annotation to bypass it.

**Fix:** Add `@noauth` to your login and register routes. Without it, users cannot authenticate because authentication is required to authenticate -- a catch-22.

### *6. Password Hash Column Too Short*

**Problem:** Registration fails with a database error about the password hash being too long.

**Cause:** bcrypt hashes are 60 characters long. If your `password_hash` column is defined as `VARCHAR(50)`, it gets truncated.

**Fix:** Use `TEXT` for the password hash column, or at minimum `VARCHAR(255)`. Never constrain the hash length.

### *7. Token in URL Query Parameters*

**Problem:** Tokens in URLs like `/api/profile?token=eyJ...` leak through browser history, server logs, and the Referer header.

**Cause:** Query parameters are visible in many places where headers are not.

**Fix:** Always send tokens in the `Authorization` header, never in the URL. The only exception is WebSocket connections, where the initial HTTP upgrade request cannot carry custom headers -- use a short-lived token for that case.

# Chapter 8: Middleware

## 1. The Gatekeepers

Your API needs CORS headers for the React frontend, rate limiting for the public endpoints, and auth checking for admin routes -- all without cluttering your route handlers. You could copy-paste the same ten lines of CORS code into every handler, but that breaks the moment you forget one. You could pile all the checks into a giant `if` tree at the top of each handler, but that buries the actual business logic under boilerplate.

Middleware solves this. It lets you wrap routes with reusable logic that runs before (or after) the handler. Each middleware does one job -- check a token, set CORS headers, log the request, enforce rate limits -- and passes control to the next layer. Your route handlers stay focused on their actual purpose.

In Chapter 2 you saw a brief introduction to middleware. This chapter goes deep: built-in middleware, custom middleware, execution order, short-circuiting, and real-world patterns.

## 2. What Middleware Is

A middleware function sits between the incoming HTTP request and your route handler. It receives the request, the response, and a `$next` function. It can:

1. Inspect or modify the request before passing it along

2. Decide not to pass it along (short-circuit)

3. Inspect or modify the response on the way back

Here is the simplest middleware that does nothing but pass through:

```
<?php

function passthrough($request, $response, $next) {
    return $next($request, $response);
}
```

And here is one that blocks everything:

```
<?php

function blockEverything($request, $response, $next) {
    return $response->json(["error" => "Service unavailable"], 503);
}
```

The `$next` function is what makes middleware composable. Call it to continue to the next middleware (or the route handler if there is no more middleware). Do not call it to stop the chain.

## 3. Built-in CorsMiddleware

Cross-Origin Resource Sharing (CORS) is the browser mechanism that controls which domains can call your API. When your React frontend at `http://localhost:3000` makes a fetch request to your Tina4 API at `http://localhost:7145`, the browser sends a preflight `OPTIONS` request first. If your API does not respond with the right headers, the browser blocks the request entirely.

Tina4 provides a built-in `CorsMiddleware` that handles this. Configure it in `.env`:

```
CORS_ORIGINS=http://localhost:3000,https://myapp.com
CORS_METHODS=GET,POST,PUT,PATCH,DELETE,OPTIONS
CORS_HEADERS=Content-Type,Authorization,X-API-Key
CORS_MAX_AGE=86400
CORS_CREDENTIALS=true
```

Apply it to a group:

```
<?php
use Tina4\Route;

Route::group("/api", function () {

    Route::get("/products", function ($request, $response) {
        return $response->json(["products" => []]);
    });

    Route::post("/products", function ($request, $response) {
        return $response->json(["created" => true], 201);
    });

}, "CorsMiddleware");
```

Test the preflight:

```
curl -X OPTIONS http://localhost:7145/api/products \
  -H "Origin: http://localhost:3000" \
  -H "Access-Control-Request-Method: POST" \
  -H "Access-Control-Request-Headers: Content-Type,Authorization" \
  -v
```

You should see these response headers:

```
Access-Control-Allow-Origin: http://localhost:3000
Access-Control-Allow-Methods: GET,POST,PUT,PATCH,DELETE,OPTIONS
Access-Control-Allow-Headers: Content-Type,Authorization,X-API-Key
Access-Control-Max-Age: 86400
Access-Control-Allow-Credentials: true
```

The `OPTIONS` request returns `204 No Content` with those headers. The browser caches the preflight result for 86400 seconds (24 hours), so subsequent requests skip the preflight entirely.

### *Wildcard Origins*

During development, you can allow all origins:

```
CORS_ORIGINS=*
```

This is the default when you scaffold a project. Do not use `*` in production -- specify your actual domains.

### *CORS Without Middleware*

If you prefer, you can skip the middleware and handle CORS globally. Add `CORS_ORIGINS` to your `.env` and Tina4 applies CORS headers to all responses automatically. The middleware approach gives you finer control -- you can apply CORS to specific route groups and leave internal routes without CORS headers.

## 4. Built-in RateLimiter

Rate limiting prevents a single client from overwhelming your API. Tina4's built-in `RateLimiter` middleware tracks requests per IP address and returns `429 Too Many Requests` when the limit is exceeded.

Configure it in `.env`:

```
TINA4_RATE_LIMIT=60
TINA4_RATE_LIMIT_WINDOW=60
```

This means 60 requests per 60 seconds per IP. Apply it:

```
<?php
use Tina4\Route;

Route::group("/api/public", function () {

    Route::get("/search", function ($request, $response) {
        $q = $request->query["q"] ?? "";
        return $response->json(["query" => $q, "results" => []]);
    });

    Route::get("/catalog", function ($request, $response) {
        return $response->json(["items" => []]);
    });

}, "RateLimiter");
```

When a client exceeds the limit:

```
curl http://localhost:7145/api/public/search?q=test
```

```
{"error":"Rate limit exceeded. Try again in 42 seconds.","retry_after":42}
```

The response includes a `Retry-After` header with the number of seconds until the limit resets.

### *Custom Limits Per Group*

You can override the global rate limit for specific groups by setting the limit in the middleware call:

```
<?php
use Tina4\Route;

// Public endpoints: 30 requests per minute
Route::group("/api/public", function () {
    Route::get("/search", function ($request, $response) {
        return $response->json(["results" => []]);
    });
}, "RateLimiter:30");

// Authenticated endpoints: 120 requests per minute
Route::group("/api/v1", function () {
    Route::get("/data", function ($request, $response) {
        return $response->json(["data" => []]);
    });
}, ["authMiddleware", "RateLimiter:120"]);
```

The syntax `"RateLimiter:30"` passes `30` as a parameter to the rate limiter. This overrides the global `TINA4_RATE_LIMIT` for that group only.

## 5. Writing Custom Middleware

Custom middleware follows the same pattern: receive `$request`, `$response`, and `$next`. Define it as a named function.

### Request Logging Middleware

```
<?php

function logRequest($request, $response, $next) {
    $start = microtime(true);
    $method = $request->method;
    $path = $request->path;
    $ip = $request->ip;

    // Log the incoming request
    error_log("[" . date("Y-m-d H:i:s") . "] " . $method . " " . $path . " from " . $ip);

    // Call the next middleware or route handler
    $result = $next($request, $response);

    // Log the completion
    $duration = round((microtime(true) - $start) * 1000, 2);
    error_log("  Completed in " . $duration . "ms");

    return $result;
}
```

Save this in `src/routes/middleware.php` (or any file in `src/routes/`). Apply it:

```
Route::get("/api/products", function ($request, $response) {
    return $response->json(["products" => []]);
}, "logRequest");
```

```
curl http://localhost:7145/api/products
```

Your server log shows:

```
[2026-03-22 14:30:00] GET /api/products from 127.0.0.1
  Completed in 2.34ms
```

### Request Timing Middleware

```
<?php

function addTiming($request, $response, $next) {
    $start = microtime(true);

    $result = $next($request, $response);

    $duration = round((microtime(true) - $start) * 1000, 2);

    // Add timing header to the response
    $response->addHeader("X-Response-Time", $duration . "ms");

    return $result;
}
```

```
curl http://localhost:7145/api/products -v
```

In the response headers:

```
X-Response-Time: 3.12ms
```

This is useful for performance monitoring. Your frontend or monitoring tools can read the `X-Response-Time` header to track endpoint performance.

### *IP Whitelist Middleware*

```php
<?php

function ipWhitelist($request, $response, $next) {
    $allowedIps = explode(",", $_ENV["ALLOWED_IPS"] ?? "127.0.0.1");

    if (!in_array($request->ip, $allowedIps)) {
        return $response->json([
            "error" => "Access denied",
            "your_ip" => $request->ip
        ], 403);
    }

    return $next($request, $response);
}
```

Configure in `.env`:

```
ALLOWED_IPS=127.0.0.1,10.0.0.5,192.168.1.100
```

Apply it to admin routes:

```php
Route::group("/admin", function () {
    Route::get("/dashboard", function ($request, $response) {
        return $response->json(["page" => "admin dashboard"]);
    });
}, "ipWhitelist");
```

```
curl http://localhost:7145/admin/dashboard
```

If your IP is in the whitelist:

```
{"page":"admin dashboard"}
```

If not:

```
{"error":"Access denied","your_ip":"203.0.113.42"}
```

### *Request Validation Middleware*

```php
<?php

function requireJson($request, $response, $next) {
    if (in_array($request->method, ["POST", "PUT", "PATCH"])) {
        $contentType = $request->headers["Content-Type"] ?? "";

        if (strpos($contentType, "application/json") === false) {
            return $response->json([
                "error" => "Content-Type must be application/json",
                "received" => $contentType
            ], 415);
        }
    }

    return $next($request, $response);
}
```

This middleware ensures all POST, PUT, and PATCH requests send JSON. Apply it to your entire API:

```php
Route::group("/api", function () {
    // All API routes here
}, "requireJson");
```

```
curl -X POST http://localhost:7145/api/products \
  -d "name=Widget"
```

```
{"error":"Content-Type must be application/json","received":"application/x-www-form-urlencoded"}
```

(Status: `415 Unsupported Media Type`)

## 6. Applying Middleware to Individual Routes

Pass middleware as the third argument to any route method:

```
<?php
use Tina4\Route;

Route::get("/api/data", function ($request, $response) {
    return $response->json(["data" => [1, 2, 3]]);
}, "logRequest");

Route::post("/api/data", function ($request, $response) {
    return $response->json(["created" => true], 201);
}, ["logRequest", "requireJson"]);
```

For a single middleware, pass a string. For multiple, pass an array. Each middleware runs in the order listed.

## 7. Route Groups with Shared Middleware

Groups apply middleware to every route inside them:

```
<?php
use Tina4\Route;

// Public API -- rate limited, CORS enabled
Route::group("/api/public", function () {

    Route::get("/products", function ($request, $response) {
        return $response->json(["products" => []]);
    });

    Route::get("/categories", function ($request, $response) {
        return $response->json(["categories" => []]);
    });

}, ["CorsMiddleware", "RateLimiter:30"]);

// Admin API -- auth required, IP restricted, logged
Route::group("/api/admin", function () {

    Route::get("/users", function ($request, $response) {
        return $response->json(["users" => []]);
    });

    Route::delete("/users/{id:int}", function ($request, $response) {
        $id = $request->params["id"];
// ...
```

Individual routes inside a group can add their own middleware too. The group middleware runs first, then the route-specific middleware:

```
Route::group("/api", function () {

    // This route gets: logRequest -> requireJson -> handler
```

```
    Route::post("/upload", function ($request, $response) {
        return $response->json(["uploaded" => true]);
    }, "requireJson");

}, "logRequest");
```

## 8. Middleware Execution Order

When you stack middleware, they execute from outer to inner -- like layers of an onion. The first middleware listed runs first on the way in and last on the way out.

Consider this setup:

```
<?php

function middlewareA($request, $response, $next) {
    error_log("A: before");
    $result = $next($request, $response);
    error_log("A: after");
    return $result;
}

function middlewareB($request, $response, $next) {
    error_log("B: before");
    $result = $next($request, $response);
    error_log("B: after");
    return $result;
}

function middlewareC($request, $response, $next) {
    error_log("C: before");
    $result = $next($request, $response);
    error_log("C: after");
    return $result;
}

Route::get("/api/test", function ($request, $response) {
    error_log("Handler");
    return $response->json(["ok" => true]);
}, ["middlewareA", "middlewareB", "middlewareC"]);

curl http://localhost:7145/api/test
```

Server log output:

```
A: before
B: before
C: before
Handler
C: after
B: after
A: after
```

The request flows inward: A, B, C, Handler. The response flows outward: C, B, A. This means:

• `middlewareA` sees the request first and the response last. Put CORS headers and request logging here.

• `middlewareC` sees the request last and the response first. Put response transformations here.

• Authentication should be early (catch unauthorized requests before doing expensive work).

• Timing should be outermost (measure the entire pipeline).

### *Group + Route Middleware Order*

```
Route::group("/api", function () {

    Route::get("/data", function ($request, $response) {
        return $response->json(["data" => true]);
    }, "middlewareC");

}, ["middlewareA", "middlewareB"]);
```

Execution order: `middlewareA` -> `middlewareB` -> `middlewareC` -> handler. Group middleware always runs before route middleware.

## 9. Short-Circuiting

When middleware does not call `$next`, the chain stops. No subsequent middleware runs and the route handler is never called. This is called short-circuiting.

### *Authentication Short-Circuit*

```
<?php

function requireAuth($request, $response, $next) {
    $token = $request->headers["Authorization"] ?? "";

    if (empty($token)) {
        // Short-circuit: return 401, do not call $next
        return $response->json(["error" => "Authentication required"], 401);
    }

    return $next($request, $response);
}
```

If the `Authorization` header is missing, the middleware returns a `401` immediately. The route handler never executes. This saves server resources -- you do not query the database or run business logic for unauthenticated requests.

### *Maintenance Mode*

```
<?php

function maintenanceMode($request, $response, $next) {
    $isMaintenanceMode = ($_ENV["MAINTENANCE_MODE"] ?? "false") === "true";

    if ($isMaintenanceMode) {
        // Allow health checks even in maintenance mode
        if ($request->path === "/health") {
            return $next($request, $response);
        }

        return $response->json([
            "error" => "Service is undergoing maintenance",
            "retry_after" => 300
        ], 503);
    }

    return $next($request, $response);
}
```

Add to `.env` when you need to take the site down:

```
MAINTENANCE_MODE=true
```

Apply it globally:

```
Route::group("/", function () {
    // All your routes
}, "maintenanceMode");
```

Every request except `/health` gets a `503 Service Unavailable` response.

### *Conditional Short-Circuiting*

Sometimes you want to short-circuit only certain methods:

```
<?php

function readOnly($request, $response, $next) {
    if (in_array($request->method, ["POST", "PUT", "PATCH", "DELETE"])) {
        return $response->json([
            "error" => "API is in read-only mode"
        ], 405);
    }

    return $next($request, $response);
}
```

GET requests pass through. Write operations are blocked. This is useful for standby replicas or demo environments.

## 10. Modifying Requests in Middleware

Middleware can add data to the request before passing it to the handler. This is how authentication middleware attaches user information:

```
<?php
use Tina4\Auth;

function attachUser($request, $response, $next) {
    $authHeader = $request->headers["Authorization"] ?? "";

    if (!empty($authHeader) && str_starts_with($authHeader, "Bearer ")) {
        $token = substr($authHeader, 7);
        if (Auth::validToken($token)) {
            $request->user = Auth::getPayload($token);
        }
    }

    // Always call $next -- this middleware does not block requests
    // Routes that need auth should check $request->user themselves
    return $next($request, $response);
}
```

This pattern is different from a blocking auth middleware. It attaches user data if a token is present but does not block unauthenticated requests. Some routes need the user data (show personalized content) but are still accessible without it.

### *Adding Request Metadata*

```
<?php

function addRequestId($request, $response, $next) {
```

```
    $requestId = bin2hex(random_bytes(8));
    $request->requestId = $requestId;

    $result = $next($request, $response);

    // Add request ID to response headers for debugging
    $response->addHeader("X-Request-Id", $requestId);

    return $result;
}
```

Your route handler can now access `$request->requestId` for logging, error tracking, or correlation:

```
Route::get("/api/data", function ($request, $response) {
    error_log("[" . $request->requestId . "] Processing data request");
    return $response->json(["request_id" => $request->requestId, "data" => []]);
}, "addRequestId");
```

```
curl http://localhost:7145/api/data -v
```

```
{"request_id":"a1b2c3d4e5f6a7b8","data":[]}
```

Response headers include:

```
X-Request-Id: a1b2c3d4e5f6a7b8
```

## 11. Real-World Middleware Stack

Here is a realistic middleware setup for a production API:

```
<?php
use Tina4\Route;

// src/routes/middleware.php

function addRequestId($request, $response, $next) {
    $request->requestId = bin2hex(random_bytes(8));
    $result = $next($request, $response);
    $response->addHeader("X-Request-Id", $request->requestId);
    return $result;
}

function logRequest($request, $response, $next) {
    $start = microtime(true);
    error_log("[" . $request->requestId . "] " . $request->method . " " . $request->path);

    $result = $next($request, $response);

    $duration = round((microtime(true) - $start) * 1000, 2);
    error_log("[" . $request->requestId . "] Completed in " . $duration . "ms");

    return $result;
}

function requireApiKey($request, $response, $next) {
// ...
```

```
<?php
use Tina4\Route;

// src/routes/api.php

Route::group("/api/v1", function () {
```

```
    Route::get("/products", function ($request, $response) {
        return $response->json(["products" => [
            ["id" => 1, "name" => "Widget", "price" => 9.99],
            ["id" => 2, "name" => "Gadget", "price" => 19.99]
        ]]);
    });

    Route::get("/products/{id:int}", function ($request, $response) {
        $id = $request->params["id"];
        return $response->json(["id" => $id, "name" => "Widget", "price" => 9.99]);
    });

    Route::post("/products", function ($request, $response) {
        $body = $request->body;
        return $response->json([
            "id" => 3,
            "name" => $body["name"] ?? "Unknown",
            "price" => (float) ($body["price"] ?? 0)
// ...
```

Test without an API key:

```
curl http://localhost:7145/api/v1/products
```

```
{"error":"Invalid or missing API key","request_id":"a1b2c3d4e5f6a7b8"}
```

(Status: `401 Unauthorized`)

Test with a valid API key:

```
curl http://localhost:7145/api/v1/products \
  -H "X-API-Key: my-secret-key"
```

```
{"products":[{"id":1,"name":"Widget","price":9.99},{"id":2,"name":"Gadget","price":19.99}]}
```

## 12. Exercise: Build an API Key Middleware

Build a middleware called `validateApiKey` that:

1. Checks for an `X-API-Key` header on every request

2. Validates the key against a comma-separated list stored in the `API_KEYS` environment variable

3. If the key is missing, returns `401` with `{"error": "API key required"}`

4. If the key is invalid, returns `403` with `{"error": "Invalid API key"}`

5. If the key is valid, attaches the key to `$request->apiKey` and continues

6. Apply this middleware to a route group with at least two endpoints

### *Setup*

Add this to your `.env`:

```
API_KEYS=key-alpha-001,key-beta-002,key-gamma-003
```

### *Test with:*

```
# No API key -- should get 401
curl http://localhost:7145/api/partner/data

# Invalid API key -- should get 403
```

```
curl http://localhost:7145/api/partner/data \
  -H "X-API-Key: wrong-key"

# Valid API key -- should get 200
curl http://localhost:7145/api/partner/data \
  -H "X-API-Key: key-alpha-001"

# Valid key on another endpoint
curl http://localhost:7145/api/partner/stats \
  -H "X-API-Key: key-beta-002"
```

## 13. Solution

Create `src/routes/api-key-middleware.php`:

```
<?php
use Tina4\Route;

function validateApiKey($request, $response, $next) {
    $apiKey = $request->headers["X-API-Key"] ?? "";

    // Check if header is present
    if (empty($apiKey)) {
        return $response->json(["error" => "API key required"], 401);
    }

    // Validate against allowed keys
    $validKeys = array_map("trim", explode(",", $_ENV["API_KEYS"] ?? ""));

    if (!in_array($apiKey, $validKeys)) {
        return $response->json(["error" => "Invalid API key"], 403);
    }

    // Attach key to request for downstream use
    $request->apiKey = $apiKey;

    return $next($request, $response);
}

Route::group("/api/partner", function () {
// ...
```

**Expected output -- no key:**

```
curl http://localhost:7145/api/partner/data
```

```
{"error":"API key required"}
```

(Status: `401 Unauthorized`)

**Expected output -- invalid key:**

```
curl http://localhost:7145/api/partner/data -H "X-API-Key: wrong-key"
```

```
{"error":"Invalid API key"}
```

(Status: `403 Forbidden`)

**Expected output -- valid key:**

```
curl http://localhost:7145/api/partner/data -H "X-API-Key: key-alpha-001"
```

```
{
```

```
  "authenticated_with": "key-alpha-001",
  "data": [
    {"id": 1, "value": "alpha"},
    {"id": 2, "value": "beta"}
  ]
}
```

**Expected output -- different valid key on stats:**

```
curl http://localhost:7145/api/partner/stats -H "X-API-Key: key-beta-002"
```

```
{
  "authenticated_with": "key-beta-002",
  "stats": {
    "total_requests": 1423,
    "avg_response_ms": 42
  }
}
```

# 14. Gotchas

### *1. Middleware Must Be a Named Function*

**Problem:** You pass an anonymous closure as middleware and get an error.

**Cause:** Tina4 expects middleware to be referenced by a string name, not as an inline closure. The string is resolved to a function at runtime.

**Fix:** Define your middleware as a named function: `function myMiddleware($request, $response, $next) { ... }` and pass `"myMiddleware"` as a string.

### *2. Forgetting to Return $next()*

**Problem:** Your middleware runs but the route handler never executes. The response is empty or a 500 error.

**Cause:** You called `$next($request, $response)` but did not `return` the result.

**Fix:** Always `return $next($request, $response)`. Without the `return`, the middleware discards the response from the handler and returns `null`.

### *3. Middleware Order Matters*

**Problem:** Your logging middleware does not see the request ID, even though `addRequestId` is in the middleware list.

**Cause:** `logRequest` runs before `addRequestId`. Middleware executes in the order listed.

**Fix:** Put `addRequestId` before `logRequest` in the array: `["addRequestId", "logRequest"]`. Think of the order as "what needs to happen first."

### *4. CORS Preflight Returns 404*

**Problem:** The browser's preflight `OPTIONS` request gets a 404, but `GET` and `POST` work fine when tested with curl.

**Cause:** You did not apply `CorsMiddleware` to the route, so the `OPTIONS` method is not handled.

**Fix:** Apply `CorsMiddleware` to the group. It automatically handles `OPTIONS` requests and returns the correct CORS headers. Alternatively, set `CORS_ORIGINS` in `.env` for global CORS handling.

### *5. Rate Limiter Counts Preflight Requests*

**Problem:** Your frontend hits the rate limit faster than expected because every `POST` request actually counts as two requests (the `OPTIONS` preflight plus the `POST`).

**Cause:** The rate limiter counts all requests, including `OPTIONS`.

**Fix:** Put `CorsMiddleware` before `RateLimiter` in the middleware chain: `["CorsMiddleware", "RateLimiter"]`. The CORS middleware handles `OPTIONS` and returns immediately, so the rate limiter only sees actual requests. If CORS is configured globally via `.env`, the preflight is handled before middleware runs.

### *6. Middleware File Not Auto-Loaded*

**Problem:** You defined middleware in a file but get "function not found" when referencing it.

**Cause:** The file is not in `src/routes/`. Tina4 auto-loads all `.php` files in `src/routes/`, but middleware defined outside that directory is not discovered.

**Fix:** Put your middleware functions in a file inside `src/routes/`, such as `src/routes/middleware.php`. The filename does not matter -- all `.php` files in the directory are loaded.

### *7. Short-Circuiting Skips Cleanup Middleware*

**Problem:** Your timing middleware logs the start time but never logs the completion time for blocked requests.

**Cause:** When an inner middleware short-circuits, the outer middleware's code after `$next()` still runs. But if the short-circuiting middleware is listed before the timing middleware, the timing middleware never executes at all.

**Fix:** Put cleanup-dependent middleware (timing, logging) at the outermost layer. They wrap the entire chain, so they always see both the start and the end -- even when an inner middleware short-circuits. The code after `$next()` in outer middleware always runs, regardless of whether the chain was short-circuited.

# Chapter 9: Sessions & Cookies

## 1. State in a Stateless World

Your e-commerce site needs a shopping cart that persists across page loads, remembers the user's language preference, and flashes success or error messages after form submissions. HTTP is stateless -- every request is independent, with no memory of what came before. Sessions and cookies solve this by giving the server a way to remember who is making requests and what they have been doing.

In Chapter 7 you saw a brief introduction to sessions for authentication. This chapter goes deeper: session backends, flash messages, cookies, remember-me tokens, and security configuration.

## 2. How Sessions Work

When a user visits your site for the first time, Tina4 generates a unique session ID (a long random string), stores it in a cookie on the user's browser, and creates a server-side storage entry keyed by that ID. On every subsequent request, the browser sends the cookie, Tina4 looks up the session data, and makes it available via `$request->session`.

The flow looks like this:

1. Browser sends first request (no session cookie)

2. Tina4 generates session ID: `abc123def456`

3. Tina4 sets cookie: `tina4_session=abc123def456`

4. Tina4 creates empty session storage for `abc123def456`

5. Browser sends second request with cookie `tina4_session=abc123def456`

6. Tina4 loads session data for `abc123def456`

7. Your route handler reads and writes `$request->session`

8. At the end of the request, Tina4 saves the updated session data

The session data is stored server-side. The browser only has the session ID -- it never sees the actual data.

## 3. File Sessions (Default)

Out of the box, Tina4 stores sessions in files. No configuration needed.

```
<?php
use Tina4\Route;

Route::get("/visit-counter", function ($request, $response) {
    $count = ($request->session["visit_count"] ?? 0) + 1;
    $request->session["visit_count"] = $count;

    return $response->json([
        "visit_count" => $count,
        "message" => "You have visited this page " . $count . " time" . ($count === 1 ? "" : "s")
    ]);
```

```
});
```

```
curl http://localhost:7145/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":1,"message":"You have visited this page 1 time"}
```

```
curl http://localhost:7145/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":2,"message":"You have visited this page 2 times"}
```

```
curl http://localhost:7145/visit-counter -c cookies.txt -b cookies.txt
```

```
{"visit_count":3,"message":"You have visited this page 3 times"}
```

The `-c cookies.txt` flag tells curl to save cookies to a file, and `-b cookies.txt` tells it to send them back. This simulates how a browser works.

Session files are stored in your system's temporary directory by default. You can change this:

```
TINA4_SESSION_PATH=/path/to/session/files
```

File sessions work perfectly for single-server deployments. They are the simplest option and require no additional software.

## 4. Redis Sessions

For production deployments with multiple servers (behind a load balancer), you need a shared session store. Redis is the most common choice.

```
TINA4_SESSION_HANDLER=redis
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
TINA4_SESSION_PASSWORD=your-redis-password
```

That is the only change. Your code stays exactly the same. `$request->session` works identically whether sessions are stored in files, Redis, MongoDB, or Valkey. The storage backend is invisible to your route handlers.

### *Why Redis?*

• Sessions are shared across all server instances

• Sub-millisecond reads and writes

• Built-in key expiry (session cleanup happens automatically)

• No disk I/O

### *Redis with a Prefix*

If you share a Redis instance with other applications, add a prefix to avoid key collisions:

```
TINA4_SESSION_HANDLER=redis
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
TINA4_SESSION_PREFIX=myapp:sess:
```

## 5. MongoDB Sessions

If your stack already uses MongoDB, you can store sessions there:

```
TINA4_SESSION_HANDLER=mongodb
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=27017
TINA4_SESSION_DATABASE=myapp
TINA4_SESSION_COLLECTION=sessions
```

MongoDB sessions support TTL indexes, so expired sessions are cleaned up automatically by MongoDB itself.

## 6. Valkey Sessions

Valkey is the open-source fork of Redis. If you use Valkey, the configuration is almost identical:

```
TINA4_SESSION_HANDLER=valkey
TINA4_SESSION_HOST=localhost
TINA4_SESSION_PORT=6379
```

Valkey is wire-compatible with Redis, so the same client library works for both.

## 7. Reading and Writing Session Data

Session data is a simple key-value store. You read and write it through `$request->session`:

```
<?php
use Tina4\Route;

// Write to session
Route::post("/api/preferences", function ($request, $response) {
    $body = $request->body;

    $request->session["language"] = $body["language"] ?? "en";
    $request->session["theme"] = $body["theme"] ?? "light";
    $request->session["items_per_page"] = (int) ($body["items_per_page"] ?? 20);

    return $response->json([
        "message" => "Preferences saved",
        "preferences" => [
            "language" => $request->session["language"],
            "theme" => $request->session["theme"],
            "items_per_page" => $request->session["items_per_page"]
        ]
    ]);
});

// Read from session
Route::get("/api/preferences", function ($request, $response) {
    return $response->json([
        "language" => $request->session["language"] ?? "en",
// ...
```

```
curl -X POST http://localhost:7145/api/preferences \
  -H "Content-Type: application/json" \
  -d '{"language": "es", "theme": "dark", "items_per_page": 50}' \
  -c cookies.txt -b cookies.txt
```

```
{
  "message": "Preferences saved",
```

```
  "preferences": {
    "language": "es",
    "theme": "dark",
    "items_per_page": 50
  }
}
```

```
curl http://localhost:7145/api/preferences -b cookies.txt
```

```
{"language":"es","theme":"dark","items_per_page":50}
```

### *Storing Complex Data*

Sessions can hold arrays and nested structures:

```
Route::post("/api/cart/add", function ($request, $response) {
    $body = $request->body;

    if (!isset($request->session["cart"])) {
        $request->session["cart"] = [];
    }

    $cart = $request->session["cart"];

    $cart[] = [
        "product_id" => (int) $body["product_id"],
        "name" => $body["name"],
        "price" => (float) $body["price"],
        "quantity" => (int) ($body["quantity"] ?? 1),
        "added_at" => date("c")
    ];

    $request->session["cart"] = $cart;

    $total = array_sum(array_map(
        fn($item) => $item["price"] * $item["quantity"],
        $cart
    ));

    return $response->json([
// ...
```

## 8. Flash Messages

Flash messages are session data that exists for exactly one request. You set a flash message before redirecting, and it is available on the very next request. After that next request reads it, it is gone.

This is the standard pattern for form submissions: submit the form, redirect to a success page, show a "Profile updated" message that disappears on refresh.

### *Setting a Flash Message*

```
<?php
use Tina4\Route;

Route::post("/profile/update", function ($request, $response) {
    $body = $request->body;

    // Update the profile (database logic here)

    // Set a flash message
```

```
    $request->session["_flash"] = [
        "type" => "success",
        "message" => "Profile updated successfully"
    ];

    return $response->redirect("/profile");
});
```

### Reading and Clearing Flash Messages

```
Route::get("/profile", function ($request, $response) {
    // Read the flash message
    $flash = $request->session["_flash"] ?? null;

    // Clear it immediately so it does not show again
    unset($request->session["_flash"]);

    return $response->render("profile.html", [
        "user" => ["name" => "Alice", "email" => "alice@example.com"],
        "flash" => $flash
    ]);
});
```

### Using Flash Messages in Templates

```
{% extends "base.html" %}

{% block content %}
    {% if flash %}
        <div class="alert alert-{{ flash.type }}">
            {{ flash.message }}
        </div>
    {% endif %}

    <h1>Profile</h1>
    <p>Name: {{ user.name }}</p>
    <p>Email: {{ user.email }}</p>
{% endblock %}
```

The alert div appears on the first load after the form submission. If the user refreshes, the flash message is gone because it was cleared when it was first read.

### Multiple Flash Messages

You can flash multiple messages by using an array:

```
$request->session["_flash"] = [
    ["type" => "success", "message" => "Email updated"],
    ["type" => "warning", "message" => "Please verify your new email address"]
];
```

```
{% if flash %}
    {% for msg in flash %}
        <div class="alert alert-{{ msg.type }}">
            {{ msg.message }}
        </div>
    {% endfor %}
{% endif %}
```

## 9. Setting and Reading Cookies

Cookies are small pieces of data stored in the browser. Unlike sessions, the data is stored client-side. Use cookies for non-sensitive preferences that should persist even after the session expires.

### Setting a Cookie

```
<?php
use Tina4\Route;

Route::post("/api/set-language", function ($request, $response) {
    $language = $request->body["language"] ?? "en";

    $response->setCookie("language", $language, [
        "expires" => time() + (365 * 24 * 60 * 60),  // 1 year
        "path" => "/",
        "httpOnly" => false,  // Allow JavaScript to read it
        "secure" => false,    // Set to true in production (HTTPS only)
        "sameSite" => "Lax"
    ]);

    return $response->json(["message" => "Language set to " . $language]);
});
```

### Reading a Cookie

```
Route::get("/api/get-language", function ($request, $response) {
    $language = $request->cookies["language"] ?? "en";

    return $response->json(["language" => $language]);
});
```

```
curl -X POST http://localhost:7145/api/set-language \
  -H "Content-Type: application/json" \
  -d '{"language": "fr"}' \
  -c cookies.txt

curl http://localhost:7145/api/get-language -b cookies.txt
```

```
{"language":"fr"}
```

### Deleting a Cookie

To delete a cookie, set it with an expiry date in the past:

```
Route::post("/api/clear-language", function ($request, $response) {
    $response->setCookie("language", "", [
        "expires" => time() - 3600,
        "path" => "/"
    ]);

    return $response->json(["message" => "Language cookie cleared"]);
});
```

### When to Use Cookies vs Sessions

| Use Cookies For | Use Sessions For |

|----------|----------|

| Language preference | Shopping cart contents |

| Theme preference (light/dark) | User authentication state |

| "Remember this device" flag | Flash messages |

| Analytics tracking consent | Form wizard progress |

| Non-sensitive, long-lived data | Sensitive, short-lived data |

## 10. Remember Me Functionality

The "remember me" pattern uses a long-lived cookie to re-authenticate users after their session expires.

```php
<?php
use Tina4\Route;
use Tina4\Auth;
use Tina4\Database;

/**
 * @noauth
 */
Route::post("/login", function ($request, $response) {
    $body = $request->body;
    $db = Database::getConnection();

    // Validate credentials
    $user = $db->fetchOne(
        "SELECT id, name, email, password_hash FROM users WHERE email = :email",
        ["email" => $body["email"]]
    );

    if ($user === null || !Auth::checkPassword($body["password"], $user["password_hash"])) {
        return $response->json(["error" => "Invalid email or password"], 401);
    }

    // Set session
    $request->session["user_id"] = $user["id"];
    $request->session["user_name"] = $user["name"];
// ...
```

The "remember me" middleware that checks the cookie:

```php
<?php
use Tina4\Database;

function rememberMeMiddleware($request, $response, $next) {
    // Already logged in via session? Continue.
    if (!empty($request->session["user_id"])) {
        return $next($request, $response);
    }

    // Check for remember_me cookie
    $rememberToken = $request->cookies["remember_me"] ?? "";

    if (empty($rememberToken)) {
        return $next($request, $response);
    }

    $db = Database::getConnection();
    $hashedToken = hash("sha256", $rememberToken);

    $user = $db->fetchOne(
        "SELECT id, name, email FROM users WHERE remember_token = :token",
        ["token" => $hashedToken]
    );

    if ($user !== null) {
```

```
// ...
```

The flow:

1. User logs in with "remember me" checked

2. Server stores a hashed token in the database and sets an unhashed token in a cookie

3. Session expires (user closes browser or session lifetime ends)

4. User returns -- session is empty, but the `remember_me` cookie is still there

5. Middleware finds the cookie, looks up the hashed token in the database, restores the session

6. User is logged in again without entering credentials

We store the hash in the database and the raw token in the cookie. If the database is breached, the attacker gets hashes, not tokens. They cannot forge the cookie.

## 11. Session Security

### *Configuration Options*

```
TINA4_SESSION_LIFETIME=3600       # Session expires after 1 hour of inactivity
TINA4_SESSION_NAME=tina4_session  # Cookie name for the session ID
TINA4_SESSION_SECURE=true         # Only send cookie over HTTPS
TINA4_SESSION_HTTPONLY=true       # JavaScript cannot access the cookie
TINA4_SESSION_SAMESITE=Lax        # CSRF protection
```

### *httpOnly*

When `TINA4_SESSION_HTTPONLY=true` (the default), the session cookie cannot be read by JavaScript. This prevents XSS attacks from stealing the session ID. There is almost never a reason to set this to `false`.

### *secure*

When `TINA4_SESSION_SECURE=true`, the session cookie is only sent over HTTPS connections. Set this to `true` in production. During development with `http://localhost`, set it to `false` or your browser will not send the cookie.

### *sameSite*

The `sameSite` attribute controls whether the browser sends the cookie with cross-site requests:

| Value | Behavior |

|-------|----------|

| `Strict` | Cookie is never sent with cross-site requests. Safest, but breaks some legitimate flows (e.g., clicking a link from an email). |

| `Lax` | Cookie is sent with top-level navigations (clicking links) but not with cross-site API calls. Good default. |

| `None` | Cookie is always sent. Requires `secure=true`. Only use if you need cross-site cookie access. |

### *Session Regeneration*

After a user logs in, regenerate the session ID to prevent session fixation attacks:

```
Route::post("/login", function ($request, $response) {
    // Validate credentials...

    // Regenerate session ID (keeps the data, changes the ID)
    $request->sessionRegenerate();

    $request->session["user_id"] = $user["id"];

    return $response->redirect("/dashboard");
});
```

Session fixation is an attack where an attacker sets a known session ID on the victim's browser before the victim logs in. After login, the attacker uses the same session ID to access the victim's account. Regenerating the session ID after login invalidates the attacker's known ID.

## 12. Exercise: Build a Shopping Cart with Session Storage

Build a shopping cart that stores items in the session. No database needed -- the cart lives entirely in session data.

### *Requirements*

Create these endpoints:

| Method | Path | Description |

|--------|------|----------|

| `POST` | `/api/cart/add` | Add an item to the cart. Body: `{"product_id": 1, "name": "Widget", "price": 9.99, "quantity": 2}` |

| `GET` | `/api/cart` | View the cart. Show items, quantities, item subtotals, and cart total. |

| `PUT` | `/api/cart/{product_id:int}` | Update quantity. Body: `{"quantity": 3}`. Remove item if quantity is 0. |

| `DELETE` | `/api/cart/{product_id:int}` | Remove an item from the cart. |

| `DELETE` | `/api/cart` | Clear the entire cart. |

### *Business Rules*

1. If adding a product that already exists in the cart, increment the quantity instead of adding a duplicate

2. Cart total should be calculated dynamically

3. Return the full cart state after every operation

### *Test with:*

```
# Add first item
curl -X POST http://localhost:7145/api/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 1}' \
  -c cookies.txt -b cookies.txt

# Add second item
curl -X POST http://localhost:7145/api/cart/add \
  -H "Content-Type: application/json" \
```

```
  -d '{"product_id": 2, "name": "USB-C Hub", "price": 49.99, "quantity": 2}' \
  -c cookies.txt -b cookies.txt

# Add more of item 1 (should increment, not duplicate)
curl -X POST http://localhost:7145/api/cart/add \
  -H "Content-Type: application/json" \
  -d '{"product_id": 1, "name": "Wireless Keyboard", "price": 79.99, "quantity": 1}' \
  -c cookies.txt -b cookies.txt

# View cart
curl http://localhost:7145/api/cart -b cookies.txt

# Update quantity
curl -X PUT http://localhost:7145/api/cart/2 \
  -H "Content-Type: application/json" \
  -d '{"quantity": 5}' \
// ...
```

## 13. Solution

Create `src/routes/cart.php`:

```
<?php
use Tina4\Route;

function getCart($session) {
    return $session["cart"] ?? [];
}

function cartResponse($cart) {
    $total = 0;
    $itemCount = 0;
    $items = [];

    foreach ($cart as $item) {
        $subtotal = $item["price"] * $item["quantity"];
        $total += $subtotal;
        $itemCount += $item["quantity"];
        $items[] = array_merge($item, ["subtotal" => $subtotal]);
    }

    return [
        "items" => $items,
        "item_count" => $itemCount,
        "unique_items" => count($cart),
        "total" => round($total, 2)
    ];
// ...
```

**Expected output after adding two items and then adding more of item 1:**

```
curl http://localhost:7145/api/cart -b cookies.txt

{
  "items": [
    {
      "product_id": 1,
      "name": "Wireless Keyboard",
      "price": 79.99,
      "quantity": 2,
      "subtotal": 159.98
    },
```

```
    {
      "product_id": 2,
      "name": "USB-C Hub",
      "price": 49.99,
      "quantity": 2,
      "subtotal": 99.98
    }
  ],
  "item_count": 4,
  "unique_items": 2,
  "total": 259.96
}
```

Notice that the Wireless Keyboard has `quantity: 2` (1 + 1 from the second add), not two separate entries.

**Expected output after updating quantity of item 2 to 5:**

```
{
  "items": [
    {
      "product_id": 1,
      "name": "Wireless Keyboard",
      "price": 79.99,
      "quantity": 2,
      "subtotal": 159.98
    },
    {
      "product_id": 2,
      "name": "USB-C Hub",
      "price": 49.99,
      "quantity": 5,
      "subtotal": 249.95
    }
  ],
  "item_count": 7,
  "unique_items": 2,
  "total": 409.93
}
```

**Expected output after clearing the cart:**

```
{
  "items": [],
  "item_count": 0,
  "unique_items": 0,
  "total": 0
}
```

# 14. Gotchas

### *1. Sessions Do Not Work with curl Without Cookie Flags*

**Problem:** Each curl request sees an empty session, as if it is a new user.

**Cause:** curl does not automatically save or send cookies. Without the `-c` and `-b` flags, every request starts a new session.

**Fix:** Use `-c cookies.txt -b cookies.txt` with curl. The `-c` flag saves cookies to a file after the response, and `-b` sends cookies from that file with the request. Browsers handle this automatically.

### *2. Session Data Disappears After Server Restart*

**Problem:** All session data is gone after restarting the dev server.

**Cause:** If you are using file sessions and the session files are stored in the system temp directory, a server restart might clear them (depending on your OS and PHP configuration). Or, if the session ID cookie has expired, the browser sends a new request without the cookie.

**Fix:** Set `TINA4_SESSION_PATH` to a persistent directory outside the temp folder. For production, use Redis or Valkey which persist data independently of the web server.

### *3. Session Cookie Not Sent Over HTTP in Production*

**Problem:** Sessions work locally but not in production. The browser does not send the session cookie.

**Cause:** `TINA4_SESSION_SECURE=true` means the cookie is only sent over HTTPS. If your production server is behind a reverse proxy that terminates SSL, the app sees HTTP and the cookie is not set.

**Fix:** Ensure your reverse proxy sets the `X-Forwarded-Proto: https` header, and that Tina4 trusts proxy headers. Or make sure `TINA4_SESSION_SECURE` is only `true` when the connection is genuinely HTTPS end-to-end.

### *4. Flash Messages Show Twice*

**Problem:** The flash message appears, then appears again on the next page load.

**Cause:** You read the flash message but did not clear it from the session. It persists until explicitly removed.

**Fix:** Always clear the flash message immediately after reading it: `unset($request->session["_flash"])`. The flash pattern requires manual cleanup -- read it, use it, delete it, all in the same request.

### *5. Large Session Data Causes Slow Requests*

**Problem:** Pages load slowly and performance degrades over time.

**Cause:** You are storing large amounts of data in the session (entire database result sets, uploaded file contents, large arrays). Session data is serialized and deserialized on every request.

**Fix:** Keep session data small. Store IDs and references, not entire objects. If a cart has 50 items, store `[{"product_id": 1, "quantity": 2}, ...]` rather than full product objects with descriptions, images, and metadata.

### *6. Remember Me Token Not Invalidated on Password Change*

**Problem:** After a user changes their password, their "remember me" cookies on other devices still work.

**Cause:** The remember-me token in the database was not cleared when the password changed.

**Fix:** Clear the `remember_token` column whenever the password is updated: `$db->execute("UPDATE users SET remember_token = NULL WHERE id = :id", ["id" => $userId])`. This forces all other devices to log in again with the new password.

### *7. Session Fixation*

**Problem:** An attacker can hijack a user's session by setting a known session ID before the user logs in.

**Cause:** The session ID is not regenerated after login. The attacker knows the session ID (because they set it), and after the user logs in, the attacker's session ID now has an authenticated session.

**Fix:** Call `$request->sessionRegenerate()` after successful login. This creates a new session ID, copies the data, and invalidates the old ID. The attacker's known ID becomes useless.

# Chapter 10: API Documentation with Swagger

## 1. The 47-Endpoint Problem

Your team has 47 API endpoints and the frontend developer keeps asking "what does this endpoint accept?" You email a spreadsheet. It goes out of date. You write a wiki page. Nobody updates it. You add comments to the code. Nobody reads them.

Swagger (also called OpenAPI) solves this permanently. It generates interactive API documentation from annotations in your route files. The docs are always up to date because they come from the code itself. Your frontend developer can browse every endpoint, see the expected request and response formats, and even test endpoints directly from the browser.

Tina4 auto-generates a Swagger UI at `/swagger` from doc-block annotations on your routes. No build step. No extra tooling. Write the annotations, and the documentation appears.

## 2. What Swagger/OpenAPI Is

OpenAPI is a specification format for describing REST APIs. Swagger is the toolset that reads OpenAPI specs and generates documentation, client SDKs, and server stubs.

An OpenAPI spec describes:

• Every endpoint (path + HTTP method)

• What parameters each endpoint accepts (path, query, header, body)

• What each endpoint returns (response codes, response bodies)

• Data schemas (what a "User" or "Product" object looks like)

• Authentication requirements

• Grouping and tagging

Tina4 builds this spec automatically from doc-block comments in your PHP code. You never write JSON or YAML by hand.

## 3. Enabling Swagger

Swagger is available out of the box when `TINA4_DEBUG=true`. Navigate to:

```
http://localhost:7145/swagger
```

You should see the Swagger UI with any routes you have already defined. If you have not added any Swagger annotations yet, you will see the routes listed with default descriptions.

For production, you can explicitly enable or disable Swagger:

```
TINA4_SWAGGER=true
```

### *The Swagger JSON Endpoint*

The raw OpenAPI spec is available at:

```
http://localhost:7145/swagger/json
```

```
curl http://localhost:7145/swagger/json
```

```
{
  "openapi": "3.0.0",
  "info": {
    "title": "My Store API",
    "version": "1.0.0"
  },
  "paths": {
    "/api/products": {
      "get": {
        "summary": "List all products",
        "responses": {
          "200": {
            "description": "Successful response"
          }
        }
      }
    }
  }
}
```

This JSON can be imported into tools like Postman, Insomnia, or used to generate client SDKs.

## 4. Adding Descriptions to Routes

Add Swagger annotations as doc-block comments above your route definitions:

```
<?php
use Tina4\Route;

/**
 * List all products
 * @description Returns a paginated list of all products in the catalog
 * @tags Products
 */
Route::get("/api/products", function ($request, $response) {
    return $response->json(["products" => []]);
});
```

The first line of the doc-block becomes the `summary`. The `@description` tag provides a longer explanation. Visit `/swagger` and you will see the endpoint listed under the "Products" group with both the summary and the description.

### *Documenting Path Parameters*

```
/**
 * Get a product by ID
 * @description Returns a single product with full details including inventory status
 * @tags Products
 * @param int $id The unique product identifier
 */
Route::get("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    return $response->json([
        "id" => $id,
        "name" => "Wireless Keyboard",
        "price" => 79.99
    ]);
```

```
});
```

The `@param` annotation tells Swagger about the path parameter. It shows up in the docs as a required field with type information.

### *Documenting Query Parameters*

```
/**
 * Search products
 * @description Search the product catalog by name, category, or price range
 * @tags Products
 * @query string $q Search query (searches product name and description)
 * @query string $category Filter by category name
 * @query float $min_price Minimum price filter
 * @query float $max_price Maximum price filter
 * @query int $page Page number (default: 1)
 * @query int $limit Items per page (default: 20, max: 100)
 */
Route::get("/api/products/search", function ($request, $response) {
    $q = $request->query["q"] ?? "";
    $page = (int) ($request->query["page"] ?? 1);
    $limit = min((int) ($request->query["limit"] ?? 20), 100);

    return $response->json([
        "query" => $q,
        "page" => $page,
        "limit" => $limit,
        "results" => [],
        "total" => 0
    ]);
});
```

Each `@query` annotation adds a parameter to the Swagger docs with its type and description.

## 5. Documenting Request and Response Schemas

### *Request Body*

Use `@body` to document what the endpoint expects:

```
/**
 * Create a new product
 * @description Creates a product in the catalog. Requires admin authentication.
 * @tags Products
 * @body {"name": "string", "category": "string", "price": "float", "in_stock": "bool", "description": "string
 * @response 201 {"id": "int", "name": "string", "category": "string", "price": "float", "in_stock": "bool",
 * @response 400 {"error": "string"}
 */
Route::post("/api/products", function ($request, $response) {
    $body = $request->body;

    if (empty($body["name"])) {
        return $response->json(["error" => "Name is required"], 400);
    }

    return $response->json([
        "id" => 1,
        "name" => $body["name"],
        "category" => $body["category"] ?? "Uncategorized",
        "price" => (float) ($body["price"] ?? 0),
        "in_stock" => (bool) ($body["in_stock"] ?? true),
```

```
        "created_at" => date("c")
    ], 201);
});
```

The `@body` annotation describes the expected JSON structure. The `@response` annotation documents possible response codes and their payloads.

### *Multiple Response Codes*

```
/**
 * Update a product
 * @description Update an existing product by ID. Only provided fields are updated.
 * @tags Products
 * @param int $id Product ID
 * @body {"name": "string", "category": "string", "price": "float", "in_stock": "bool"}
 * @response 200 {"id": "int", "name": "string", "category": "string", "price": "float", "in_stock": "bool", "
 * @response 404 {"error": "string", "id": "int"}
 * @response 400 {"error": "string"}
 */
Route::put("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $body = $request->body;

    // Simulate not found
    if ($id > 100) {
        return $response->json(["error" => "Product not found", "id" => $id], 404);
    }

    return $response->json([
        "id" => $id,
        "name" => $body["name"] ?? "Widget",
        "category" => $body["category"] ?? "General",
        "price" => (float) ($body["price"] ?? 9.99),
        "in_stock" => (bool) ($body["in_stock"] ?? true),
// ...
```

Swagger UI shows each response code with its corresponding schema. Developers can see at a glance what a `200` looks like versus a `404`.

## 6. Tags for Grouping Endpoints

Tags group related endpoints in the Swagger UI. Without tags, all endpoints appear in a flat list. With tags, they are organized into collapsible sections.

```
/**
 * List all users
 * @tags Users
 */
Route::get("/api/users", function ($request, $response) {
    return $response->json(["users" => []]);
});

/**
 * Get user by ID
 * @tags Users
 */
Route::get("/api/users/{id:int}", function ($request, $response) {
    return $response->json(["id" => $request->params["id"], "name" => "Alice"]);
});

/**
```

```
 * List all orders
 * @tags Orders
 */
Route::get("/api/orders", function ($request, $response) {
    return $response->json(["orders" => []]);
});

/**
// ...
```

In the Swagger UI, you will see three sections: "Users", "Orders", and "Products". Each section expands to show its endpoints. This makes navigating a large API manageable.

### *Multiple Tags*

An endpoint can belong to multiple groups:

```
/**
 * Get user's orders
 * @tags Users, Orders
 */
Route::get("/api/users/{id:int}/orders", function ($request, $response) {
    return $response->json(["orders" => []]);
});
```

This endpoint appears in both the "Users" and "Orders" sections.

## 7. Example Values

Add example values to make the docs more useful. Developers can see realistic data instead of just type names:

```
/**
 * Create a new product
 * @description Creates a product in the catalog
 * @tags Products
 * @example request {"name": "Ergonomic Keyboard", "category": "Electronics", "price": 89.99, "in_stock": true
 * @example response {"id": 42, "name": "Ergonomic Keyboard", "category": "Electronics", "price": 89.99, "in_s
 */
Route::post("/api/products", function ($request, $response) {
    $body = $request->body;

    return $response->json([
        "id" => 42,
        "name" => $body["name"],
        "category" => $body["category"] ?? "Uncategorized",
        "price" => (float) ($body["price"] ?? 0),
        "in_stock" => (bool) ($body["in_stock"] ?? true),
        "created_at" => date("c")
    ], 201);
});
```

The `@example` annotations populate the "Example Value" section in the Swagger UI. When a developer clicks "Try it out", the example request is pre-filled in the input field, so they can test immediately.

## 8. Try-It-Out from the Swagger UI

The Swagger UI includes a "Try it out" button on every endpoint. Clicking it:

1. Expands the endpoint with editable input fields
2. Pre-fills example values (if provided)
3. Lets you edit the parameters, headers, and request body
4. Sends the actual HTTP request to your running server
5. Shows the response status, headers, and body

This is a live testing tool built into your documentation. No need for Postman or curl -- frontend developers can test your API directly from the docs page.

### *Authentication in Try-It-Out*

If your endpoints require authentication, click the "Authorize" button at the top of the Swagger UI. Enter your JWT token or API key, and all subsequent "Try it out" requests will include the authentication header.

Tina4 auto-detects authentication requirements from your route annotations (`@secured`, `@noauth`) and shows the lock icon on endpoints that require auth.

## 9. Customizing the Swagger Info Block

Configure the top-level API information in `.env`:

```
TINA4_SWAGGER_TITLE=My Store API
TINA4_SWAGGER_DESCRIPTION=API for managing products, orders, and users
TINA4_SWAGGER_VERSION=1.0.0
TINA4_SWAGGER_CONTACT_EMAIL=api@mystore.com
TINA4_SWAGGER_LICENSE=MIT
```

This information appears in the header of the Swagger UI and in the OpenAPI spec:

```
{
  "openapi": "3.0.0",
  "info": {
    "title": "My Store API",
    "description": "API for managing products, orders, and users",
    "version": "1.0.0",
    "contact": {
      "email": "api@mystore.com"
    },
    "license": {
      "name": "MIT"
    }
  }
}
```

## 10. Generating Client SDKs from the Spec

The OpenAPI spec at `/swagger/json` can be used with code generation tools to create client libraries in any language.

### *Using OpenAPI Generator*

```
# Install the OpenAPI Generator CLI
npm install -g @openapitools/openapi-generator-cli
```

```
# Generate a TypeScript client
openapi-generator-cli generate \
  -i http://localhost:7145/swagger/json \
  -g typescript-fetch \
  -o ./frontend/api-client

# Generate a Python client
openapi-generator-cli generate \
  -i http://localhost:7145/swagger/json \
  -g python \
  -o ./python-client
```

This generates typed client code with methods for every endpoint. Your frontend developer gets:

```
// Auto-generated TypeScript client
const api = new ProductsApi();

// Fully typed -- IDE autocompletion works
const product = await api.getProductById({ id: 42 });
console.log(product.name);  // TypeScript knows this is a string

const newProduct = await api.createProduct({
    name: "Widget",
    category: "General",
    price: 9.99,
    inStock: true
});
```

Every time you update your Swagger annotations and regenerate, the client stays in sync with the server.

## 11. A Complete Documented API

Here is a full example showing all the annotation features together:

```
<?php
use Tina4\Route;

/**
 * List all users
 * @description Returns a paginated list of users. Supports filtering by role and searching by name.
 * @tags Users
 * @query int $page Page number (default: 1)
 * @query int $limit Items per page (default: 20)
 * @query string $role Filter by role (admin, user, moderator)
 * @query string $search Search by name or email
 * @response 200 {"users": [{"id": "int", "name": "string", "email": "string", "role": "string"}], "total": "i
 * @example response {"users": [{"id": 1, "name": "Alice", "email": "alice@example.com", "role": "admin"}, {"i
 */
Route::get("/api/users", function ($request, $response) {
    $page = (int) ($request->query["page"] ?? 1);
    $limit = (int) ($request->query["limit"] ?? 20);

    return $response->json([
        "users" => [
            ["id" => 1, "name" => "Alice", "email" => "alice@example.com", "role" => "admin"],
            ["id" => 2, "name" => "Bob", "email" => "bob@example.com", "role" => "user"]
        ],
        "total" => 42,
        "page" => $page,
// ...
```

Visit `/swagger` and you will see all four endpoints grouped under "Users", each with full parameter documentation, example values, and multiple response codes.

## 12. Exercise: Document a Complete User API

Take the User API from the example above and extend it with the following endpoints. Write full Swagger annotations for each one.

### *Requirements*

Document these additional endpoints (you can use hardcoded data in the handlers):

| Method | Path | Description |

|--------|------|-------------|

| `PUT` | `/api/users/{id}` | Update a user. Body: name, email, role. Response: updated user. |

| `GET` | `/api/users/{id}/orders` | List a user's orders. Query: status filter, pagination. |

| `POST` | `/api/users/{id}/avatar` | Upload user avatar. Body: avatar_url string. |

Each endpoint should have:

1. A summary (first line of doc-block)

2. A `@description`

3. A `@tags` annotation

4. `@param` for path parameters

5. `@query` for query parameters (where applicable)

6. `@body` for request body (where applicable)

7. `@response` for each possible response code

8. `@example` for request and response (where applicable)

### *Test by visiting:*

```
http://localhost:7145/swagger
```

Verify that all endpoints appear with correct documentation, examples, and response codes.

## 13. Solution

Create `src/routes/user-api-documented.php`:

```
<?php
use Tina4\Route;

/**
 * Update a user
 * @description Updates an existing user's profile information. Only provided fields are updated. Email must
 * @tags Users
 * @param int $id User ID
 * @body {"name": "string", "email": "string", "role": "string"}
```

```
 * @response 200 {"id": "int", "name": "string", "email": "string", "role": "string", "updated_at": "string"}
 * @response 404 {"error": "string"}
 * @response 409 {"error": "string"}
 * @example request {"name": "Alice Smith", "email": "alice.smith@example.com", "role": "admin"}
 * @example response {"id": 1, "name": "Alice Smith", "email": "alice.smith@example.com", "role": "admin", "up
 */
Route::put("/api/users/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $body = $request->body;

    if ($id > 100) {
        return $response->json(["error" => "User not found"], 404);
    }

    return $response->json([
        "id" => $id,
// ...
```

Visit `http://localhost:7145/swagger` and verify:

• The "Users" section now has six endpoints (list, get, create, update, delete, and avatar)

• The "Orders" section shows the "List user orders" endpoint (because it has both `Users` and `Orders` tags)

• Each endpoint has its summary, description, parameters, request body schema, and response codes

• The "Try it out" button works for each endpoint

• Example values are pre-filled when you click "Try it out"

## 14. Gotchas

### *1. Annotations Must Be Directly Above the Route*

**Problem:** Your Swagger annotations do not appear in the docs.

**Cause:** There is a blank line or other code between the doc-block and the `Route::` call. Tina4 only reads doc-blocks that are immediately above the route definition.

**Fix:** Make sure the `*/` closing of the doc-block is on the line directly before `Route::get(...)` with no blank lines in between.

### *2. Missing @tags Makes Endpoints Hard to Find*

**Problem:** All endpoints appear in one giant flat list in the Swagger UI.

**Cause:** You did not add `@tags` to your routes. Without tags, Swagger groups everything under "default".

**Fix:** Add `@tags ResourceName` to every route doc-block. Group related endpoints under the same tag.

### *3. @body Must Be Valid JSON*

**Problem:** The Swagger UI shows the body schema as empty or broken.

**Cause:** The JSON in your `@body` annotation is malformed. A trailing comma, missing quotes, or unescaped characters will break the parser.

**Fix:** Validate your `@body` JSON. Every key and string value must be in double quotes. No trailing commas. Use `"string"`, `"int"`, `"float"`, `"bool"` for type placeholders.

### *4. Swagger Shows Routes You Did Not Annotate*

**Problem:** Unannotated routes appear in the Swagger UI with minimal documentation.

**Cause:** Tina4 includes all registered routes in the Swagger spec, not just annotated ones. Unannotated routes get a default summary based on the HTTP method and path.

**Fix:** This is by design -- it ensures nothing is hidden. Add annotations to improve the documentation quality. If you want to hide a route from Swagger, add `@hidden` to its doc-block.

### *5. Response Examples Do Not Match Actual Responses*

**Problem:** The example response in Swagger shows different fields than the actual API response.

**Cause:** The `@example response` was written once and never updated when the handler changed. Swagger annotations are comments -- they are not validated against the actual code.

**Fix:** Treat annotations as part of the code. When you change a handler's response format, update the annotations at the same time. Consider adding integration tests that compare actual responses to documented schemas.

### *6. Swagger UI Not Available in Production*

**Problem:** `/swagger` returns a 404 in production.

**Cause:** By default, Swagger is only available when `TINA4_DEBUG=true`. Production deployments with `TINA4_DEBUG=false` disable the Swagger UI.

**Fix:** If you want Swagger in production (for example, on a staging server), explicitly set `TINA4_SWAGGER=true` in your `.env`. Be aware that exposing your API documentation publicly may reveal implementation details.

### *7. SDK Generation Produces Incorrect Types*

**Problem:** The generated TypeScript client has `any` types instead of proper interfaces.

**Cause:** Your `@body` and `@response` annotations use generic strings instead of typed schemas. `"name": "string"` tells the generator the type is a literal string "string", not a TypeScript `string` type.

**Fix:** Use the correct OpenAPI type format in your annotations. For complex schemas, define them with proper type notation: `"name": "string"` (lowercase), `"price": "number"`, `"in_stock": "boolean"`, `"tags": ["string"]` (array of strings).

# Chapter 11: Queue System

## 1. Do Not Make the User Wait

Your app sends welcome emails on signup, generates PDF invoices, and resizes uploaded images. Each of these tasks takes between two and thirty seconds. If you do them inside the HTTP request, the user stares at a loading spinner while the server processes their email, renders their invoice, and resizes their photo. That is a terrible experience.

Queues solve this by moving slow work to a background process. The HTTP handler drops a job onto a queue and immediately responds to the user. A separate consumer process picks up the job and does the work at its own pace. The user sees "Welcome! Check your email." in under 100 milliseconds, and the email arrives 5 seconds later.

Tina4 has a built-in queue system that works out of the box with SQLite. No Redis, no RabbitMQ, no external services. Just add jobs and process them.

## 2. Why Queues Matter

Without queues:

```
User clicks "Sign Up"
  → Server validates input (10ms)
  → Server creates user in database (20ms)
  → Server sends welcome email (3000ms)
  → Server generates PDF welcome kit (2000ms)
  → Server resizes avatar (1500ms)
  → User sees response (6530ms later)
```

With queues:

```
User clicks "Sign Up"
  → Server validates input (10ms)
  → Server creates user in database (20ms)
  → Server queues: send welcome email (1ms)
  → Server queues: generate PDF (1ms)
  → Server queues: resize avatar (1ms)
  → User sees response (33ms later)

Meanwhile, in the background:
  → Consumer sends welcome email
  → Consumer generates PDF
  → Consumer resizes avatar
```

The user's experience goes from 6.5 seconds to 33 milliseconds. The work still gets done -- just not during the HTTP request.

Beyond speed, queues provide:

• **Retry logic**: If the email server is down, the job retries automatically

• **Rate limiting**: Process jobs at a controlled pace to avoid overwhelming external services

• **Fault isolation**: A failed PDF generation does not crash the signup request

• **Scaling**: Run more consumer processes to handle higher load

## 3. SQLite Queue (Default)

Tina4's queue system uses SQLite by default. No configuration needed. When you produce your first job, Tina4 creates a `data/queue.db` database automatically.

### *Producing a Job*

```
<?php
use Tina4\Route;
use Tina4\Queue;

Route::post("/api/register", function ($request, $response) {
    $body = $request->body;

    // Create the user (database logic)
    $userId = 42; // Simulated

    // Queue a welcome email
    Queue::produce("send-welcome-email", [
        "user_id" => $userId,
        "email" => $body["email"],
        "name" => $body["name"]
    ]);

    return $response->json([
        "message" => "Registration successful. Welcome email will arrive shortly.",
        "user_id" => $userId
    ], 201);
});
```

```
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful. Welcome email will arrive shortly.",
  "user_id": 42
}
```

The response returns immediately. The email job is sitting in the queue, waiting to be processed.

### *The Queue::produce() Method*

```
Queue::produce($queueName, $payload, $options = []);
```

• **$queueName**: A string that identifies which queue to put the job on. Consumers subscribe to specific queues by name.

• **$payload**: An associative array of data the consumer needs to process the job. Must be JSON-serializable.

• **$options**: Optional settings like delay, priority, and max retries.

### *Producing with Options*

```
// Delay the job by 60 seconds (useful for "send reminder in 1 minute")
Queue::produce("send-reminder", [
    "user_id" => 42,
    "message" => "Do not forget to verify your email!"
], ["delay" => 60]);

// Set max retries (default is 3)
Queue::produce("generate-invoice", [
```

```
    "order_id" => 101,
    "format" => "pdf"
], ["max_retries" => 5]);

// Set priority (lower number = higher priority)
Queue::produce("resize-image", [
    "image_path" => "/uploads/photo.jpg",
    "sizes" => [100, 300, 600]
], ["priority" => 1]);
```

## 4. Consuming Jobs

A consumer listens for jobs on a specific queue and processes them. Define consumers in `src/routes/` (or any auto-loaded file):

```
<?php
use Tina4\Queue;

Queue::consume("send-welcome-email", function ($job) {
    $email = $job->payload["email"];
    $name = $job->payload["name"];

    // Send the email (using Messenger, covered in Chapter 13)
    error_log("Sending welcome email to " . $email . " for " . $name);

    // Simulate email sending
    // In a real app, you would use Tina4\Messenger here

    // Return true to mark the job as completed
    return true;
});
```

### *Starting the Consumer*

Run the consumer process with the Tina4 CLI:

```
tina4 queue:work
```

```
Queue worker started
  Listening on: all queues
  Backend: sqlite:///data/queue.db
  Polling interval: 1s

[2026-03-22 14:30:01] Processing job #1 on "send-welcome-email"
[2026-03-22 14:30:01] Job #1 completed in 45ms
```

The worker polls the queue every second, picks up pending jobs, and calls your consumer callback.

### *Listening to Specific Queues*

```
# Process only email jobs
tina4 queue:work --queue send-welcome-email

# Process multiple specific queues
tina4 queue:work --queue send-welcome-email,generate-invoice
```

### *Running Multiple Workers*

For higher throughput, run multiple worker processes:

```
# Terminal 1
tina4 queue:work --queue send-welcome-email
```

```
# Terminal 2
tina4 queue:work --queue generate-invoice

# Terminal 3
tina4 queue:work --queue resize-image
```

Each worker handles one queue. Jobs on different queues are processed in parallel.

## 5. Job Lifecycle

Every job moves through a series of states:

```
pending → reserved → completed
                   → failed → pending (retry)
                            → dead (max retries exceeded)
```

### *Pending*

The job is in the queue, waiting to be picked up. It stays here until a worker claims it.

### *Reserved*

A worker has claimed the job and is processing it. Other workers will not pick it up. If the worker crashes during processing, the job returns to `pending` after a timeout.

### *Completed*

The consumer returned `true`. The job is done and removed from the active queue.

### *Failed*

The consumer returned `false` or threw an exception. The job moves to `failed` state and is scheduled for retry.

### *Dead Letter*

A job that has exceeded its maximum retry count moves to the dead letter queue. It will not be retried automatically. You can inspect and re-queue dead letter jobs manually.

### *Inspecting Job State*

```php
<?php
use Tina4\Queue;

// Get queue statistics
$stats = Queue::stats("send-welcome-email");
```

```
[
    "queue" => "send-welcome-email",
    "pending" => 12,
    "reserved" => 2,
    "completed" => 1453,
    "failed" => 3,
    "dead" => 1
]
```

You can also see these stats in the dev dashboard at `/tina4/console` under the "Queue Manager" section.

## 6. Retry Logic and Max Retries

When a job fails, Tina4 automatically retries it with exponential backoff:

• Retry 1: after 10 seconds

• Retry 2: after 30 seconds

• Retry 3: after 90 seconds

The default max retry count is 3. After the third failure, the job moves to the dead letter queue.

### *Handling Failures in Consumers*

```
<?php
use Tina4\Queue;

Queue::consume("send-welcome-email", function ($job) {
    $email = $job->payload["email"];

    try {
        // Attempt to send the email
        $success = sendEmail($email, "Welcome!", "Welcome to our platform.");

        if (!$success) {
            error_log("Failed to send email to " . $email . " (attempt " . $job->attempts . ")");
            return false; // Triggers retry
        }

        return true; // Job completed

    } catch (\Exception $e) {
        error_log("Exception sending email to " . $email . ": " . $e->getMessage());
        return false; // Triggers retry
    }
});
```

### *Custom Max Retries*

Set max retries when producing the job:

```
// Critical emails: retry up to 10 times
Queue::produce("send-password-reset", [
    "email" => "alice@example.com",
    "token" => "abc123"
], ["max_retries" => 10]);

// Non-critical: retry only once
Queue::produce("send-marketing-email", [
    "email" => "bob@example.com",
    "campaign" => "spring-sale"
], ["max_retries" => 1]);
```

### *Accessing Attempt Count*

The `$job` object tells you which attempt this is:

```
Queue::consume("generate-invoice", function ($job) {
    error_log("Processing invoice (attempt " . $job->attempts . " of " . $job->maxRetries . ")");

    if ($job->attempts >= 3) {
        // On the last attempt, try a fallback approach
        error_log("Using fallback invoice generator");
    }
```

```
    return true;
});
```

## 7. Dead Letter Queue

Jobs that exceed their max retries land in the dead letter queue. These jobs need human attention -- maybe the email server is permanently down, the PDF template is broken, or the input data is invalid.

### *Viewing Dead Letter Jobs*

```
tina4 queue:dead
```

```
Dead Letter Queue
-----------------
Job #42  Queue: send-welcome-email  Failed: 3 times  Last error: "Connection refused"
  Payload: {"user_id": 42, "email": "alice@example.com", "name": "Alice"}
  First attempt: 2026-03-22 14:30:00
  Last attempt:  2026-03-22 14:35:30

Job #67  Queue: generate-invoice  Failed: 3 times  Last error: "Template not found"
  Payload: {"order_id": 101, "format": "pdf"}
  First attempt: 2026-03-22 15:00:00
  Last attempt:  2026-03-22 15:05:00
```

### *Re-Queuing Dead Letter Jobs*

Once you have fixed the underlying issue (started the email server, fixed the template), re-queue the dead letter jobs:

```
# Re-queue a specific job
tina4 queue:retry 42

# Re-queue all dead letter jobs for a specific queue
tina4 queue:retry --queue send-welcome-email

# Re-queue all dead letter jobs
tina4 queue:retry --all
```

### *Clearing Dead Letter Jobs*

If a dead letter job is no longer relevant (the user signed up again, the order was cancelled), remove it:

```
# Remove a specific dead letter job
tina4 queue:clear 42

# Remove all dead letter jobs older than 7 days
tina4 queue:clear --older-than 7d
```

## 8. Switching to RabbitMQ

For production deployments that need higher throughput, message durability across server restarts, and distributed processing, switch to RabbitMQ. Change one line in `.env`:

```
TINA4_QUEUE_BACKEND=rabbitmq
TINA4_QUEUE_HOST=localhost
TINA4_QUEUE_PORT=5672
TINA4_QUEUE_USERNAME=guest
TINA4_QUEUE_PASSWORD=guest
TINA4_QUEUE_VHOST=/
```

Your code does not change. The same `Queue::produce()` and `Queue::consume()` calls work with RabbitMQ as the backend. Tina4 handles the protocol differences internally.

### *Why Switch?*

| Feature | SQLite Queue | RabbitMQ |

|---------|-------------|----------|

| Setup | Zero config | Requires RabbitMQ server |

| Throughput | Hundreds of jobs/sec | Tens of thousands of jobs/sec |

| Multi-server | Single server only | Multiple servers, distributed |

| Persistence | File-based | Durable queues, survives restarts |

| Monitoring | Dev dashboard | RabbitMQ Management UI |

| Best for | Development, small apps | Production, high-volume apps |

## 9. Switching to Kafka

For event streaming at scale (millions of events per second, event replay, multiple consumer groups), switch to Kafka:

```
TINA4_QUEUE_BACKEND=kafka
TINA4_QUEUE_HOST=localhost
TINA4_QUEUE_PORT=9092
TINA4_QUEUE_GROUP_ID=my-app-workers
```

Again, your code stays the same. The `Queue::produce()` and `Queue::consume()` API is identical across all backends.

### *When to Use Kafka*

• You need to process millions of events per second

• Multiple independent consumers need to read the same events

• You need event replay (re-process historical events)

• You are building event-driven microservices

For most applications, SQLite (development) and RabbitMQ (production) are the right choices. Kafka is for when you outgrow RabbitMQ.

## 10. Monitoring via Dev Dashboard

When `TINA4_DEBUG=true`, the dev dashboard at `/tina4/console` shows a "Queue Manager" section with:

• **Queue overview**: pending, reserved, completed, failed, and dead counts for each queue

• **Recent jobs**: the last 50 processed jobs with their status, duration, and payload

• **Failed jobs**: jobs that failed with their error messages and retry count

• **Dead letter queue**: jobs that exceeded max retries

• **Throughput graph**: jobs processed per minute over the last hour

This gives you visibility into your queue system without command-line tools.

## 11. Producing Multiple Jobs

Sometimes a single action triggers multiple background tasks:

```
<?php
use Tina4\Route;
use Tina4\Queue;

Route::post("/api/orders", function ($request, $response) {
    $body = $request->body;

    // Create the order (database logic)
    $orderId = 101;
    $userId = $body["user_id"];

    // Queue multiple background tasks
    Queue::produce("send-order-confirmation", [
        "order_id" => $orderId,
        "email" => $body["email"]
    ]);

    Queue::produce("generate-invoice", [
        "order_id" => $orderId,
        "format" => "pdf"
    ]);

    Queue::produce("update-inventory", [
        "items" => $body["items"]
    ]);
// ...
```

```
curl -X POST http://localhost:7145/api/orders \
  -H "Content-Type: application/json" \
  -d '{"user_id": 42, "email": "alice@example.com", "items": [{"product_id": 1, "quantity": 2}], "shipping_ad
```

```
{"message":"Order placed successfully","order_id":101}
```

Four jobs are queued in under 5 milliseconds. The user gets an instant response. The email, invoice, inventory update, and warehouse notification happen in the background.

## 12. Exercise: Build an Email Queue

Build a queue-based email system for user signup. When a user registers, queue a welcome email. Write a consumer that processes the email jobs.

### *Requirements*

1. Create a `POST /api/signup` endpoint that:

• Accepts `name`, `email`, and `password`

• Queues a job on the `welcome-emails` queue

• Returns immediately with a success message

2. Create a consumer for the `welcome-emails` queue that:

• Logs the email details (simulating sending)

• Includes the user's name in the log message

• Returns `true` on success

3. Create a `GET /api/queue/stats` endpoint that shows the queue statistics

### *Test with:*

```
# Register a user
curl -X POST http://localhost:7145/api/signup \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'

# Register another user
curl -X POST http://localhost:7145/api/signup \
  -H "Content-Type: application/json" \
  -d '{"name": "Bob", "email": "bob@example.com", "password": "anotherPass456"}'

# Check queue stats (should show 2 pending)
curl http://localhost:7145/api/queue/stats

# Start the worker in another terminal
tina4 queue:work --queue welcome-emails

# Check stats again (should show 2 completed)
curl http://localhost:7145/api/queue/stats
```

## 13. Solution

Create `src/routes/email-queue.php`:

```
<?php
use Tina4\Route;
use Tina4\Queue;

/**
 * @noauth
 */
Route::post("/api/signup", function ($request, $response) {
    $body = $request->body;

    // Validate input
    if (empty($body["name"]) || empty($body["email"]) || empty($body["password"])) {
        return $response->json(["error" => "Name, email, and password are required"], 400);
    }

    // Simulate user creation (in a real app, save to database)
    $userId = rand(1, 10000);

    // Queue the welcome email
    Queue::produce("welcome-emails", [
        "user_id" => $userId,
        "name" => $body["name"],
        "email" => $body["email"],
        "signed_up_at" => date("c")
    ]);
```

```
// ...
```

### Expected output -- signup:

```
curl -X POST http://localhost:7145/api/signup \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Signup successful! A welcome email will be sent shortly.",
  "user_id": 4829
}
```

(Status: `201 Created`)

### Expected output -- stats before processing:

```
curl http://localhost:7145/api/queue/stats
```

```
{
  "queue": "welcome-emails",
  "stats": {
    "queue": "welcome-emails",
    "pending": 2,
    "reserved": 0,
    "completed": 0,
    "failed": 0,
    "dead": 0
  }
}
```

### Expected worker output:

```
[2026-03-22 14:30:01] Processing job #1 on "welcome-emails"
=== Welcome Email ===
To: alice@example.com
Subject: Welcome to our platform, Alice!
Body: Hi Alice, your account (ID: 4829) has been created.
Signed up: 2026-03-22T14:30:00+00:00
=====================
[2026-03-22 14:30:01] Job #1 completed in 2ms

[2026-03-22 14:30:02] Processing job #2 on "welcome-emails"
=== Welcome Email ===
To: bob@example.com
Subject: Welcome to our platform, Bob!
Body: Hi Bob, your account (ID: 7213) has been created.
Signed up: 2026-03-22T14:30:00+00:00
=====================
[2026-03-22 14:30:02] Job #2 completed in 1ms
```

### Expected output -- stats after processing:

```
{
  "queue": "welcome-emails",
  "stats": {
    "queue": "welcome-emails",
    "pending": 0,
    "reserved": 0,
    "completed": 2,
    "failed": 0,
    "dead": 0
  }
}
```

## 14. Gotchas

### *1. Worker Must Be Running Separately*

**Problem:** You produce jobs but nothing happens. The queue fills up but jobs are never processed.

**Cause:** The queue worker is not running. Producing a job only adds it to the queue -- you need a separate process to consume it.

**Fix:** Run `tina4 queue:work` in a separate terminal. In production, use a process manager like `supervisord` or `systemd` to keep the worker running.

### *2. Consumer Not Registered*

**Problem:** The worker starts but reports "No consumer found for queue: my-queue".

**Cause:** The `Queue::consume()` call is in a file that the worker did not load. The worker must discover the consumer registration.

**Fix:** Make sure your consumer is registered in a file inside `src/routes/` so it is auto-loaded. The `tina4 queue:work` command loads the same files as the web server.

### *3. Job Payload Is Not Serializable*

**Problem:** `Queue::produce()` throws an error about serialization.

**Cause:** You passed an object, a database connection, a file handle, or another non-serializable value in the payload.

**Fix:** Only pass simple data types in the payload: strings, numbers, booleans, and arrays of these types. If you need to reference a database record, pass the ID, not the record itself. The consumer can look up the record by ID.

### *4. Jobs Process in Wrong Order*

**Problem:** Jobs are processed in a different order than they were produced.

**Cause:** With multiple workers, jobs are distributed across workers and processed in parallel. There is no guaranteed ordering.

**Fix:** If order matters, use a single worker for that queue. Or include a sequence number in the payload and handle ordering in the consumer logic. For most use cases (sending emails, generating PDFs), order does not matter.

### *5. SQLite Queue Lock Contention*

**Problem:** Multiple workers on the same SQLite queue cause "database is locked" errors.

**Cause:** SQLite supports only one writer at a time. Multiple workers competing for the same SQLite database create contention.

**Fix:** For multiple workers, switch to RabbitMQ or Kafka. SQLite queues are designed for single-worker setups and development environments.

### *6. Consumer Returns Nothing*

**Problem:** Jobs are processed but immediately marked as failed and retried.

**Cause:** Your consumer function does not return `true`. Without an explicit `return true`, PHP returns `null`, which Tina4 interprets as failure.

**Fix:** Always `return true` when the job succeeds and `return false` when it fails. Do not forget the `return` statement.

### *7. Dead Letter Jobs Accumulate*

**Problem:** The dead letter queue grows over time and nobody notices.

**Cause:** No monitoring or alerting is set up for dead letter jobs.

**Fix:** Check `Queue::stats()` periodically or set up an alert when the dead count exceeds a threshold. In production, integrate with your monitoring system (Prometheus, Datadog, etc.) to track dead letter counts. Review dead letter jobs regularly -- they often reveal bugs in your consumer logic or issues with external services.

# Chapter 12: Real-time with WebSocket

## 1. The Refresh Button Problem

Your project management app needs live updates. When someone moves a card from "In Progress" to "Done", everyone else on the team should see it instantly -- no page refresh, no polling, no waiting. Traditional HTTP is request-response: the client asks, the server answers. The server cannot push data to the client on its own.

WebSocket solves this by establishing a persistent, bi-directional connection between the browser and the server. Once connected, either side can send messages at any time. The connection stays open until explicitly closed.

Tina4 treats WebSocket like routing. You define a WebSocket handler the same way you define an HTTP route. The path determines which handler processes the connection.

## 2. What WebSocket Is

HTTP works like this:

```
Client: "Give me /api/products"
Server: "Here are the products" (connection closes)
Client: "Give me /api/products" (new connection)
Server: "Here are the products" (connection closes)
```

WebSocket works like this:

```
Client: "I want to upgrade to WebSocket on /ws/chat"
Server: "Upgrade accepted. Connection open."
Client: "Hello everyone!"
Server: "Alice says: Hello everyone!"  (pushed to all clients)
Server: "Bob joined the chat"  (pushed to all clients, at any time)
Client: "Goodbye!"
Server: "Alice says: Goodbye!"
Client: (closes connection)
```

Key differences:

• **Persistent**: The connection stays open. No repeated handshakes.

• **Bi-directional**: The server can push data without the client asking.

• **Low overhead**: After the initial handshake, messages are tiny (no HTTP headers per message).

• **Real-time**: Messages arrive within milliseconds.

## 3. Router::websocket() -- WebSocket as a Route

In Tina4, you define WebSocket handlers using `Route::websocket()`:

```
<?php
use Tina4\Route;

Route::websocket("/ws/echo", function ($connection, $event, $data) {
    if ($event === "message") {
        $connection->send("Echo: " . $data);
```

```
    }
});
```

This is the simplest WebSocket handler: it receives a message and sends it back with "Echo: " prepended. The handler receives three arguments:

• **$connection**: The WebSocket connection object. Use it to send messages.

• **$event**: The event type: `"open"`, `"message"`, or `"close"`.

• **$data**: The message data (only present for `"message"` events).

### *Starting the Server*

WebSocket runs alongside your HTTP server:

```
tina4 serve

  Tina4 PHP v3.0.0
  HTTP server running at http://0.0.0.0:7145
  WebSocket server running at ws://0.0.0.0:7145
  Press Ctrl+C to stop
```

Both HTTP and WebSocket share the same port. The server detects the protocol upgrade request and routes it to the correct handler.

## 4. Connection Events

Every WebSocket connection goes through three lifecycle events:

### *Open*

Fires when a client connects:

```
<?php
use Tina4\Route;

Route::websocket("/ws/notifications", function ($connection, $event, $data) {
    if ($event === "open") {
        error_log("Client connected: " . $connection->id);
        $connection->send(json_encode([
            "type" => "welcome",
            "message" => "Connected to notifications",
            "connection_id" => $connection->id
        ]));
    }
});
```

### *Message*

Fires when a client sends data:

```
Route::websocket("/ws/notifications", function ($connection, $event, $data) {
    if ($event === "open") {
        $connection->send(json_encode([
            "type" => "welcome",
            "connection_id" => $connection->id
        ]));
    }

    if ($event === "message") {
        $message = json_decode($data, true);
```

```
        error_log("Received from " . $connection->id . ": " . $data);

        // Process the message based on its type
        if (($message["type"] ?? "") === "ping") {
            $connection->send(json_encode([
                "type" => "pong",
                "timestamp" => date("c")
            ]));
        }
    }
});
```

### *Close*

Fires when a client disconnects:

```
Route::websocket("/ws/notifications", function ($connection, $event, $data) {
    if ($event === "open") {
        error_log("Client connected: " . $connection->id);
    }

    if ($event === "message") {
        error_log("Message from " . $connection->id . ": " . $data);
    }

    if ($event === "close") {
        error_log("Client disconnected: " . $connection->id);
        // Clean up: remove from tracking, notify others, etc.
    }
});
```

### *A Complete Handler*

Here is a handler that responds to all three events:

```
<?php
use Tina4\Route;

Route::websocket("/ws/chat", function ($connection, $event, $data) {
    switch ($event) {
        case "open":
            error_log("[Chat] New connection: " . $connection->id);
            $connection->send(json_encode([
                "type" => "system",
                "message" => "Welcome to the chat!",
                "your_id" => $connection->id
            ]));
            break;

        case "message":
            $message = json_decode($data, true);
            error_log("[Chat] " . $connection->id . ": " . ($message["text"] ?? $data));

            // Echo back with sender info
            $connection->send(json_encode([
                "type" => "message",
                "from" => $connection->id,
                "text" => $message["text"] ?? $data,
                "timestamp" => date("c")
            ]));
// ...
```

## 5. Sending to a Single Client

`$connection->send()` sends a message to the specific client that triggered the event:

```
Route::websocket("/ws/private", function ($connection, $event, $data) {
    if ($event === "message") {
        $message = json_decode($data, true);
        $action = $message["action"] ?? "";

        if ($action === "get-time") {
            $connection->send(json_encode([
                "type" => "time",
                "server_time" => date("c"),
                "timezone" => date_default_timezone_get()
            ]));
        }

        if ($action === "get-status") {
            $connection->send(json_encode([
                "type" => "status",
                "uptime" => 3600,
                "connections" => 42,
                "memory_mb" => round(memory_get_usage() / 1024 / 1024, 2)
            ]));
        }
    }
});
```

Only the client that sent the message receives the response. Other connected clients do not see it.

## 6. Broadcasting to All Clients

`$connection->broadcast()` sends a message to every client connected to the same WebSocket path:

```
<?php
use Tina4\Route;

Route::websocket("/ws/announcements", function ($connection, $event, $data) {
    if ($event === "open") {
        // Tell everyone about the new connection
        $connection->broadcast(json_encode([
            "type" => "system",
            "message" => "A new user joined",
            "online_count" => $connection->connectionCount()
        ]));
    }

    if ($event === "message") {
        $message = json_decode($data, true);

        // Broadcast the message to everyone (including sender)
        $connection->broadcast(json_encode([
            "type" => "announcement",
            "from" => $connection->id,
            "text" => $message["text"] ?? "",
            "timestamp" => date("c")
        ]));
    }

// ...
```

When one client sends a message, every client connected to `/ws/announcements` receives it. This is the foundation for chat rooms, live dashboards, and collaborative editing.

### *Broadcast Excluding Sender*

Sometimes you want to send to everyone except the client that triggered the event:

```
if ($event === "message") {
    $message = json_decode($data, true);

    // Send to sender (confirmation)
    $connection->send(json_encode([
        "type" => "sent",
        "text" => $message["text"]
    ]));

    // Send to everyone else
    $connection->broadcast(json_encode([
        "type" => "message",
        "from" => $message["username"] ?? "Anonymous",
        "text" => $message["text"],
        "timestamp" => date("c")
    ]), true); // true = exclude sender
}
```

The second argument to `broadcast()` excludes the sender when set to `true`. The sender gets a "sent" confirmation, and everyone else gets the "message".

## 7. Path-Scoped Isolation

Different WebSocket paths are completely isolated. Clients connected to `/ws/chat/room-1` do not see messages from `/ws/chat/room-2`:

```
<?php
use Tina4\Route;

Route::websocket("/ws/chat/{room}", function ($connection, $event, $data) {
    $room = $connection->params["room"];

    if ($event === "open") {
        error_log("[Room " . $room . "] New connection: " . $connection->id);
        $connection->broadcast(json_encode([
            "type" => "system",
            "message" => "Someone joined room " . $room,
            "room" => $room,
            "online" => $connection->connectionCount()
        ]));
    }

    if ($event === "message") {
        $message = json_decode($data, true);

        $connection->broadcast(json_encode([
            "type" => "message",
            "room" => $room,
            "from" => $message["username"] ?? "Anonymous",
            "text" => $message["text"] ?? "",
            "timestamp" => date("c")
// ...
```

Connect to different rooms:

```
ws://localhost:7145/ws/chat/general    -- general chat
ws://localhost:7145/ws/chat/random     -- random chat
ws://localhost:7145/ws/chat/dev-team   -- dev team chat
```

Broadcasting in `/ws/chat/general` only reaches clients connected to `/ws/chat/general`. The `dev-team` and `random` rooms are separate.

This gives you chat rooms, project-specific notifications, and per-user channels without any extra configuration. The URL path is the isolation boundary.

## 8. Building a Live Chat

Here is a complete chat application with usernames, typing indicators, and message history:

### *WebSocket Handler*

Create `src/routes/chat-ws.php`:

```php
<?php
use Tina4\Route;

$chatUsers = [];

Route::websocket("/ws/livechat/{room}", function ($connection, $event, $data) use (&$chatUsers) {
    $room = $connection->params["room"];

    if ($event === "open") {
        // User has not set their name yet
        $chatUsers[$connection->id] = [
            "id" => $connection->id,
            "username" => "Anonymous",
            "room" => $room,
            "joined_at" => date("c")
        ];

        $connection->send(json_encode([
            "type" => "welcome",
            "message" => "Connected to room: " . $room,
            "your_id" => $connection->id,
            "online" => $connection->connectionCount()
        ]));
    }

// ...
```

## 9. Live Notifications

WebSocket is great for pushing notifications to users in real time:

```php
<?php
use Tina4\Route;
use Tina4\Queue;

// WebSocket handler for notifications
Route::websocket("/ws/notifications/{userId}", function ($connection, $event, $data) {
    $userId = $connection->params["userId"];
```

```
    if ($event === "open") {
        error_log("[Notifications] User " . $userId . " connected");
        $connection->send(json_encode([
            "type" => "connected",
            "message" => "Listening for notifications"
        ]));
    }

    if ($event === "message") {
        // Client can send acknowledgments or mark notifications as read
        $message = json_decode($data, true);
        if (($message["type"] ?? "") === "mark-read") {
            error_log("[Notifications] User " . $userId . " read notification " . $message["id"]);
        }
    }
});

// ...
```

The `Route::pushToWebSocket()` method lets your HTTP handlers send messages to WebSocket clients. This bridges the gap between traditional request-response endpoints and real-time notifications.

## 10. Connecting from JavaScript

Tina4 provides `frond.js`, a built-in JavaScript helper library that includes WebSocket support:

### *Basic Connection*

```
<script src="/js/frond.js"></script>
<script>
    const ws = frond.ws("/ws/chat/general");

    ws.on("open", function () {
        console.log("Connected to chat");
    });

    ws.on("message", function (data) {
        const message = JSON.parse(data);
        console.log("Received:", message);

        if (message.type === "message") {
            addMessageToUI(message.from, message.text, message.timestamp);
        }

        if (message.type === "system") {
            addSystemMessage(message.message);
        }
    });

    ws.on("close", function () {
        console.log("Disconnected from chat");
    });

// ...
```

### *Auto-Reconnect*

`frond.js` automatically reconnects when the connection drops:

```
const ws = frond.ws("/ws/notifications/42", {
    reconnect: true,           // Enable auto-reconnect (default: true)
```

```
    reconnectInterval: 3000,   // Retry every 3 seconds
    maxReconnectAttempts: 10   // Give up after 10 attempts
});

ws.on("reconnect", function (attempt) {
    console.log("Reconnecting... attempt " + attempt);
});

ws.on("reconnect_failed", function () {
    console.log("Failed to reconnect after maximum attempts");
});
```

### *Using Native WebSocket*

If you prefer not to use `frond.js`, the native WebSocket API works too:

```
const ws = new WebSocket("ws://localhost:7145/ws/chat/general");

ws.onopen = function () {
    console.log("Connected");
    ws.send(JSON.stringify({ type: "set-username", username: "Alice" }));
};

ws.onmessage = function (event) {
    const message = JSON.parse(event.data);
    console.log("Received:", message);
};

ws.onclose = function () {
    console.log("Disconnected");
};

ws.onerror = function (error) {
    console.error("WebSocket error:", error);
};
```

The advantage of `frond.js` is auto-reconnect, message buffering during reconnection, and a cleaner event API.

## 11. A Complete Chat Page

Here is a full chat page using templates and WebSocket:

Create `src/templates/chat.html`:

```
{% extends "base.html" %}

{% block title %}Chat - {{ room }}{% endblock %}

{% block content %}
    <h1>Chat Room: {{ room }}</h1>
    <p id="status">Connecting...</p>
    <p id="online">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: scroll; padding: 12px; margin
    </div>

    <div id="typing" style="height: 20px; color: #888; font-style: italic; margin-bottom: 4px;"></div>

    <form id="chat-form" style="display: flex; gap: 8px;">
        <input type="text" id="message-input" placeholder="Type a message..."
```

```
                style="flex: 1; padding: 8px; border: 1px solid #ddd; border-radius: 4px;">
        <button type="submit" style="padding: 8px 16px; background: #333; color: white; border: none; border-r
            Send
        </button>
    </form>

    <script src="/js/frond.js"></script>
    <script>
        const room = "{{ room }}";
// ...
```

Create the route to serve the page:

```
<?php
use Tina4\Route;

Route::get("/chat/{room}", function ($request, $response) {
    $room = $request->params["room"];
    return $response->render("chat.html", ["room" => $room]);
});
```

Visit `http://localhost:7145/chat/general` in two browser tabs. Type in one tab and watch the message appear in the other instantly.

## 12. Exercise: Build a Real-Time Chat Room

Build a WebSocket chat room with the following features:

### *Requirements*

1. WebSocket endpoint at `/ws/room/{roomName}` that handles:

• `open`: Send welcome message with connection count

• `message` with type `set-name`: Set the user's display name

• `message` with type `chat`: Broadcast the message to all clients with the sender's name

• `close`: Broadcast that the user left

2. HTTP endpoint at `GET /room/{roomName}` that serves an HTML chat page

3. The chat page should:

• Prompt for a username on load

• Display messages in real time

• Show system messages (join/leave) in a different style

• Show the count of online users

### *Test by:*

1. Open `http://localhost:7145/room/test` in two browser tabs

2. Set different usernames in each

3. Send messages from both tabs and verify they appear in both

4. Close one tab and verify the "user left" message appears in the other

## 13. Solution

Create `src/routes/chat-room.php`:

```
<?php
use Tina4\Route;

$roomUsers = [];

Route::websocket("/ws/room/{roomName}", function ($connection, $event, $data) use (&$roomUsers) {
    $room = $connection->params["roomName"];
    $key = $room . ":" . $connection->id;

    if ($event === "open") {
        $roomUsers[$key] = "Anonymous";

        $connection->send(json_encode([
            "type" => "system",
            "message" => "Welcome to room: " . $room . ". Set your name with: {\"type\": \"set-name\", \"name\
            "online" => $connection->connectionCount()
        ]));

        $connection->broadcast(json_encode([
            "type" => "system",
            "message" => "A new user joined",
            "online" => $connection->connectionCount()
        ]), true);
    }

// ...
```

Create `src/templates/room.html`:

```
{% extends "base.html" %}

{% block title %}Room: {{ room }}{% endblock %}

{% block content %}
    <h1>Room: {{ room }}</h1>
    <p id="online" style="color: #666;">Online: 0</p>

    <div id="messages" style="border: 1px solid #ddd; height: 400px; overflow-y: auto; padding: 12px; margin-b
    </div>

    <form id="form" style="display: flex; gap: 8px;">
        <input type="text" id="input" placeholder="Type a message..."
               style="flex: 1; padding: 10px; border: 1px solid #ddd; border-radius: 4px; font-size: 14px;">
        <button type="submit"
                style="padding: 10px 20px; background: #1a1a2e; color: white; border: none; border-radius: 4px
            Send
        </button>
    </form>

    <script src="/js/frond.js"></script>
    <script>
        const room = "{{ room }}";
        const username = prompt("Choose a username:") || "Anonymous";
        const ws = frond.ws("/ws/room/" + room);
// ...
```

Open `http://localhost:7145/room/test` in two browser tabs. Set different usernames. Send messages from one tab and watch them appear in both. Close one tab and verify the "left the room" message appears.

## 14. Gotchas

### *1. WebSocket Needs a Persistent Server*

**Problem:** WebSocket connections drop immediately or do not work at all.

**Cause:** You are running behind a traditional PHP-FPM + Nginx setup. PHP-FPM processes terminate after each HTTP request, so there is no persistent process to maintain WebSocket connections.

**Fix:** Use `tina4 serve` which runs a persistent server that handles both HTTP and WebSocket. For production, run the Tina4 server directly (not behind PHP-FPM). You can still use Nginx as a reverse proxy, but it must be configured for WebSocket proxying with `proxy_set_header Upgrade $http_upgrade` and `proxy_set_header Connection "upgrade"`.

### *2. Messages Are Strings, Not Objects*

**Problem:** `$data` in the message handler is a string, not a PHP array.

**Cause:** WebSocket transmits raw strings. If the client sends JSON, you receive the JSON string, not a decoded array.

**Fix:** Always `json_decode($data, true)` when you expect JSON messages. Always `json_encode()` when you send structured data. WebSocket does not know about content types -- it is just bytes.

### *3. Connection Count Is Per-Path*

**Problem:** `$connection->connectionCount()` returns a lower number than expected.

**Cause:** Connection count is scoped to the WebSocket path. Clients on `/ws/chat/room-1` are counted separately from `/ws/chat/room-2`.

**Fix:** This is by design. Each path is an isolated group. If you need a global connection count across all paths, maintain a counter in a shared variable.

### *4. Broadcasting Does Not Scale Across Servers*

**Problem:** Users connected to different server instances do not see each other's messages.

**Cause:** `$connection->broadcast()` only sends to clients connected to the same server process. With multiple server instances behind a load balancer, each instance has its own set of connections.

**Fix:** Use a pub/sub backend like Redis to relay messages across server instances. Each server subscribes to a Redis channel, and broadcast messages are published to Redis so all servers receive them.

### *5. Large Messages Cause Disconnects*

**Problem:** The connection drops when sending a large message.

**Cause:** WebSocket servers typically have a maximum message size. The default is usually around 1MB. If your message exceeds this, the server closes the connection.

**Fix:** Keep messages small (under 64KB is a good target). For large data transfers, use HTTP endpoints instead of WebSocket. WebSocket is designed for small, frequent messages -- not bulk data transfer.

### *6. Memory Leak from Tracking Connected Users*

**Problem:** The server's memory usage grows over time and eventually crashes.

**Cause:** You store user data in a PHP array (like `$chatUsers`) but do not clean it up when users disconnect. If the `close` event handler does not remove the user from the array, the array grows indefinitely.

**Fix:** Always clean up in the `close` handler. Remove disconnected users from tracking arrays: `unset($chatUsers[$connection->id])`. Test by connecting and disconnecting repeatedly to verify memory stays stable.

### *7. No Authentication on WebSocket Connect*

**Problem:** Anyone can connect to your WebSocket endpoint and see all messages.

**Cause:** The WebSocket upgrade request does not carry your JWT token in the `Authorization` header (browsers do not support custom headers on WebSocket connections).

**Fix:** Pass the token as a query parameter: `ws://localhost:7145/ws/chat?token=eyJ...`. In your `open` handler, validate the token and disconnect if invalid. Use a short-lived token specifically for WebSocket connections. Alternatively, authenticate via an HTTP endpoint first, store the session, and check the session cookie during the WebSocket upgrade.

# Chapter 13: Email with Messenger

## 1. Every App Sends Email

Your SaaS app needs to send signup confirmations, password resets, and weekly digest emails -- with attachments, HTML templates, and reliable delivery. Email is one of those features that every application needs but nobody enjoys implementing. SMTP configuration, plain text fallbacks, attachment encoding, connection timeouts, bounce handling -- the details pile up fast.

Tina4's `Messenger` class handles all of this. Configure it via `.env`, create a Messenger instance, and send. In development mode, emails are intercepted and shown in the dev dashboard so you can inspect them without setting up a real SMTP server.

## 2. Messenger Configuration via .env

All email configuration lives in `.env`:

```
TINA4_MAIL_SMTP_HOST=smtp.example.com
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_USERNAME=your-email@example.com
TINA4_MAIL_SMTP_PASSWORD=your-app-password
TINA4_MAIL_SMTP_ENCRYPTION=tls
TINA4_MAIL_FROM_ADDRESS=noreply@example.com
TINA4_MAIL_FROM_NAME=My Store
```

| Variable | Description | Common Values |

|----------|----------|----------|

| `TINA4_MAIL_SMTP_HOST` | SMTP server hostname | `smtp.gmail.com`, `smtp.mailgun.org`, `smtp.sendgrid.net` |

| `TINA4_MAIL_SMTP_PORT` | SMTP port | `587` (TLS), `465` (SSL), `25` (unencrypted) |

| `TINA4_MAIL_SMTP_USERNAME` | Login username | Usually your email address |

| `TINA4_MAIL_SMTP_PASSWORD` | Login password or app-specific password | App passwords for Gmail |

| `TINA4_MAIL_SMTP_ENCRYPTION` | Encryption method | `tls` (recommended), `ssl`, `none` |

| `TINA4_MAIL_FROM_ADDRESS` | Default "From" address | `noreply@yourdomain.com` |

| `TINA4_MAIL_FROM_NAME` | Default "From" display name | `My Store`, `Acme Corp` |

### *Common Provider Configurations*

**Gmail:**

```
TINA4_MAIL_SMTP_HOST=smtp.gmail.com
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_USERNAME=your-email@gmail.com
TINA4_MAIL_SMTP_PASSWORD=your-app-password
TINA4_MAIL_SMTP_ENCRYPTION=tls
```

Note: Gmail requires an "App Password" (not your regular password) when two-factor authentication is enabled.

**Mailgun:**

```
TINA4_MAIL_SMTP_HOST=smtp.mailgun.org
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_USERNAME=postmaster@mg.yourdomain.com
TINA4_MAIL_SMTP_PASSWORD=your-mailgun-smtp-password
TINA4_MAIL_SMTP_ENCRYPTION=tls
```

**SendGrid:**

```
TINA4_MAIL_SMTP_HOST=smtp.sendgrid.net
TINA4_MAIL_SMTP_PORT=587
TINA4_MAIL_SMTP_USERNAME=apikey
TINA4_MAIL_SMTP_PASSWORD=your-sendgrid-api-key
TINA4_MAIL_SMTP_ENCRYPTION=tls
```

## 3. Constructor Override Pattern

If you need to use different SMTP settings for different purposes (transactional emails from one account, marketing from another), override the configuration in the constructor:

```php
<?php
use Tina4\Messenger;

// Uses .env defaults
$mailer = new Messenger();

// Override specific settings
$marketingMailer = new Messenger([
    "host" => "smtp.mailgun.org",
    "port" => 587,
    "username" => "marketing@mg.yourdomain.com",
    "password" => "marketing-smtp-password",
    "encryption" => "tls",
    "from_address" => "newsletter@yourdomain.com",
    "from_name" => "My Store Newsletter"
]);
```

The constructor accepts an associative array that overrides any `.env` values. Unspecified keys fall back to the `.env` configuration.

## 4. Sending Plain Text Email

The simplest email you can send:

```php
<?php
use Tina4\Route;
use Tina4\Messenger;

Route::post("/api/contact", function ($request, $response) {
    $body = $request->body;

    $mailer = new Messenger();

    $result = $mailer->send(
        $body["email"],                    // To
        "Contact Form Submission",         // Subject
        "Name: " . $body["name"] . "\n" .  // Body (plain text)
        "Email: " . $body["email"] . "\n" .
```

```
        "Message:\n" . $body["message"]
    );

    if ($result["success"]) {
        return $response->json(["message" => "Email sent successfully"]);
    }

    return $response->json(["error" => "Failed to send email", "details" => $result["error"]], 500);
});
```

```
curl -X POST http://localhost:7145/api/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice", "email": "alice@example.com", "message": "I love your products!"}'
```

```
{"message":"Email sent successfully"}
```

The `send()` method returns an array with `"success"` (boolean) and `"error"` (string, only present on failure).

### *The send() Method*

```
$mailer->send($to, $subject, $body, $options = []);
```

• **$to**: Recipient email address (string)

• **$subject**: Email subject line (string)

• **$body**: Email body content (string -- plain text or HTML)

• **$options**: Optional settings (array)

## 5. Sending HTML Email with Text Fallback

Most emails should be HTML with a plain text fallback for email clients that do not render HTML:

```
<?php
use Tina4\Messenger;

$mailer = new Messenger();

$htmlBody = '
<html>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
    <div style="background: #1a1a2e; color: white; padding: 20px; text-align: center;">
        <h1 style="margin: 0;">Welcome to My Store!</h1>
    </div>
    <div style="padding: 20px;">
        <p>Hi Alice,</p>
        <p>Thank you for creating your account. We are excited to have you!</p>
        <p>Here is what you can do next:</p>
        <ul>
            <li>Browse our <a href="https://mystore.com/products">product catalog</a></li>
            <li>Set up your <a href="https://mystore.com/profile">profile</a></li>
            <li>Check out our <a href="https://mystore.com/deals">current deals</a></li>
        </ul>
        <p>If you have any questions, reply to this email and we will get back to you within 24 hours.</p>
        <p>Cheers,<br>The My Store Team</p>
    </div>
    <div style="background: #f5f5f5; padding: 12px; text-align: center; font-size: 12px; color: #888;">
        <p>You received this because you signed up at mystore.com</p>
// ...
```

The `text_body` option provides the plain text fallback. Email clients that cannot render HTML will show the text version instead.

## 6. Adding Attachments

Attach files by providing their paths:

```
<?php
use Tina4\Messenger;

$mailer = new Messenger();

$result = $mailer->send(
    "accounting@example.com",
    "Monthly Invoice #1042",
    "<h2>Invoice #1042</h2><p>Please find the invoice attached.</p>",
    [
        "attachments" => [
            "/path/to/invoices/invoice-1042.pdf",
            "/path/to/reports/monthly-summary.csv"
        ]
    ]
);
```

Each attachment is the absolute file path. Tina4 reads the file, determines the MIME type, and encodes it for email transmission.

### *Inline Attachments with Custom Names*

```
$result = $mailer->send(
    "alice@example.com",
    "Your Export",
    "<p>Here is your data export.</p>",
    [
        "attachments" => [
            [
                "path" => "/tmp/export-20260322.csv",
                "name" => "my-store-export.csv"  // Custom filename
            ]
        ]
    ]
);
```

The recipient sees the file named `my-store-export.csv` regardless of its actual filename on disk.

## 7. CC and BCC

```
<?php
use Tina4\Messenger;

$mailer = new Messenger();

$result = $mailer->send(
    "alice@example.com",
    "Team Meeting Notes",
    "<p>Here are the notes from today's meeting.</p>",
    [
        "cc" => ["bob@example.com", "charlie@example.com"],
```

```
        "bcc" => ["manager@example.com"],
        "reply_to" => "alice@example.com"
    ]
);
```

• **cc**: Array of email addresses to carbon copy. All recipients can see CC addresses.

• **bcc**: Array of email addresses to blind carbon copy. Recipients cannot see BCC addresses.

• **reply_to**: When the recipient clicks "Reply", this address is used instead of the "From" address.

## 8. Reading Inbox via IMAP

Tina4's Messenger can also read emails via IMAP:

```
TINA4_MAIL_IMAP_HOST=imap.example.com
TINA4_MAIL_IMAP_PORT=993
TINA4_MAIL_IMAP_USERNAME=support@example.com
TINA4_MAIL_IMAP_PASSWORD=your-imap-password
TINA4_MAIL_IMAP_ENCRYPTION=ssl
```

```
<?php
use Tina4\Route;
use Tina4\Messenger;

Route::get("/api/inbox", function ($request, $response) {
    $mailer = new Messenger();

    $emails = $mailer->getInbox([
        "limit" => 20,
        "unread_only" => true
    ]);

    $messages = [];
    foreach ($emails as $email) {
        $messages[] = [
            "id" => $email["id"],
            "from" => $email["from"],
            "subject" => $email["subject"],
            "date" => $email["date"],
            "preview" => substr($email["text_body"], 0, 200),
            "has_attachments" => !empty($email["attachments"])
        ];
    }

    return $response->json(["messages" => $messages, "count" => count($messages)]);
// ...
```

```
curl http://localhost:7145/api/inbox
```

```
{
  "messages": [
    {
      "id": "12345",
      "from": "customer@example.com",
      "subject": "Order question",
      "date": "2026-03-22T10:30:00+00:00",
      "preview": "Hi, I have a question about my recent order...",
      "has_attachments": false
    }
  ],
  "count": 1
```

```
}
```

### *Reading a Specific Email*

```
Route::get("/api/inbox/{id}", function ($request, $response) {
    $mailer = new Messenger();
    $emailId = $request->params["id"];

    $email = $mailer->getMessage($emailId);

    if ($email === null) {
        return $response->json(["error" => "Email not found"], 404);
    }

    return $response->json([
        "id" => $email["id"],
        "from" => $email["from"],
        "to" => $email["to"],
        "subject" => $email["subject"],
        "date" => $email["date"],
        "html_body" => $email["html_body"],
        "text_body" => $email["text_body"],
        "attachments" => array_map(fn($a) => [
            "name" => $a["name"],
            "size" => $a["size"],
            "type" => $a["type"]
        ], $email["attachments"] ?? [])
    ]);
});
```

## 9. Dev Mode: Email Interception

When `TINA4_DEBUG=true`, all outgoing emails are intercepted and shown in the dev dashboard instead of being sent to real recipients. This prevents accidentally emailing users during development.

Navigate to `/tina4/console` and look for the "Mail" section. You will see:

• Every email that was "sent" during the current session

• The To, CC, and BCC addresses

• The subject and body (both HTML and plain text)

• Attachments (viewable inline)

• The timestamp

This is invaluable for testing email functionality without configuring a real SMTP server or polluting someone's inbox.

### *Disabling Interception*

If you need to test real email delivery during development, override the interception:

```
TINA4_MAIL_INTERCEPT=false
```

With this set, emails are sent to real recipients even when `TINA4_DEBUG=true`. Use with caution -- you do not want to accidentally email your entire user base from a dev machine.

# 10. Using Templates for Email Content

Hardcoding HTML in PHP strings is ugly and hard to maintain. Use Frond templates for email content:

Create `src/templates/emails/welcome.html`:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
</head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; background: #f5f5f5; padding: 2
    <div style="background: #1a1a2e; color: white; padding: 24px; text-align: center; border-radius: 8px 8px 0
        <h1 style="margin: 0; font-size: 24px;">Welcome, {{ name }}!</h1>
    </div>
    <div style="background: white; padding: 24px; border-radius: 0 0 8px 8px;">
        <p>Hi {{ name }},</p>
        <p>Your account has been created successfully. Here are your details:</p>

        <table style="width: 100%; border-collapse: collapse; margin: 16px 0;">
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Email</td>
                <td style="padding: 8px; border-bottom: 1px solid #eee;">{{ email }}</td>
            </tr>
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Account ID</td>
                <td style="padding: 8px; border-bottom: 1px solid #eee;">#{{ user_id }}</td>
            </tr>
            <tr>
                <td style="padding: 8px; border-bottom: 1px solid #eee; font-weight: bold;">Signed Up</td>
                <td style="padding: 8px; border-bottom: 1px solid #eee;">{{ signed_up_at }}</td>
// ...
```

### *Rendering and Sending*

```
<?php
use Tina4\Route;
use Tina4\Messenger;
use Tina4\Frond;

Route::post("/api/register", function ($request, $response) {
    $body = $request->body;

    // Create user (database logic)
    $userId = 42;

    // Render the email template
    $emailData = [
        "name" => $body["name"],
        "email" => $body["email"],
        "user_id" => $userId,
        "signed_up_at" => date("F j, Y"),
        "base_url" => $_ENV["APP_URL"] ?? "http://localhost:7145",
        "app_name" => "My Store",
        "promo_code" => "WELCOME10",
        "unsubscribe_token" => bin2hex(random_bytes(16))
    ];

    $htmlBody = Frond::render("emails/welcome.html", $emailData);

// ...
```

```
curl -X POST http://localhost:7145/api/register \
  -H "Content-Type: application/json" \
```

```
  -d '{"name": "Alice", "email": "alice@example.com", "password": "securePass123"}'
```

```
{
  "message": "Registration successful",
  "email_sent": true,
  "user_id": 42
}
```

With `TINA4_DEBUG=true`, the email appears in the dev dashboard instead of being sent. You can inspect the rendered HTML, check that template variables were substituted correctly, and verify the layout.

## 11. Sending Email via Queues

For production, never send email synchronously in a route handler. Use the queue system from Chapter 11:

```
<?php
use Tina4\Route;
use Tina4\Queue;
use Tina4\Messenger;
use Tina4\Frond;

// In the route handler, just queue the email
Route::post("/api/register", function ($request, $response) {
    $body = $request->body;
    $userId = 42; // Simulated

    Queue::produce("emails", [
        "template" => "emails/welcome.html",
        "to" => $body["email"],
        "subject" => "Welcome to My Store, " . $body["name"] . "!",
        "data" => [
            "name" => $body["name"],
            "email" => $body["email"],
            "user_id" => $userId,
            "signed_up_at" => date("F j, Y"),
            "base_url" => $_ENV["APP_URL"] ?? "http://localhost:7145",
            "app_name" => "My Store",
            "promo_code" => "WELCOME10"
        ]
    ]);
// ...
```

The route handler returns in under 50 milliseconds. The email is sent by the queue worker, with automatic retries if the SMTP server is temporarily unavailable.

## 12. Exercise: Build a Contact Form with Email Notification

Build a contact form that sends an email notification when submitted.

### *Requirements*

1. Create a `GET /contact` page that renders a contact form with fields: name, email, subject, and message

2. Create a `POST /contact` endpoint that:

• Validates all fields are present

• Sends an email notification to the site admin (`admin@example.com`)

• The email should include all form fields, nicely formatted

• Shows a flash message on success

• Redirects back to the contact page

3. Create an email template at `src/templates/emails/contact-notification.html` that formats the contact submission

### *Test with:*

```
# View the form
curl http://localhost:7145/contact

# Submit the form
curl -X POST http://localhost:7145/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Bob", "email": "bob@example.com", "subject": "Product inquiry", "message": "Do you ship intern

# Check the dev dashboard for the intercepted email
# Navigate to http://localhost:7145/tina4/console
```

## 13. Solution

Create `src/templates/emails/contact-notification.html`:

```
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"></head>
<body style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
    <div style="background: #1a1a2e; color: white; padding: 16px; text-align: center;">
        <h2 style="margin: 0;">New Contact Form Submission</h2>
    </div>
    <div style="padding: 20px; background: white;">
        <table style="width: 100%; border-collapse: collapse;">
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold; width: 100px;">Fro
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ name }} ({{ email }})</td>
            </tr>
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold;">Subject</td>
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ subject }}</td>
            </tr>
            <tr>
                <td style="padding: 10px; border-bottom: 1px solid #eee; font-weight: bold;">Date</td>
                <td style="padding: 10px; border-bottom: 1px solid #eee;">{{ submitted_at }}</td>
            </tr>
        </table>

        <div style="margin-top: 16px; padding: 16px; background: #f8f9fa; border-radius: 4px;">
            <h3 style="margin-top: 0;">Message</h3>
// ...
```

Create `src/templates/contact.html`:

```
{% extends "base.html" %}

{% block title %}Contact Us{% endblock %}

{% block content %}
```

```
    <h1>Contact Us</h1>

    {% if flash %}
        <div style="padding: 12px; border-radius: 4px; margin-bottom: 16px;
            {% if flash.type == 'success' %}background: #d4edda; color: #155724;{% endif %}
            {% if flash.type == 'error' %}background: #f8d7da; color: #721c24;{% endif %}">
            {{ flash.message }}
        </div>
    {% endif %}

    <form method="POST" action="/contact" style="max-width: 500px;">
        <div style="margin-bottom: 12px;">
            <label for="name" style="display: block; margin-bottom: 4px; font-weight: bold;">Name</label>
            <input type="text" name="name" id="name" required
                   style="width: 100%; padding: 8px; border: 1px solid #ddd; border-radius: 4px;">
        </div>

        <div style="margin-bottom: 12px;">
            <label for="email" style="display: block; margin-bottom: 4px; font-weight: bold;">Email</label>
            <input type="email" name="email" id="email" required
// ...
```

Create `src/routes/contact.php`:

```
<?php
use Tina4\Route;
use Tina4\Messenger;
use Tina4\Frond;

Route::get("/contact", function ($request, $response) {
    $flash = $request->session["_flash"] ?? null;
    unset($request->session["_flash"]);

    return $response->render("contact.html", [
        "flash" => $flash
    ]);
});

Route::post("/contact", function ($request, $response) {
    $body = $request->body;

    // Validate
    $errors = [];
    if (empty($body["name"])) $errors[] = "Name is required";
    if (empty($body["email"])) $errors[] = "Email is required";
    if (empty($body["subject"])) $errors[] = "Subject is required";
    if (empty($body["message"])) $errors[] = "Message is required";

    if (!empty($errors)) {
// ...
```

**Testing:**

1. Open `http://localhost:7145/contact` in your browser

2. Fill in the form and submit

3. You should see a green "Thank you" flash message

4. Open `http://localhost:7145/tina4/console` to see the intercepted email

5. The email should show the sender details, subject, message, and formatted HTML

**API test:**

```
curl -X POST http://localhost:7145/contact \
  -H "Content-Type: application/json" \
  -d '{"name": "Bob", "email": "bob@example.com", "subject": "Product inquiry", "message": "Do you ship intern
  -c cookies.txt -b cookies.txt
```

The response is a `302` redirect to `/contact`. Follow the redirect to see the flash message:

```
curl http://localhost:7145/contact -b cookies.txt
```

The HTML response includes the success flash message.

## 14. Gotchas

### *1. Gmail Blocks "Less Secure" Apps*

**Problem:** Sending via Gmail fails with "Authentication failed" or "Username and Password not accepted".

**Cause:** Gmail blocks SMTP access from apps that do not use OAuth2 by default. Using your regular password will not work if two-factor authentication is enabled.

**Fix:** Generate an "App Password" in your Google Account settings (Security > 2-Step Verification > App Passwords). Use this 16-character password as `TINA4_MAIL_SMTP_PASSWORD`. This is separate from your regular Google password.

### *2. Emails Go to Spam*

**Problem:** Emails are delivered but land in the recipient's spam folder.

**Cause:** Your sending domain lacks proper DNS records (SPF, DKIM, DMARC) or you are sending from a free email provider (Gmail, Yahoo).

**Fix:** Use a dedicated sending domain with proper DNS records. Set up SPF, DKIM, and DMARC records. Use a transactional email service like Mailgun, SendGrid, or Amazon SES that handles email reputation for you.

### *3. HTML Email Looks Broken*

**Problem:** The email looks fine in Gmail but broken in Outlook or Apple Mail.

**Cause:** Email clients have wildly different HTML/CSS support. CSS flexbox, grid, and many modern properties do not work in email.

**Fix:** Use inline styles (not external stylesheets or `<style>` blocks). Use table-based layouts for complex designs. Test with an email preview tool. Keep it simple -- most transactional emails do not need elaborate designs.

### *4. Attachment File Not Found*

**Problem:** `Messenger::send()` returns an error about a missing file.

**Cause:** The attachment path is relative or incorrect. The file does not exist at the specified location.

**Fix:** Use absolute paths for attachments. Verify the file exists before calling `send()`: `if (!file_exists($path)) { ... }`. If the file is generated dynamically (like a PDF), make sure the generation completes before sending.

### *5. Dev Mode Silently Intercepts Emails*

**Problem:** You set up SMTP correctly but no emails arrive. No errors either.

**Cause:** `TINA4_DEBUG=true` intercepts all emails and shows them in the dev dashboard. The email is never sent to the SMTP server.

**Fix:** Check the dev dashboard at `/tina4/console` for intercepted emails. If you want to send real emails during development, set `TINA4_MAIL_INTERCEPT=false`. Remember to remove this setting before committing.

### *6. Email Template Variables Not Substituted*

**Problem:** The email body shows `{{ name }}` literally instead of the user's name.

**Cause:** You passed the raw template file content instead of rendering it through Frond. The template engine was not invoked.

**Fix:** Use `Frond::render("emails/template.html", $data)` to render the template with variables substituted. Do not read the file with `file_get_contents()` -- that gives you the raw template source.

### *7. Connection Timeout on Send*

**Problem:** `Messenger::send()` hangs for 30 seconds and then fails with a timeout error.

**Cause:** The SMTP server is unreachable from your network, the port is blocked by a firewall, or the hostname is wrong.

**Fix:** Test SMTP connectivity: `telnet smtp.example.com 587`. Verify the hostname, port, and encryption settings. Check that your firewall allows outbound connections on the SMTP port. If you are behind a corporate firewall, port 587 or 465 might be blocked -- ask your network administrator.

# Chapter 14: Caching

## 1. From 800ms to 3ms

Your product catalog page runs 12 database queries and takes 800 milliseconds to render. Every visitor triggers the same queries, the same template rendering, the same JSON serialization -- for data that changes maybe once a day. After adding caching, the first request takes 800ms and the next 10,000 requests take 3ms each. That is a 266x improvement for one line of configuration.

Caching stores the result of expensive operations so they can be reused without recomputing. Tina4 provides caching at multiple levels: response caching (entire HTTP responses), database query caching, and a direct cache API for custom use cases.

## 2. Response Caching with ResponseCache Middleware

The fastest way to cache is at the HTTP response level. The `ResponseCache` middleware stores the complete response (headers and body) and serves it directly on subsequent requests without calling your route handler at all.

```
<?php
use Tina4\Route;

Route::get("/api/products", function ($request, $response) {
    // This handler runs 12 database queries and takes 800ms
    // With ResponseCache, it only runs once every 5 minutes

    error_log("Handler called -- this should only appear once every 5 minutes");

    $products = [
        ["id" => 1, "name" => "Wireless Keyboard", "price" => 79.99],
        ["id" => 2, "name" => "USB-C Hub", "price" => 49.99],
        ["id" => 3, "name" => "Monitor Stand", "price" => 129.99]
    ];

    return $response->json(["products" => $products, "generated_at" => date("c")]);
}, "ResponseCache:300");
```

The `"ResponseCache:300"` middleware caches the response for 300 seconds (5 minutes). During those 5 minutes:

• The first request runs the handler (800ms)

• The next 10,000 requests serve the cached response (3ms each)

• After 300 seconds, the cache expires and the next request runs the handler again

```
curl http://localhost:7145/api/products

{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
    {"id": 2, "name": "USB-C Hub", "price": 49.99},
    {"id": 3, "name": "Monitor Stand", "price": 129.99}
  ],
  "generated_at": "2026-03-22T14:30:00+00:00"
}
```

Call it again within 5 minutes:

```
curl http://localhost:7145/api/products
```

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "price": 79.99},
    {"id": 2, "name": "USB-C Hub", "price": 49.99},
    {"id": 3, "name": "Monitor Stand", "price": 129.99}
  ],
  "generated_at": "2026-03-22T14:30:00+00:00"
}
```

Notice the `generated_at` timestamp is the same. The handler did not run -- the response came from cache.

### *Cache Headers*

The `ResponseCache` middleware automatically sets cache-related headers:

```
X-Cache: HIT
X-Cache-TTL: 247
Cache-Control: public, max-age=300
```

- `X-Cache: HIT` or `X-Cache: MISS` tells you whether the response came from cache
- `X-Cache-TTL` shows the remaining time-to-live in seconds
- `Cache-Control` enables browser and CDN caching

### *Caching with Query Parameters*

By default, the cache key includes the full URL with query parameters. `/api/products?page=1` and `/api/products?page=2` are cached separately:

```
Route::get("/api/products", function ($request, $response) {
    $page = (int) ($request->query["page"] ?? 1);
    $limit = 20;
    $offset = ($page - 1) * $limit;

    // Simulate database query
    return $response->json([
        "page" => $page,
        "products" => [],
        "generated_at" => date("c")
    ]);
}, "ResponseCache:300");
```

```
curl "http://localhost:7145/api/products?page=1"  # Cache MISS, stores for page=1
curl "http://localhost:7145/api/products?page=2"  # Cache MISS, stores for page=2
curl "http://localhost:7145/api/products?page=1"  # Cache HIT
```

### *What Not to Cache*

Do not use `ResponseCache` on:

- **POST, PUT, PATCH, DELETE routes**: Only GET responses should be cached
- **User-specific endpoints**: `/api/profile` returns different data for each user
- **Real-time data**: Stock prices, live scores, chat messages
- **Authenticated endpoints**: Unless the cache is scoped per user

```
// GOOD: Public, rarely changing data
```

```
Route::get("/api/categories", function ($request, $response) {
    return $response->json(["categories" => []]);
}, "ResponseCache:3600"); // Cache for 1 hour

// BAD: User-specific data -- do NOT cache
Route::get("/api/profile", function ($request, $response) {
    return $response->json($request->user);
}, "authMiddleware"); // No ResponseCache here
```

## 3. Memory Cache (Default)

Tina4's cache system stores data in memory by default. No configuration needed.

```
# This is the default -- you do not need to set it explicitly
TINA4_CACHE_BACKEND=memory
```

Memory cache is the fastest option (no disk I/O, no network calls) but it resets when the server restarts. It is ideal for development and single-server deployments where losing the cache on restart is acceptable.

## 4. Redis Cache

For production deployments where you want cache persistence across server restarts and shared cache across multiple server instances, use Redis:

```
TINA4_CACHE_BACKEND=redis
TINA4_CACHE_HOST=localhost
TINA4_CACHE_PORT=6379
TINA4_CACHE_PASSWORD=your-redis-password
TINA4_CACHE_PREFIX=myapp:cache:
```

Your code does not change. The `cache_get`, `cache_set`, and `ResponseCache` middleware all work identically. Only the storage backend changes.

### *Why Redis?*

• Cache survives server restarts

• Shared across multiple server instances (behind a load balancer)

• Sub-millisecond reads and writes

• Built-in key expiry (TTL cleanup is automatic)

• Same Redis instance can serve sessions, cache, and queues

## 5. File Cache

If you want cache persistence but do not have Redis, use file-based caching:

```
TINA4_CACHE_BACKEND=file
TINA4_CACHE_PATH=/path/to/cache/directory
```

File cache stores each cache entry as a file on disk. It is slower than memory or Redis but survives server restarts without extra infrastructure.

### *When to Use File Cache*

- You need cache persistence but cannot run Redis
- Your hosting environment is limited (shared hosting, no external services)
- Cache entries are large and you do not want them in memory

## 6. Direct Cache API

For custom caching logic, use the `cache_get`, `cache_set`, and `cache_delete` functions directly:

### *cache_set*

```
<?php
use function Tina4\cache_set;

// Cache a value for 300 seconds
cache_set("product:42", [
    "id" => 42,
    "name" => "Wireless Keyboard",
    "price" => 79.99,
    "in_stock" => true
], 300);

// Cache a string
cache_set("exchange_rate:USD_EUR", "0.92", 3600);

// Cache indefinitely (no TTL)
cache_set("app:config", ["theme" => "dark", "lang" => "en"]);
```

### *cache_get*

```
<?php
use function Tina4\cache_get;

$product = cache_get("product:42");
// Returns the cached array, or null if not found or expired

if ($product === null) {
    // Cache miss -- fetch from database
    $product = fetchProductFromDatabase(42);
    cache_set("product:42", $product, 300);
}

return $response->json($product);
```

### *cache_delete*

```
<?php
use function Tina4\cache_delete;

// Delete a specific key
cache_delete("product:42");

// Delete multiple keys
cache_delete("product:42");
cache_delete("product:43");
cache_delete("product:44");
```

### *Real-World Pattern: Cache-Aside*

The most common caching pattern is cache-aside (also called lazy loading):

```php
<?php
use Tina4\Route;
use Tina4\Database;
use function Tina4\cache_get;
use function Tina4\cache_set;

Route::get("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $cacheKey = "product:" . $id;

    // 1. Try the cache first
    $product = cache_get($cacheKey);

    if ($product !== null) {
        // Cache hit -- return immediately
        return $response->json(array_merge($product, ["source" => "cache"]));
    }

    // 2. Cache miss -- fetch from database
    $db = Database::getConnection();
    $product = $db->fetchOne(
        "SELECT id, name, category, price, in_stock FROM products WHERE id = :id",
        ["id" => $id]
    );

// ...
```

```
curl http://localhost:7145/api/products/42
```

First call (cache miss):

```
{
  "id": 42,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "source": "database"
}
```

Second call (cache hit):

```
{
  "id": 42,
  "name": "Wireless Keyboard",
  "category": "Electronics",
  "price": 79.99,
  "in_stock": true,
  "source": "cache"
}
```

## 7. Database Query Caching

Tina4 can cache database query results automatically. Enable it in `.env`:

```
TINA4_DB_CACHE=true
TINA4_DB_CACHE_TTL=300
```

With database caching enabled, identical queries return cached results instead of hitting the database:

```php
<?php
use Tina4\Route;
use Tina4\Database;
```

```
Route::get("/api/categories", function ($request, $response) {
    $db = Database::getConnection();

    // First call: executes the query (20ms)
    // Subsequent calls within 300 seconds: returns cached result (0.1ms)
    $categories = $db->fetchAll("SELECT * FROM categories ORDER BY name");

    return $response->json(["categories" => $categories]);
});
```

The cache key is derived from the SQL query and its parameters. Different queries or different parameters produce different cache keys:

```
// These are cached separately:
$db->fetchAll("SELECT * FROM products WHERE category = :cat", ["cat" => "Electronics"]);
$db->fetchAll("SELECT * FROM products WHERE category = :cat", ["cat" => "Fitness"]);
```

### *When to Use DB Cache*

• Read-heavy applications where the same queries run repeatedly

• Reference data that changes infrequently (categories, countries, settings)

• Dashboard queries that aggregate large datasets

### *When Not to Use DB Cache*

• Write-heavy applications where data changes constantly

• Queries with real-time requirements (inventory counts, live prices)

• Queries that must always return the latest data

### *Skipping Cache for Specific Queries*

```
// Force a fresh query, bypassing the cache
$freshData = $db->fetchAll("SELECT * FROM products", [], ["no_cache" => true]);
```

## 8. Cache Invalidation Strategies

The hardest problem in caching is knowing when to invalidate (clear) the cache. Stale cache serves outdated data. Premature invalidation reduces cache effectiveness.

### *Strategy 1: Time-Based Expiry (TTL)*

The simplest strategy. Set a TTL and let the cache expire naturally:

```
cache_set("products:featured", $featuredProducts, 600); // Expires in 10 minutes
```

Good for data where near-real-time accuracy is acceptable. A 10-minute delay in updating the featured products list is usually fine.

### *Strategy 2: Event-Based Invalidation*

Clear the cache when the underlying data changes:

```
<?php
use Tina4\Route;
use Tina4\Database;
use function Tina4\cache_set;
use function Tina4\cache_delete;
```

```
// Update a product
Route::put("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $body = $request->body;
    $db = Database::getConnection();

    $db->execute(
        "UPDATE products SET name = :name, price = :price WHERE id = :id",
        ["name" => $body["name"], "price" => $body["price"], "id" => $id]
    );

    // Invalidate the cache for this product
    cache_delete("product:" . $id);

    // Also invalidate any list caches that might include this product
    cache_delete("products:all");
    cache_delete("products:featured");

    $updated = $db->fetchOne("SELECT * FROM products WHERE id = :id", ["id" => $id]);
// ...
```

This is the most accurate strategy -- the cache is always fresh after a write. The downside is you must remember to invalidate everywhere the data could be cached.

### *Strategy 3: Write-Through Cache*

Update the cache at the same time as the database:

```
Route::put("/api/products/{id:int}", function ($request, $response) {
    $id = $request->params["id"];
    $body = $request->body;
    $db = Database::getConnection();

    $db->execute(
        "UPDATE products SET name = :name, price = :price WHERE id = :id",
        ["name" => $body["name"], "price" => $body["price"], "id" => $id]
    );

    $updated = $db->fetchOne("SELECT * FROM products WHERE id = :id", ["id" => $id]);

    // Write the new data directly to cache (instead of deleting)
    cache_set("product:" . $id, $updated, 600);

    return $response->json($updated);
});
```

This ensures the cache always has the latest data. No cache miss after an update -- the next read comes from the already-warm cache.

## 9. TTL Management

Choosing the right TTL depends on how often the data changes and how acceptable stale data is:

| Data Type | Suggested TTL | Reasoning |

|----------|----------|----------|

| Static config (categories, countries) | 3600 (1 hour) | Changes rarely, stale data is harmless |

| Product catalog | 300 (5 min) | Updates several times per day |

| User profile | 60 (1 min) | Users expect changes to appear quickly |

| Search results | 120 (2 min) | Balance between freshness and performance |

| Dashboard stats | 30 (30 sec) | Near-real-time but expensive to compute |

| Exchange rates | 60 (1 min) | Updates frequently, slight delay is acceptable |

| Shopping cart | 0 (no cache) | Must always reflect current state |

### *Dynamic TTL*

Adjust TTL based on data characteristics:

```php
<?php
use function Tina4\cache_set;
use function Tina4\cache_get;

function getCachedProduct($id) {
    $cacheKey = "product:" . $id;
    $product = cache_get($cacheKey);

    if ($product !== null) {
        return $product;
    }

    $product = fetchProductFromDatabase($id);

    // Popular products: shorter TTL (more likely to change)
    // Inactive products: longer TTL (rarely change)
    $ttl = ($product["view_count"] > 1000) ? 60 : 3600;

    cache_set($cacheKey, $product, $ttl);

    return $product;
}
```

## 10. Cache Statistics

Monitor cache performance to verify that caching is actually helping:

```php
<?php
use Tina4\Route;
use function Tina4\cache_stats;

Route::get("/api/cache/stats", function ($request, $response) {
    $stats = cache_stats();

    return $response->json($stats);
});
```

```
curl http://localhost:7145/api/cache/stats
```

```
{
  "backend": "memory",
  "entries": 42,
  "hits": 15234,
  "misses": 891,
  "hit_rate": "94.5%",
  "memory_bytes": 524288,
  "oldest_entry_age": 3542
}
```

A hit rate above 90% means your caching strategy is working well. Below 80% suggests your TTLs are too short, your cache is too small, or you are caching data that is not accessed frequently enough to benefit.

The dev dashboard at `/tina4/console` also shows cache statistics, including per-key hit counts and miss counts, so you can identify which keys benefit most from caching.

## 11. Combining Cache Layers

For maximum performance, layer multiple cache strategies:

```
<?php
use Tina4\Route;
use Tina4\Database;
use function Tina4\cache_get;
use function Tina4\cache_set;

Route::get("/api/catalog", function ($request, $response) {
    $page = (int) ($request->query["page"] ?? 1);
    $cacheKey = "catalog:page:" . $page;

    // Layer 1: Check application cache
    $catalog = cache_get($cacheKey);

    if ($catalog !== null) {
        return $response->json(array_merge($catalog, ["cache" => "application"]));
    }

    // Layer 2: Database query (with DB-level caching if TINA4_DB_CACHE=true)
    $db = Database::getConnection();
    $limit = 20;
    $offset = ($page - 1) * $limit;

    $products = $db->fetchAll(
        "SELECT p.*, c.name as category_name
         FROM products p
// ...
```

This creates three cache layers:

1. **ResponseCache** (60 seconds): The entire HTTP response is cached. No PHP code runs at all.

2. **Application cache** (300 seconds): If the response cache expired but the app cache is still fresh, skip the database queries.

3. **DB query cache** (if enabled): Individual query results are cached even if the application cache missed.

The first visitor after a full cache expiry waits 800ms. Everyone else gets the response in under 5ms.

## 12. Exercise: Cache an Expensive Product Listing Endpoint

Build a product listing endpoint that uses caching at multiple levels.

### *Requirements*

1. Create a `GET /api/store/products` endpoint that:

• Accepts query parameters: `category`, `page`, `limit`

• Returns a list of products with pagination metadata

• Uses the direct cache API (`cache_get`/`cache_set`) with a 5-minute TTL

• Includes a `source` field in the response (`"cache"` or `"database"`)

2. Create a `POST /api/store/products` endpoint that:

• Creates a new product

• Invalidates the relevant cache entries

3. Create a `GET /api/store/cache-stats` endpoint that shows cache statistics

### *Test with:*

```
# First call -- cache miss, slow
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"

# Second call -- cache hit, fast
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"

# Different category -- cache miss
curl "http://localhost:7145/api/store/products?category=Fitness&page=1"

# Create a product -- should invalidate cache
curl -X POST http://localhost:7145/api/store/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Smart Watch", "category": "Electronics", "price": 299.99}'

# Same query again -- cache miss (invalidated by the POST)
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"

# Check cache stats
curl http://localhost:7145/api/store/cache-stats
```

## 13. Solution

Create `src/routes/store-cached.php`:

```
<?php
use Tina4\Route;
use function Tina4\cache_get;
use function Tina4\cache_set;
use function Tina4\cache_delete;
use function Tina4\cache_stats;

// Simulated product database
function getProductStore() {
    return [
        ["id" => 1, "name" => "Wireless Keyboard", "category" => "Electronics", "price" => 79.99, "in_stock" =
        ["id" => 2, "name" => "Yoga Mat", "category" => "Fitness", "price" => 29.99, "in_stock" => true],
        ["id" => 3, "name" => "Coffee Grinder", "category" => "Kitchen", "price" => 49.99, "in_stock" => false
        ["id" => 4, "name" => "Standing Desk", "category" => "Electronics", "price" => 549.99, "in_stock" => t
        ["id" => 5, "name" => "Running Shoes", "category" => "Fitness", "price" => 119.99, "in_stock" => true
        ["id" => 6, "name" => "Bluetooth Speaker", "category" => "Electronics", "price" => 39.99, "in_stock" =
        ["id" => 7, "name" => "Resistance Bands", "category" => "Fitness", "price" => 14.99, "in_stock" => tru
        ["id" => 8, "name" => "French Press", "category" => "Kitchen", "price" => 34.99, "in_stock" => true]
    ];
}

// List products with caching
```

```
Route::get("/api/store/products", function ($request, $response) {
    $category = $request->query["category"] ?? null;
    $page = (int) ($request->query["page"] ?? 1);
// ...
```

**Expected output -- first call (cache miss):**

```
curl "http://localhost:7145/api/store/products?category=Electronics&page=1"
```

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": true},
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": true},
    {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "in_stock": true}
  ],
  "page": 1,
  "limit": 20,
  "total": 3,
  "pages": 1,
  "generated_at": "2026-03-22T14:30:00+00:00",
  "source": "database"
}
```

**Expected output -- second call (cache hit):**

```
{
  "products": [
    {"id": 1, "name": "Wireless Keyboard", "category": "Electronics", "price": 79.99, "in_stock": true},
    {"id": 4, "name": "Standing Desk", "category": "Electronics", "price": 549.99, "in_stock": true},
    {"id": 6, "name": "Bluetooth Speaker", "category": "Electronics", "price": 39.99, "in_stock": true}
  ],
  "page": 1,
  "limit": 20,
  "total": 3,
  "pages": 1,
  "generated_at": "2026-03-22T14:30:00+00:00",
  "source": "cache"
}
```

Notice: same `generated_at`, but `source` changed from `"database"` to `"cache"`. The handler did not run.

**Expected output -- after creating a product:**

```
curl -X POST http://localhost:7145/api/store/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Smart Watch", "category": "Electronics", "price": 299.99}'
```

```
{
  "message": "Product created",
  "product": {
    "id": 4721,
    "name": "Smart Watch",
    "category": "Electronics",
    "price": 299.99,
    "in_stock": true
  },
  "cache_invalidated": true
}
```

The next request to the same URL will be a cache miss again, because the POST invalidated the cache.

## 14. Gotchas

### *1. Caching Authenticated Responses*

**Problem:** User A's profile is served to User B because the response was cached.

**Cause:** `ResponseCache` caches by URL only. If `/api/profile` returns different data per user but all requests hit the same URL, the first user's response is served to everyone.

**Fix:** Do not use `ResponseCache` on user-specific endpoints. Use the direct cache API with user-specific keys instead: `cache_set("profile:" . $userId, $data, 300)`.

### *2. Cache Stampede*

**Problem:** When a popular cache key expires, hundreds of requests hit the database simultaneously (all experiencing a cache miss at the same moment).

**Cause:** All requests see the cache miss and all try to rebuild the cache independently.

**Fix:** Use cache locking or "stale-while-revalidate". One request rebuilds the cache while others serve the stale value. Tina4's `ResponseCache` handles this automatically by serving the expired response to concurrent requests while one request refreshes it.

### *3. Memory Cache Lost on Restart*

**Problem:** After restarting the server, performance drops until the cache warms up.

**Cause:** Memory cache is lost when the process restarts. Every request is a cache miss until data is cached again.

**Fix:** For production, use Redis cache (`TINA4_CACHE_BACKEND=redis`). It persists across server restarts. Alternatively, implement a cache warmup script that pre-populates the cache with frequently accessed data.

### *4. Stale Data After Database Update*

**Problem:** You updated a product's price in the database, but the API still returns the old price.

**Cause:** The cache still has the old data and has not expired yet.

**Fix:** Always invalidate (or update) the cache when you modify the underlying data. Use `cache_delete("product:" . $id)` after an update, or use write-through caching with `cache_set()` to update the cache with the new value.

### *5. Cache Key Collisions*

**Problem:** Two different queries return the same cached data.

**Cause:** Your cache keys are not specific enough. Using `"products"` as a key for both the full list and a filtered list causes collisions.

**Fix:** Include all relevant parameters in the cache key: `"products:category:Electronics:page:1:limit:20"`. Or use an MD5 hash of the parameters: `"products:" . md5(json_encode($params))`.

### *6. Serialization Overhead*

**Problem:** Caching makes certain requests slower, not faster.

**Cause:** The cached object is very large. Serializing and deserializing it takes more time than re-computing it.

**Fix:** Only cache data that is expensive to compute. If the original operation takes 5ms and cache serialization takes 10ms, caching is counterproductive. Profile before and after caching to verify the improvement.

### *7. Forgetting to Set TTL*

**Problem:** Cache entries never expire and the server's memory grows until it crashes.

**Cause:** You called `cache_set("key", $value)` without a TTL. The entry lives forever (or until the server restarts).

**Fix:** Always set a TTL: `cache_set("key", $value, 300)`. Even for data that "never changes," set a long TTL like 86400 (24 hours). This provides a safety net against stale data and prevents unbounded memory growth.

# Chapter 15: Frontend with tina4css

## 1. The Problem with Frontend Toolchains

Your client wants a beautiful dashboard. You know the drill -- install Node.js, run `npm install`, wait for 200MB of `node_modules` to download, configure webpack or Vite, set up PostCSS, add a CSS framework, maybe Tailwind with its purge config, and then pray nothing breaks when you upgrade a dependency six months from now.

Tina4 takes a different approach. The framework ships with **tina4css** -- a Bootstrap-compatible CSS framework -- and **frond.js** -- a lightweight JavaScript helper library. Both are included automatically when you scaffold a project. No npm. No webpack. No build step for development. Just link the files and start building.

By the end of this chapter, you will have a complete admin dashboard with a sidebar, navigation, cards, tables, modals, and dark mode support -- all without touching npm.

## 2. What Ships with Tina4

When you run `tina4 init`, two files appear in your project:

```
src/public/
■■■ css/
■   ■■■ tina4.css        # The CSS framework
■■■ js/
■   ■■■ frond.js         # AJAX helpers, form submission, token management
■■■ scss/
    ■■■ tina4.scss       # SCSS source (optional, for customization)
```

Include them in any template:

```
<link rel="stylesheet" href="/css/tina4.css">
<script src="/js/frond.js"></script>
```

That is all you need. No CDN, no package manager, no version conflicts.

## 3. The Grid System

tina4css uses a 12-column responsive grid, compatible with Bootstrap's class names. If you already know Bootstrap, you already know tina4css.

```
<div class="container">
    <div class="row">
        <div class="col-md-4">
            <p>One third</p>
        </div>
        <div class="col-md-4">
            <p>One third</p>
        </div>
        <div class="col-md-4">
            <p>One third</p>
        </div>
    </div>
</div>
```

Breakpoints:

| Class Prefix | Screen Width | Typical Device |

|----------|----------|----------|

| `col-` | All sizes | Phones and up |

| `col-sm-` | >= 576px | Large phones |

| `col-md-` | >= 768px | Tablets |

| `col-lg-` | >= 992px | Desktops |

| `col-xl-` | >= 1200px | Large desktops |

Columns stack vertically on screens smaller than their breakpoint. A `col-md-6` element takes half the row on tablets and up, but full width on phones.

## 4. Components

### *Navbar*

```
<nav class="navbar navbar-dark bg-dark">
    <div class="container">
        <a class="navbar-brand" href="/">My Dashboard</a>
        <ul class="navbar-nav">
            <li class="nav-item">
                <a class="nav-link active" href="/dashboard">Dashboard</a>
            </li>
            <li class="nav-item">
                <a class="nav-link" href="/users">Users</a>
            </li>
            <li class="nav-item">
                <a class="nav-link" href="/settings">Settings</a>
            </li>
        </ul>
    </div>
</nav>
```

Use `navbar-light bg-light` for a light theme, or `navbar-dark bg-primary` for a colored background.

### *Buttons*

```
<button class="btn btn-primary">Save</button>
<button class="btn btn-secondary">Cancel</button>
<button class="btn btn-danger">Delete</button>
<button class="btn btn-success">Approve</button>
<button class="btn btn-warning">Warning</button>
<button class="btn btn-outline-primary">Outlined</button>
<button class="btn btn-sm btn-primary">Small</button>
<button class="btn btn-lg btn-primary">Large</button>
```

### *Cards*

```
<div class="card">
    <div class="card-header">
        Monthly Revenue
    </div>
    <div class="card-body">
        <h5 class="card-title">$12,450</h5>
        <p class="card-text">Up 8% from last month</p>
```

```
    </div>
    <div class="card-footer text-muted">
        Updated 5 minutes ago
    </div>
</div>
```

### Forms

```
<form>
    <div class="form-group">
        <label for="email">Email address</label>
        <input type="email" class="form-control" id="email"
               placeholder="you@example.com">
    </div>
    <div class="form-group">
        <label for="password">Password</label>
        <input type="password" class="form-control" id="password">
    </div>
    <div class="form-group form-check">
        <input type="checkbox" class="form-check-input" id="remember">
        <label class="form-check-label" for="remember">Remember me</label>
    </div>
    <button type="submit" class="btn btn-primary">Sign In</button>
</form>
```

### Tables

```
<table class="table table-striped table-hover">
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Role</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>1</td>
            <td>Alice Johnson</td>
            <td>alice@example.com</td>
            <td><span class="badge badge-primary">Admin</span></td>
            <td>
                <button class="btn btn-sm btn-outline-primary">Edit</button>
                <button class="btn btn-sm btn-outline-danger">Delete</button>
            </td>
        </tr>
        <tr>
            <td>2</td>
            <td>Bob Smith</td>
            <td>bob@example.com</td>
// ...
```

Table variants: `table-bordered`, `table-striped`, `table-hover`, `table-sm` (compact), `table-responsive` (wraps in a scrollable container on small screens).

### Badges

```
<span class="badge badge-primary">Primary</span>
<span class="badge badge-success">Active</span>
<span class="badge badge-warning">Pending</span>
<span class="badge badge-danger">Overdue</span>
<span class="badge badge-info">Info</span>
```

```
<span class="badge badge-dark">Dark</span>
```

### Alerts

```
<div class="alert alert-success">
    Product saved successfully.
</div>

<div class="alert alert-danger">
    Error: could not connect to the database.
</div>

<div class="alert alert-warning alert-dismissible">
    Your trial expires in 3 days.
    <button type="button" class="close" data-dismiss="alert">&times;</button>
</div>

<div class="alert alert-info">
    Tip: you can drag and drop items to reorder them.
</div>
```

### Modals

```
<button class="btn btn-primary" data-toggle="modal" data-target="#confirmModal">
    Delete User
</button>

<div class="modal" id="confirmModal">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h5 class="modal-title">Confirm Deletion</h5>
                <button type="button" class="close" data-dismiss="modal">&times;</button>
            </div>
            <div class="modal-body">
                <p>Are you sure you want to delete this user? This action cannot be undone.</p>
            </div>
            <div class="modal-footer">
                <button class="btn btn-secondary" data-dismiss="modal">Cancel</button>
                <button class="btn btn-danger">Delete</button>
            </div>
        </div>
    </div>
</div>
```

tina4css includes the JavaScript needed for modal toggling. No jQuery required.

### Progress Bars

```
<div class="progress">
    <div class="progress-bar bg-success" style="width: 75%">75%</div>
</div>

<div class="progress">
    <div class="progress-bar bg-warning progress-bar-striped" style="width: 45%">
        45% - Uploading...
    </div>
</div>
```

## 5. SCSS Customization

The SCSS source lives in `src/public/scss/tina4.scss`. If you want to customize colors, fonts, or spacing, edit this file and compile it to CSS.

### Variables

At the top of `tina4.scss`, you will find variables you can override:

```
// Colors
$primary:   #007bff;
$secondary: #6c757d;
$success:   #28a745;
$danger:    #dc3545;
$warning:   #ffc107;
$info:      #17a2b8;
$dark:      #343a40;
$light:     #f8f9fa;

// Typography
$font-family-base: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
$font-size-base: 1rem;
$line-height-base: 1.5;

// Spacing
$spacer: 1rem;

// Border radius
$border-radius: 0.25rem;

// Breakpoints
$breakpoint-sm: 576px;
$breakpoint-md: 768px;
$breakpoint-lg: 992px;
// ...
```

### Compiling SCSS

If you edit the SCSS files, compile them with the Tina4 CLI:

```
tina4 scss
```

```
Compiling SCSS...
  src/public/scss/tina4.scss → src/public/css/tina4.css
Done.
```

This reads all `.scss` files from `src/public/scss/` and writes compiled CSS to `src/public/css/`. No Sass gem, no Node, no additional dependencies -- the Tina4 CLI includes a built-in SCSS compiler.

You can also run it in watch mode during development:

```
tina4 scss --watch
```

```
Watching src/public/scss/ for changes...
  [14:30:05] Compiled tina4.scss → tina4.css (42ms)
```

Every time you save a `.scss` file, it recompiles automatically.

### Custom Theme Example

Create `src/public/scss/custom.scss`:

```
// Override variables before importing tina4
$primary: #6f42c1;
$font-family-base: "Inter", sans-serif;
$border-radius: 0.5rem;
```

```
// Import the base framework
@import "tina4";

// Add your own styles
.sidebar {
    background: $dark;
    color: $light;
    min-height: 100vh;
    padding: 1rem;

    .nav-link {
        color: rgba(255, 255, 255, 0.7);
        padding: 0.5rem 1rem;
        border-radius: $border-radius;

        &:hover, &.active {
            color: #fff;
            background: rgba(255, 255, 255, 0.1);
        }
    }
// ...
```

Compile:

```
tina4 scss
```

Then reference your custom CSS in your template:

```
<link rel="stylesheet" href="/css/custom.css">
```

## 6. frond.js -- The JavaScript Helper

frond.js is a lightweight JavaScript library that ships with Tina4. It provides AJAX helpers, form submission, and JWT token management without requiring jQuery, Axios, or any other library.

### *AJAX Requests*

```
// GET request
frond.get("/api/products", function (data) {
    console.log(data.products);
});

// GET with error handling
frond.get("/api/products", function (data) {
    console.log(data);
}, function (error) {
    console.error("Failed:", error);
});

// POST request
frond.post("/api/products", {
    name: "Widget",
    price: 9.99
}, function (data) {
    console.log("Created:", data);
});

// PUT request
frond.put("/api/products/1", {
    name: "Updated Widget",
    price: 12.99
```

```
}, function (data) {
// ...
```

### Form Submission

frond.js can intercept form submissions and send them via AJAX instead of a full page reload:

```
<form id="createProduct" data-frond-submit="/api/products" data-frond-method="POST">
    <div class="form-group">
        <label for="name">Product Name</label>
        <input type="text" class="form-control" name="name" id="name">
    </div>
    <div class="form-group">
        <label for="price">Price</label>
        <input type="number" class="form-control" name="price" id="price" step="0.01">
    </div>
    <button type="submit" class="btn btn-primary">Create Product</button>
</form>

<div id="result"></div>

<script>
    frond.onSubmit("createProduct", function (response) {
        document.getElementById("result").innerHTML =
            '<div class="alert alert-success">Product created: ' + response.name + '</div>';
    }, function (error) {
        document.getElementById("result").innerHTML =
            '<div class="alert alert-danger">Error: ' + error.message + '</div>';
    });
</script>
```

The `data-frond-submit` attribute tells frond.js the URL to POST to. The `data-frond-method` attribute specifies the HTTP method. frond.js serializes all form fields as JSON automatically.

### Token Management

When your application uses JWT authentication (Chapter 7), frond.js manages tokens automatically:

```
// Store the token after login
frond.setToken("eyJhbGciOiJIUzI1NiIs...");

// All subsequent requests include the Authorization header automatically
frond.get("/api/profile", function (data) {
    // The request included: Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
    console.log(data);
});

// Clear the token on logout
frond.clearToken();
```

frond.js stores the token in `localStorage` and includes it as a `Bearer` token in the `Authorization` header on every request.

### Loading Indicators

```
// Show a loading state while fetching
frond.get("/api/products", function (data) {
    renderProducts(data.products);
}, null, {
    loading: "#loadingSpinner"  // CSS selector for loading element
});
```

The element with id `loadingSpinner` will be shown while the request is in flight and hidden when it completes.

## 7. Building a Dashboard Layout

Let us build a real admin dashboard. We will create a template layout with a sidebar, a top navbar, and a content area.

### *The Base Layout*

Create `src/templates/admin/layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}Admin Dashboard{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <style>
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; }

        .app-wrapper { display: flex; min-height: 100vh; }

        .sidebar {
            width: 250px;
            background: #1a1a2e;
            color: #e0e0e0;
            padding: 0;
            flex-shrink: 0;
        }
        .sidebar-header {
            padding: 20px;
            font-size: 1.2em;
            font-weight: bold;
            color: #fff;
// ...
```

### *The Dashboard Page*

Create `src/templates/admin/dashboard.html`:

```
{% extends "admin/layout.html" %}

{% block title %}Dashboard - Admin{% endblock %}
{% block page_title %}Dashboard{% endblock %}

{% block content %}
    <div class="stats-grid">
        {% for stat in stats %}
            <div class="stat-card">
                <div class="stat-value">{{ stat.value }}</div>
                <div class="stat-label">{{ stat.label }}</div>
                <div class="stat-change {{ stat.direction }}">
                    {{ stat.direction == "up" ? "+" : "" }}{{ stat.change }}% from last month
                </div>
            </div>
        {% endfor %}
    </div>
```

```
    <div class="row">
        <div class="col-md-8">
            <div class="card">
                <div class="card-header">Recent Orders</div>
                <div class="card-body" style="padding: 0;">
                    <table class="table table-hover" style="margin: 0;">
                        <thead>
// ...
```

### *The Dashboard Route*

Create `src/routes/admin.php`:

```
<?php
use Tina4\Route;

Route::get("/admin", function ($request, $response) {
    $stats = [
        ["label" => "Total Users", "value" => "1,247", "change" => 12, "direction" => "up"],
        ["label" => "Revenue", "value" => "$34,500", "change" => 8, "direction" => "up"],
        ["label" => "Orders", "value" => "456", "change" => 3, "direction" => "down"],
        ["label" => "Conversion Rate", "value" => "3.2%", "change" => 0.5, "direction" => "up"]
    ];

    $recentOrders = [
        ["id" => 1042, "customer" => "Alice Johnson", "total" => 149.99, "status" => "completed"],
        ["id" => 1041, "customer" => "Bob Smith", "total" => 89.50, "status" => "pending"],
        ["id" => 1040, "customer" => "Carol Davis", "total" => 234.00, "status" => "completed"],
        ["id" => 1039, "customer" => "David Lee", "total" => 45.99, "status" => "cancelled"],
        ["id" => 1038, "customer" => "Eva Martinez", "total" => 178.25, "status" => "pending"]
    ];

    return $response->render("admin/dashboard.html", [
        "active_page" => "dashboard",
        "user_name" => "Admin",
        "pending_orders" => 2,
        "stats" => $stats,
        "recent_orders" => $recentOrders,
// ...
```

Start the server and visit `http://localhost:7145/admin`. You should see:

• A dark sidebar on the left with navigation links

• A top bar with a welcome message and logout button

• Four stat cards showing users, revenue, orders, and conversion rate

• A recent orders table with color-coded status badges

• Quick action buttons in a card on the right

• System health progress bars

All of this with zero npm dependencies.

## 8. Dark Mode Support

tina4css includes built-in dark mode. Add the `data-theme="dark"` attribute to the `<html>` element:

```
<html lang="en" data-theme="dark">
```

This inverts the color scheme -- light backgrounds become dark, dark text becomes light, and all components adapt automatically.

### *Toggle with JavaScript*

Add a dark mode toggle button to your layout:

```
<button class="btn btn-sm btn-outline-secondary" onclick="toggleDarkMode()">
    Toggle Dark Mode
</button>

<script>
    function toggleDarkMode() {
        const html = document.documentElement;
        const current = html.getAttribute("data-theme");
        const next = current === "dark" ? "light" : "dark";
        html.setAttribute("data-theme", next);
        localStorage.setItem("theme", next);
    }

    // Load saved preference
    const saved = localStorage.getItem("theme");
    if (saved) {
        document.documentElement.setAttribute("data-theme", saved);
    }
</script>
```

### *Respecting System Preference*

If the user has not set a preference, respect their operating system setting:

```
const saved = localStorage.getItem("theme");
if (saved) {
    document.documentElement.setAttribute("data-theme", saved);
} else if (window.matchMedia("(prefers-color-scheme: dark)").matches) {
    document.documentElement.setAttribute("data-theme", "dark");
}
```

### *Custom Dark Mode Colors*

In your SCSS, you can customize the dark mode palette:

```
[data-theme="dark"] {
    --bg-primary: #1a1a2e;
    --bg-secondary: #16213e;
    --text-primary: #e0e0e0;
    --text-secondary: #a0a0a0;
    --border-color: #2d2d4a;
    --card-bg: #16213e;
}
```

## 9. Responsive Design

tina4css is mobile-first. Components stack vertically on small screens by default and expand horizontally on larger screens.

### *Responsive Utilities*

Show and hide elements at different breakpoints:

```
<!-- Only visible on medium screens and up -->
```

```
<div class="d-none d-md-block">
    Desktop sidebar content
</div>

<!-- Only visible on small screens -->
<div class="d-block d-md-none">
    Mobile navigation
</div>
```

### *Responsive Tables*

Wrap tables in a `.table-responsive` container to make them scroll horizontally on small screens:

```
<div class="table-responsive">
    <table class="table">
        <!-- Wide table content -->
    </table>
</div>
```

### *Spacing Utilities*

Use margin and padding utilities that respond to breakpoints:

```
<div class="p-2 p-md-4">Padding: 0.5rem on mobile, 1.5rem on desktop</div>
<div class="mt-2 mt-lg-5">Margin-top: 0.5rem on mobile, 3rem on large screens</div>
```

Spacing scale:

| Class | Size |

|-------|------|

| `*-0` | 0 |

| `*-1` | 0.25rem |

| `*-2` | 0.5rem |

| `*-3` | 1rem |

| `*-4` | 1.5rem |

| `*-5` | 3rem |

Directions: `m` (margin), `p` (padding), `t` (top), `b` (bottom), `l` (left), `r` (right), `x` (horizontal), `y` (vertical).

## 10. Building a Users Page with AJAX

Let us build a users management page that loads data via AJAX using frond.js.

Create `src/templates/admin/users.html`:

```
{% extends "admin/layout.html" %}

{% block title %}Users - Admin{% endblock %}
{% block page_title %}User Management{% endblock %}

{% block content %}
    <div class="card">
        <div class="card-header" style="display: flex; justify-content: space-between; align-items: center;">
            <span>All Users</span>
            <button class="btn btn-sm btn-primary" data-toggle="modal" data-target="#addUserModal">
```

```
                Add User
            </button>
        </div>
        <div class="card-body" style="padding: 0;">
            <div id="loadingSpinner" class="text-center p-4" style="display: none;">
                Loading...
            </div>
            <table class="table table-hover" id="usersTable" style="margin: 0;">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Email</th>
                        <th>Created</th>
                        <th>Actions</th>
// ...
```

Create the route for the users page:

```
Route::get("/admin/users", function ($request, $response) {
    return $response->render("admin/users.html", [
        "active_page" => "users",
        "user_name" => "Admin",
        "pending_orders" => 2
    ]);
});
```

This page loads users via an AJAX call to `/api/users` (provided by auto-CRUD on the User model from Chapter 6). The table populates dynamically, and users can be added through a modal form and deleted with confirmation -- all without full page reloads.

## 11. Exercise: Build a Complete Admin Dashboard

Build an admin dashboard with the following features:

### *Requirements*

1. **Layout** -- Sidebar with navigation links for Dashboard, Products, and Orders. Top bar with the current user name and a dark mode toggle button.

2. **Dashboard Page** (`GET /admin`) -- Four stat cards (Total Products, Total Orders, Revenue, Low Stock Items). A table of the 5 most recent orders. A progress bar showing inventory health (percentage of products in stock).

3. **Products Page** (`GET /admin/products`) -- A table listing all products with columns: ID, Name, Category, Price, Stock Status. An "Add Product" button that opens a modal with a form. The form submits via frond.js AJAX to `POST /api/products`. After submission, the table reloads without a page refresh.

4. **Dark Mode** -- The toggle button switches between light and dark themes. The preference persists in localStorage.

### *Seed Data*

Create a route that populates sample data:

```
Route::get("/admin/seed", function ($request, $response) {
    $products = [
        ["name" => "Wireless Keyboard", "category" => "Electronics", "price" => 79.99, "in_stock" => true],
        ["name" => "USB-C Hub", "category" => "Electronics", "price" => 49.99, "in_stock" => true],
```

```
        ["name" => "Standing Desk", "category" => "Furniture", "price" => 549.99, "in_stock" => false],
        ["name" => "Monitor Light", "category" => "Electronics", "price" => 39.99, "in_stock" => true],
        ["name" => "Ergonomic Chair", "category" => "Furniture", "price" => 399.99, "in_stock" => false]
    ];

    foreach ($products as $data) {
        $product = new Product();
        $product->name = $data["name"];
        $product->category = $data["category"];
        $product->price = $data["price"];
        $product->inStock = $data["in_stock"];
        $product->save();
    }

    return $response->json(["message" => "Seeded " . count($products) . " products"]);
});
```

### *Expected Result*

Visit `http://localhost:7145/admin` and see:

• A sidebar with navigation (Dashboard highlighted)

• Four stat cards showing product and order counts

• A recent orders table with status badges

• A dark mode toggle that switches the entire UI

Visit `http://localhost:7145/admin/products` and see:

• A table of all products loaded via AJAX

• An "Add Product" button that opens a modal

• Form submission creates the product and refreshes the table

## 12. Solution

The solution follows the same patterns shown in sections 7 and 10. Create the layout template extending the base layout from section 7. Create the dashboard page template with stat cards using `{% for stat in stats %}`. Create the products page template using the AJAX pattern from section 10, but for products instead of users.

For the products page, use auto-CRUD on the Product model (`$autoCrud = true`) so the API endpoints are available at `/api/products`. Load the table with `frond.get("/api/products", ...)` and handle form submission with `frond.post("/api/products", ...)`.

For the dark mode toggle, add the JavaScript from section 8 to the base layout template inside the `{% block extra_js %}` block.

The key route handlers are:

```
<?php
use Tina4\Route;

Route::get("/admin", function ($request, $response) {
    $product = new Product();
    $allProducts = $product->select("*");
    $inStockCount = count(array_filter($allProducts, fn($p) => $p->inStock));
```

```
    $totalProducts = count($allProducts);
    $stockHealth = $totalProducts > 0 ? round(($inStockCount / $totalProducts) * 100) : 0;

    $stats = [
        ["label" => "Total Products", "value" => $totalProducts, "change" => 5, "direction" => "up"],
        ["label" => "Total Orders", "value" => 156, "change" => 12, "direction" => "up"],
        ["label" => "Revenue", "value" => "$8,450", "change" => 3, "direction" => "up"],
        ["label" => "Low Stock", "value" => $totalProducts - $inStockCount, "change" => 2, "direction" => "dow
    ];

    return $response->render("admin/dashboard.html", [
        "active_page" => "dashboard",
        "stats" => $stats,
        "stock_health" => $stockHealth,
        "recent_orders" => [],
        "pending_orders" => 0
    ]);
});
// ...
```

## 13. Gotchas

### *1. CSS File Not Loading*

**Problem:** The browser shows unstyled HTML. The CSS file returns a 404.

**Cause:** The CSS file must be inside `src/public/css/`. If you put it in `src/css/` or `css/` at the root, Tina4 will not serve it.

**Fix:** Make sure the file is at `src/public/css/tina4.css`. Static files are served from `src/public/`.

### *2. SCSS Changes Not Reflected*

**Problem:** You edited `tina4.scss` but the browser still shows the old styles.

**Cause:** SCSS must be compiled to CSS. Editing the `.scss` file does not automatically update the `.css` file unless you are running `tina4 scss --watch`.

**Fix:** Run `tina4 scss` to compile, or use `tina4 scss --watch` during development.

### *3. Modal Does Not Open*

**Problem:** Clicking a button with `data-toggle="modal"` does nothing.

**Cause:** frond.js is not loaded, or the `data-target` value does not match the modal's `id`.

**Fix:** Check that `<script src="/js/frond.js"></script>` is included before `</body>`. Verify that `data-target="#myModal"` matches `<div class="modal" id="myModal">` (the `#` prefix is required in `data-target`).

### *4. frond.js AJAX Returns HTML Instead of JSON*

**Problem:** `frond.get("/api/products", ...)` receives an HTML page instead of JSON data.

**Cause:** The route handler returns `$response->render(...)` or `$response->html(...)` instead of `$response->json(...)`. Or the URL is wrong and hitting a catch-all route that returns HTML.

**Fix:** Verify the API route returns JSON. Check the URL in the frond.js call matches the route path exactly.

### *5. Dark Mode Flickers on Load*

**Problem:** The page loads in light mode for a split second before switching to dark mode.

**Cause:** The theme is set by JavaScript after the page renders. The initial HTML does not include `data-theme="dark"`.

**Fix:** Move the theme detection script to the `<head>` section so it runs before the body renders:

```
<head>
    <script>
        (function() {
            var t = localStorage.getItem("theme");
            if (t) document.documentElement.setAttribute("data-theme", t);
            else if (window.matchMedia("(prefers-color-scheme: dark)").matches)
                document.documentElement.setAttribute("data-theme", "dark");
        })();
    </script>
</head>
```

### *6. Form Data Not Reaching the Server*

**Problem:** `frond.post()` sends the request but `$request->body` is empty on the server.

**Cause:** frond.js sends JSON by default. If you are building the data object incorrectly or passing a FormData object, the server may not parse it correctly.

**Fix:** Pass a plain JavaScript object to `frond.post()`. Do not use `new FormData()` -- frond.js handles serialization internally. Make sure your object keys match what the server expects:

```
frond.post("/api/products", {
    name: document.getElementById("name").value,
    price: parseFloat(document.getElementById("price").value)
}, callback);
```

### *7. Responsive Grid Not Working*

**Problem:** Columns do not stack on mobile. They overflow horizontally instead.

**Cause:** Missing the viewport meta tag. Without it, mobile browsers render the page at desktop width.

**Fix:** Add this to your `<head>`:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

This tag is included in all the examples above but is easy to forget when creating templates from scratch.

# Chapter 16: GraphQL

## 1. The Problem GraphQL Solves

Your mobile app needs a list of products. Each product has a name, price, 20 image URLs, a full description, 15 review objects, and 8 other fields you do not need right now. With REST, you call `GET /api/products` and receive all of it -- 50KB of JSON when you only needed 2KB. On a spotty mobile connection, that wasted bandwidth matters.

Now your web dashboard needs the same products, but it also needs the category, stock status, and supplier info. With REST, you either make three requests (products, categories, suppliers) and stitch them together on the client, or you build a custom endpoint like `GET /api/products?include=category,supplier` and handle the query parameter parsing on the server.

GraphQL solves both problems. The client asks for exactly the fields it needs, and the server returns exactly those fields. One endpoint, one request, one response with the right shape.

Tina4 includes a built-in GraphQL engine. No external packages. No Apollo Server. No `graphql-php` library. It is part of the framework.

## 2. GraphQL vs REST -- A Quick Comparison

| Aspect | REST | GraphQL |

|--------|------|---------|

| Endpoints | One per resource (`/products`, `/users`) | One endpoint (`/graphql`) |

| Data shape | Server decides | Client decides |

| Over-fetching | Common (get all fields) | Never (get only requested fields) |

| Under-fetching | Common (multiple requests needed) | Never (get related data in one query) |

| Versioning | `/api/v1/`, `/api/v2/` | Schema evolves, no versioning needed |

| Caching | HTTP caching works naturally | Requires client-side cache management |

| Learning curve | Low | Moderate |

REST is still great for simple APIs. GraphQL shines when clients have diverse data needs -- mobile apps, dashboards, third-party integrations.

## 3. Enabling GraphQL

GraphQL is available by default in Tina4. The engine serves requests at `/graphql`. If you want to change the endpoint, set it in `.env`:

```
TINA4_GRAPHQL_ENDPOINT=/graphql
```

To verify it is working, start the server and send a test query:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ __schema { queryType { name } } }"}'
```

```
{
  "data": {
    "__schema": {
      "queryType": {
        "name": "Query"
      }
    }
  }
}
```

If you see that response, GraphQL is running.

## 4. Defining a Schema

A GraphQL schema defines the types of data your API can return and the queries and mutations clients can execute.

Create `src/graphql/schema.graphql`:

```
type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

type Query {
    products: [Product!]!
    product(id: Int!): Product
}
```

This schema says:

• A `Product` has six fields. The `!` means the field is non-nullable.

• The `Query` type has two operations: `products` returns a list of products, and `product(id)` returns a single product (or null if not found).

Tina4 auto-discovers `.graphql` files in `src/graphql/`. You do not need to register them.

## 5. Writing Resolvers

A resolver is the function that runs when a query or mutation is executed. Resolvers live in `src/graphql/` as PHP files.

Create `src/graphql/resolvers.php`:

```
<?php
use Tina4\GraphQL;

// Resolve the "products" query
GraphQL::resolve("Query", "products", function ($root, $args) {
    $product = new Product();
    $products = $product->select("*", "", [], "name ASC");

    return array_map(fn($p) => $p->toDict(), $products);
});
```

```
// Resolve the "product" query
GraphQL::resolve("Query", "product", function ($root, $args) {
    $product = new Product();
    $product->load($args["id"]);

    if (empty($product->id)) {
        return null;
    }

    return $product->toDict();
});
```

The `GraphQL::resolve()` method takes three arguments:

1. **Type name** -- The GraphQL type this resolver belongs to (`Query`, `Mutation`, or a custom type).

2. **Field name** -- The field within the type this resolver handles.

3. **Resolver function** -- Receives `$root` (the parent value, if any) and `$args` (the query arguments).

### *Testing the Queries*

List all products:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ products { id name price inStock } }"}'
```

```
{
  "data": {
    "products": [
      {"id": 1, "name": "Wireless Keyboard", "price": 79.99, "inStock": true},
      {"id": 2, "name": "USB-C Hub", "price": 49.99, "inStock": true},
      {"id": 3, "name": "Standing Desk", "price": 549.99, "inStock": false}
    ]
  }
}
```

Notice: the response only contains the four fields we asked for (`id`, `name`, `price`, `inStock`), not `category` or `createdAt`. That is GraphQL in action.

Get a single product with all fields:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ product(id: 1) { id name category price inStock createdAt } }"}'
```

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Wireless Keyboard",
      "category": "Electronics",
      "price": 79.99,
      "inStock": true,
      "createdAt": "2026-03-22 14:30:00"
    }
  }
}
```

Request a product that does not exist:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ product(id: 999) { id name } }"}'
```

```
{
  "data": {
    "product": null
  }
}
```

## 6. Mutations

Mutations are how clients create, update, or delete data. They are defined in the schema alongside queries.

Update `src/graphql/schema.graphql`:

```
type Product {
    id: Int!
    name: String!
    category: String!
    price: Float!
    inStock: Boolean!
    createdAt: String
}

input ProductInput {
    name: String!
    category: String
    price: Float!
    inStock: Boolean
}

input ProductUpdateInput {
    name: String
    category: String
    price: Float
    inStock: Boolean
}

type DeleteResult {
    success: Boolean!
// ...
```

Key concepts:

• **`input` types** define the shape of data the client sends. They are like regular types but used for arguments.

• **`Mutation` type** lists all write operations. By convention, mutations are named with verbs: `createProduct`, `updateProduct`, `deleteProduct`.

• Fields in `ProductUpdateInput` are all optional (no `!`) because the client may only want to update one field.

### *Mutation Resolvers*

Add to `src/graphql/resolvers.php`:

```
// Create a product
GraphQL::resolve("Mutation", "createProduct", function ($root, $args) {
    $input = $args["input"];

    $product = new Product();
```

```
    $product->name = $input["name"];
    $product->category = $input["category"] ?? "Uncategorized";
    $product->price = (float) $input["price"];
    $product->inStock = (bool) ($input["inStock"] ?? true);
    $product->save();

    return $product->toDict();
});

// Update a product
GraphQL::resolve("Mutation", "updateProduct", function ($root, $args) {
    $product = new Product();
    $product->load($args["id"]);

    if (empty($product->id)) {
        return null;
    }

    $input = $args["input"];
    if (isset($input["name"])) $product->name = $input["name"];
// ...
```

### *Testing Mutations*

Create a product:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { createProduct(input: { name: \"Desk Lamp\", category: \"Office\", price: 39.99 }) { i
  }'
```

```
{
  "data": {
    "createProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 39.99
    }
  }
}
```

Update a product:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "mutation { updateProduct(id: 4, input: { price: 44.99, inStock: false }) { id name price inStock
  }'
```

```
{
  "data": {
    "updateProduct": {
      "id": 4,
      "name": "Desk Lamp",
      "price": 44.99,
      "inStock": false
    }
  }
}
```

Delete a product:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
```

```
  -d '{
    "query": "mutation { deleteProduct(id: 4) { success message } }"
  }'
```

```
{
  "data": {
    "deleteProduct": {
      "success": true,
      "message": "Product deleted"
    }
  }
}
```

## 7. Nested Types and Relationships

GraphQL's real power comes from traversing relationships in a single query. Let us add authors and comments to a blog schema.

Update `src/graphql/schema.graphql`:

```
type User {
    id: Int!
    name: String!
    email: String!
    posts: [Post!]!
}

type Post {
    id: Int!
    title: String!
    body: String!
    published: Boolean!
    author: User!
    comments: [Comment!]!
    commentCount: Int!
}

type Comment {
    id: Int!
    authorName: String!
    body: String!
    post: Post!
}

type Product {
// ...
```

Add resolvers for the nested types:

```
// Posts query
GraphQL::resolve("Query", "posts", function ($root, $args) {
    $post = new Post();
    $posts = $post->select("*", "published = :pub", ["pub" => 1], "created_at DESC");
    return array_map(fn($p) => $p->toDict(), $posts);
});

// Single post query
GraphQL::resolve("Query", "post", function ($root, $args) {
    $post = new Post();
    $post->load($args["id"]);
    return empty($post->id) ? null : $post->toDict();
});
```

```
// Resolve Post.author (nested field)
GraphQL::resolve("Post", "author", function ($post, $args) {
    $user = new User();
    $user->load($post["user_id"]);
    return $user->toDict();
});

// Resolve Post.comments (nested field)
GraphQL::resolve("Post", "comments", function ($post, $args) {
    $comment = new Comment();
    $comments = $comment->select("*", "post_id = :postId", ["postId" => $post["id"]]);
// ...
```

Now clients can query across relationships in a single request:

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "{ posts { id title author { name email } comments { authorName body } commentCount } }"
  }'
```

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
        "author": {
          "name": "Alice",
          "email": "alice@example.com"
        },
        "comments": [
          {"authorName": "Bob", "body": "Great post!"}
        ],
        "commentCount": 1
      }
    ]
  }
}
```

One request. Exactly the fields the client needs. No over-fetching. No under-fetching.

## 8. Auto-Generating Schema from ORM Models

If your ORM models have `$autoCrud = true`, Tina4 can auto-generate GraphQL types and basic CRUD resolvers for them. Enable this in `.env`:

```
TINA4_GRAPHQL_AUTO_SCHEMA=true
```

With this setting, every ORM model with `$autoCrud = true` automatically gets:

• A GraphQL type with fields matching the model properties

• A query to list all records (e.g., `products`)

• A query to get one by ID (e.g., `product(id: Int!)`)

• A mutation to create (e.g., `createProduct(input: ProductInput!)`)

• A mutation to update (e.g., `updateProduct(id: Int!, input: ProductUpdateInput!)`)

• A mutation to delete (e.g., `deleteProduct(id: Int!)`)

You can still define custom resolvers that override the auto-generated ones. Custom resolvers always take precedence.

## 9. The GraphiQL Playground

When `TINA4_DEBUG=true`, Tina4 serves a GraphiQL interactive playground at:

```
http://localhost:7145/graphql/playground
```

GraphiQL gives you:

• A query editor with syntax highlighting and auto-completion

• A documentation explorer showing all types, queries, and mutations

• A results panel showing the response

• Query history

This is the fastest way to explore and test your GraphQL API during development. Type a query on the left, click the play button, and see the results on the right. The documentation explorer on the right sidebar shows every type, field, and argument available in your schema.

GraphiQL is only available when `TINA4_DEBUG=true`. In production, it is disabled automatically.

## 10. Query Variables

For production use, clients should send query variables separately from the query string. This prevents injection and allows query caching.

```
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{
    "query": "query GetProduct($id: Int!) { product(id: $id) { id name price } }",
    "variables": {"id": 1}
  }'
```

```
{
  "data": {
    "product": {
      "id": 1,
      "name": "Wireless Keyboard",
      "price": 79.99
    }
  }
}
```

The query uses `$id` as a placeholder, and the actual value comes from the `variables` object. This is safer and more efficient than string interpolation.

## 11. Exercise: Build a GraphQL API for a Blog

Build a complete GraphQL API for a blog with posts, authors, and comments.

### *Requirements*

1. **Schema** -- Define types for `User`, `Post`, and `Comment` with the following fields:

• `User`: id, name, email, posts (nested)

• `Post`: id, title, body, published, author (nested), comments (nested), commentCount (computed)

• `Comment`: id, authorName, body, createdAt

2. **Queries**:

• `posts` -- List all published posts

• `post(id: Int!)` -- Get a single post by ID

• `user(id: Int!)` -- Get a user with their posts

3. **Mutations**:

• `createPost(userId: Int!, title: String!, body: String!, published: Boolean)` -- Create a new post

• `addComment(postId: Int!, authorName: String!, body: String!)` -- Add a comment to a post

4. Use the ORM models from Chapter 6 (User, Post, Comment).

### *Test with these queries:*

```
# List published posts with author names
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ posts { id title author { name } commentCount } }"}'

# Get a specific post with all comments
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ post(id: 1) { title body author { name email } comments { authorName body } } }"}'

# Create a post
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { createPost(userId: 1, title: \"GraphQL is great\", body: \"Here is why...\", publi

# Add a comment
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "mutation { addComment(postId: 1, authorName: \"Carol\", body: \"Nice article!\") { id authorN

# Get a user with all their posts
curl -X POST http://localhost:7145/graphql \
  -H "Content-Type: application/json" \
  -d '{"query": "{ user(id: 1) { name email posts { title published } } }"}'
```

## 12. Solution

### *Schema*

Create `src/graphql/blog-schema.graphql`:

```
type User {
    id: Int!
    name: String!
    email: String!
    posts: [Post!]!
```

```
}

type Post {
    id: Int!
    title: String!
    body: String!
    published: Boolean!
    author: User!
    comments: [Comment!]!
    commentCount: Int!
    createdAt: String
}

type Comment {
    id: Int!
    authorName: String!
    body: String!
    createdAt: String
}

// ...
```

### *Resolvers*

Create `src/graphql/blog-resolvers.php`:

```
<?php
use Tina4\GraphQL;

// Query: list published posts
GraphQL::resolve("Query", "posts", function ($root, $args) {
    $post = new Post();
    $posts = $post->select("*", "published = :pub", ["pub" => 1], "created_at DESC");
    return array_map(fn($p) => $p->toDict(), $posts);
});

// Query: single post
GraphQL::resolve("Query", "post", function ($root, $args) {
    $post = new Post();
    $post->load($args["id"]);
    return empty($post->id) ? null : $post->toDict();
});

// Query: single user
GraphQL::resolve("Query", "user", function ($root, $args) {
    $user = new User();
    $user->load($args["id"]);
    return empty($user->id) ? null : $user->toDict();
});

// Post.author resolver
// ...
```

**Expected output for `{ posts { id title author { name } commentCount } }`:**

```
{
  "data": {
    "posts": [
      {
        "id": 1,
        "title": "My First Post",
        "author": {"name": "Alice"},
        "commentCount": 1
      },
```

```
      {
        "id": 2,
        "title": "GraphQL is great",
        "author": {"name": "Alice"},
        "commentCount": 0
      }
    ]
  }
}
```

## 13. Gotchas

### *1. Schema File Not Found*

**Problem:** Queries return errors about unknown types or fields.

**Cause:** The `.graphql` file is not in `src/graphql/`, or has a syntax error that prevents parsing.

**Fix:** Make sure schema files are in `src/graphql/` and end with `.graphql`. Check for syntax errors -- a missing `!` or unmatched brace will cause the entire schema to fail silently.

### *2. Resolver Not Called*

**Problem:** A query returns `null` even though data exists.

**Cause:** The resolver is not registered, or the type/field names do not match the schema exactly. GraphQL is case-sensitive.

**Fix:** Verify that `GraphQL::resolve("Query", "products", ...)` matches `type Query { products: ... }` exactly. Check capitalization.

### *3. Nested Resolver Returns Wrong Data*

**Problem:** `post.author` returns the entire post instead of the user.

**Cause:** The nested resolver receives the parent object as its first argument (`$post`), not a fresh model. If you return the parent instead of loading the related record, you get the wrong data.

**Fix:** In a nested resolver, always load the related record from the database using the foreign key from the parent:

```
GraphQL::resolve("Post", "author", function ($post, $args) {
    $user = new User();
    $user->load($post["user_id"]); // Use the foreign key from the parent
    return $user->toDict();
});
```

### *4. Mutation Input Not Parsed*

**Problem:** `$args["input"]` is null inside a mutation resolver.

**Cause:** The mutation signature in the schema uses an `input` type, but the client query passes arguments directly instead of wrapping them in an `input` object.

**Fix:** Match the query to the schema. If the schema says `createProduct(input: ProductInput!)`, the client must send `createProduct(input: { name: "Widget", price: 9.99 })`, not `createProduct(name: "Widget", price: 9.99)`.

### *5. N+1 Query Problem*

**Problem:** Loading 50 posts with their authors generates 51 database queries (1 for posts + 50 for authors).

**Cause:** Each nested resolver runs independently. When you resolve `author` for each post, that is one query per post.

**Fix:** Use data loader patterns or batch loading. For simple cases, you can pre-load all related records in the parent resolver and pass them down. For complex cases, consider using the REST API with eager loading instead of GraphQL for that particular endpoint.

### *6. GraphQL Playground Returns 404*

**Problem:** `/graphql/playground` returns a 404 error.

**Cause:** `TINA4_DEBUG` is set to `false`. The playground is only available in debug mode.

**Fix:** Set `TINA4_DEBUG=true` in your `.env` file. The playground is intentionally disabled in production.

### *7. Type Mismatch Between Schema and Resolver*

**Problem:** A field declared as `Int!` in the schema receives a string from the resolver, causing a type error.

**Cause:** PHP does not enforce types as strictly as GraphQL. If your resolver returns `"42"` (a string) for an `Int!` field, GraphQL rejects it.

**Fix:** Cast values to the correct type in your resolvers. Use `(int)`, `(float)`, `(bool)` casts explicitly:

```
return [
    "id" => (int) $product->id,
    "price" => (float) $product->price,
    "inStock" => (bool) $product->inStock
];
```

The `toDict()` method handles this automatically for model properties with type declarations, but computed or derived fields need manual casting.

# Chapter 17: Testing

## 1. Why Tests Matter More Than You Think

It is Friday afternoon. Your client just reported a critical bug in production. You fix it -- one line of code. But did that fix break anything else? You have 47 routes, 12 ORM models, and 3 middleware functions. Manually clicking through every page would take an hour. Running the test suite takes 2 seconds.

```
tina4 test

Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product
    [PASS] test_update_product
    [PASS] test_delete_product
    [PASS] test_list_products_with_filter

  AuthTest
    [PASS] test_login_with_valid_credentials
    [PASS] test_login_with_invalid_password
    [PASS] test_protected_route_without_token
    [PASS] test_protected_route_with_valid_token

  9 tests, 9 passed, 0 failed (0.34s)
```

Everything still works. You deploy with confidence and enjoy your weekend.

Tina4 includes an inline testing framework. No external packages required. No PHPUnit configuration. Write a test, run it, done.

## 2. Your First Test

Tests live in the `tests/` directory. Every `.php` file in that directory is auto-discovered when you run `tina4 test`.

Create `tests/BasicTest.php`:

```php
<?php
use Tina4\Test;

class BasicTest extends Test
{
    public function testAddition()
    {
        $this->assertEqual(2 + 2, 4, "Basic addition should work");
    }

    public function testStringContains()
    {
        $greeting = "Hello, World!";
        $this->assertTrue(str_contains($greeting, "World"), "Greeting should contain 'World'");
    }

    public function testArrayLength()
    {
        $items = [1, 2, 3, 4, 5];
        $this->assertEqual(count($items), 5, "Array should have 5 items");
```

```
    }
}
```

Run it:

```
tina4 test

Running tests...

  BasicTest
    [PASS] test_addition
    [PASS] test_string_contains
    [PASS] test_array_length

  3 tests, 3 passed, 0 failed (0.02s)
```

### *How It Works*

1. Your test class extends `Tina4\Test`.

2. Every method that starts with `test` is a test case. The method name is converted to a readable label: `testAddition` becomes `test_addition`.

3. Inside each test, you call assertion methods to verify behavior.

4. If all assertions pass, the test passes. If any assertion fails, the test fails and you see the failure message.

## 3. Assertion Methods

Tina4's Test class provides these assertion methods:

### *assertEqual($actual, $expected, $message)*

Checks that two values are equal.

```
$this->assertEqual(4, 4, "Should be equal");              // PASS
$this->assertEqual("hello", "hello", "Strings match");    // PASS
$this->assertEqual(4, 5, "Not equal");                     // FAIL
```

### *assertTrue($value, $message)*

Checks that a value is truthy.

```
$this->assertTrue(true, "Should be true");                // PASS
$this->assertTrue(1, "1 is truthy");                      // PASS
$this->assertTrue("yes", "Non-empty string is truthy");   // PASS
$this->assertTrue(false, "This fails");                   // FAIL
$this->assertTrue(0, "Zero is falsy");                    // FAIL
```

### *assertFalse($value, $message)*

Checks that a value is falsy.

```
$this->assertFalse(false, "Should be false");             // PASS
$this->assertFalse(0, "Zero is falsy");                   // PASS
$this->assertFalse("", "Empty string is falsy");          // PASS
$this->assertFalse(true, "This fails");                   // FAIL
```

### *assertRaises($callable, $exceptionClass, $message)*

Checks that a function throws a specific exception.

```
$this->assertRaises(function () {
    throw new \InvalidArgumentException("Bad input");
}, \InvalidArgumentException::class, "Should throw InvalidArgumentException");

$this->assertRaises(function () {
    $result = 10 / 0;
}, \DivisionByZeroError::class, "Should throw on division by zero");
```

### *assertNotEqual($actual, $expected, $message)*

Checks that two values are not equal.

```
$this->assertNotEqual("hello", "world", "Strings differ");  // PASS
$this->assertNotEqual(4, 4, "Same values");                  // FAIL
```

### *assertNull($value, $message)*

Checks that a value is null.

```
$this->assertNull(null, "Should be null");     // PASS
$this->assertNull("hello", "Not null");         // FAIL
```

### *assertNotNull($value, $message)*

Checks that a value is not null.

```
$this->assertNotNull("hello", "Has value");    // PASS
$this->assertNotNull(null, "Is null");          // FAIL
```

## 4. Testing ORM Models

Let us test a Product model. The test creates records, loads them, updates them, and deletes them.

Create `tests/ProductTest.php`:

```
<?php
use Tina4\Test;

class ProductTest extends Test
{
    private ?int $testProductId = null;

    public function testCreateProduct()
    {
        $product = new Product();
        $product->name = "Test Widget";
        $product->category = "Testing";
        $product->price = 19.99;
        $product->inStock = true;
        $product->save();

        $this->assertNotNull($product->id, "Product should have an ID after save");
        $this->assertTrue($product->id > 0, "Product ID should be positive");

        $this->testProductId = $product->id;
    }

    public function testLoadProduct()
    {
        // Create a product to load
// ...
```

Run it:

```
tina4 test

Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product
    [PASS] test_update_product
    [PASS] test_delete_product
    [PASS] test_select_with_filter

  5 tests, 5 passed, 0 failed (0.18s)
```

### *Test Database*

By default, `tina4 test` uses a separate test database so your development data is not affected. The test database is created at `data/test.db` (SQLite) and is reset before each test run. If you want to use a different database for tests, set it in `.env`:

```
TINA4_TEST_DATABASE_URL=sqlite:///data/test.db
```

## 5. Testing Routes

Tina4 provides a test client for making HTTP requests to your routes without starting a server.

Create `tests/RouteTest.php`:

```php
<?php
use Tina4\Test;

class RouteTest extends Test
{
    public function testHealthEndpoint()
    {
        $response = $this->get("/health");

        $this->assertEqual($response->statusCode, 200, "Health check should return 200");

        $body = json_decode($response->body, true);
        $this->assertEqual($body["status"], "ok", "Status should be 'ok'");
        $this->assertNotNull($body["version"], "Should include version");
    }

    public function testGetProducts()
    {
        $response = $this->get("/api/products");

        $this->assertEqual($response->statusCode, 200, "Should return 200");

        $body = json_decode($response->body, true);
        $this->assertTrue(isset($body["data"]) || isset($body["products"]), "Should contain product data");
    }
// ...
```

### *Test Client Methods*

The test client provides methods for all HTTP verbs:

```php
// GET request
$response = $this->get("/api/products");
```

```
// GET with query parameters
$response = $this->get("/api/products?category=Electronics&page=2");

// POST with JSON body
$response = $this->post("/api/products", ["name" => "Widget", "price" => 9.99]);

// PUT with JSON body
$response = $this->put("/api/products/1", ["name" => "Updated Widget"]);

// PATCH with JSON body
$response = $this->patch("/api/products/1", ["price" => 12.99]);

// DELETE
$response = $this->delete("/api/products/1");

// Request with custom headers
$response = $this->get("/api/profile", [
    "Authorization" => "Bearer eyJhbGciOiJIUzI1NiIs..."
]);
```

### *Response Object*

The response object has these properties:

```
$response->statusCode;   // HTTP status code (200, 201, 404, etc.)
$response->body;         // Response body as a string
$response->headers;      // Response headers as an associative array
$response->contentType;  // Content-Type header value
```

## 6. Testing Authentication

Create `tests/AuthTest.php`:

```
<?php
use Tina4\Test;

class AuthTest extends Test
{
    private ?string $token = null;

    public function testLoginWithValidCredentials()
    {
        $response = $this->post("/api/auth/login", [
            "email" => "admin@example.com",
            "password" => "correct-password"
        ]);

        $this->assertEqual($response->statusCode, 200, "Login should succeed");

        $body = json_decode($response->body, true);
        $this->assertNotNull($body["token"], "Should return a JWT token");
        $this->assertTrue(strlen($body["token"]) > 50, "Token should be a substantial string");

        $this->token = $body["token"];
    }

    public function testLoginWithInvalidPassword()
    {
// ...
```

## 7. Setup and Teardown

Use `setUp()` and `tearDown()` methods to run code before and after each test:

```
<?php
use Tina4\Test;

class UserTest extends Test
{
    private ?int $userId = null;

    public function setUp(): void
    {
        // Runs before each test
        $user = new User();
        $user->name = "Test User";
        $user->email = "test-" . time() . "@example.com";
        $user->save();
        $this->userId = $user->id;
    }

    public function tearDown(): void
    {
        // Runs after each test
        if ($this->userId) {
            $user = new User();
            $user->load($this->userId);
            if (!empty($user->id)) {
                $user->delete();
// ...
```

`setUp()` runs before every test method, and `tearDown()` runs after every test method, regardless of whether the test passed or failed. This keeps tests isolated -- each test starts with a clean state.

## 8. Running Tests

### *Run All Tests*

```
tina4 test
```

### *Run a Specific Test File*

```
tina4 test --file tests/ProductTest.php
```

### *Run a Specific Test Method*

```
tina4 test --file tests/ProductTest.php --method testCreateProduct
```

### *Verbose Output*

```
tina4 test --verbose
```

```
Running tests...

  ProductTest
    [PASS] test_create_product (0.03s)
      assertEqual: Product should have an ID after save
      assertTrue: Product ID should be positive
    [PASS] test_load_product (0.02s)
      assertEqual: Name should match
      assertEqual: Category should match
      assertEqual: Price should match
```

```
      assertTrue: Should be in stock by default
    [PASS] test_update_product (0.04s)
      assertEqual: Name should be updated
      assertEqual: Price should be updated
    [PASS] test_delete_product (0.02s)
      assertTrue: Deleted product should not be loadable
    [PASS] test_select_with_filter (0.05s)
      assertTrue: Should find at least 2 FilterCat products
      assertTrue: Should include Filter Test A
      assertTrue: Should include Filter Test B

  5 tests, 5 passed, 0 failed (0.16s)
```

Verbose mode shows each assertion within each test, along with timing information.

### *Failed Test Output*

When a test fails, you see exactly what went wrong:

```
  ProductTest
    [PASS] test_create_product
    [FAIL] test_load_product
      assertEqual FAILED: Name should match
        Expected: "Load Test Widget"
        Actual:   "Wrong Name"
        File: tests/ProductTest.php:34
    [PASS] test_update_product

  3 tests, 2 passed, 1 failed (0.12s)
```

The failure message shows the assertion that failed, what was expected, what was actually received, and the file and line number.

## 9. PHPUnit Integration

If your team already uses PHPUnit, Tina4 tests can run alongside it. Install PHPUnit via Composer:

```
composer require --dev phpunit/phpunit
```

Create `phpunit.xml` at the project root:

```
<?xml version="1.0" encoding="UTF-8"?>
<phpunit bootstrap="vendor/autoload.php"
         colors="true"
         stopOnFailure="false">
    <testsuites>
        <testsuite name="Application">
            <directory>tests</directory>
        </testsuite>
    </testsuites>
</phpunit>
```

Tina4's `Test` class is compatible with PHPUnit. Your test files work with both `tina4 test` and `vendor/bin/phpunit` without modification.

```
vendor/bin/phpunit

PHPUnit 10.5.0 by Sebastian Bergmann and contributors.

.....                                                             5 / 5 (100%)

Time: 00:00.340, Memory: 8.00 MB
```

```
OK (5 tests, 12 assertions)
```

### *Code Coverage*

With PHPUnit, you can generate code coverage reports:

```
vendor/bin/phpunit --coverage-text
```

```
Code Coverage Report:
  Summary:
    Classes:  60.00% (3/5)
    Methods:  75.00% (15/20)
    Lines:    82.14% (115/140)

  Product
    Methods:  100.00% ( 5/ 5)   Lines: 100.00% ( 28/ 28)
  User
    Methods:  66.67% ( 4/ 6)   Lines:  78.95% ( 30/ 38)
```

For HTML coverage reports:

```
vendor/bin/phpunit --coverage-html coverage/
```

Open `coverage/index.html` in your browser to see a visual breakdown of which lines are covered by tests.

Note: Code coverage requires the Xdebug or PCOV extension. Install one of them:

```
# Xdebug
pecl install xdebug

# PCOV (faster for coverage only)
pecl install pcov
```

## 10. Testing Best Practices

### *Test One Thing Per Test*

Each test method should verify one behavior. If it fails, you know exactly what broke.

```
// Good: each test verifies one thing
public function testCreateProductReturns201()
{
    $response = $this->post("/api/products", ["name" => "Widget", "price" => 9.99]);
    $this->assertEqual($response->statusCode, 201, "Should return 201");
}

public function testCreateProductReturnsCreatedProduct()
{
    $response = $this->post("/api/products", ["name" => "Widget", "price" => 9.99]);
    $body = json_decode($response->body, true);
    $this->assertEqual($body["name"], "Widget", "Should return the product name");
}

// Avoid: testing multiple unrelated things
public function testEverything()
{
    // Creates, reads, updates, deletes, checks auth, validates input...
    // When this fails, you don't know which part broke
}
```

### *Use Descriptive Assertion Messages*

```
// Good: tells you what went wrong
```

```
$this->assertEqual($user->name, "Alice", "User name should be 'Alice' after creation");

// Bad: unhelpful when it fails
$this->assertEqual($user->name, "Alice", "Failed");
```

### *Isolate Tests*

Each test should create its own data and clean up after itself. Never depend on data from another test or from the development database.

```
// Good: creates its own data
public function testDeleteProduct()
{
    $product = new Product();
    $product->name = "Temporary";
    $product->price = 1.00;
    $product->save();

    $product->delete();

    $check = new Product();
    $check->load($product->id);
    $this->assertTrue(empty($check->id), "Product should be deleted");
}

// Bad: depends on data from another test or the dev database
public function testDeleteProduct()
{
    $product = new Product();
    $product->load(1);  // Assumes product with ID 1 exists
    $product->delete();
}
```

## 11. Exercise: Test a User Model and Authentication Flow

Write a complete test suite for a User model and authentication system.

### *Requirements*

Create `tests/UserModelTest.php` with these tests:

1. **testCreateUser** -- Create a user with name, email. Verify the user gets an ID and all fields are correct.

2. **testDuplicateEmail** -- Try to create two users with the same email. Verify the second one raises an exception or returns an error.

3. **testUpdateUser** -- Create a user, update their name, reload and verify.

4. **testDeleteUser** -- Create a user, delete them, verify they cannot be loaded.

5. **testSelectUsers** -- Create 3 users, query all, verify count is at least 3.

Create `tests/AuthFlowTest.php` with these tests:

1. **testRegisterNewUser** -- POST to `/api/auth/register` with name, email, password. Verify 201 status.

2. **testRegisterDuplicateEmail** -- Register the same email twice. Verify the second returns 409 Conflict.

3. **testLoginSuccess** -- Login with correct credentials. Verify you get a JWT token.

4. **testLoginFailure** -- Login with wrong password. Verify 401 status.

5. **testAccessProtectedRoute** -- Use the token from login to access `/api/profile`. Verify 200 status and correct user data.

6. **testAccessWithExpiredToken** -- Use a manually crafted expired token. Verify 401 status.

### *Expected output:*

```
tina4 test

Running tests...

  UserModelTest
    [PASS] test_create_user
    [PASS] test_duplicate_email
    [PASS] test_update_user
    [PASS] test_delete_user
    [PASS] test_select_users

  AuthFlowTest
    [PASS] test_register_new_user
    [PASS] test_register_duplicate_email
    [PASS] test_login_success
    [PASS] test_login_failure
    [PASS] test_access_protected_route
    [PASS] test_access_with_expired_token

  11 tests, 11 passed, 0 failed (0.52s)
```

## 12. Solution

### *tests/UserModelTest.php*

```php
<?php
use Tina4\Test;

class UserModelTest extends Test
{
    public function testCreateUser()
    {
        $user = new User();
        $user->name = "Test User";
        $user->email = "testuser-" . uniqid() . "@example.com";
        $user->save();

        $this->assertNotNull($user->id, "User should have an ID after save");
        $this->assertEqual($user->name, "Test User", "Name should match");
        $this->assertTrue(str_contains($user->email, "@example.com"), "Email should be set");
    }

    public function testDuplicateEmail()
    {
        $email = "duplicate-" . uniqid() . "@example.com";

        $user1 = new User();
        $user1->name = "First User";
        $user1->email = $email;
        $user1->save();
// ...
```

### *tests/AuthFlowTest.php*

```php
<?php
use Tina4\Test;

class AuthFlowTest extends Test
{
    private string $testEmail;
    private string $testPassword = "SecurePassword123!";

    public function setUp(): void
    {
        $this->testEmail = "auth-test-" . uniqid() . "@example.com";
    }

    public function testRegisterNewUser()
    {
        $response = $this->post("/api/auth/register", [
            "name" => "Auth Test User",
            "email" => $this->testEmail,
            "password" => $this->testPassword
        ]);

        $this->assertEqual($response->statusCode, 201, "Registration should return 201");

        $body = json_decode($response->body, true);
        $this->assertEqual($body["name"], "Auth Test User", "Should return user name");
// ...
```

## 13. Gotchas

### *1. Tests Run in Order*

**Problem:** Test B depends on data created in Test A, but sometimes Test B fails.

**Cause:** While tests within a class run in the order they are defined, test classes may run in any order. If Test B depends on Test A being in a different class, this is fragile.

**Fix:** Each test should create its own data. Never depend on another test's side effects. Use `setUp()` to create the data each test needs.

### *2. Test Database vs Development Database*

**Problem:** Running tests deletes your development data.

**Cause:** Tests are running against the same database as your development server.

**Fix:** Tina4 uses a separate test database by default (`data/test.db`). If your tests are modifying development data, check that `TINA4_TEST_DATABASE_URL` is set correctly in `.env`, or that you are running `tina4 test` (not executing test files manually).

### *3. Unique Constraint Failures*

**Problem:** Tests fail with "UNIQUE constraint failed" on the email field.

**Cause:** Previous test runs left data in the test database, or two tests create records with the same email.

**Fix:** Use `uniqid()` or `time()` to generate unique values in tests:

```php
$user->email = "test-" . uniqid() . "@example.com";
```

### *4. Test Method Not Discovered*

**Problem:** You wrote a test method but it does not appear in the output.

**Cause:** The method name does not start with `test`. Tina4 only runs methods that begin with `test` (case-sensitive).

**Fix:** Rename the method to start with `test`: `testCreateProduct`, `testLoginFailure`, etc.

### *5. Assertion Arguments Reversed*

**Problem:** The failure message shows "Expected: 404, Actual: 200" but that seems backward.

**Cause:** You passed the arguments in the wrong order. `assertEqual($actual, $expected)` -- actual comes first, expected comes second.

**Fix:** Follow the convention: `$this->assertEqual($response->statusCode, 200, "message")`. The first argument is what you got, the second is what you expected.

### *6. Cannot Test Routes That Require Authentication Setup*

**Problem:** Tests for protected routes fail because the authentication middleware expects a database table or JWT secret that does not exist in the test environment.

**Cause:** The test database is empty -- it does not have the users table or the JWT secret is not configured.

**Fix:** Use `setUp()` to create the necessary data:

```
public function setUp(): void
{
    // Create the users table if it does not exist
    $user = new User();
    $user->createTable();

    // Create a test user
    $user->name = "Test Admin";
    $user->email = "admin@test.com";
    $user->passwordHash = password_hash("password", PASSWORD_DEFAULT);
    $user->save();
}
```

### *7. Tests Pass Locally but Fail in CI*

**Problem:** All tests pass on your machine but fail in continuous integration.

**Cause:** The CI environment may have a different PHP version, missing extensions, or different filesystem permissions. Also, tests that depend on time or random values can be flaky.

**Fix:** Make tests deterministic. Avoid relying on the current time, filesystem state, or external services. If you must test time-dependent behavior, mock the clock or use fixed timestamps. Check that your CI environment matches your local PHP version and has all required extensions.

# Chapter 18: Dev Tools

## 1. Debugging at 2am

It is 2am. Your production monitoring just pinged you -- a 500 error on the checkout endpoint. You pull up the dev dashboard, find the failing request in the request inspector, and see the full stack trace with source code context. Line 47 of `src/routes/checkout.php` -- a null reference on the shipping address because the user did not fill in the form. You add a null check, push the fix, and go back to sleep. Total time: 30 seconds.

Tina4's dev tools are not an afterthought bolted on later. They are built into the framework from day one. When `TINA4_DEBUG=true`, you get a full development dashboard, an error overlay with source code, live reload, a request inspector, a SQL query runner, and more -- all without installing any additional packages.

## 2. Enabling the Dev Dashboard

The dev dashboard is available when you enable the console in your `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=my-dev-token
```

Restart your server and navigate to:

```
http://localhost:7145/tina4/console
```

Enter your console token (`my-dev-token`) when prompted. You are now in the dev dashboard.

The console token protects the dashboard from unauthorized access. In development, any simple string works. In shared environments, use a strong random token. In production, set `TINA4_DEBUG=false` and the entire dashboard is disabled.

## 3. Dashboard Overview

The dev dashboard has several sections, accessible from the navigation tabs at the top:

### *System Overview*

The landing page shows at a glance:

• **Framework version** -- The installed Tina4 PHP version

• **PHP version** -- The running PHP version and loaded extensions

• **Uptime** -- How long the server has been running

• **Memory usage** -- Current and peak memory consumption

• **Database status** -- Connection status, database engine, file size (for SQLite)

• **Environment** -- Current `.env` variables (sensitive values are masked)

• **Project structure** -- Directory listing of your project with file counts

This is the first thing to check when something feels off. Is the database connected? Is the right PHP version running? Are the environment variables loaded?

## 4. The Dev Toolbar

When you visit any HTML page in your application (like `/products` or `/admin`), a debug toolbar appears at the bottom of the page. It is a thin bar that expands when you click on it.

The toolbar shows:

| Field | What It Means |

|-------|--------------|

| **Request** | HTTP method and URL of the current request |

| **Status** | HTTP response status code |

| **Time** | Total request processing time in milliseconds |

| **DB** | Number of database queries executed and total query time |

| **Memory** | PHP memory used for this request |

| **Template** | The template rendered and how long rendering took |

| **Session** | Session ID and session data summary |

| **Route** | Which route handler matched this request |

Click any section to expand it and see details. For example, clicking "DB" shows every SQL query that ran during the request, with the query text, parameters, execution time, and number of rows returned.

### *Disabling the Toolbar*

The toolbar is automatically hidden when `TINA4_DEBUG=false`. You can also hide it for specific routes by returning a response with the `X-Debug-Toolbar: off` header:

```
Route::get("/api/data", function ($request, $response) {
    return $response->json(["data" => "no toolbar"], 200, [
        "X-Debug-Toolbar" => "off"
    ]);
});
```

This is useful for API endpoints that return JSON -- the toolbar is only meaningful for HTML pages.

## 5. Error Overlay

When an unhandled exception occurs, Tina4 does not show a generic "500 Internal Server Error" page. Instead, it shows a detailed error overlay with:

• **Exception type and message** -- What went wrong, in plain language

• **Stack trace** -- Every function call that led to the error, from the entry point to the crash

• **Source code** -- The actual PHP code around the line that threw the exception, with the failing line highlighted

• **Request data** -- The HTTP method, URL, headers, body, and query parameters of the request that triggered the error

• **Environment** -- Relevant `.env` variables at the time of the error

### *Example Error*

If you accidentally write:

```
Route::get("/api/broken", function ($request, $response) {
    $user = null;
    return $response->json(["name" => $user->name]);
});
```

Visiting `/api/broken` shows the error overlay:

```
TypeError: Cannot access property "name" on null

  File: src/routes/api.php
  Line: 4

  2 | Route::get("/api/broken", function ($request, $response) {
  3 |     $user = null;
> 4 |     return $response->json(["name" => $user->name]);
  5 | });

  Stack Trace:
    #0 src/routes/api.php:4 — {closure}()
    #1 vendor/tina4/tina4-php/src/Router.php:142 — call_user_func()
    #2 vendor/tina4/tina4-php/src/Server.php:89 — Router->dispatch()

  Request: GET /api/broken
  Headers: Accept: */*
  Query: (none)
  Body: (none)
```

The highlighted line (line 4) makes it immediately obvious: `$user` is null and you are trying to access `->name` on it.

### *Error Overlay in Production*

When `TINA4_DEBUG=false`, the error overlay is disabled. Instead, users see a clean error page (`src/templates/errors/500.html` if it exists, or a generic message). The full error details are written to the log file at `logs/error.log`.

## 6. Request Inspector

The request inspector in the dev dashboard records every HTTP request to your application. It shows:

• **Method and URL** -- GET /api/products, POST /api/users, etc.

• **Status code** -- 200, 201, 404, 500, etc. (color-coded: green for 2xx, yellow for 4xx, red for 5xx)

• **Response time** -- How many milliseconds the request took

• **Request ID** -- A unique identifier for correlating logs

• **Timestamp** -- When the request was received

Click on any request to see its full details:

### *Request Details Panel*

• **Headers** -- All request headers (Accept, Content-Type, Authorization, etc.)

• **Body** -- The request body (for POST/PUT/PATCH), formatted as JSON if applicable

• **Query parameters** -- Parsed URL query parameters

• **Route match** -- Which route definition matched this request

• **Middleware** -- Which middleware ran and how long each took

• **Database queries** -- Every SQL query executed during this request, with timing

• **Template renders** -- Which templates were rendered and how long each took

• **Response headers** -- The response headers sent back to the client

• **Response body** -- The first 1000 characters of the response body

### *Filtering Requests*

The inspector supports filtering:

• **By status**: Click the status code badges at the top to filter (e.g., show only 5xx errors)

• **By method**: Filter by GET, POST, PUT, DELETE

• **By path**: Search for a URL pattern (e.g., `/api/` to show only API requests)

• **By time range**: Show requests from the last 5 minutes, 1 hour, or all time

### *Request Replay*

Click the "Replay" button on any request to re-send the exact same request. This is invaluable for debugging -- you can reproduce an error without manually constructing the curl command.

## 7. SQL Query Runner

The dev dashboard includes an interactive SQL query runner. Navigate to the "SQL" tab and you can:

• **Execute queries** directly against your database

• **See results** in a formatted table

• **View query timing** -- how long each query took

• **Browse tables** -- a sidebar lists all tables in your database with their column definitions

• **Export results** -- download query results as CSV

### *Example*

Type into the query editor:

```
SELECT name, price, in_stock FROM products WHERE price > 50 ORDER BY price DESC
```

Click "Run" and see:

```
Results (3 rows, 2.1ms):

| name              | price  | in_stock |
```

```
|-------------------|--------|----------|
| Standing Desk     | 549.99 | 0        |
| Ergonomic Chair   | 399.99 | 1        |
| Wireless Keyboard | 79.99  | 1        |
```

This is faster than opening a separate database client. You can test queries, check data, and debug issues without leaving the browser.

### *Safety*

The query runner is read-write in development. You can run INSERT, UPDATE, and DELETE statements. Be careful -- there is no undo. In shared environments, consider using a read-only database user for the dev dashboard.

## 8. Live Reload

When you edit a file and save it, the browser updates automatically. No manual refresh needed.

### *How It Works*

1. The dev server watches your project directory for file changes (routes, templates, ORM models, static files).

2. When a file changes, the server pushes a notification to the browser via WebSocket.

3. The browser receives the notification and reloads the page (for route/template changes) or hot-swaps the resource (for CSS/JS changes).

### *What Triggers a Reload*

| File Type | Action |

|----------|--------|

| `src/routes/*.php` | Full page reload |

| `src/orm/*.php` | Full page reload |

| `src/templates/*.html` | Full page reload |

| `src/public/css/*.css` | CSS hot-swap (no reload) |

| `src/public/js/*.js` | Full page reload |

| `src/public/images/*` | No automatic reload |

| `.env` | Server restart required |

CSS hot-swapping is seamless -- your page layout stays the same, only the styles update. This is great when you are tweaking colors, spacing, or fonts.

### *Disabling Live Reload*

If live reload interferes with your workflow (e.g., you are filling out a form and do not want it to reset), disable it in `.env`:

```
TINA4_LIVE_RELOAD=false
```

## 9. Hot-Patching

Hot-patching takes live reload a step further. When you edit a route handler or template, the dev server applies the change without restarting. This means:

• The server stays up

• Active WebSocket connections are preserved

• In-memory state (like queue workers) is not reset

• The change takes effect on the next request

Hot-patching works because Tina4's dev server re-reads route files and templates on every request when `TINA4_DEBUG=true`. It is not a separate feature to enable -- it is the default development behavior.

In production (`TINA4_DEBUG=false`), routes and templates are cached for performance. Changes require a server restart.

## 10. Gallery -- Interactive Examples

The dev dashboard includes a Gallery section where you can deploy interactive code examples. This is useful for:

• Demonstrating API endpoints to team members

• Creating runnable documentation

• Prototyping features quickly

### *Creating a Gallery Entry*

Create a file in `src/gallery/`:

```
<?php
// src/gallery/product-search.php
// title: Product Search
// description: Search products by name and category

use Tina4\Route;

Route::get("/gallery/product-search", function ($request, $response) {
    $q = $request->query["q"] ?? "";
    $category = $request->query["category"] ?? "";

    $product = new Product();
    $conditions = [];
    $params = [];

    if (!empty($q)) {
        $conditions[] = "name LIKE :q";
        $params["q"] = "%" . $q . "%";
    }

    if (!empty($category)) {
        $conditions[] = "category = :category";
        $params["category"] = $category;
    }

// ...
```

The comments at the top (`// title:` and `// description:`) are parsed by the gallery and shown in the dashboard. Navigate to the Gallery tab to see all your interactive examples, with a "Try It" button that opens the endpoint in a new tab.

## 11. Queue Monitor

If your application uses background job queues (Chapter 14 -- Queues), the dev dashboard includes a queue monitor showing:

• **Pending jobs** -- Jobs waiting to be processed

• **Active jobs** -- Jobs currently being processed

• **Completed jobs** -- Recently completed jobs with timing

• **Failed jobs** -- Jobs that threw exceptions, with error details

• **Dead-letter queue** -- Jobs that failed too many times and were moved to the dead-letter queue

For each job, you can see:

• The job class or function name

• The payload (serialized arguments)

• When it was enqueued

• When it started processing (if active)

• The error message and stack trace (if failed)

• How many times it has been retried

You can also:

• **Retry a failed job** -- Click "Retry" to move it back to the pending queue

• **Delete a job** -- Remove it from any queue

• **Pause/resume the queue** -- Temporarily stop processing without losing jobs

## 12. System Info

The System Info tab shows detailed information about your environment:

• **PHP Configuration** -- Version, loaded extensions, php.ini location, memory limit, max execution time, upload limits

• **Database Info** -- Engine, version, connection details, table sizes, index information

• **Server Info** -- OS, hostname, server software, document root

• **Tina4 Config** -- All loaded `.env` variables (sensitive values masked), auto-discovered routes, registered middleware, ORM models

• **Disk Usage** -- Size of your project directory, data directory, logs directory, and vendor directory

This is especially useful when debugging environment-specific issues. If a colleague says "it works on my machine," compare the System Info output between both environments to spot differences.

## 13. Exercise: Debug a Failing Route

Set up a project with the following intentionally broken route, then use the dev tools to find and fix the bug.

### *Setup*

Create `src/routes/orders.php` with this intentionally broken code:

```
<?php
use Tina4\Route;

Route::get("/api/orders/summary", function ($request, $response) {
    $product = new Product();
    $products = $product->select("*");

    $total = 0;
    foreach ($products as $p) {
        $total += $p->price * $p->quantity; // Bug: Product has no "quantity" field
    }

    $orderCount = count($products);
    $averageValue = $total / $orderCount; // Bug: division by zero if no products

    return $response->json([
        "total_value" => $total,
        "order_count" => $orderCount,
        "average_value" => round($averageValue, 2)
    ]);
});
```

### *Task*

1. Start the dev server with `tina4 serve`.

2. Visit `http://localhost:7145/api/orders/summary`.

3. Observe the error overlay.

4. Use the error overlay to identify the exact line and cause of the error.

5. Fix both bugs:

• Replace `$p->quantity` with `1` (or remove the multiplication) since products do not have a quantity field.

• Add a check for zero `$orderCount` before dividing.

6. Reload the page and verify the response is correct.

7. Open the dev dashboard at `/tina4/console` and find the failed request in the request inspector.

8. Verify the fixed request now appears as a 200 in the inspector.

### *Expected Steps*

1. The error overlay shows: `Undefined property: Product::$quantity` on the line `$total += $p->price * $p->quantity;`

2. You fix the line to `$total += $p->price;`

3. You reload and get a new error (if there are zero products): `Division by zero`

4. You add `if ($orderCount > 0)` before the division

5. The endpoint returns valid JSON

## 14. Solution

The fixed route:

```
<?php
use Tina4\Route;

Route::get("/api/orders/summary", function ($request, $response) {
    $product = new Product();
    $products = $product->select("*");

    $total = 0.0;
    foreach ($products as $p) {
        $total += $p->price;
    }

    $orderCount = count($products);
    $averageValue = $orderCount > 0 ? $total / $orderCount : 0;

    return $response->json([
        "total_value" => round($total, 2),
        "order_count" => $orderCount,
        "average_value" => round($averageValue, 2)
    ]);
});
```

**Expected output (with 5 seeded products):**

```
{
  "total_value": 909.95,
  "order_count": 5,
  "average_value": 181.99
}
```

### *What You Practiced*

• Reading the error overlay to find the exact line and error type

• Understanding stack traces

• Using the request inspector to view failed and successful requests

• Iterative debugging -- fixing one error, reloading, fixing the next

## 15. Gotchas

### *1. Dev Dashboard Returns 404*

**Problem:** Navigating to `/tina4/console` returns a 404 page.

**Cause:** Either `TINA4_CONSOLE` is not set to `true`, or `TINA4_DEBUG` is `false`.

**Fix:** Add both to your `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=your-token
```

Restart the server after changing `.env`.

### 2. Console Token Not Accepted

**Problem:** You enter the token but the dashboard rejects it.

**Cause:** The token in your `.env` has extra whitespace or quotes. Environment values should not be quoted in `.env` files.

**Fix:** Check your `.env`:

```
# Wrong
TINA4_CONSOLE_TOKEN="my-token"

# Right
TINA4_CONSOLE_TOKEN=my-token
```

### 3. Live Reload Not Working

**Problem:** You save a file but the browser does not reload.

**Cause:** Live reload uses WebSocket. If your browser blocks WebSocket connections (some corporate proxies do), or if you are accessing the site via a reverse proxy that does not forward WebSocket, live reload will not work.

**Fix:** Check the browser console for WebSocket errors. If you are behind a proxy, configure it to forward WebSocket connections. As a workaround, manually refresh with `Ctrl+R`.

### 4. Error Overlay Shows in Production

**Problem:** Users see stack traces and source code on error pages.

**Cause:** `TINA4_DEBUG=true` in the production `.env`.

**Fix:** Always set `TINA4_DEBUG=false` in production. This hides all debug information and shows the custom error page from `src/templates/errors/500.html` instead.

### 5. Request Inspector Shows Too Many Requests

**Problem:** The request inspector is flooded with entries, making it hard to find the one you care about.

**Cause:** Static file requests (CSS, JS, images, favicon) are all recorded.

**Fix:** Use the path filter to narrow down the list. Type `/api/` in the filter box to show only API requests. You can also set `TINA4_CONSOLE_IGNORE_STATIC=true` in `.env` to exclude static file requests from the inspector.

### 6. SQL Runner Modifies Data Accidentally

**Problem:** You ran a DELETE query in the SQL runner and lost data.

**Cause:** The SQL runner executes queries directly against the database with full read-write access.

**Fix:** There is no undo. For safety, always include a WHERE clause with DELETE and UPDATE statements. If you are worried about accidental modifications, use a read-only database connection for the dev dashboard by setting `TINA4_CONSOLE_READ_ONLY=true` in `.env`.

### *7. Debug Toolbar Breaks Layout*

**Problem:** The debug toolbar at the bottom of the page overlaps with your content or breaks the layout.

**Cause:** The toolbar adds a fixed-position element at the bottom of the page. If your page uses `position: fixed` elements at the bottom (like a footer or chat widget), they may conflict.

**Fix:** The toolbar adds a `data-tina4-debug` attribute to the body. Use this in your CSS to adjust:

```
body[data-tina4-debug] .my-footer {
    bottom: 40px; /* Make room for the debug toolbar */
}
```

Alternatively, collapse the toolbar by clicking on it -- it minimizes to a thin line.

# Chapter 19: CLI & Scaffolding

## 1. Getting a New Developer Up to Speed

A new developer joins your team on Monday morning. You hand them the repo URL, and by 10am they have a running project, a new database model, CRUD routes, a migration, and a deployment to staging. All from the command line. No hunting through documentation. No copy-pasting boilerplate from old projects. The Tina4 CLI handles the scaffolding.

The Tina4 CLI is a single Rust binary that manages all four Tina4 frameworks (PHP, Python, Ruby, Node.js). The commands are identical across languages. Once you learn the CLI for PHP, you know it for Python too.

## 2. tina4 init -- Project Scaffolding

You saw this in Chapter 1, but let us look at it in detail.

```
tina4 init my-project

Creating Tina4 project in ./my-project ...
  Detected language: PHP (composer.json)
  Created .env
  Created .env.example
  Created .gitignore
  Created src/routes/
  Created src/orm/
  Created src/migrations/
  Created src/seeds/
  Created src/templates/
  Created src/templates/errors/
  Created src/public/
  Created src/public/js/
  Created src/public/css/
  Created src/public/scss/
  Created src/public/images/
  Created src/public/icons/
  Created src/locales/
  Created data/
  Created logs/
  Created secrets/
  Created tests/

Project created! Next steps:
  cd my-project
// ...
```

### *Language Detection*

The CLI detects the language from existing files in the directory:

| File Present | Language |

|----------|----------|

| `composer.json` | PHP |

| `pyproject.toml` or `requirements.txt` | Python |

| `Gemfile` | Ruby |

| `package.json` | Node.js |

If no language-specific file exists, the CLI asks you:

```
tina4 init my-project

No language detected. Which language?
  1. PHP
  2. Python
  3. Ruby
  4. Node.js
> 1

Creating Tina4 PHP project in ./my-project ...
```

### *Init into an Existing Directory*

If you already have a project and want to add Tina4 structure:

```
cd existing-project
tina4 init .
```

The CLI only creates files and directories that do not already exist. It never overwrites existing files.

## 3. tina4 serve -- Dev Server

```
tina4 serve

 _____ _              _  _
|_   _(_)_ __   __ _| || |
  | | | | '_ \ / _` | || |_
  | | | | | | | (_| |__   _|
  |_| |_|_| |_|\__,_|  |_|

  Tina4 PHP v3.0.0
  Server running at http://0.0.0.0:7145
  Debug mode: ON
  Database: sqlite:///data/app.db
  Press Ctrl+C to stop
```

### *Options*

| Flag | Description | Example |

|------|----------|---------|

| `--port` | Custom port (default: 7145) | `tina4 serve --port 8080` |

| `--host` | Bind address (default: 0.0.0.0) | `tina4 serve --host 127.0.0.1` |

| `--no-reload` | Disable live reload | `tina4 serve --no-reload` |

### *Production Mode Detection*

```
tina4 serve --production
```

When the `--production` flag is passed, the CLI:

1. Checks for `TINA4_DEBUG=false` in `.env` (warns if debug is on)

2. Uses FrankenPHP if available for better performance

3. Enables response caching and template pre-compilation

4. Disables the dev toolbar and error overlay

5. Enables graceful shutdown handling

In practice, you rarely use `tina4 serve --production` directly. Instead, you use Docker or a process manager (Chapter 20). But this flag is useful for quick production testing.

## 4. tina4 generate model -- ORM Scaffolding

Generate a new ORM model:

```
tina4 generate model Order
```

```
Created src/orm/Order.php
Created src/migrations/20260322100000_create_orders_table.sql
```

The generated model:

```
<?php
use Tina4\ORM;

class Order extends ORM
{
    public int $id;
    public string $createdAt;
    public string $updatedAt;

    public string $tableName = "orders";
    public string $primaryKey = "id";
}
```

The generated migration:

```
-- UP
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS orders;
```

### *Adding Fields*

You can specify fields on the command line:

```
tina4 generate model Order --fields "userId:int,total:float,status:string,paid:bool"
```

```
Created src/orm/Order.php
Created src/migrations/20260322100000_create_orders_table.sql
```

The generated model now includes all the fields:

```
<?php
use Tina4\ORM;

class Order extends ORM
{
    public int $id;
    public int $userId;
```

```
    public float $total;
    public string $status;
    public bool $paid;
    public string $createdAt;
    public string $updatedAt;

    public string $tableName = "orders";
    public string $primaryKey = "id";
}
```

And the migration:

```
-- UP
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    total REAL NOT NULL DEFAULT 0,
    status TEXT NOT NULL DEFAULT '',
    paid INTEGER NOT NULL DEFAULT 0,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- DOWN
DROP TABLE IF EXISTS orders;
```

### *Field Types*

| CLI Type | PHP Type | SQLite Column |

|----------|----------|---------------|

| `string` | `string` | `TEXT` |

| `int` | `int` | `INTEGER` |

| `float` | `float` | `REAL` |

| `bool` | `bool` | `INTEGER` |

| `text` | `string` | `TEXT` |

| `date` | `string` | `TEXT` |

### *Options*

| Flag | Description | Example |

|------|-------------|---------|

| `--fields` | Comma-separated field definitions | `--fields "name:string,price:float"` |

| `--auto-crud` | Enable auto-CRUD on the model | `--auto-crud` |

| `--soft-delete` | Add soft delete support | `--soft-delete` |

| `--no-migration` | Skip migration generation | `--no-migration` |

## 5. tina4 generate route -- CRUD Route Scaffolding

Generate CRUD routes for a model:

```
tina4 generate route Order
```

```
Created src/routes/orders.php
```

The generated route file:

```
<?php
use Tina4\Route;

Route::group("/api", function () {

    // List all orders
    Route::get("/orders", function ($request, $response) {
        $order = new Order();
        $page = (int) ($request->query["page"] ?? 1);
        $perPage = (int) ($request->query["per_page"] ?? 20);
        $offset = ($page - 1) * $perPage;

        $orders = $order->select("*", "", [], "created_at DESC", $perPage, $offset);
        $results = array_map(fn($o) => $o->toDict(), $orders);

        return $response->json([
            "data" => $results,
            "page" => $page,
            "per_page" => $perPage,
            "count" => count($results)
        ]);
    });

    // Get a single order
    Route::get("/orders/{id:int}", function ($request, $response) {
// ...
```

The generator reads the model's properties and creates appropriate routes with type casting and null checks. You can customize the generated code immediately -- it is regular PHP, not magic.

### *Options*

| Flag | Description | Example |

|------|----------|---------|

| `--prefix` | Custom route prefix (default: `/api`) | `--prefix /api/v2` |

| `--middleware` | Add middleware to all routes | `--middleware requireAuth` |

## 6. tina4 generate migration -- Migration Scaffolding

Create an empty migration file:

```
tina4 generate migration "add email to orders"
```

```
Created src/migrations/20260322101500_add_email_to_orders.sql
```

The generated file:

```
-- UP
-- Add your forward migration SQL here


-- DOWN
-- Add your rollback migration SQL here
```

You fill in the SQL yourself:

```
-- UP
ALTER TABLE orders ADD COLUMN email TEXT DEFAULT '';

-- DOWN
ALTER TABLE orders DROP COLUMN email;
```

The timestamp prefix (`20260322101500`) ensures migrations run in order. Each migration runs exactly once -- the framework tracks which migrations have been applied.

## 7. tina4 generate middleware -- Middleware Scaffolding

```
tina4 generate middleware rateLimiter
```

```
Created src/middleware/rateLimiter.php
```

The generated file:

```
<?php

function rateLimiter($request, $response, $next)
{
    // Add your middleware logic here

    // Continue to the next middleware or route handler
    return $next($request, $response);

    // Or return early to block the request:
    // return $response->json(["error" => "Rate limit exceeded"], 429);
}
```

The middleware is a named function that you can reference in route definitions:

```
Route::get("/api/data", function ($request, $response) {
    return $response->json(["data" => "protected"]);
}, "rateLimiter");
```

## 8. tina4 doctor -- Environment Health Check

```
tina4 doctor
```

```
Tina4 Doctor - Environment Health Check

  PHP Version ............... 8.3.4       [OK]
  Composer ................. 2.7.2       [OK]
  ext-json ................. loaded      [OK]
  ext-mbstring ............. loaded      [OK]
  ext-openssl .............. loaded      [OK]
  ext-sqlite3 .............. loaded      [OK]
  ext-fileinfo ............. loaded      [OK]
  ext-pdo_sqlite ........... loaded      [OK]
  .env file ................ found       [OK]
  data/ directory .......... writable    [OK]
  logs/ directory .......... writable    [OK]
  secrets/ directory ....... writable    [OK]
  Database connection ...... connected   [OK]
  Routes discovered ........ 14 routes   [OK]
  ORM models discovered .... 5 models    [OK]
  Templates directory ...... found       [OK]
  tina4.css ................ found       [OK]
  frond.js ................. found       [OK]
```

```
All checks passed. Your environment is ready.
```

If something is wrong, the doctor tells you:

```
ext-mbstring ............. missing     [FAIL]
  Fix: apt install php8.3-mbstring (Ubuntu)
       brew install php (macOS, includes mbstring)

data/ directory .......... not writable [FAIL]
  Fix: chmod 755 data/

Database connection ...... failed      [FAIL]
  Error: could not open database file
  Fix: Check DATABASE_URL in .env. Current value: sqlite:///data/app.db
       Make sure the data/ directory exists and is writable.
```

The doctor checks everything a new developer might get wrong and provides specific fix instructions for each issue.

## 9. tina4 test -- Running Tests

```
tina4 test
```

```
Running tests...

  ProductTest
    [PASS] test_create_product
    [PASS] test_load_product

  AuthTest
    [PASS] test_login

  3 tests, 3 passed, 0 failed (0.21s)
```

See Chapter 17 for the full testing guide. The CLI command discovers all test files in `tests/` and runs them.

### *Options*

| Flag | Description | Example |

|------|----------|---------|

| `--file` | Run a specific test file | `--file tests/ProductTest.php` |

| `--method` | Run a specific test method | `--method testCreateProduct` |

| `--verbose` | Show all assertions | `--verbose` |

## 10. tina4 routes -- List All Routes

```
tina4 routes
```

```
Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /health                       -                   public
GET      /api/products                 -                   public
GET      /api/products/{id:int}        -                   public
POST     /api/products                 -                   secured
```

```
PUT      /api/products/{id:int}        -                    secured
DELETE   /api/products/{id:int}        -                    secured
GET      /api/orders                   -                    public
POST     /api/orders                   requireAuth          secured
GET      /admin                        -                    public
GET      /admin/users                  -                    public
```

### *Filtering*

```
# Filter by HTTP method
tina4 routes --method POST

# Filter by path pattern
tina4 routes --filter orders

# Filter by middleware
tina4 routes --middleware requireAuth
```

This is invaluable when debugging routing issues. If a route is not matching, check `tina4 routes` to see if it was registered and what middleware is attached.

## 11. tina4 migrate -- Run Database Migrations

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322100000_create_orders_table.sql
  [APPLIED] 20260322101500_add_email_to_orders.sql
  [SKIPPED] 20260322080000_create_products_table.sql (already applied)
Migrations complete. 2 applied, 1 skipped.
```

### *Options*

| Flag | Description | Example |

|------|----------|---------|

| `--rollback` | Undo the last migration | `tina4 migrate --rollback` |

| `--rollback-all` | Undo all migrations | `tina4 migrate --rollback-all` |

| `--status` | Show migration status | `tina4 migrate --status` |

### *Migration Status*

```
tina4 migrate --status
```

```
Migration Status:

  [APPLIED]  20260322080000_create_products_table.sql     (applied 2026-03-22 08:15:00)
  [APPLIED]  20260322100000_create_orders_table.sql       (applied 2026-03-22 10:05:00)
  [APPLIED]  20260322101500_add_email_to_orders.sql       (applied 2026-03-22 10:20:00)
  [PENDING]  20260322120000_add_status_to_products.sql    (not yet applied)

  3 applied, 1 pending
```

## 12. Exercise: Scaffold a Complete Feature in 5 Commands

You need to add a "Tasks" feature to your project. Each task has a title, description, priority (string), completed status (boolean), and belongs to a user.

### *Requirements*

Using only CLI commands, do the following:

1. Generate the Task model with fields
2. Generate CRUD routes for the Task model
3. Run the migration to create the table
4. Verify the routes are registered
5. Run the doctor to make sure everything is healthy

### *Expected Commands*

Complete this in exactly 5 commands. After running them, test the API with curl to verify everything works.

### *Test with:*

```
# Create a task
curl -X POST http://localhost:7145/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Write chapter 19", "description": "CLI and scaffolding", "priority": "high", "user_id": 1}'

# List tasks
curl http://localhost:7145/api/tasks

# Get one task
curl http://localhost:7145/api/tasks/1
```

## 13. Solution

The five commands:

```
# 1. Generate the model with fields (creates model + migration)
tina4 generate model Task --fields "userId:int,title:string,description:text,priority:string,completed:bool"

# 2. Generate CRUD routes
tina4 generate route Task

# 3. Run the migration
tina4 migrate

# 4. Verify routes are registered
tina4 routes --filter tasks

# 5. Health check
tina4 doctor
```

**Command 1 output:**

```
Created src/orm/Task.php
Created src/migrations/20260322140000_create_tasks_table.sql
```

**Command 2 output:**

```
Created src/routes/tasks.php
```

**Command 3 output:**

```
Running migrations...
  [APPLIED] 20260322140000_create_tasks_table.sql
Migrations complete. 1 applied.
```

**Command 4 output:**

```
Method   Path                          Middleware          Auth
------   ----                          ----------          ----
GET      /api/tasks                    -                   public
GET      /api/tasks/{id:int}           -                   public
POST     /api/tasks                    -                   secured
PUT      /api/tasks/{id:int}           -                   secured
DELETE   /api/tasks/{id:int}           -                   secured
```

**Command 5 output:**

```
Tina4 Doctor - Environment Health Check
  ...
  All checks passed. Your environment is ready.
```

**Test - create a task:**

```
curl -X POST http://localhost:7145/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Write chapter 19", "description": "CLI and scaffolding", "priority": "high", "user_id": 1}'
```

```
{
  "id": 1,
  "user_id": 1,
  "title": "Write chapter 19",
  "description": "CLI and scaffolding",
  "priority": "high",
  "completed": false,
  "created_at": "2026-03-22 14:05:00",
  "updated_at": "2026-03-22 14:05:00"
}
```

From zero to a working CRUD API in 5 commands and under 2 minutes.

# 14. Gotchas

### *1. Model Name Must Be PascalCase*

**Problem:** `tina4 generate model order_item` creates a model class named `order_item` which is not valid PHP.

**Cause:** The CLI uses the argument as-is for the class name if it cannot determine the PascalCase form.

**Fix:** Always use PascalCase for model names: `tina4 generate model OrderItem`. The CLI converts it to snake_case for the table name (`order_items`).

### *2. Migration Already Applied*

**Problem:** You regenerated a model and the migration has the same name as an existing one, but the CLI says it was already applied.

**Cause:** Tina4 tracks applied migrations by filename. If the filename matches, it considers it already applied.

**Fix:** Each migration should have a unique timestamp prefix. If you need to modify a table after the initial migration, create a new migration: `tina4 generate migration "add status to tasks"`.

### *3. Generated Routes Conflict with Auto-CRUD*

**Problem:** You generated routes for a model that has `$autoCrud = true`, and now requests hit the wrong handler.

**Cause:** Both the generated routes and auto-CRUD routes register at the same paths. The first one registered wins.

**Fix:** Either use generated routes or auto-CRUD, not both for the same model. If you use `tina4 generate route`, set `$autoCrud = false` on the model (or do not set it at all -- it defaults to false).

### *4. Port Already in Use*

**Problem:** `tina4 serve` fails with "Address already in use."

**Cause:** Another process is using port 7145 (or whichever port you configured).

**Fix:** Find and stop the other process, or use a different port:

```
tina4 serve --port 8080
```

To find what is using the port:

```
lsof -i :7145
```

### *5. CLI Not Found After Installation*

**Problem:** `tina4: command not found` after running the install script.

**Cause:** The CLI binary is not in your `PATH`. The install script puts it in `~/.tina4/bin/` which may not be in your shell's PATH.

**Fix:** Add the binary location to your PATH. For zsh (macOS default):

```
echo 'export PATH="$HOME/.tina4/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

For bash:

```
echo 'export PATH="$HOME/.tina4/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

# Chapter 20: Deployment

## 1. From Development to Production

You have built the app. It works perfectly on `localhost:7145`. Now it needs to run 24/7 on a real server, handle 10,000 concurrent users, survive server restarts, and not leak memory. The gap between "it works on my machine" and "it works in production" is where most projects stumble.

This chapter covers everything you need to deploy a Tina4 PHP application to production: environment configuration, Docker packaging, web server setup, SSL, scaling, monitoring, and graceful shutdown handling.

## 2. Production .env Configuration

The first step is configuring your `.env` for production. Development defaults are optimized for debugging. Production defaults are optimized for performance and security.

Create a production `.env`:

```
# Core
TINA4_DEBUG=false
TINA4_LOG_LEVEL=WARNING
TINA4_PORT=7145

# Database
DATABASE_URL=sqlite:///data/app.db

# Security
CORS_ORIGINS=https://yourdomain.com
JWT_SECRET=your-long-random-secret-at-least-32-characters
TINA4_RATE_LIMIT=120

# Performance
TINA4_CACHE_TEMPLATES=true
TINA4_MINIFY_HTML=true

# Console (disabled in production)
TINA4_CONSOLE=false
```

### *Key Differences from Development*

| Setting | Development | Production | Why |

|---------|------------|------------|-----|

| `TINA4_DEBUG` | `true` | `false` | Hides stack traces, disables toolbar |

| `TINA4_LOG_LEVEL` | `ALL` | `WARNING` | Reduces log noise |

| `CORS_ORIGINS` | `*` | Your domain | Prevents cross-origin abuse |

| `TINA4_CONSOLE` | `true` | `false` | Hides dev dashboard |

| `TINA4_CACHE_TEMPLATES` | `false` | `true` | Caches compiled templates |

| `TINA4_MINIFY_HTML` | `false` | `true` | Reduces response size |

### *Sensitive Values*

Never commit production secrets to version control. The `.env` file is gitignored by default. For deployment, use environment variables from your hosting platform, CI/CD secrets, or a secrets management tool.

```
# Docker: pass env vars at runtime
docker run -e JWT_SECRET=your-secret -e DATABASE_URL=sqlite:///data/app.db my-app

# Fly.io: set secrets
fly secrets set JWT_SECRET=your-secret

# Railway: use the dashboard or CLI
railway variables set JWT_SECRET=your-secret
```

## 3. FrankenPHP Auto-Detection

Tina4 PHP auto-detects FrankenPHP at startup. FrankenPHP is a modern PHP application server built on Caddy that provides:

• Worker mode (keeps PHP in memory between requests)

• Built-in HTTPS with automatic certificate management

• HTTP/2 and HTTP/3 support

• Dramatically better performance than PHP-FPM for long-running applications

### *How Auto-Detection Works*

When you run `tina4 serve --production`, the CLI checks for FrankenPHP:

1. If `frankenphp` is in your PATH, it uses FrankenPHP in worker mode

2. If not, it falls back to PHP's built-in server with production optimizations

```
# With FrankenPHP installed
tina4 serve --production

  Tina4 PHP v3.0.0
  Server: FrankenPHP (worker mode)
  Workers: 4
  Running at https://0.0.0.0:7145
  TLS: automatic (Let's Encrypt)

# Without FrankenPHP
tina4 serve --production

  Tina4 PHP v3.0.0
  Server: PHP built-in
  Running at http://0.0.0.0:7145
  Warning: For production, consider FrankenPHP or PHP-FPM + nginx
```

### *Installing FrankenPHP*

```
# macOS
brew install dunglas/tap/frankenphp

# Linux
curl -fsSL https://frankenphp.dev/install.sh | bash

# Docker (no installation needed)
docker pull dunglas/frankenphp
```

## 4. Docker Deployment

Docker is the most reliable way to deploy any application. It packages your code, dependencies, and runtime into a single container that runs identically everywhere.

### *Dockerfile*

Create `Dockerfile` at the project root:

```
FROM dunglas/frankenphp:latest-php8.3-alpine

# Install PHP extensions
RUN install-php-extensions \
    pdo_sqlite \
    mbstring \
    openssl \
    fileinfo

# Set working directory
WORKDIR /app

# Copy Composer files first (for better caching)
COPY composer.json composer.lock ./

# Install dependencies
RUN curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=composer \
    && composer install --no-dev --optimize-autoloader --no-interaction

# Copy application code
COPY . .

# Create required directories
RUN mkdir -p data logs secrets \
    && chown -R www-data:www-data data logs secrets
// ...
```

### *docker-compose.yml*

Create `docker-compose.yml`:

```
version: "3.8"

services:
  app:
    build: .
    ports:
      - "7145:7145"
    environment:
      - TINA4_DEBUG=false
      - TINA4_LOG_LEVEL=WARNING
      - JWT_SECRET=${JWT_SECRET}
      - DATABASE_URL=sqlite:///data/app.db
      - CORS_ORIGINS=https://yourdomain.com
    volumes:
      - app-data:/app/data
      - app-logs:/app/logs
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:7145/health"]
      interval: 30s
      timeout: 5s
```

```
      retries: 3
      start_period: 10s

volumes:
// ...
```

### Build and Run

```
# Build the image
docker build -t my-tina4-app .

# Run it
docker run -d \
  --name my-app \
  -p 7145:7145 \
  -e JWT_SECRET=your-production-secret \
  -v app-data:/app/data \
  my-tina4-app

# Or use docker-compose
docker compose up -d
```

### Verify

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 5,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

### .dockerignore

Create `.dockerignore` to keep the image small:

```
.git
.env
data/
logs/
secrets/
node_modules/
vendor/
tests/
*.md
.DS_Store
```

## 5. PHP-FPM + nginx Configuration

If you prefer a traditional setup without Docker, use PHP-FPM with nginx.

### PHP-FPM Pool Configuration

Create `/etc/php/8.3/fpm/pool.d/tina4.conf`:

```
[tina4]
user = www-data
group = www-data
listen = /run/php/tina4.sock
listen.owner = www-data
```

```
listen.group = www-data

pm = dynamic
pm.max_children = 50
pm.start_servers = 5
pm.min_spare_servers = 5
pm.max_spare_servers = 35
pm.max_requests = 1000

env[DATABASE_URL] = sqlite:///var/www/tina4-app/data/app.db
env[JWT_SECRET] = your-production-secret
env[TINA4_DEBUG] = false
env[TINA4_LOG_LEVEL] = WARNING
```

### *nginx Configuration*

Create `/etc/nginx/sites-available/tina4`:

```
server {
    listen 80;
    server_name yourdomain.com;
    root /var/www/tina4-app/src/public;
    index index.php;

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;

    # Static files
    location ~* \.(css|js|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        expires 30d;
        add_header Cache-Control "public, immutable";
        try_files $uri =404;
    }

    # All other requests go to Tina4
    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    # PHP processing
// ...
```

Enable the site and restart:

```
sudo ln -s /etc/nginx/sites-available/tina4 /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
sudo systemctl restart php8.3-fpm
```

## 6. Health Check Endpoint

Tina4 includes a built-in health check endpoint at `/health`:

```
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 3600,
```

```
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

If the database is disconnected or the application is in a bad state, the health check returns a non-200 status:

```
{
  "status": "error",
  "database": "disconnected",
  "error": "Could not connect to database",
  "uptime_seconds": 3600,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

Use this endpoint for:

• **Docker HEALTHCHECK** (shown in the Dockerfile above)

• **Load balancer health checks** (AWS ALB, nginx upstream, HAProxy)

• **Monitoring tools** (Uptime Robot, Pingdom, custom scripts)

• **Kubernetes liveness and readiness probes**

### *Custom Health Checks*

You can extend the health check to include your own checks:

```
<?php
use Tina4\Route;

Route::get("/health/detailed", function ($request, $response) {
    $checks = [
        "database" => false,
        "cache" => false,
        "disk_space" => false
    ];

    // Check database
    try {
        $product = new Product();
        $product->select("count(*) as cnt");
        $checks["database"] = true;
    } catch (\Exception $e) {
        // Database check failed
    }

    // Check disk space
    $freeBytes = disk_free_space("/");
    $checks["disk_space"] = $freeBytes > 100 * 1024 * 1024; // 100MB minimum

    $allOk = !in_array(false, $checks);

// ...
```

## 7. Graceful Shutdown

When a container or process receives a shutdown signal (SIGTERM), Tina4 handles it gracefully:

1. Stops accepting new connections
2. Finishes processing in-flight requests (up to a configurable timeout)
3. Closes database connections cleanly
4. Flushes logs
5. Exits with status code 0

This prevents data corruption and dropped requests during deployments.

### *Shutdown Timeout*

Set the maximum time to wait for in-flight requests in `.env`:

```
TINA4_SHUTDOWN_TIMEOUT=30
```

If requests are still processing after 30 seconds, the server force-kills them and exits. The default is 30 seconds, which is enough for most applications.

### *Docker Stop Behavior*

Docker sends SIGTERM, waits 10 seconds (by default), then sends SIGKILL. Match the Docker timeout to your shutdown timeout:

```
services:
  app:
    stop_grace_period: 35s   # Slightly more than TINA4_SHUTDOWN_TIMEOUT
```

## 8. Log Rotation

In production, log files grow indefinitely unless rotated. Tina4 writes logs to `logs/app.log`.

### *Using logrotate (Linux)*

Create `/etc/logrotate.d/tina4`:

```
/var/www/tina4-app/logs/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 640 www-data www-data
    postrotate
        # Tina4 re-opens log files automatically
    endscript
}
```

This rotates logs daily, keeps 14 days of compressed history, and creates new log files with the correct permissions.

### *Docker Logging*

In Docker, application logs go to stdout/stderr by default. Configure Docker's logging driver to manage rotation:

```
services:
  app:
```

```
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "5"
```

This keeps a maximum of 50MB of logs (5 files of 10MB each).

## 9. Environment-Specific Configuration

Use different `.env` files for different environments:

```
.env                    # Development (gitignored)
.env.example            # Template (committed to git)
.env.staging            # Staging config (gitignored)
.env.production         # Production config (gitignored)
```

### *Loading a Specific .env File*

Set the `TINA4_ENV_FILE` environment variable before starting:

```
# Staging
TINA4_ENV_FILE=.env.staging tina4 serve

# Production
TINA4_ENV_FILE=.env.production tina4 serve --production
```

In Docker, pass environment variables directly:

```
docker run -e TINA4_DEBUG=false -e JWT_SECRET=xxx my-app
```

Environment variables set via `docker run -e` or `docker-compose environment:` take precedence over `.env` file values. This lets you use a generic `.env` file in the image and override specific values at runtime.

## 10. SSL/TLS with Let's Encrypt

### *With FrankenPHP (Automatic)*

FrankenPHP handles SSL automatically. Just set your domain:

```
TINA4_HOST=yourdomain.com
```

FrankenPHP uses Let's Encrypt to provision and renew certificates automatically. No manual configuration needed.

### *With nginx (Manual)*

Install Certbot:

```
sudo apt install certbot python3-certbot-nginx
```

Obtain a certificate:

```
sudo certbot --nginx -d yourdomain.com
```

Certbot modifies your nginx configuration to include SSL settings and sets up automatic renewal. Verify auto-renewal:

```
sudo certbot renew --dry-run
```

### *With Docker (Using a Reverse Proxy)*

Use Traefik or Caddy as a reverse proxy in front of your Tina4 container:

```
version: "3.8"

services:
  reverse-proxy:
    image: traefik:v3.0
    command:
      - "--providers.docker=true"
      - "--entrypoints.web.address=:80"
      - "--entrypoints.websecure.address=:443"
      - "--certificatesresolvers.letsencrypt.acme.tlschallenge=true"
      - "--certificatesresolvers.letsencrypt.acme.email=you@example.com"
      - "--certificatesresolvers.letsencrypt.acme.storage=/letsencrypt/acme.json"
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - letsencrypt:/letsencrypt

  app:
    build: .
    labels:
      - "traefik.http.routers.app.rule=Host(`yourdomain.com`)"
      - "traefik.http.routers.app.tls=true"
      - "traefik.http.routers.app.tls.certresolver=letsencrypt"
// ...
```

## 11. Scaling

### *Multiple Workers*

FrankenPHP runs multiple worker threads by default. Configure the count in `.env`:

```
TINA4_WORKERS=4
```

A good starting point is the number of CPU cores on your server. For I/O-heavy applications (database queries, API calls), you can go higher -- 2x to 4x the core count.

### *Load Balancing with nginx*

If you run multiple Tina4 instances, use nginx as a load balancer:

```
upstream tina4_backend {
    server 127.0.0.1:7145;
    server 127.0.0.1:7146;
    server 127.0.0.1:7147;
    server 127.0.0.1:7148;
}

server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://tina4_backend;
```

```
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Start four instances on different ports:

```
TINA4_PORT=7145 tina4 serve --production &
TINA4_PORT=7146 tina4 serve --production &
TINA4_PORT=7147 tina4 serve --production &
TINA4_PORT=7148 tina4 serve --production &
```

### *Docker Scaling*

With Docker Compose, scale horizontally:

```
docker compose up -d --scale app=4
```

Use a load balancer (Traefik, nginx, or a cloud load balancer) in front of the containers.

## 12. Monitoring

### *Log Aggregation*

In production, send logs to a centralized service:

```
TINA4_LOG_FORMAT=json
```

JSON-formatted logs can be ingested by services like:

• Elastic Stack (ELK)

• Grafana Loki

• Datadog

• AWS CloudWatch

Example JSON log entry:

```
{
  "timestamp": "2026-03-22T14:30:00.123Z",
  "level": "WARNING",
  "message": "Rate limit exceeded",
  "request_id": "req-abc123",
  "ip": "203.0.113.42",
  "path": "/api/products",
  "method": "GET"
}
```

### *Uptime Monitoring*

Point an external monitoring service at your health endpoint:

```
https://yourdomain.com/health
```

Services like Uptime Robot, Pingdom, or Better Uptime will ping this endpoint every 30-60 seconds and alert you if it stops responding or returns a non-200 status.

### *Application Performance Monitoring (APM)*

For deeper insights, add APM instrumentation. Since Tina4 is standard PHP, any PHP APM agent works:

• New Relic APM

• Datadog APM

• Elastic APM

These agents track request latency, database query performance, error rates, and memory usage automatically.

## 13. Exercise: Deploy a Tina4 PHP App with Docker

Deploy the task management application you have been building throughout this book.

### *Requirements*

1. Create a `Dockerfile` that:

• Uses FrankenPHP as the base image

• Installs Composer and dependencies

• Copies the application code

• Exposes port 7145

• Includes a health check

• Starts the server in production mode

2. Create a `docker-compose.yml` that:

• Builds and runs the application

• Maps port 7145

• Uses environment variables for secrets

• Persists the database and logs via volumes

• Includes restart policy

3. Create a production `.env.production` with:

• Debug mode off

• Warning-level logging

• Template caching enabled

• Appropriate CORS settings

4. Build and run the container.

5. Verify the health check works.

6. Verify you can create and list products through the containerized API.

### *Test with:*

```
# Build
docker compose build
```

```
# Start
docker compose up -d

# Health check
curl http://localhost:7145/health

# Create a product
curl -X POST http://localhost:7145/api/products \
  -H "Content-Type: application/json" \
  -d '{"name": "Docker Widget", "price": 19.99}'

# List products
curl http://localhost:7145/api/products

# View logs
docker compose logs app

# Stop
docker compose down
```

# 14. Solution

## *Dockerfile*

```
FROM dunglas/frankenphp:latest-php8.3-alpine

RUN install-php-extensions \
    pdo_sqlite \
    mbstring \
    openssl \
    fileinfo

WORKDIR /app

COPY composer.json composer.lock ./
RUN curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=composer \
    && composer install --no-dev --optimize-autoloader --no-interaction

COPY . .

RUN mkdir -p data logs secrets \
    && chown -R www-data:www-data data logs secrets

EXPOSE 7145

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD curl -f http://localhost:7145/health || exit 1

CMD ["tina4", "serve", "--production"]
```

## *docker-compose.yml*

```
version: "3.8"

services:
  app:
    build: .
    ports:
      - "7145:7145"
    environment:
      - TINA4_DEBUG=false
```

```
      - TINA4_LOG_LEVEL=WARNING
      - TINA4_CACHE_TEMPLATES=true
      - JWT_SECRET=change-this-to-a-real-secret
      - DATABASE_URL=sqlite:///data/app.db
      - CORS_ORIGINS=http://localhost:7145
    volumes:
      - app-data:/app/data
      - app-logs:/app/logs
    restart: unless-stopped
    stop_grace_period: 35s

volumes:
  app-data:
  app-logs:
```

### *.env.production*

```
TINA4_DEBUG=false
TINA4_LOG_LEVEL=WARNING
TINA4_CACHE_TEMPLATES=true
TINA4_MINIFY_HTML=true
TINA4_CONSOLE=false
TINA4_RATE_LIMIT=120
CORS_ORIGINS=https://yourdomain.com
```

**Expected output for health check:**

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 8,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

**Expected output for creating a product:**

```
{
  "id": 1,
  "name": "Docker Widget",
  "price": 19.99,
  "in_stock": true,
  "created_at": "2026-03-22 14:30:00",
  "updated_at": "2026-03-22 14:30:00"
}
```

## 15. Gotchas

### *1. SQLite Concurrency in Production*

**Problem:** Under high load, SQLite throws "database is locked" errors.

**Cause:** SQLite uses file-level locking. Multiple concurrent writes block each other.

**Fix:** For low-to-medium traffic (under 100 concurrent users), SQLite works fine. For higher traffic, switch to PostgreSQL or MySQL:

```
DATABASE_URL=pgsql://user:pass@localhost:5432/myapp
```

If you must use SQLite under load, enable WAL mode by adding this to your application startup:

```
$db = Tina4\Database::getConnection();
```

```
$db->exec("PRAGMA journal_mode=WAL");
```

### 2. Data Volume Not Persisted

**Problem:** You restart the Docker container and the database is empty.

**Cause:** The `data/` directory is inside the container. When the container is recreated, the data is lost.

**Fix:** Use a Docker volume to persist the data directory, as shown in the docker-compose.yml above:

```
volumes:
  - app-data:/app/data
```

### 3. .env File Not Loaded in Docker

**Problem:** Environment variables from `.env` are not available in the container.

**Cause:** Docker does not automatically load `.env` files inside the container. The `.env` file is for the host machine (used by `docker compose`).

**Fix:** Pass environment variables via the `environment:` section in `docker-compose.yml`, or use `env_file:`:

```
services:
  app:
    env_file:
      - .env.production
```

### 4. Permission Denied on data/ Directory

**Problem:** The application cannot write to `data/` or `logs/` inside the container.

**Cause:** The directories were created by root during the Docker build, but the application runs as `www-data`.

**Fix:** Add `chown` to your Dockerfile:

```
RUN mkdir -p data logs secrets \
    && chown -R www-data:www-data data logs secrets
```

### 5. Health Check Fails During Startup

**Problem:** The container restarts in a loop because the health check fails before the application is ready.

**Cause:** The health check starts immediately, but the application needs a few seconds to initialize.

**Fix:** Use `start_period` in the health check to give the application time to start:

```
HEALTHCHECK --start-period=10s --interval=30s --timeout=5s --retries=3 \
    CMD curl -f http://localhost:7145/health || exit 1
```

The `start-period` tells Docker to ignore health check failures during the first 10 seconds.

### 6. CORS Errors in Production

**Problem:** Your frontend gets "No 'Access-Control-Allow-Origin' header" errors.

**Cause:** `CORS_ORIGINS` is set to `*` in development but to a specific domain in production. If your frontend is served from a different domain or port, CORS blocks the requests.

**Fix:** Set `CORS_ORIGINS` to include all domains that need access:

```
CORS_ORIGINS=https://yourdomain.com,https://admin.yourdomain.com
```

### *7. SSL Certificate Not Renewing*

**Problem:** Your HTTPS certificate expires and the site goes down.

**Cause:** The auto-renewal process (Certbot or FrankenPHP) failed silently. Common reasons: DNS changes, firewall blocking port 80 (needed for ACME challenge), or the renewal service crashed.

**Fix:** Set up monitoring for certificate expiry. Use a tool like `ssl-cert-check` or an online service that alerts you before the certificate expires. Check renewal logs:

```
# Certbot
sudo certbot renew --dry-run

# Check crontab for renewal
sudo systemctl list-timers | grep certbot
```

For FrankenPHP, ensure port 80 and 443 are accessible from the internet for ACME challenges.

# Chapter 21: Building a Complete App

## 1. Putting It All Together

Over the last 20 chapters, you have learned routing, templates, databases, ORM, authentication, middleware, queues, WebSocket, caching, frontend, GraphQL, testing, dev tools, CLI scaffolding, and deployment. Now let us use all of it to build a real application from scratch.

We are building **TaskFlow** -- a task management system with:

• User registration and JWT authentication

• Task creation, assignment, and tracking

• A dashboard with real-time updates

• Email notifications when tasks are assigned

• Caching for dashboard performance

• A full test suite

• Docker deployment

This is not a toy project. It is a complete, production-ready application that demonstrates every major Tina4 feature working together.

## 2. Planning the App

### *Models*

| Model | Table | Fields |

|-------|-------|--------|

| User | users | id, name, email, passwordHash, role, createdAt |

| Task | tasks | id, title, description, status, priority, createdBy, assignedTo, dueDate, completedAt, createdAt, updatedAt |

### *Relationships*

• A User has many Tasks (created by them)

• A User has many Tasks (assigned to them)

• A Task belongs to a User (creator)

• A Task belongs to a User (assignee)

### *Routes*

| Method | Path | Description | Auth |

|--------|------|-------------|------|

| POST | /api/auth/register | Register a new user | public |

| POST | /api/auth/login | Login, get JWT | public |

| GET | /api/profile | Get current user profile | secured |

| GET | /api/tasks | List tasks (with filters) | secured |

| GET | /api/tasks/{id} | Get a single task | secured |

| POST | /api/tasks | Create a task | secured |

| PUT | /api/tasks/{id} | Update a task | secured |

| DELETE | /api/tasks/{id} | Delete a task | secured |

| GET | /api/dashboard/stats | Dashboard statistics | secured |

| GET | /admin | Dashboard HTML page | public |

### *Templates*

• `base.html` -- Base layout with sidebar and topbar

• `dashboard.html` -- Dashboard with stats, task list, quick actions

• `login.html` -- Login page

• `register.html` -- Registration page

## 3. Step 1: Init Project and Set Up Database

```
tina4 init taskflow
cd taskflow
composer install
```

Update `.env`:

```
TINA4_DEBUG=true
TINA4_CONSOLE=true
TINA4_CONSOLE_TOKEN=dev-token
JWT_SECRET=taskflow-dev-secret-change-in-production
JWT_EXPIRY=86400
```

### *Create Migrations*

Create `src/migrations/20260322000100_create_users_table.sql`:

```
-- UP
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL,
    role TEXT NOT NULL DEFAULT 'user',
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);

-- DOWN
DROP TABLE IF EXISTS users;
```

Create `src/migrations/20260322000200_create_tasks_table.sql`:

```
-- UP
CREATE TABLE tasks (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT DEFAULT '',
    status TEXT NOT NULL DEFAULT 'todo',
    priority TEXT NOT NULL DEFAULT 'medium',
    created_by INTEGER NOT NULL,
    assigned_to INTEGER,
    due_date TEXT,
    completed_at TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES users(id),
    FOREIGN KEY (assigned_to) REFERENCES users(id)
);

CREATE INDEX idx_tasks_status ON tasks(status);
CREATE INDEX idx_tasks_assigned_to ON tasks(assigned_to);
CREATE INDEX idx_tasks_created_by ON tasks(created_by);

-- DOWN
DROP TABLE IF EXISTS tasks;
```

Run the migrations:

```
tina4 migrate
```

```
Running migrations...
  [APPLIED] 20260322000100_create_users_table.sql
  [APPLIED] 20260322000200_create_tasks_table.sql
Migrations complete. 2 applied.
```

## 4. Step 2: User Model with Registration and Login

Create `src/orm/User.php`:

```
<?php
use Tina4\ORM;

class User extends ORM
{
    public int $id;
    public string $name;
    public string $email;
    public string $passwordHash;
    public string $role = "user";
    public string $createdAt;

    public string $tableName = "users";
    public string $primaryKey = "id";

    /**
     * Get tasks created by this user
     */
    public function createdTasks(): array
    {
        return $this->hasMany(Task::class, "created_by");
    }

    /**
     * Get tasks assigned to this user
// ...
```

### Authentication Routes

Create `src/routes/auth.php`:

```
<?php
use Tina4\Route;
use Tina4\Auth;

/**
 * @noauth
 */
Route::post("/api/auth/register", function ($request, $response) {
    $body = $request->body;

    // Validate input
    if (empty($body["name"]) || empty($body["email"]) || empty($body["password"])) {
        return $response->json(["error" => "name, email, and password are required"], 400);
    }

    if (strlen($body["password"]) < 8) {
        return $response->json(["error" => "Password must be at least 8 characters"], 400);
    }

    if (!filter_var($body["email"], FILTER_VALIDATE_EMAIL)) {
        return $response->json(["error" => "Invalid email address"], 400);
    }

    // Check for existing user
    $existing = new User();
// ...
```

### Test Registration and Login

```
# Start the server
tina4 serve

# Register a user
curl -X POST http://localhost:7145/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice Johnson", "email": "alice@example.com", "password": "securepass123"}'

{
  "id": 1,
  "name": "Alice Johnson",
  "email": "alice@example.com",
  "role": "user",
  "created_at": "2026-03-22 10:00:00"
}

# Login
curl -X POST http://localhost:7145/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email": "alice@example.com", "password": "securepass123"}'

{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "role": "user"
  }
}

# Access protected route
```

```
curl http://localhost:7145/api/profile \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

```
{
  "user": {
    "id": 1,
    "name": "Alice Johnson",
    "email": "alice@example.com",
    "role": "user"
  }
}
```

## 5. Step 3: Task Model with CRUD

Create `src/orm/Task.php`:

```
<?php
use Tina4\ORM;

class Task extends ORM
{
    public int $id;
    public string $title;
    public string $description = "";
    public string $status = "todo";
    public string $priority = "medium";
    public int $createdBy;
    public ?int $assignedTo = null;
    public ?string $dueDate = null;
    public ?string $completedAt = null;
    public string $createdAt;
    public string $updatedAt;

    public string $tableName = "tasks";
    public string $primaryKey = "id";

    /**
     * Get the user who created this task
     */
    public function creator(): ?User
    {
// ...
```

### *Task Routes*

Create `src/routes/tasks.php`:

```
<?php
use Tina4\Route;

Route::group("/api", function () {

    // List tasks with filters
    /**
     * @secured
     */
    Route::get("/tasks", function ($request, $response) {
        $userId = $request->user["user_id"];

        $status = $request->query["status"] ?? "";
        $priority = $request->query["priority"] ?? "";
        $assigned = $request->query["assigned"] ?? "";
```

```
        $page = (int) ($request->query["page"] ?? 1);
        $perPage = (int) ($request->query["per_page"] ?? 20);

        $conditions = [];
        $params = [];

        // Show tasks created by or assigned to the current user
        $conditions[] = "(created_by = :userId OR assigned_to = :userId2)";
        $params["userId"] = $userId;
        $params["userId2"] = $userId;
// ...
```

### *Test the Task API*

```
# Get token (from login)
TOKEN="eyJhbGciOiJIUzI1NiIs..."

# Create tasks
curl -X POST http://localhost:7145/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Design database schema", "priority": "high", "status": "done"}'

curl -X POST http://localhost:7145/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Build API endpoints", "priority": "high", "status": "in_progress"}'

curl -X POST http://localhost:7145/api/tasks \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"title": "Write documentation", "priority": "medium", "due_date": "2026-04-01"}'

# List all tasks
curl http://localhost:7145/api/tasks \
  -H "Authorization: Bearer $TOKEN"

# Filter by status
curl "http://localhost:7145/api/tasks?status=in_progress" \
// ...
```

## 6. Step 4: Dashboard Template with tina4css

Create `src/templates/app/layout.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>{% block title %}TaskFlow{% endblock %}</title>
    <link rel="stylesheet" href="/css/tina4.css">
    <script>
        (function() {
            var t = localStorage.getItem("theme");
            if (t) document.documentElement.setAttribute("data-theme", t);
            else if (window.matchMedia("(prefers-color-scheme: dark)").matches)
                document.documentElement.setAttribute("data-theme", "dark");
        })();
    </script>
    <style>
```

```
        * { margin: 0; padding: 0; box-sizing: border-box; }
        body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif; }
        .app { display: flex; min-height: 100vh; }
        .sidebar {
            width: 240px; background: #1a1a2e; color: #ccc; flex-shrink: 0;
        }
        .sidebar-brand {
            padding: 20px; font-size: 1.3em; font-weight: bold; color: #fff;
            border-bottom: 1px solid rgba(255,255,255,0.1);
// ...
```

Create `src/templates/app/dashboard.html`:

```
{% extends "app/layout.html" %}

{% block title %}Dashboard - TaskFlow{% endblock %}
{% block page_title %}Dashboard{% endblock %}

{% block content %}
    <div class="stats-row">
        <div class="stat-card">
            <div class="value" id="statTotal">--</div>
            <div class="label">Total Tasks</div>
        </div>
        <div class="stat-card">
            <div class="value" id="statTodo">--</div>
            <div class="label">To Do</div>
        </div>
        <div class="stat-card">
            <div class="value" id="statInProgress">--</div>
            <div class="label">In Progress</div>
        </div>
        <div class="stat-card">
            <div class="value" id="statDone">--</div>
            <div class="label">Completed</div>
        </div>
    </div>

// ...
```

### *Dashboard Stats Route*

Create `src/routes/dashboard.php`:

```
<?php
use Tina4\Route;

/**
 * @secured
 */
Route::get("/api/dashboard/stats", function ($request, $response) {
    $userId = $request->user["user_id"];

    $task = new Task();
    $filter = "(created_by = :uid OR assigned_to = :uid2)";
    $params = ["uid" => $userId, "uid2" => $userId];

    $all = $task->select("*", $filter, $params);

    $stats = [
        "total" => count($all),
        "todo" => 0,
        "in_progress" => 0,
        "review" => 0,
```

```
        "done" => 0
    ];

    foreach ($all as $t) {
        if (isset($stats[$t->status])) {
// ...
```

## 7. Step 5: Real-Time Updates via WebSocket

Add real-time task updates so all connected dashboard users see changes instantly.

Create `src/routes/websocket.php`:

```
<?php
use Tina4\WebSocket;

WebSocket::on("connect", function ($client) {
    error_log("Client connected: " . $client->id);
});

WebSocket::on("disconnect", function ($client) {
    error_log("Client disconnected: " . $client->id);
});

WebSocket::on("message", function ($client, $message) {
    $data = json_decode($message, true);

    if ($data["type"] === "subscribe" && $data["channel"] === "tasks") {
        WebSocket::subscribe($client, "tasks");
    }
});
```

Broadcast task changes by updating the task routes. Add this helper function at the top of `src/routes/tasks.php`:

```
function broadcastTaskUpdate(string $event, array $taskData): void
{
    \Tina4\WebSocket::broadcast("tasks", json_encode([
        "type" => "task_update",
        "event" => $event,
        "task" => $taskData
    ]));
}
```

Call it after each write operation:

```
// After creating a task
$task->save();
broadcastTaskUpdate("created", $task->toDetailedDict());

// After updating a task
$task->save();
broadcastTaskUpdate("updated", $task->toDetailedDict());

// After deleting a task
$task->delete();
broadcastTaskUpdate("deleted", ["id" => $request->params["id"]]);
```

Add WebSocket client code to the dashboard template. Add this inside the `{% block extra_js %}` block:

```
// WebSocket connection for real-time updates
var ws = new WebSocket("ws://localhost:7145/ws");
```

```
ws.onopen = function () {
    ws.send(JSON.stringify({ type: "subscribe", channel: "tasks" }));
};

ws.onmessage = function (event) {
    var data = JSON.parse(event.data);
    if (data.type === "task_update") {
        loadTasks();
        loadStats();
        showAlert("Task " + data.event + ": " + (data.task.title || ""), "info");
    }
};

ws.onclose = function () {
    // Reconnect after 3 seconds
    setTimeout(function () {
        ws = new WebSocket("ws://localhost:7145/ws");
    }, 3000);
};
```

Now when any user creates, updates, or deletes a task, all connected dashboard users see the change immediately.

## 8. Step 6: Email Notifications on Task Assignment

When a task is assigned to a user, send them an email notification.

Create `src/routes/notifications.php`:

```
<?php
use Tina4\Mail;

function sendTaskAssignmentEmail(Task $task, User $assignee, User $assigner): void
{
    $subject = "New task assigned: " . $task->title;

    $body = "Hi " . $assignee->name . ",\n\n"
        . $assigner->name . " has assigned you a new task:\n\n"
        . "Title: " . $task->title . "\n"
        . "Priority: " . strtoupper($task->priority) . "\n"
        . "Due: " . ($task->dueDate ?? "No due date") . "\n\n"
        . "Description:\n" . ($task->description ?: "(No description)") . "\n\n"
        . "View it at: http://localhost:7145/admin\n";

    Mail::send(
        $assignee->email,
        $subject,
        $body
    );
}
```

In the task creation and update routes, call this function when `assigned_to` is set:

```
// After setting assigned_to and saving
if ($task->assignedTo && $task->assignedTo !== $userId) {
    $assignee = new User();
    $assignee->load($task->assignedTo);

    $assigner = new User();
    $assigner->load($userId);

    if (!empty($assignee->id) && !empty($assigner->id)) {
```

```
        sendTaskAssignmentEmail($task, $assignee, $assigner);
    }
}
```

Configure email in `.env`:

```
TINA4_MAIL_HOST=smtp.example.com
TINA4_MAIL_PORT=587
TINA4_MAIL_USER=notifications@example.com
TINA4_MAIL_PASS=your-email-password
TINA4_MAIL_FROM=notifications@example.com
TINA4_MAIL_FROM_NAME=TaskFlow
```

For development, you can use a local mail trap like MailHog or Mailtrap.io so emails are captured without actually sending.

## 9. Step 7: Add Caching for the Dashboard

The dashboard stats query runs on every page load. With many tasks, this can be slow. Add caching so the stats are computed once and served from cache for subsequent requests.

Update the dashboard stats route:

```
/**
 * @secured
 */
Route::get("/api/dashboard/stats", function ($request, $response) {
    $userId = $request->user["user_id"];
    $cacheKey = "dashboard_stats_" . $userId;

    // Try cache first
    $cached = \Tina4\Cache::get($cacheKey);
    if ($cached !== null) {
        return $response->json($cached);
    }

    // Compute stats
    $task = new Task();
    $filter = "(created_by = :uid OR assigned_to = :uid2)";
    $params = ["uid" => $userId, "uid2" => $userId];

    $all = $task->select("*", $filter, $params);

    $stats = [
        "total" => count($all),
        "todo" => 0,
        "in_progress" => 0,
        "review" => 0,
// ...
```

Invalidate the cache when tasks change. Add this to the `broadcastTaskUpdate` function:

```
function broadcastTaskUpdate(string $event, array $taskData): void
{
    // Invalidate dashboard cache for all affected users
    if (isset($taskData["created_by"])) {
        \Tina4\Cache::delete("dashboard_stats_" . $taskData["created_by"]);
    }
    if (isset($taskData["assigned_to"]) && $taskData["assigned_to"]) {
        \Tina4\Cache::delete("dashboard_stats_" . $taskData["assigned_to"]);
    }
```

```
    \Tina4\WebSocket::broadcast("tasks", json_encode([
        "type" => "task_update",
        "event" => $event,
        "task" => $taskData
    ]));
}
```

## 10. Step 8: Write Tests

Create `tests/TaskFlowTest.php`:

```
<?php
use Tina4\Test;

class TaskFlowTest extends Test
{
    private ?string $token = null;
    private ?int $userId = null;

    public function setUp(): void
    {
        // Register a test user
        $email = "test-" . uniqid() . "@taskflow.test";
        $regResponse = $this->post("/api/auth/register", [
            "name" => "Test User",
            "email" => $email,
            "password" => "testpassword123"
        ]);

        $regBody = json_decode($regResponse->body, true);
        $this->userId = $regBody["id"] ?? null;

        // Login to get token
        $loginResponse = $this->post("/api/auth/login", [
            "email" => $email,
            "password" => "testpassword123"
// ...
```

Run the tests:

```
tina4 test
```

```
Running tests...

  TaskFlowTest
    [PASS] test_registration_returns_201
    [PASS] test_registration_rejects_short_password
    [PASS] test_login_returns_token
    [PASS] test_profile_requires_auth
    [PASS] test_profile_with_token
    [PASS] test_create_task
    [PASS] test_create_task_requires_title
    [PASS] test_list_tasks
    [PASS] test_update_task_status
    [PASS] test_delete_task
    [PASS] test_dashboard_stats

  11 tests, 11 passed, 0 failed (0.62s)
```

## 11. Step 9: Deploy with Docker

Create `Dockerfile`:

```
FROM dunglas/frankenphp:latest-php8.3-alpine

RUN install-php-extensions \
    pdo_sqlite \
    mbstring \
    openssl \
    fileinfo

WORKDIR /app

COPY composer.json composer.lock ./
RUN curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=composer \
    && composer install --no-dev --optimize-autoloader --no-interaction

COPY . .

RUN mkdir -p data logs secrets \
    && chown -R www-data:www-data data logs secrets

EXPOSE 7145

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
    CMD curl -f http://localhost:7145/health || exit 1

CMD ["tina4", "serve", "--production"]
```

Create `docker-compose.yml`:

```
version: "3.8"

services:
  app:
    build: .
    ports:
      - "7145:7145"
    environment:
      - TINA4_DEBUG=false
      - TINA4_LOG_LEVEL=WARNING
      - TINA4_CACHE_TEMPLATES=true
      - JWT_SECRET=${JWT_SECRET:-change-me-in-production}
      - DATABASE_URL=sqlite:///data/app.db
    volumes:
      - taskflow-data:/app/data
      - taskflow-logs:/app/logs
    restart: unless-stopped
    stop_grace_period: 35s

volumes:
  taskflow-data:
  taskflow-logs:
```

Build and deploy:

```
# Build
docker compose build

# Start
docker compose up -d

# Run migrations inside the container
```

```
docker compose exec app tina4 migrate

# Verify
curl http://localhost:7145/health
```

```
{
  "status": "ok",
  "database": "connected",
  "uptime_seconds": 5,
  "version": "3.0.0",
  "framework": "tina4-php"
}
```

## 12. The Complete Project Structure

```
taskflow/
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ composer.json
■■■ composer.lock
■■■ Dockerfile
■■■ docker-compose.yml
■■■ vendor/
■■■ src/
■   ■■■ routes/
■   ■   ■■■ auth.php              # Registration, login, profile
■   ■   ■■■ tasks.php             # Task CRUD
■   ■   ■■■ dashboard.php         # Dashboard stats + page
■   ■   ■■■ notifications.php     # Email notification helpers
■   ■   ■■■ websocket.php         # WebSocket event handlers
■   ■■■ orm/
■   ■   ■■■ User.php              # User model with auth methods
■   ■   ■■■ Task.php              # Task model with relationships
■   ■■■ migrations/
■   ■   ■■■ 20260322000100_create_users_table.sql
■   ■   ■■■ 20260322000200_create_tasks_table.sql
■   ■■■ templates/
■   ■   ■■■ app/
■   ■   ■   ■■■ layout.html       # Base layout with sidebar
// ...
```

Every file has a purpose. Every directory follows the convention. A new developer can look at this structure and know exactly where to find things.

## 13. What to Build Next

TaskFlow is a solid foundation. Here are ideas for extending it:

**Features:**

• **Task comments** -- Add a Comment model with a `task_id` foreign key. Display comments on the task detail page.

• **File attachments** -- Let users upload files to tasks. Store them in `data/uploads/` and serve them via a route.

• **Team management** -- Add a Team model. Users belong to teams. Tasks are scoped to teams.

• **Task labels/tags** -- Many-to-many relationship between tasks and labels for categorization.

• **Due date reminders** -- Use the queue system to schedule reminder emails 24 hours before a task's due date.

• **Activity log** -- Record every change to a task (who changed what, when) for audit trails.

• **Search** -- Full-text search across task titles and descriptions.

• **Calendar view** -- Render tasks on a calendar based on their due dates.

• **Mobile API** -- The API already works for mobile apps. Add push notification support via Firebase Cloud Messaging.

**Technical improvements:**

• **Rate limiting per user** -- Replace the global rate limiter with per-user limits.

• **Database upgrade** -- Switch from SQLite to PostgreSQL for better concurrency.

• **CI/CD pipeline** -- Add GitHub Actions to run tests automatically on every push.

• **API documentation** -- Generate OpenAPI/Swagger docs from your route definitions.

• **Internationalization** -- Add `src/locales/` files for multiple languages.

## 14. Closing Thoughts -- The Tina4 Philosophy

You have just built a complete application -- user auth, CRUD, real-time updates, email, caching, tests, deployment -- and your project has exactly one dependency: `tina4/tina4-php`.

No ORM package. No template engine package. No authentication library. No WebSocket server. No caching library. No testing framework. No CLI tool. No CSS framework. No JavaScript helpers. They are all built in.

This is the Tina4 philosophy:

**One framework. Zero dependencies. Everything you need.**

The same patterns you used here work identically in Python, Ruby, and Node.js. The project structure is the same. The CLI commands are the same. The `.env` variables are the same. The template syntax is the same. Learn Tina4 once, use it everywhere.

Your `vendor/` directory is small. Your `composer.lock` has one entry. When PHP 9.0 ships, you update one package and everything works. No dependency tree to untangle. No abandoned transitive dependency to replace. No security advisory for a package four levels deep that you did not even know you were using.

Simple does not mean limited. TaskFlow has authentication, real-time WebSocket, email, caching, GraphQL, and a test suite. It deploys in a Docker container and handles thousands of concurrent users. It does all of this in under 5,000 lines of framework code.

Build things. Ship them. Keep it simple.